



CHESS NEA

BY DANIEL LAWRENCE

Contents

Current System.....	8
Summary of Task.....	8
Description:.....	8
Walkthrough:	10
DFD (level 0 - current system)	13
DFD (Level 1 - current system)	14
Analysis techniques	15
Formal analysis	15
Proposed solution.....	18
Description.....	18
DFD (level 0 - proposed solution)	21
DFD (level 1 - proposed solution)	22
UML.....	23
E-R Diagrams.....	23
Sign up/Login.....	23
Game	24
Proposed User	24
Investigation.....	25
Project Management	27
Critical Path Analysis	27
Objectives	28
Analysis Log	30
Overview.....	31
System Design	32
DFD of proposed system	32
Hierarchy charts	33
Whole system.....	33
Play	33
Customise.....	34
Leaderboard	34
Replays.....	34
Data Structures.....	35
Database design & dictionaries.....	35

Object model and class definitions	36
Pawn.....	36
Rook.....	37
Knight	38
Bishop	39
Queen	40
King.....	41
Game	42
User defined structures	43
HCI.....	44
Login.....	44
Sign-Up.....	46
Main menu	47
Play	48
Singleplayer	48
Multiplayer.....	49
Game	51
Pawn Promotion.....	53
Customise.....	55
Leaderboard	56
Replays.....	57
Rules	58
Settings.....	59
Objective.....	60
1 - Signing up.....	60
Summary	60
Research.....	60
UML.....	61
Pseudo Code.....	62
VB.Net Code.....	63
Test Plan.....	66
2 - Logging in.....	67
Summary	67
Research.....	67
Pseudo Code(Main)	69

Pseudo Code(Forgotten password)	69
VB.Net Code.....	70
Test Plan.....	73
3 - Game form.....	74
Summary	74
Research.....	74
UML(Account/Board Loading)	74
UML(End of game).....	75
Pseudo Code.....	75
VB.Net Code.....	76
Test Plan.....	105
4 - Game logic.....	107
Summary	107
Research.....	107
UML.....	107
VB.Net Code.....	108
Test Plan.....	125
5 - AI opponent.....	128
Summary	128
Research.....	128
UML.....	128
Pseudo Code.....	129
VB.Net Code(Easy AI)	131
VB.Net Code(Hard AI).....	132
Test Plan.....	151
6 - Replays	152
Summary	152
Research.....	152
UML.....	153
Pseudo Code.....	153
VB.Net Code.....	156
Test Plan.....	166
7 - Leaderboard.....	167
Summary	167
Research.....	167

UML.....	167
Pseudo Code.....	168
VB.Net Code.....	168
Test Plan.....	170
8 - Customisation	171
Summary	171
Research.....	171
UML.....	171
Pseudo Code.....	172
VB.Net Code.....	172
Test Plan.....	179
9 - Rules	180
Summary	180
Research.....	180
UML.....	180
Pseudo Code.....	180
VB.Net Code.....	180
Test Plan.....	180
10 - Settings	181
Summary	181
Research.....	181
UML(Change password).....	181
UML(Delete account)	181
Pseudo Code.....	182
Pseudo Code(Delete account).....	182
VB.Net Code.....	182
Test Plan(Change Password)	184
Test Plan(Delete Account).....	184
Full VB.Net Source Code	185
Forms	186
Account Settings	186
BPawnChange.....	188
WPawnChange.....	191
Customisation	193
Forgotten Password.....	201

GameForm1	203
Home.....	246
Leaderboard	246
Leaderboard	249
Main Menu.....	252
Multiplayer.....	259
Play	262
Replay	263
Rules	274
Settle	274
Sign-up.....	277
Singleplayer	281
Classes	283
Bishop	283
Game	300
King.....	373
Knight	391
Pawn.....	420
Queen	440
Rook.....	472
Signing-up systems testing	489
Username	489
Password	492
Email	494
Logging in system testing	495
Login	495
Forgotten Password.....	497
Game form system testing	500
Loading account.....	500
Loading pieces onto the board	502
Saving replays.....	503
Game logic system testing	505
Pawn.....	505
Rook.....	514
Knight	518

Bishop	520
Queen	524
King	532
AI Opponent system testing.....	534
Minimax.....	534
Replays system testing.....	538
Loading Replays	538
Leaderboard system testing.....	540
Loading Leaderboard	540
Customisation system testing.....	541
Profile Picture	541
Board Colour.....	543
Piece Set.....	544
Settings system testing.....	545
Change Password	545
Appraisal	546
Evaluation against objectives.....	547
User feedback.....	Error! Bookmark not defined.
Analysis of user feedback	Error! Bookmark not defined.
During implementation	Error! Bookmark not defined.
From user testing.....	Error! Bookmark not defined.
Improvements	Error! Bookmark not defined.

Current System

Summary of Task

Chess is a beautiful game, regarding its complexity and enjoyment. I will develop a chess game in which the audience (adults and children - however as chess is not a simple game to play, very young children wouldn't be recommended to play it) will play either against another person in a 'multiplayer' mode, or against a Chess AI algorithm in a 'singleplayer' mode.

Before the user plays the game, they will have to sign up with personal details about themselves to create an account. This will be stored into a database (using Microsoft Access), with fields such as the username, password, wins, loses etc. A leaderboard could be created in order to implement some sort of ranking system, i.e., ranked on users with the most wins. As I store the users' statistics in the database, I can create an unlocking system with some user customisations, giving the user their own personal feel on their chess experience – *side note, incorporate a word document the user can access to read about the rules of chess.*

When playing the game, I will need a menu choice where the user decides whether they want to play with another user or against the AI. If the user chooses the AI, it could be split up into two levels: one level could consist of the AI just making random moves (easy mode – low level intelligence, no strategy), and the other level could consist of the AI making moves dependent on the best possible moves available (hard mode – high level intelligence, with strategy). There are two ways of creating an AI, self-learning using pattern recognition and neural networks, or the method my program will use, minimax algorithms. Both extremely useful methods, however I chose the minimax algorithm as it seemed more appropriate after planning out the basic code aspects.

The purpose of the game chess is to strategically defeat your opponent (checkmate), which happens when the king (and no other piece for that matter), can move on the board. The reason why chess is such a brilliant game to play is that it's discovered to develop better brain function, improved memory, strategic thinking and improvements in attention. Scientists have claimed that it can improve mental age by up to 14 years, therefore being a recommended game to play.

Description:

- **Graphics and visual displacement** – whilst researching about how I was going to add in the graphics of each piece, I found out that one way of adding the graphical side of things was to import the images through Unicode. As each chess piece has a unique Unicode tag it seemed quite a simple task, however another option was to add chess images through picture boxes within visual basic. To create a basic board to play on, there must be an 8 by 8 grid with light and dark tiles alongside each other. The board will have a coordinate system – the program will use a 2D array, whilst the user will see a letter and number system. Letters will be on the horizontal axis and numbers will be on the vertical. Whilst the game is being played, a square can either have a piece in it (value not equal to 0), or a square can be blank (value equal to 0).
- **Movement of pieces** – there are 4 types of movement that can happen in chess: Vertically (up and down, used by rooks, kings, queens and pawns), diagonally (used by bishops, queens, kings and pawns), and L shape movement (only by knight). A pawn has a unique movement, where it can move either 2 spaces forward from the start position, or 1 space forward in any other position. There are also special moves such as Pawn Promotion (when a pawn reaches the opposite end of the board and can be promoted into a different piece type) and Castling (where the king swaps position with one of the starting rooks). These movement choices are special cases and should be

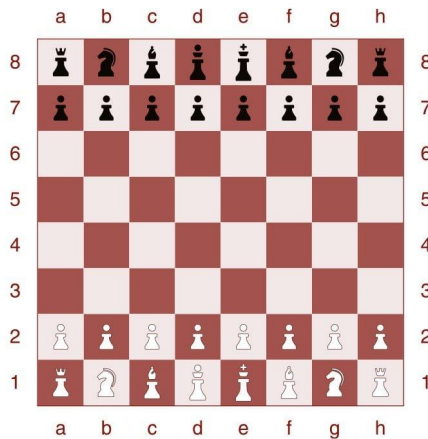
added towards the latter end of the coding. *Reminder – pieces cannot jump/travel through each other, with the only exception being the movement of the knight, and when capturing opponents' pieces.*

- **Ending of games** – 2 outcomes can happen in chess, either someone wins, or someone draws. A user can win if they checkmate their opponent (or the other resigns), and a draw can happen if either both users agree to a stalemate, or a player cannot make any legal moves on the board and is not in check.
- **Concept of AI chess** – computing all possible moves and select one either randomly (easy mode), or strategically (hard move). I have listed the simplified steps to create an AI below:
 1. Generation and visualisation of the board and pieces
 2. Position evaluation – start by giving each piece a strength value, i.e., pawn = 10, queen = 90, knight = 30.
 3. Use search trees using minimax algorithm so the AI can choose which is the best move. In the algorithm, the recursive tree of all possible moves is explored with a depth (how many moves ahead it looks), it will analyse all possible outcomes and choose the 'least losing' move.
- **Rules** – 2 opponents, one white and one black, with white always playing the first move. Each side will have a king, queen, two rooks, two bishops, and eight pawns. The initial positions at the start of each new game are always the same. The basic position set up for the game is on each end of the board there is a row of pawns and the other pieces in fixed positions behind them.
- **Disadvantages of current system** – chess takes a good amount of time to learn and become a decent level. You can become frustrated quite easily if the player you're playing, or the AI are skillful. This means that people would give up quite easily and those people who don't have a long attention span wouldn't be interested in playing.
- **Advantages of current system** – game helps train your mind with memory, focus, and boost your planning skills. Chess is such a mind working game that the best players look numerous moves ahead. When developed correctly, chess is one of the most successful games of all time. When I develop mine, I want it to be the most user-friendly experience as possible to create a good experience for them. My system will have to have a good balance of sophisticatedly and simple features for all age ranges of users. Finally, a good AI can really improve the skill level of players rapidly, as they don't make many 'bad' moves.

Walkthrough:

- **Graphics and visual displacement**

1. **Board visualisation** – the graphical side of the board the user sees, can be implemented by changing the background image (of a suitable space) into a simple chess board image found. To apply graphical pieces, a method used is to import the Unicode values of each piece, or to import graphical images into picture boxes.
2. **Computational board visualisation** – two dimensional arrays are used for the computer to process the board and pieces within it. An 8 by 8 array should be used to cover every square on the board, with each component either containing a certain number if it contains a piece, or a zero if the space is blank. When the user selects a square on the board, each square will have x and y coordinates, which will be how the computer algorithmically calculates the possible moves for the piece.



Simple representation of the board and the start position of the pieces (what the user will see)

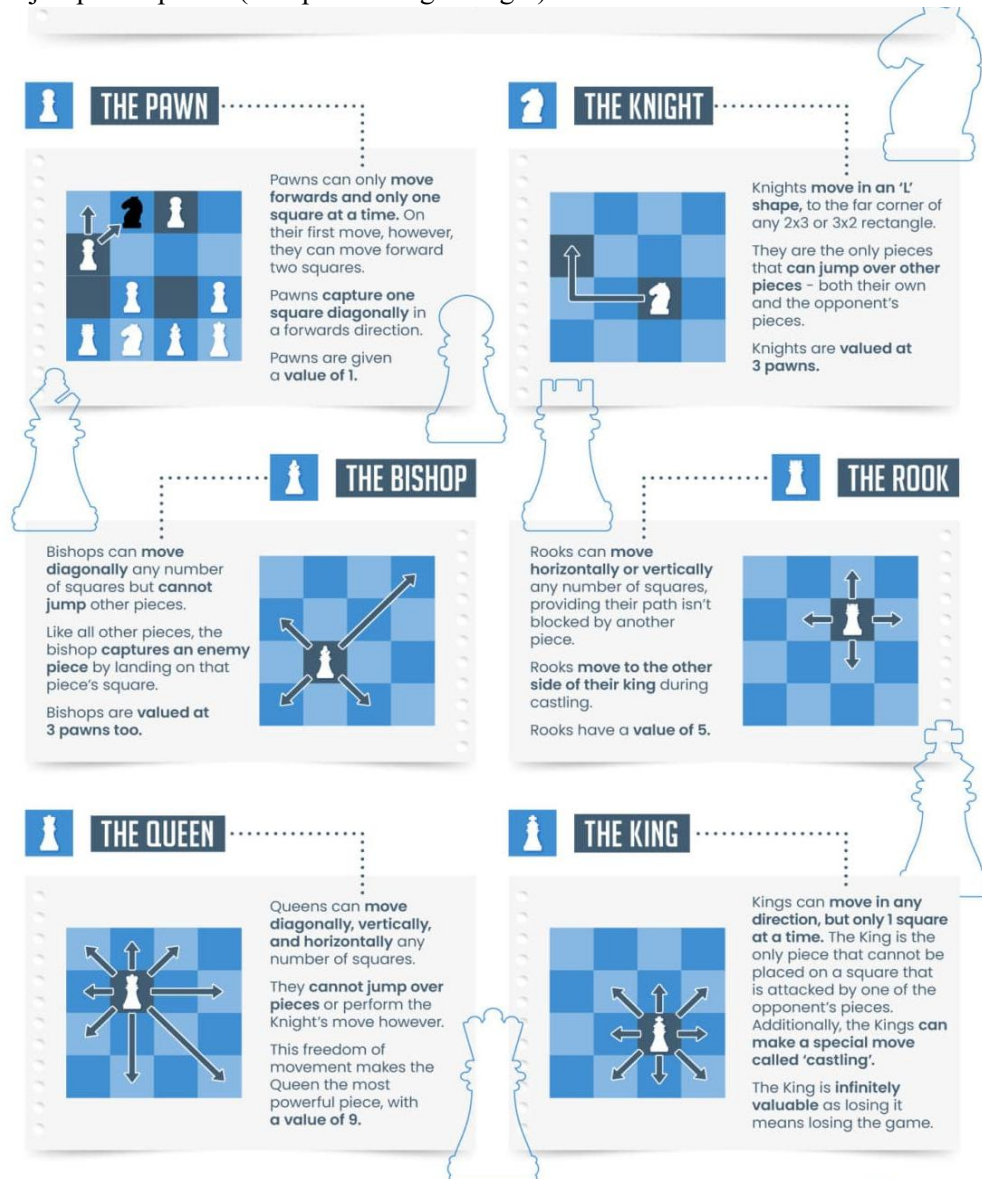
	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

Example of what a two-dimensional array looks like. Each element is given a coordinate for the computer to work with, e.g., element (0,0) has coordinates x = 0, y = 0, element (0, 1) has coordinates x = 0, y = 1...

- **Movement of pieces**

1. **The pawn** - pawns can move vertically forward two squares if the pawn is still in its starting position, otherwise one move forward. They are the only piece that can capture pieces in a different way to how they move. Pawns can only move forwards, however when capturing pieces, they move one space diagonally. Under no circumstances can the pawn move backwards, with the only exception being if the pawn reaches the opposite end of the board (*Pawn Promotion*) and turns into a different type of piece.
2. **The rook** - rooks can move vertically or horizontally for any number of squares and in any direction. The special case of movement (*Castling*) is a common moved in high level chess, where the king swaps position with the rook.
3. **The knight** - knights move in an 'L' shape: two squares in a horizontal or vertical direction, then move one square horizontally or vertically. This is the only piece on the board that can jump over other pieces.
4. **The bishop** - bishops move diagonally for any number of squares and in any direction.
5. **The queen** - queens can move vertically, horizontally, or diagonally for any number of squares. The queen is the most powerful piece in the game, making it the most valuable piece to have (after the obvious king).

6. **The king** - kings can move one square in any direction, if that square is not 'attacked' by an enemy piece. They can also (*mentioned under the rook*), perform a special move (*Castling*), where the king swaps position with the rook.
7. **All pieces** - a rule for all pieces is that they cannot travel through pieces (no exceptions), or jump over pieces (exception being a knight)



- **Ending of games**

Winning/Losing

1. **Checkmate** - checkmate is the most common way that a game ends in chess. It happens when one player is threatening in the other king and it cannot move to any square available, cannot be blocked by another piece, and the checking piece cannot be captured.
2. **Resignation** - sometimes a player believes that they will be checkmated soon and resigns instead of playing the rest of the game.

3. **Timeout** - if you run out of time you will automatically lose the game. Managing your time is therefore very crucial in a game.

Drawing

4. **Stalemate** - this can happen when a player has no possible moves to make, and the king is not in check
5. **Insufficient material** - this happens when the only pieces on the board are a king for black and a king for white
6. **Agreement** - when both players decide they want to draw the game, there is a draw by agreement. Most of the time this is because they believe that neither player can obtain an advantage.

• Concept of AI chess

1. **Piece selection** - the first step to create an AI is for the computer to select a piece
2. **Calculate possible moves** - with the piece that the AI has selected, it should call upon a routine that calculates every single move that is possible for that piece
3. **Moving selected piece** - move the piece the AI has selected to ever possible square
4. **Calculate a board score** - after the AI has moved its piece to a square, calculate the score of the board and select the move with the highest points

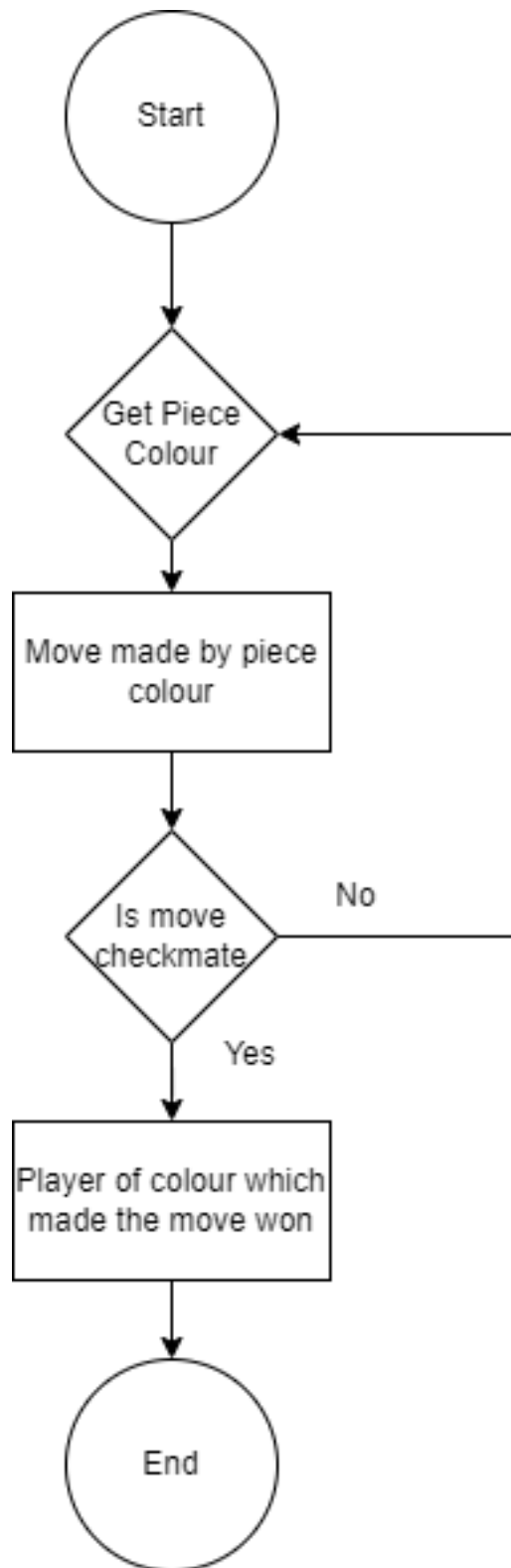


Simple concept of how the AI can look several moves ahead, calculating the best move that it can make.

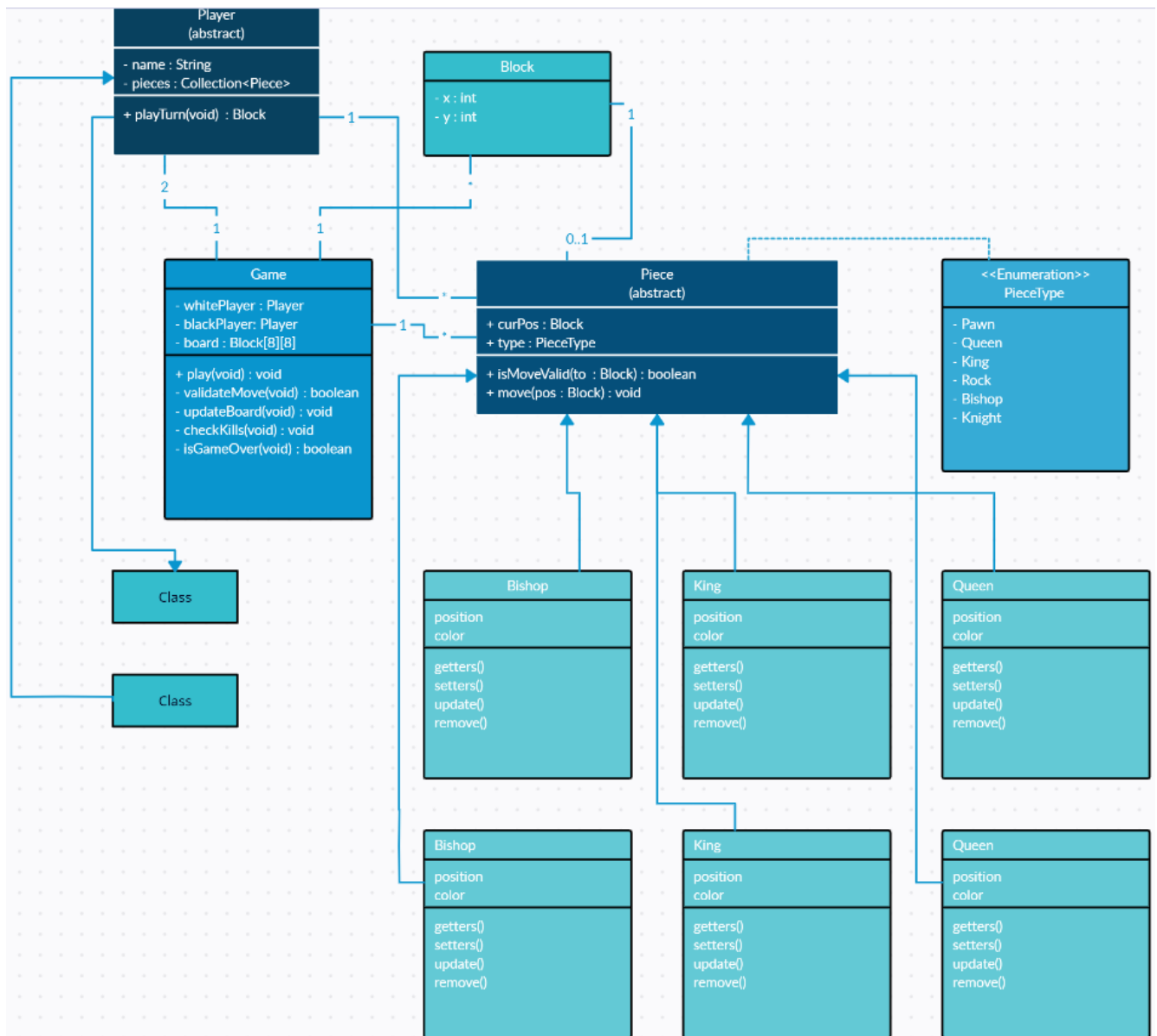
• Timer

1. Some chess games don't have a timer, and some do. The problem with games which don't have a timer is that the game could get tediously long.
2. If a game has a timer, each player is given a timer with the same amount of time. When it is your turn, your timer will go down and vice versa.
3. If your timer is near 0 seconds, and you move a piece, your timer will add some bonus seconds onto it
4. If your timer runs out, no matter how frustrating, the other player wins.

DFD (level 0 - current system)



DFD (Level 1 - current system)



Analysis techniques

Formal analysis

(Document: <https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977/>)

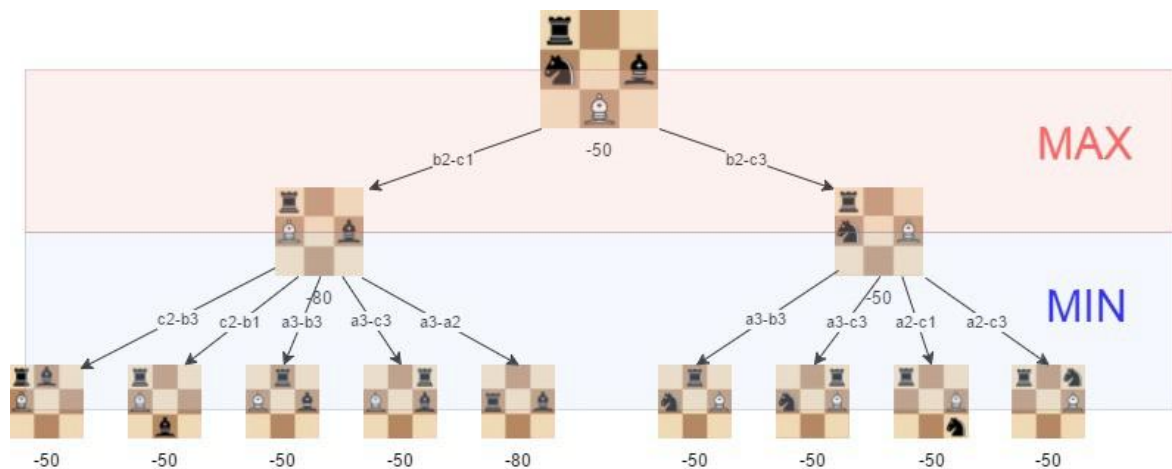
1. **Board visualisation and move generation** - for the move generation library, it will need to include all the rules of chess (<https://github.com/jhlywa/chess.js>). For the visualisation of the board a different library was used (<https://github.com/oakmac/chessboardjs/>). Using these two libraries we can calculate all legal moves for a given board state for each piece. If movement of pieces work, then you can move onto developing the interesting AI algorithm. A way to start to develop the algorithm is to create simple AI which plays moves randomly. This algorithm is obviously not complex; however, it is a good base to start on.

2. **Position evaluation** - we need to be able to comprehend which side is stronger in certain

	10		-10
	30		-30
	30		-30
	50		-50
	90		-90
	900		-900

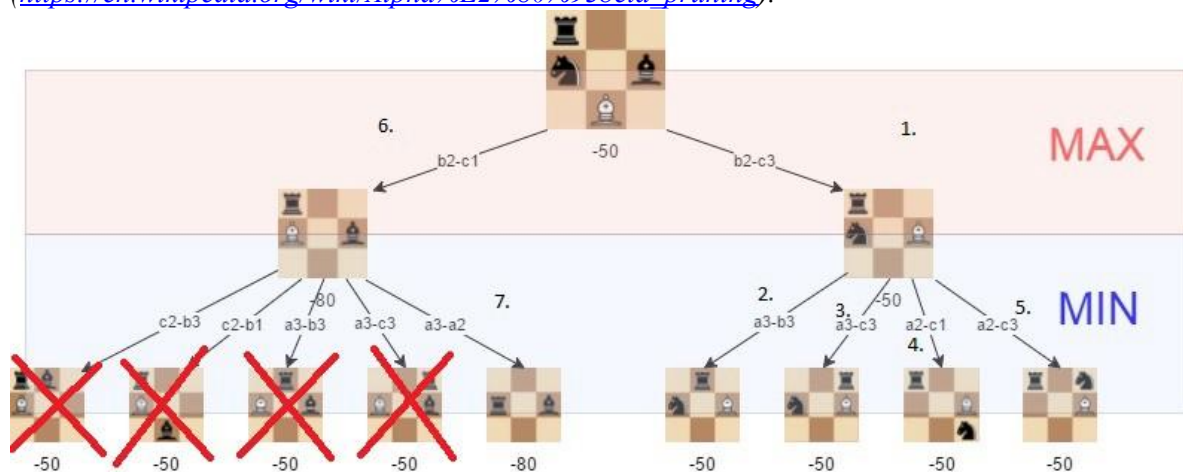
positions. The best way to achieve this is to assign each piece an integer value. With these values an algorithm can be created that chooses the move with the highest evaluation/outcome. The only noticeable improvement now is that the algorithm will always capture a piece if it can. There is still not really any intelligence at this point until we get to the minimax algorithm. The points for the pieces are created due to the pieces power on the board. The king obviously is the highest point piece as the game is based around it, then the queen as it is the most powerful moving piece on the board.

3. **Search tree using minimax** - the purpose of the search tree is to choose which is the best move for the ones available. Minimax is a decision rule used in AI, decision and game theory and statistics. It is used for minimizing the possible loss for a worst-case scenario (<https://en.wikipedia.org/wiki/Minimax>). In this algorithm, it uses a recursive technique to explore all possible moves in depth. The position is evaluated at the ending “leaves” of the tree. After, we return either the smallest or largest value of the child to the parent node, depending on whether it is a white or black move.



With this minimax algorithm working, the chess AI will start to understand some basic tactics of chess. The effectiveness of this algorithm is mainly based on the search depth we go into. The more depth we go into when searching for moves and outcomes, the more understanding the AI becomes.

4. **Alpha-beta pruning** - the purpose of this algorithm is that it decreases the number of nodes that are evaluated by the minimax algorithm in its search tree. It stops evaluating a move when at least one possibility has been found that proves the move to be worse than a previously examined one (https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning).



(Interview: <https://www.thenakedscientists.com/articles/interviews/ai-chess>)

Traditional engines that use the minimax algorithm had values for pieces. However, built by DeepMind, was a program called AlphaZero. This program is a self-taught chess AI, so it didn't know the rules of chess. What it did was then play 44 million games at lightning-fast speed against itself starting completely random. So, at first, most of the games would end in a draw because it didn't really know anything about chess and couldn't make any intelligent moves. Due to the moves being completely random, sometimes the game would end through a checkmate. When this would happen, the AI would look at the king of moves that it had made and would try to implement it into the next game. It would then play more and more and slowly build up a picture of what patterns were good.

Summary of the key points:

- **Board visualisation and move generation** - making sure the board is loaded in correctly, with the pieces in the correct position and the piece rules loaded in too. Finally, calculate every possible move each piece can make
- **Position evaluation** - the evaluation of what position is stronger depending on the move that is made. The AI can decide whether to take the player's piece
- **Minimax** - decision rule used in AI; this will help the AI to decide what move is the best value
- **Alpha-beta pruning** - improves efficiency of the AI when it uses the Minimax
- **Neural networks** - a different way to create a chess AI. Where it uses self-taught algorithms and pattern matching to decide on moves

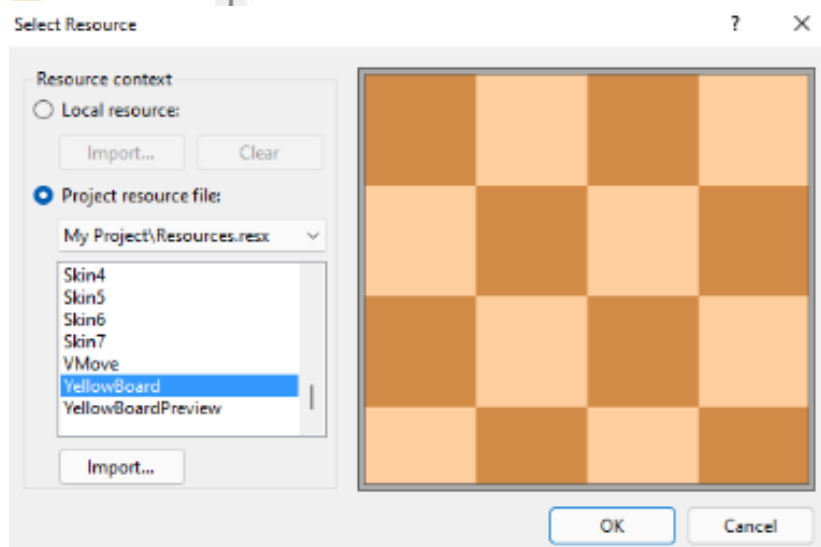
Proposed solution

Description

- **Board** - some engines use 64 squares to represent the 8 by 8 chess board. Although this method would work to implement the board, the more efficient way creating a board is to gather/create an entire board onto one image and import it onto a forms background. Finally, importing transparent picture boxes means that you can select each square on the board.

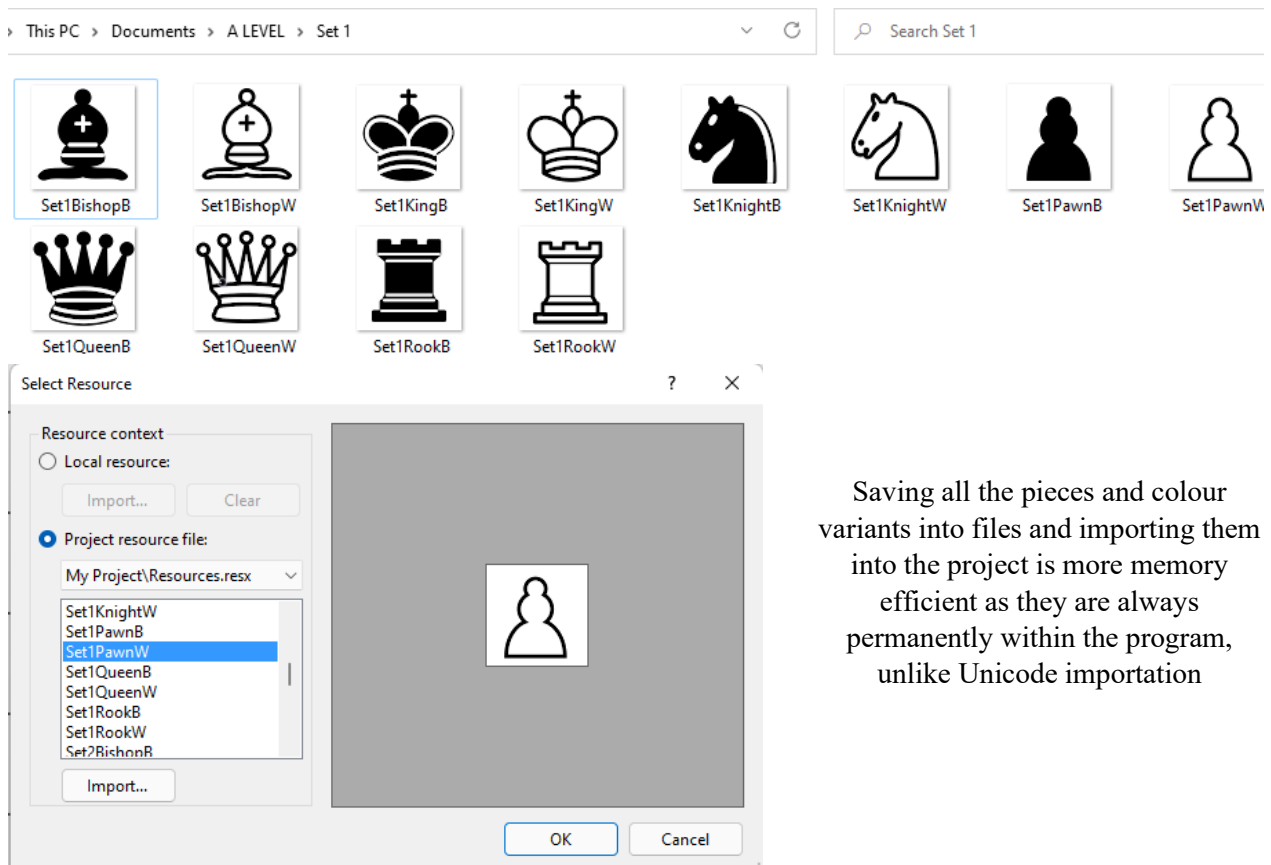


Saving a background picture of a blank chess board into your files



Importing your save file onto your game form

- **Pieces** - most engines use Unicode to import chess pieces into their game as it is relatively simple to do. However, the disadvantage with Unicode is that because it has a lot more characters, it uses a lot more memory than other methods. Another downfall is that you would have to manipulate the size of the chess pieces to fit appropriately within a square, but the issue is with Unicode you wouldn't know the initial size of the piece you are importing. A solution for this is to either create/obtain your own pieces and add them to the project's resources (being above the background image when doing so). This means you can easily alter the size of the pieces before importing them and makes them more memory efficient.

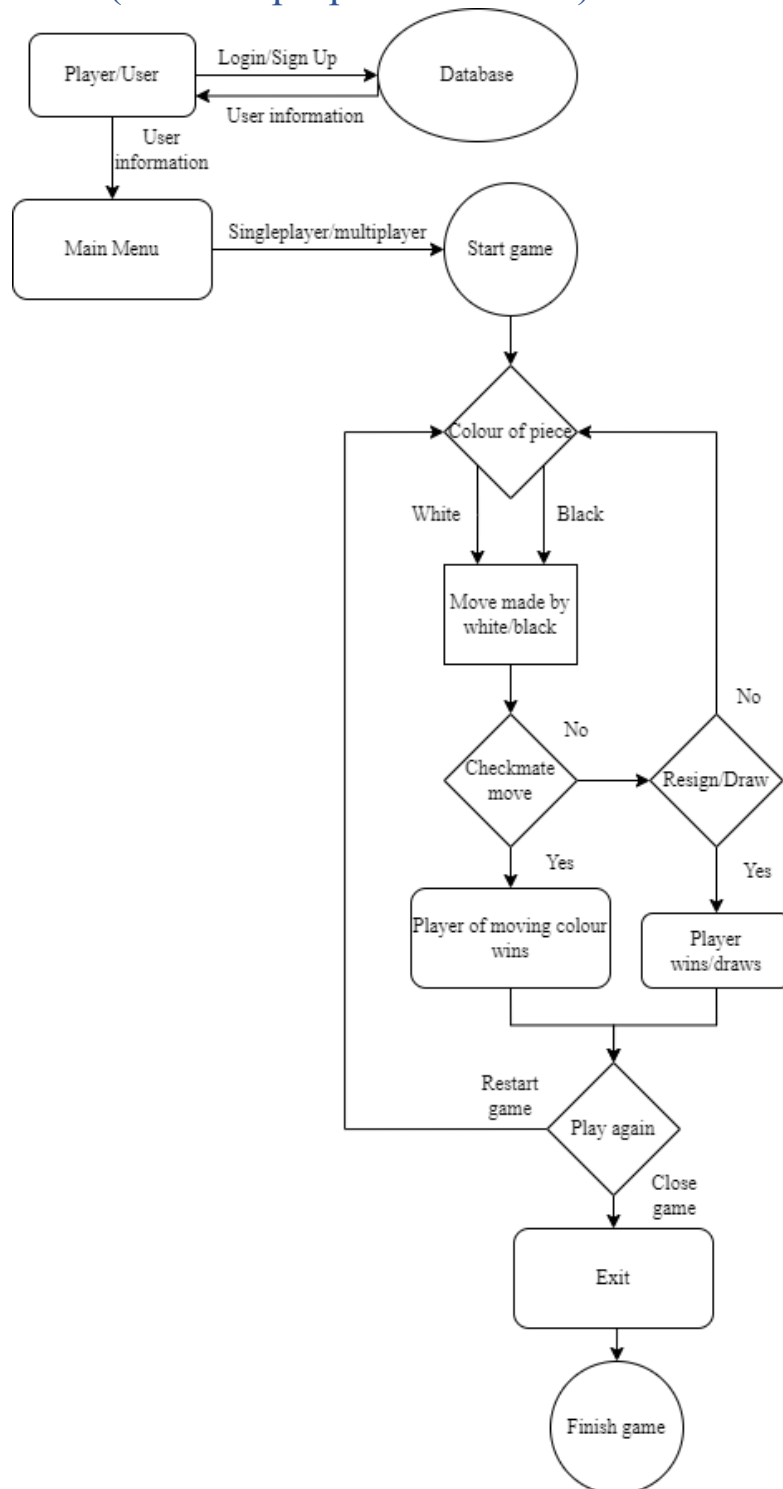


Saving all the pieces and colour variants into files and importing them into the project is more memory efficient as they are always permanently within the program, unlike Unicode importation

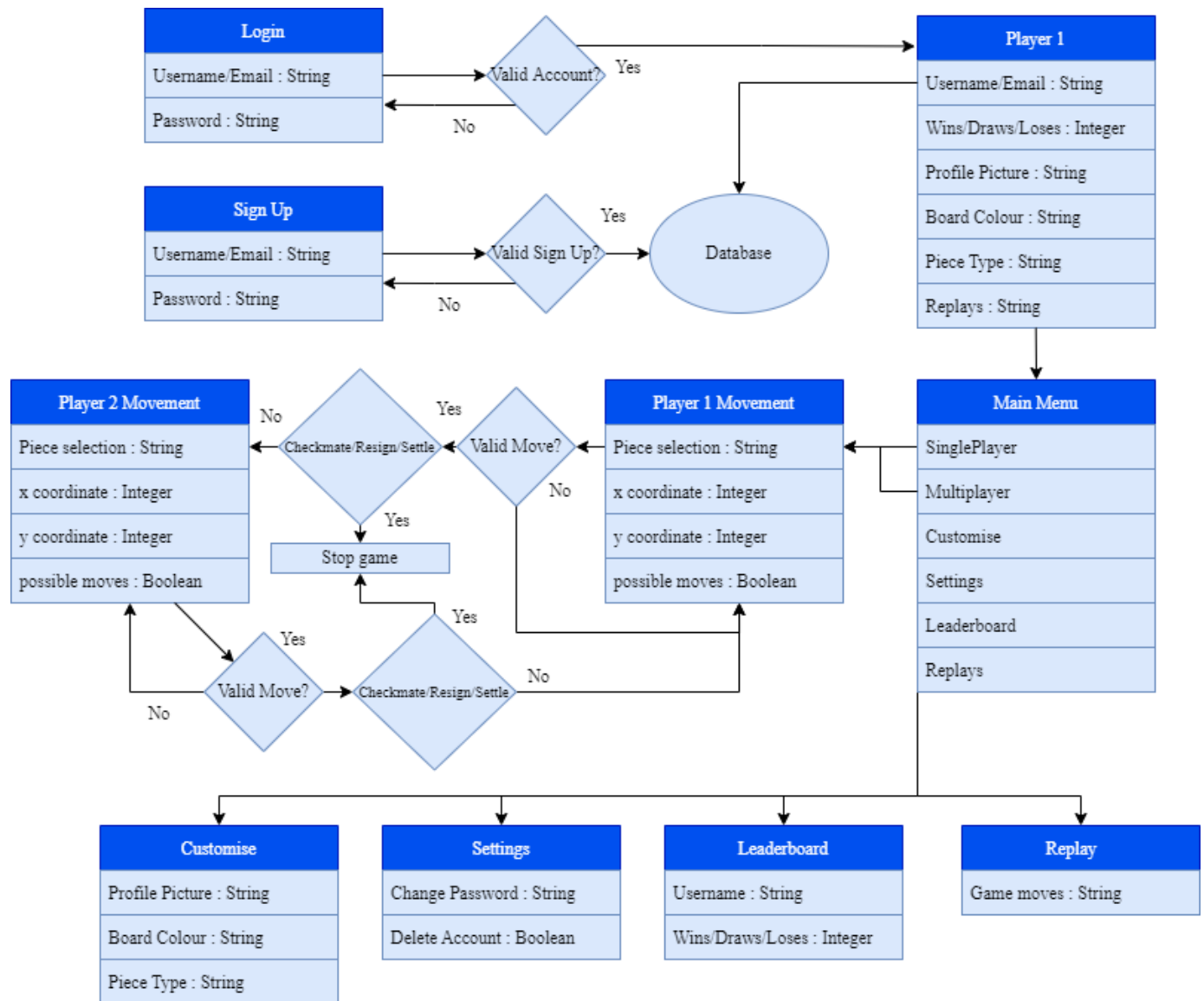
- **Move generation** - researching about self-taught AI's, they start of knowing nothing about chess. The problem with this is that it must play millions of games just to understand how the game is played. Therefore, move generation is most efficient by creating separate classes for each type of piece i.e., a pawn class, a rook class... This means that when moving a piece, the AI will know the rules of the game already before playing millions of games which is a good advantage to have. The solution to not using neural networks is to use the minmax algorithm which is a very suitable and efficient alternative.
- **Ending to games**
 1. Game time - some chess engines don't have a way for the game to end on time. This can be very frustrating for players, and a solution for this is to add a timer. The timer I will be implementing is one where you can select from a few options of time (2 minutes, 5 minutes, 10 minutes, unlimited time and so on).
 2. Settling - there are many ways in chess for the players to force the game into a draw. However, some programs don't have a way for both players to agree to draw no matter the move. If both players think that the result of their game would be a draw, instead of waiting for it to happen I can implement a settlement form feature, where both players must agree to settle for it to happen (*otherwise settlement/draw would be denied*).
 3. Resignation - in basic chess games, a player can only win from checkmate. This could be frustrating as this means that the whole game always must be played out. Therefore, if a user feels like they are clearly going to lose, there should be a resignation button to halt dragging the game out.

- **Player customisation** - not many chess programs give users their own profile and are able to customise it. My solution to this is to get the user their own personal account, with things such as profile pictures, board colours, and piece types, being customisable. This gives every player their own aspect to the game when playing my program.
- **Analysis of games** - a good section to include in my chess game is to help the user understand where they went wrong in the game, in order to improve. The best way to improve at anything is to look over your mistakes and find out how to fix them. Consequently, a replay section would be very useful to benefit the user.
- **Competitiveness** - a way to make my game more competitive for users is to create a leaderboard section so they can compete against the best players. This gives the player something to work towards as they get up the leaderboard with time.

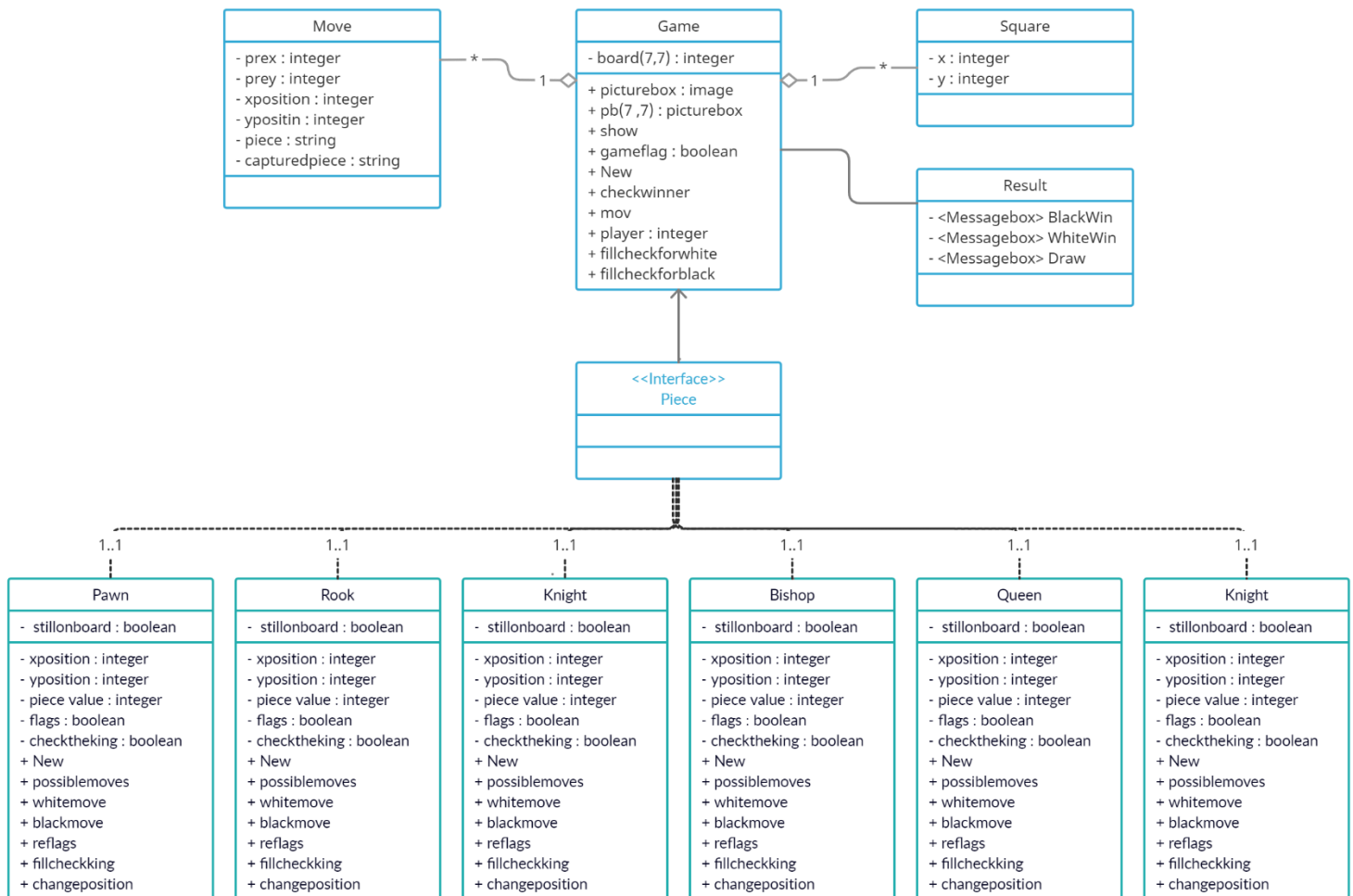
DFD (level 0 - proposed solution)



DFD (level 1 - proposed solution)

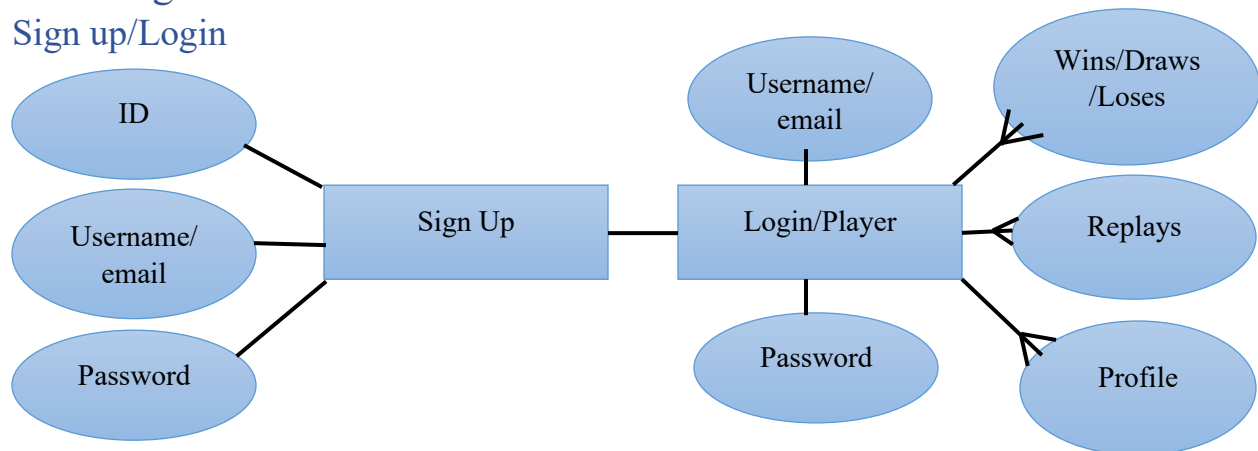


UML

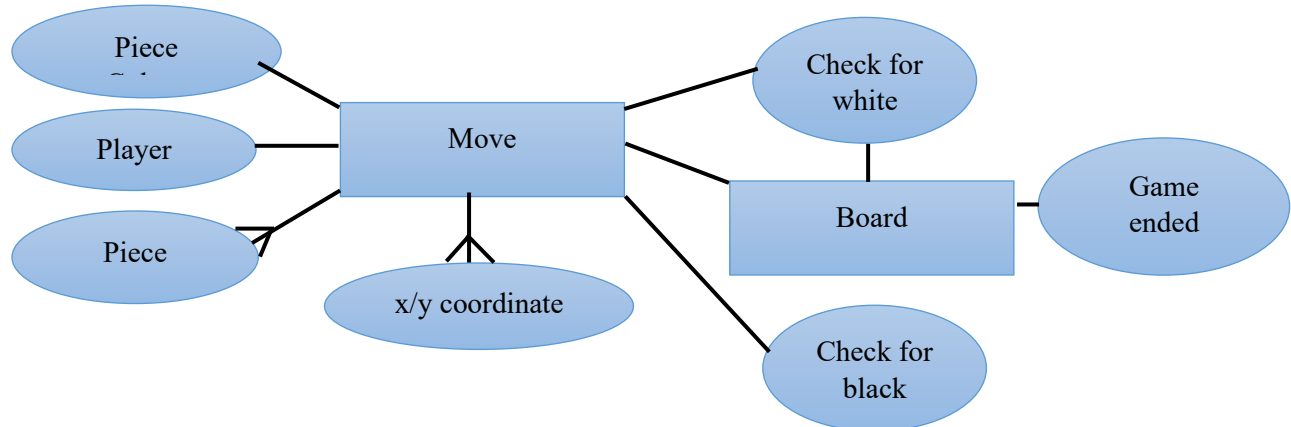


E-R Diagrams

Sign up/Login



Game



Proposed User

1. Audience

Document: (<https://www.parents.com/kids/development/intellectual/benefits-of-chess/#:~:text=While%20some%20children%20will%20be,the%20ideal%20time%20to%20start>)

Chess does not have a set age to play, therefore many experts disagree with the optimal age for children to start learning to play. Some people believe that the younger the child plays, the greater impact it will have on them, as it is known that young children's brains pick up things very easily. Tom Brownscombe is a scholastic director of the U.S. Chess Federation, and he found mixed results comparing children learning at the age of 4 and the age of 8. He concluded that although some children can learn much younger, the optimum age to start learning is 8. Consequently, my project is suitable for all age ranges, but the recommendation is 8+ years.

2. Role

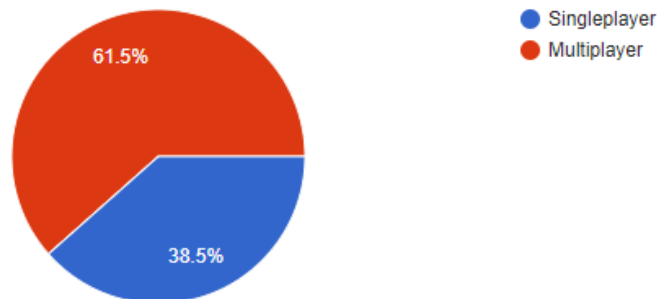
The role of the user within my project is to be a player playing the game chess. They will sign up with a username, email, and password, which will be stored within a game database so the player can sign out and sign in with their personal account. The main role of the player is to either play a game against an AI, or a real opponent (*singleplayer/multiplayer*). Finally, there should be some added in tasks the user can perform, such as their statistics, leaderboard, customisation etc.

Investigation

Questionnaire: *Created by Daniel Lawrence*

Which game mode do you prefer

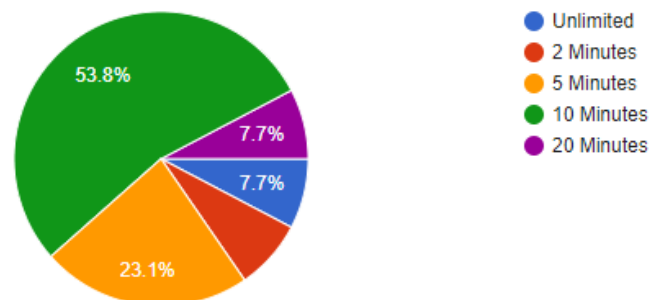
13 responses



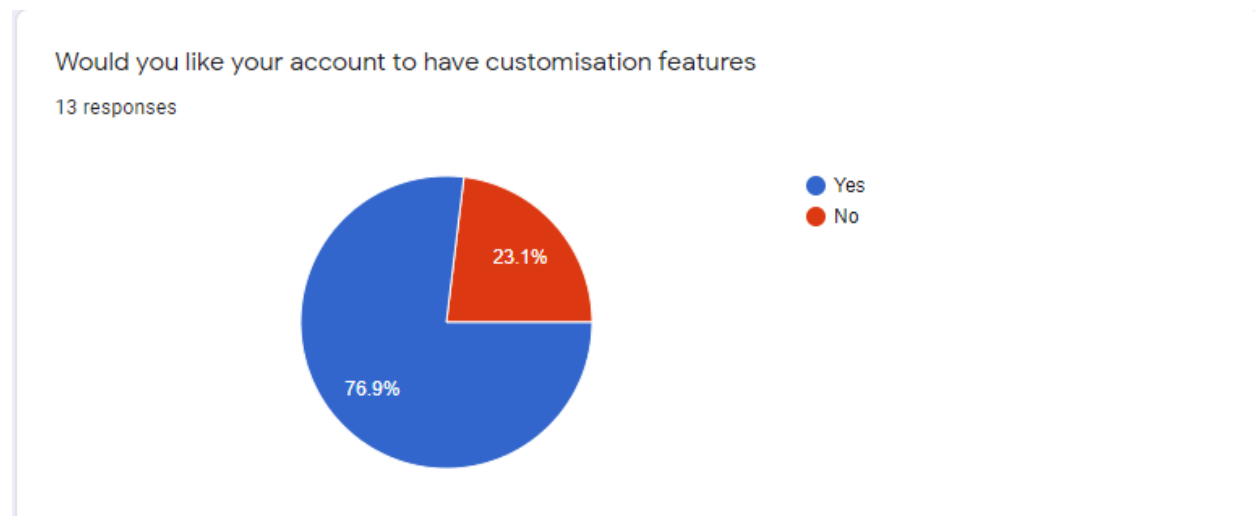
Although the singleplayer is where the complex algorithms take place, for the sample I took it shows that 61.5% of players prefer multiplayer. Therefore, as well as focusing on the AI, I will need to make the multiplayer aspect of the program just as enjoyable for the players.

What is your preferred time for a game

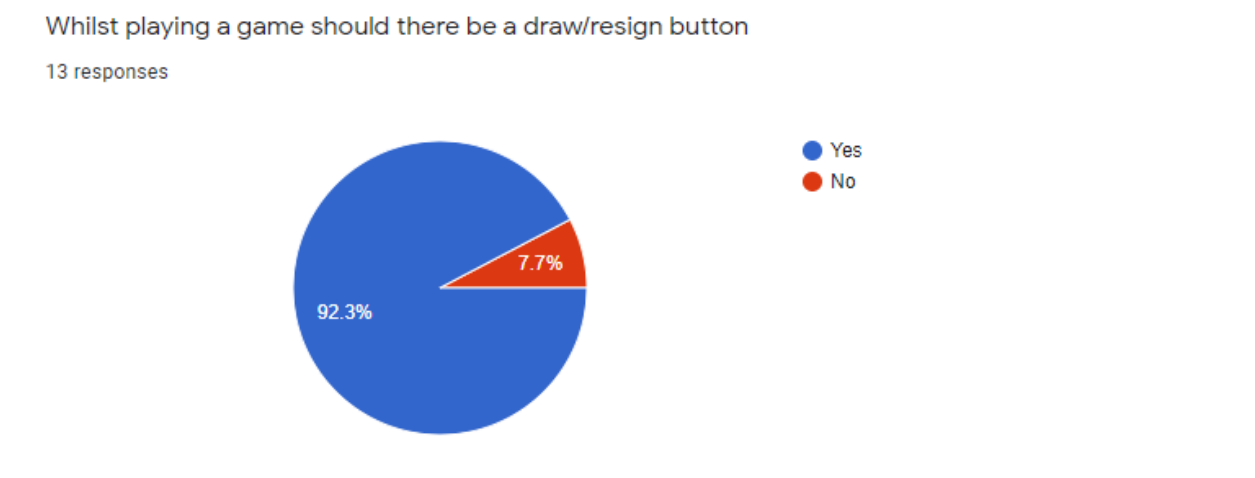
13 responses



Game time is an issue, I have previously stated, about how some programs let their chess games last forever, or until someone wins/draws. As long games cause frustration, which was a problem with other chess engines, my solution for was to implement a timer system. The image above shows the preferred game length, 10 minutes (53.8%), and all the others, which I will be adding into my program



Player customisation is a sub section of my program which will increase the user enjoyment of the game. Other chess engines have a complete working chess game, but I feel lack identity with the user which will cause them to stop playing the game relatively quickly. So, I proposed a customisation system for the user to have their own account and features (76.9% of players agreed).



In addition to the long games being made by no timer, there is no point playing out a game which is clearly playing out a certain way. Virtually all (92.3%) players wanted a resignation/settlement button to stop the prolonging of games.

Document: (<https://www.chess.com/article/view/how-to-get-better-at-chess>)

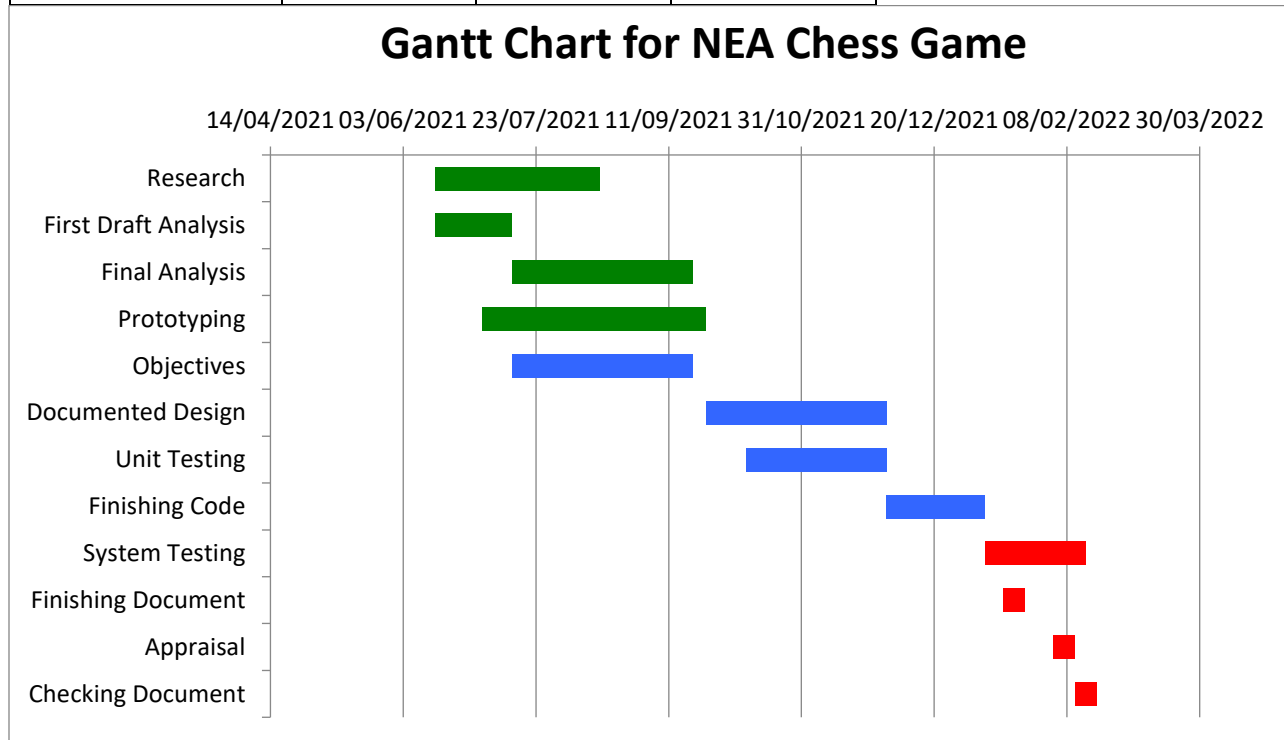
This document talks about the best tips to get better at chess. One of the tips is to learn from your own games and analyse why you lost. Replays is the best way for this to happen as every game contains many mistakes and opportunities, so in order to improve you need to learn from both playing and reviewing. I feel replays are massively beneficial with games, which is why I chose to add a section into my game for them.

Project Management

Critical Path Analysis

Task Name	Start	End	Duration (days)
Research	15/06/2021	16/08/2021	62
First Draft Analysis	15/06/2021	14/07/2021	29
Final Analysis	14/07/2021	20/09/2021	68
Prototyping	03/07/2021	25/09/2021	84
Objectives	14/07/2021	20/09/2021	68
Documented Design	25/09/2021	02/12/2021	68
Unit Testing	10/10/2021	02/12/2021	53
Finishing Code	02/12/2021	08/01/2022	37
System Testing	08/01/2022	15/02/2022	38
Finishing Document	15/01/2022	23/01/2022	8
Appraisal	03/02/2022	11/02/2022	8
Checking Document	11/02/2022	19/02/2022	8

Gantt Chart for NEA Chess Game



Objectives

1. Signing up

- This is the first task that every user will have to do. A new player will have to enter a username, email, and password in order to sign up for an account
- Existing account - for a sign up to be allowed, check within the database whether the username or email is taken. If either the username/email is already in use, then the program should reject the sign up and the user should be made sure of that
- Valid password - a secure password is one of the most important things about creating an account. Therefore, my project should not allow poor password, such as a password shorter than 5 characters or so, and a ridiculous password, longer than 30 characters or so
- Shortcuts - on a lot of professional sign-up forms, whenever you enter your email there is usually some sort of shortcut to help the user (e.g., automatically puts .com). The idea of shortcuts is to make life easier for the player, so I could implement some of my own.

2. Logging in

- This is the first task that every returning user will have to do. The returning player will have to enter a Username/Email and their password
- Accept/Reject - if the user has entered a Username/Email and password, then check both against the database. If what the user has entered, matches with an element within the database, then accept the login, otherwise reject it
- Password recovery - if the user has forgotten password, without a way to recover the password, their account would be permanently lost. If the user has forgotten their password, the user should type in their email, and the program should mail them a 4-digit reset password code, in which they have a certain number of attempts to get it right, in order to change the password.

3. Game form

- Board - correctly importing the board as a background image onto the form, with an 8 by 8 square of picture boxes to display the piece when the user plays the game.
- Pieces - loading the pieces onto the board is very important. As the images for the pieces will be stored in the project resources, import them into the required picture boxes at the start of the game
- Timer - before the game is allowed to be started, get the user to select a present time slot (2, 5, 10, 20, or unlimited minutes). Each player should have their own separate timer
- Starting the game - after the user has selected the timer period, allow the player 1 timer to start counting down (white always goes first). As the game is started enable the entire form so the user/users can move their pieces and use extra tasks on the form
- Finishing the game
 - Checkmate - if a user has checkmated someone, stop the game and display that the user has won, storing a win for that user in the database, and storing a loss for the user that has received the checkmate
 - Draw - if that board has resulted in a stalemate, store a draw for both users. A draw should also occur if both users agree to a settlement
 - Resign - to reduce the time of games. If the opponent has resigned then add a loss for that user, and a win for the other user
 - Replay - after the game has ended, prompt the user to store the replay giving them a choice whether they want to

- Chess notation - give each square a letter and a number for chess notation to be displayed on the board. This will mean that the movements in the game are easy to follow.
- Loading accounts - for the program to feel user friendly whilst playing the game, load the players usernames on to the form with their profile pictures. Furthermore, select the board colour and piece types they have chosen.

4. Game logic

- Movement - when the game is being played make sure the piece movement is all legit.
- Check - after every move calculate if the king is in danger, and if it is show that is in check on the board.
- Pawn promotion - after every move see if a pawn is at the opposite end of the board, and if this is true then allow the pawn to be changed into any piece bar the king
- Castling - if the king or rook is selected and the condition for a castle is met, then allow a castle to take place

5. AI opponent

- *Easy mode* - a randomly moving AI opponent. There's no logic to the movements for this AI, however it is for the most basic of players
- *Hard mode* - using a minimax algorithm so the program has some strategy behind its movements

6. Replays

- Board - correctly importing the board as a background image onto the form, with an 8 by 8 square of picture boxes to display the piece when the user plays the game, making sure the picture boxes are disabled so the user cannot move the pieces
- Play - for the user to play the replay, there must be a button which moves the pieces for them. If a user wants to skip through the game, there could be a fast forward button on the form as well as a normal paced button
- Pause - pauses the replay
- Rewind - if the user wants to go to a previous state of the game, they should be allowed to rewind the board, as they could have missed the move they wanted to analyse

7. Leaderboard

- Correctly displayed leaderboard, with the users ordered by number of wins

8. Customisation

- Correctly gather data from the database, and return data to the database
- Display the number of wins/draws/loses for the correct user
- Create an unlock system for the user so they have something to work towards and encourages more gameplay
- When the user selects a profile picture/board colour/piece type, load that data into the database for the correct user

9. Rules

- Correctly opening a word document that contains the correct rules of chess for the user to read

10. Settings

- Option for the user to change their password whilst signed in
- Option for the user to delete their account whilst signed in

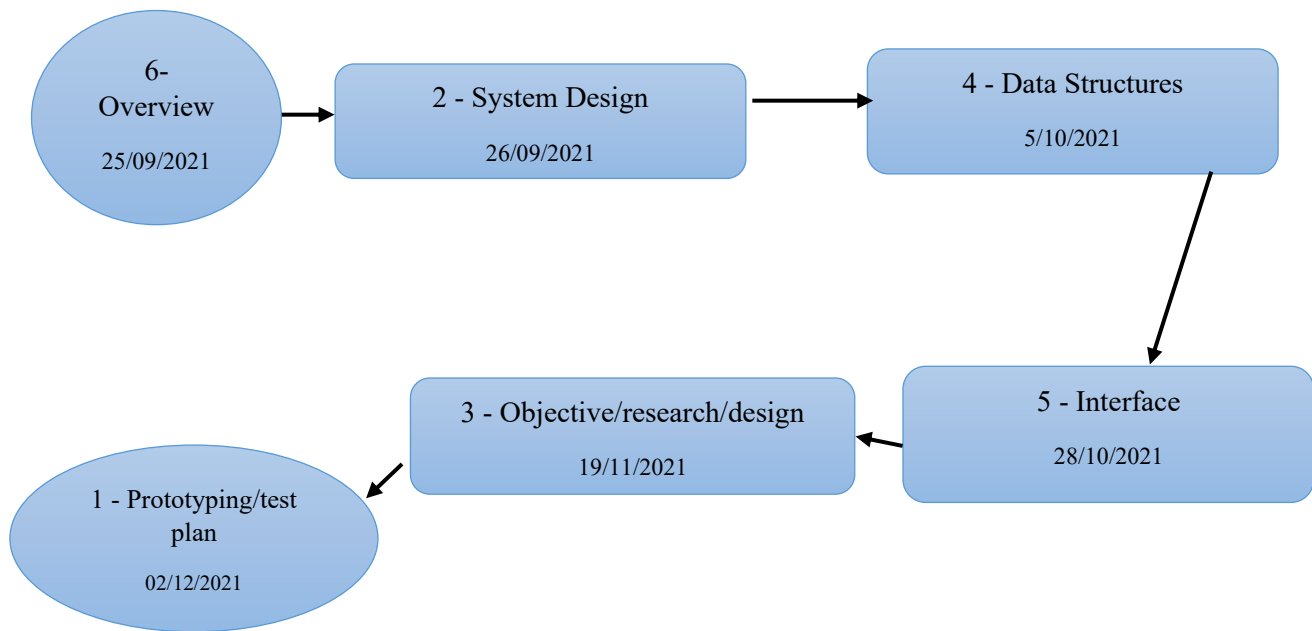
○

Analysis Log

Date	Activity <i>What you did</i>	Findings <i>What you found out</i>
15/06/2021	https://www.chess.com/	I used chess.com to help me with the understand the current system. It gave me inspiration when I was writing my proposed solution and has vast information about chess.
30/06/2021	https://www.ichess.net/blog/chess-pieces-moves/	I used this website to give me a good understanding about the legal moves each piece can make on the board
30/06/2021	https://www.chess.com/	I used chess.com to help me comprehend every possible way a game can end in chess
10/07/2021	https://app.diagrams.net/	Using a flow chart diagram website, it allowed be to create suitable diagrams that represent the current system
20/08/2021	https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977/	This document helped me analyse the creation of the chess game, and the AI using the minimax algorithm
21/08/2021	https://www.thenakedscientists.com/articles/interviews/ai-chess	This interview explained the neural network side of AI, helping me decide to choose to use minimax
26/08/2021	https://www.parents.com/kids/development/intellectual/benefits-of-chess/#:~:text=While%20some%20children%20will%20be,the%20ideal%20time%20to%20start	I found an interview which explained the audience that would most likely being chess, which was good for analysing my user and their role
10/09/2021	<i>Questionnaire</i>	Extremely useful in determining what to have in my proposed solution in contrast to some of the current systems.
12/09/2021	https://www.chess.com/article/view/how-to-get-better-at-chess	The best way for someone to improve at chess is for them to analyse their own mistakes, so it gave me the idea of a replay system

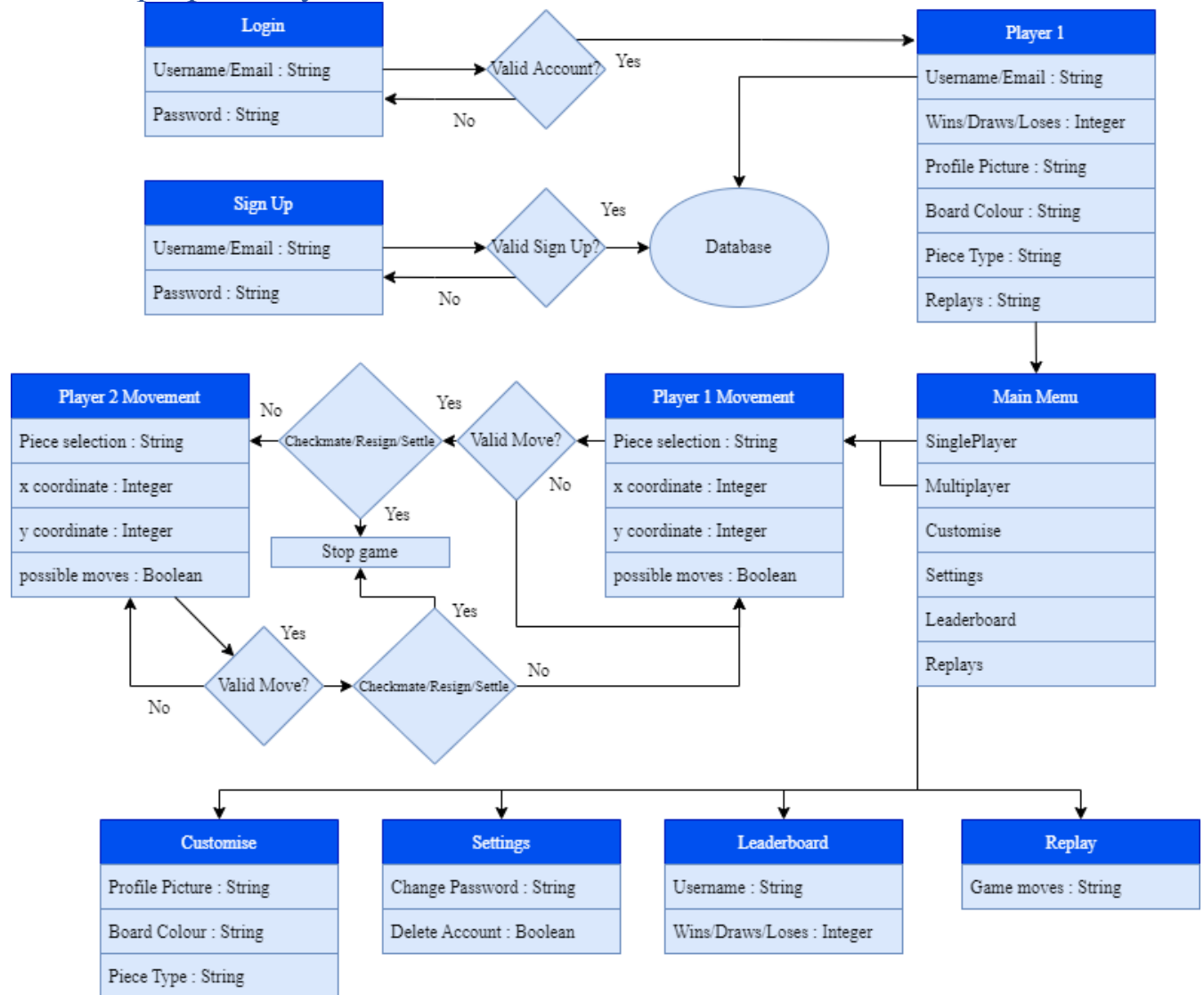
Overview

Below is a flow diagram that shows the development of the sections within the documented design, and their finish dates. They are numbered from numbers 1-6, 1 being the most important and 6 being the least.



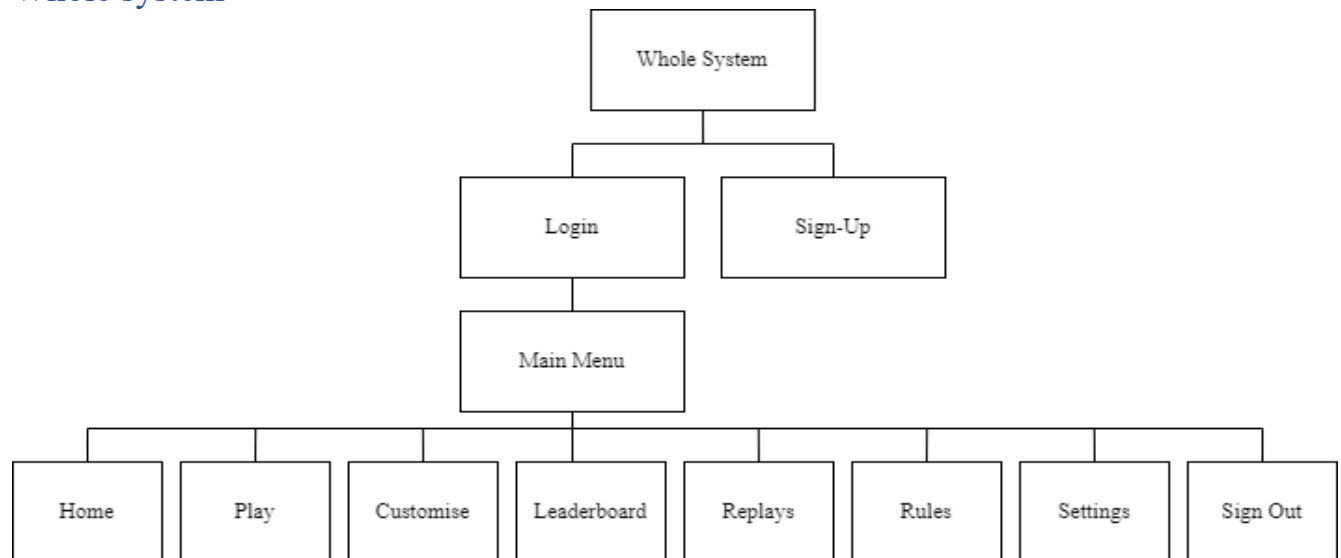
System Design

DFD of proposed system

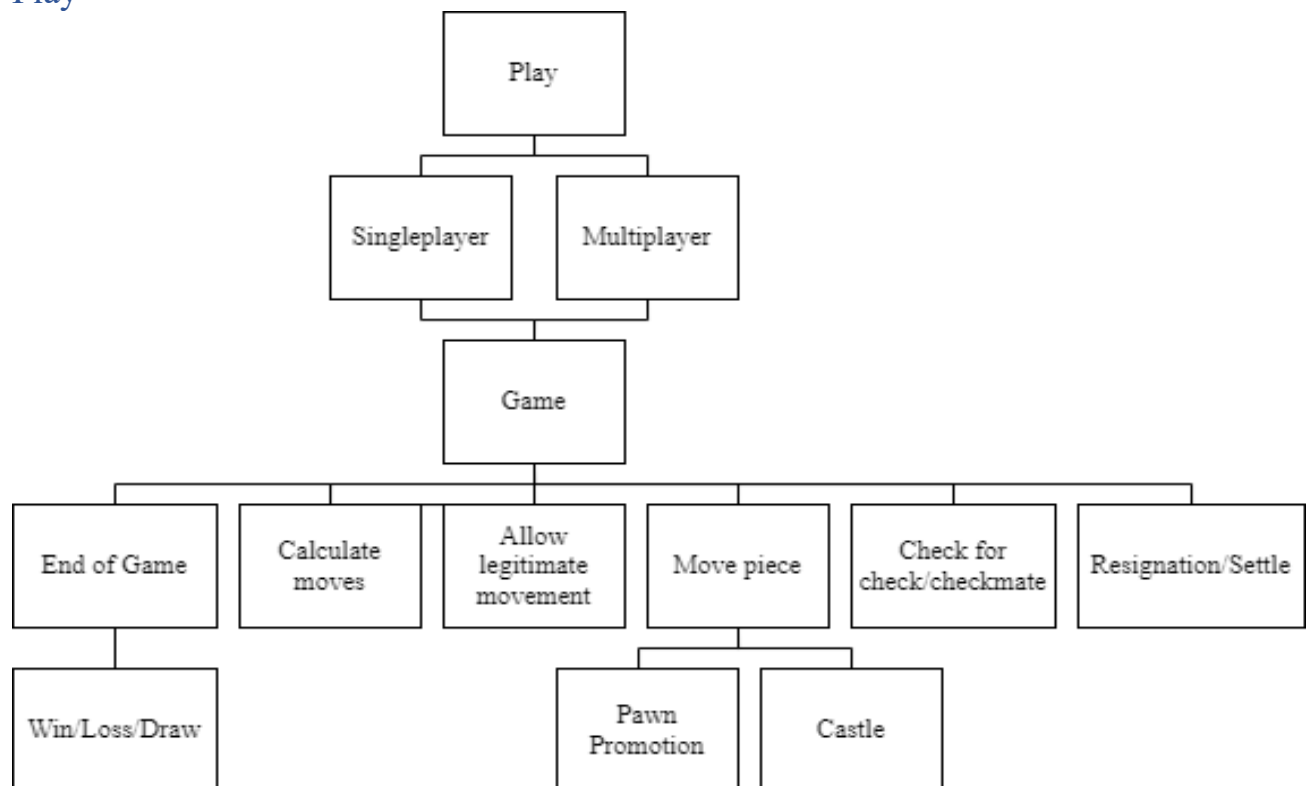


Hierarchy charts

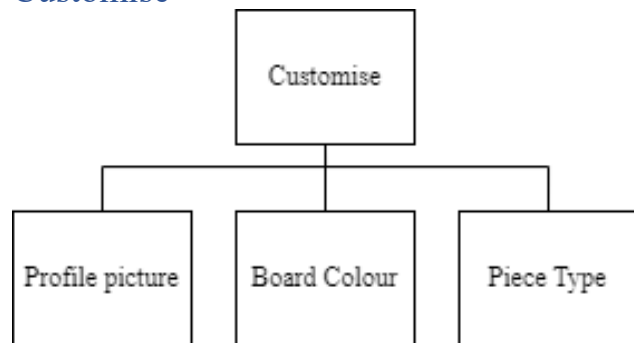
Whole system



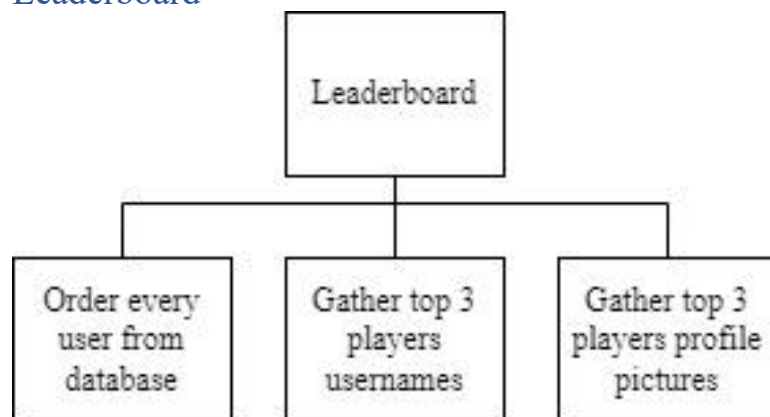
Play



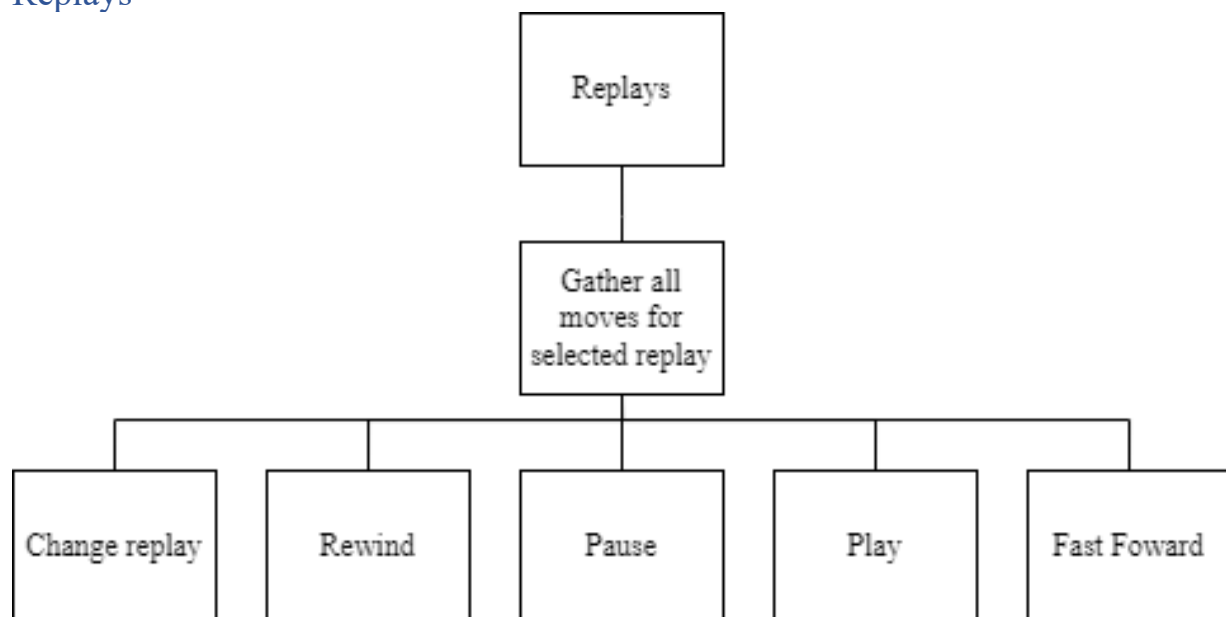
Customise



Leaderboard



Replays



Data Structures

Database design & dictionaries

Field Name	Data Type	Description	Example
ID	AutoNumber	Unique number ID for all players	3
Username	Short Text	Unique username for players	Admin
Password	Short Text	Stores the password for the players	Admin
Email	Long Text	Unique email for players	admin@icloud.com
Wins	Number	Stores the number of wins for the players	8
Draws	Number	Stores the number of draws for the players	5
Loses	Number	Stores the number of loses for the players	2
ProfilePicture	Short Text	Stores the chosen profile picture for the players	Default
BoardColour	Short Text	Stores the chosen board colour for the players	BlueBoard
PieceType	Short Text	Stores the chosen piece type for the players	Set2
PreMove1	Long Text	Stores the movement for replay 1	A3
Move1	Long Text	Stores the movement for replay 1	A4
PreMove2	Long Text	Stores the movement for replay 2	B3
Move2	Long Text	Stores the movement for replay 2	B4
PreMove3	Long Text	Stores the movement for replay 3	C4
Move3	Long Text	Stores the movement for replay 3	C5
PreMove4	Long Text	Stores the movement for replay 4	D4
Move4	Long Text	Stores the movement for replay 4	D5
PreMove5	Long Text	Stores the movement for replay 5	E4
Move4	Long Text	Stores the movement for replay 5	E5

Object model and class definitions

Pawn

Object	Data Type	Description
-xposition	Integer	x-coordinate for pawn
-yposition	Integer	y-coordinate for pawn
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the pawn can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the pawn can check the king
-stillonboard	Boolean	Set to false if the pawn is no longer on the board
-allcheckmoves	Boolean	Used to calculate moves when opposite colour is moving
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new pawn onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+diagonalchecktest()	Sub	Calculates the moves for the opposite-coloured king being in check
+reflags()	Sub	Setting all elements in the flag array to false
+recheckking()	Sub	Setting all elements in the checktheking array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

Rook

Object	Data Type	Description
-xposition	Integer	x-coordinate for rook
-yposition	Integer	y-coordinate for rook
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the rook can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the rook can check the king
-AllowedToCastle	Boolean	Used when calculating if the rook can castle or not
-stillonboard	Boolean	Set to false if the rook is no longer on the board
-allcheckmoves	Boolean	Used to calculate moves when opposite colour is moving
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new rook onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+reflags()	Sub	Setting all elements in the flag array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

Knight

Object	Data Type	Description
-xposition	Integer	x-coordinate for knight
-yposition	Integer	y-coordinate for knight
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the knight can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the knight can check the king
-stillonboard	Boolean	Set to false if the knight is no longer on the board
-allcheckmoves	Boolean	Used to calculate moves when opposite colour is moving
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new knight onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+reflags()	Sub	Setting all elements in the flag array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

Bishop

Object	Data Type	Description
-xposition	Integer	x-coordinate for bishop
-yposition	Integer	y-coordinate for bishop
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the bishop can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the bishop can check the king
-stillonboard	Boolean	Set to false if the bishop is no longer on the board
-allcheckmoves	Boolean	Used to calculate moves when opposite colour is moving
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new bishop onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+reflags()	Sub	Setting all elements in the flag array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

Queen

Object	Data Type	Description
-xposition	Integer	x-coordinate for queen
-yposition	Integer	y-coordinate for queen
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the queen can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the queen can check the king
-stillonboard	Boolean	Set to false if the queen is no longer on the board
-allcheckmoves	Boolean	Used to calculate moves when opposite colour is moving
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new queen onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+reflags()	Sub	Setting all elements in the flag array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

King

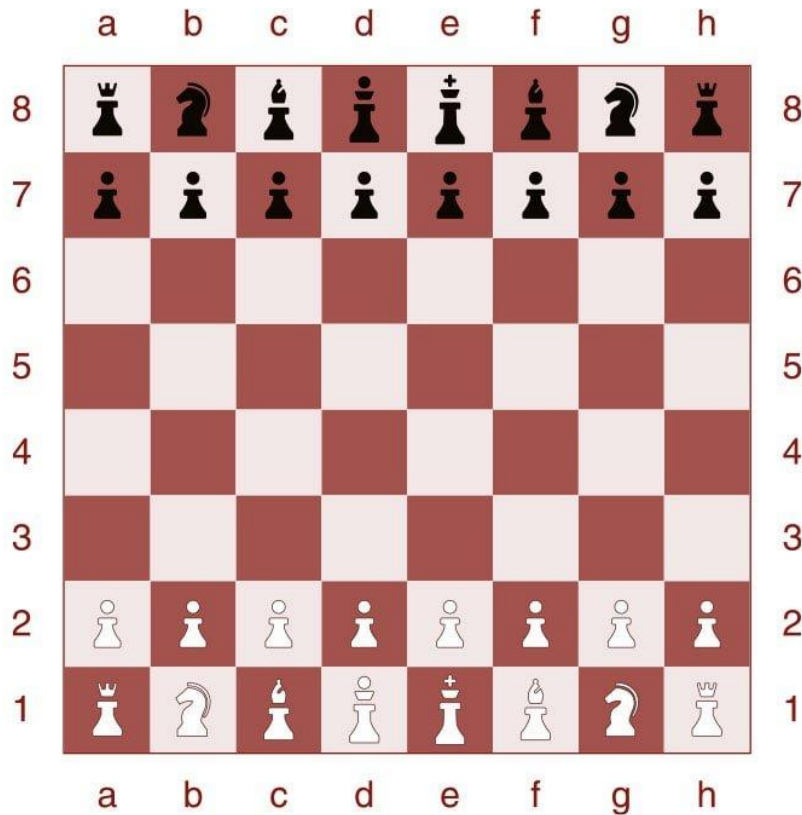
Object	Data Type	Description
-xposition	Integer	x-coordinate for king
-yposition	Integer	y-coordinate for king
-piecevalue	Integer	Negative means black piece and positive means white piece
-flags(7,7)	2D Array	Used when calculating if the king can move to a space.
-checktheking(7,7)	2D Array	Used when calculating if the king can check the king
-AllowedToCastle	Boolean	Used when calculating if the king can castle or not
Object	Subroutine/function	Description
+New(x, y, pvalue)	Sub	Used when loading a new king onto the board
+returnx()	Function - returns xposition	Returns the xposition of the piece
+returny()	Function - returns yposition	Returns the yposition of the piece
+returnvalue()	Function - returns piecevalue	Returns the piece value
+possiblemoves()	Function - returns flags	Returns the flags of all the possible moves for that piece
+whitemove()	Sub	Calculates the possible moves for a white piece
+blackmove()	Sub	Calculates the possible moves for a black piece
+reflags()	Sub	Setting all elements in the flag array to false
+fillcheckking()	Sub	Calculates all the moves which could check the king
+changeposition(x, y, prex, prey)	Sub	Change the position of the piece

Game

Object	Data Type	Description
-board(7, 7)	2D Array	Representing the board
-canmove(7, 7)	2D Array	Used when calculates the moves which a piece can move to on the board
-checkforwhite(7, 7)	2D Array	Used for calculating for check
-checkforblack(7, 7)	2D Array	Used for calculating check
-gameflag	Boolean	Used to calculate if the game is over or not
-player	Integer	Player 1 = White, Player 2 = Black
-blackindanger	Boolean	Warning the program black is in check
-whiteindanger	Boolean	Warning the program white is in check
-movecounter	Integer	Counts the moves made on the board
-xpositionCheck	Integer	X position for the piece checking the king
-ypositionCheck	Integer	Y position for the piece checking the king
-recursionstop	Boolean	Stops the recursion issue
-castleincheck(1)	Boolean	Determines if the castle was made would the king be in check
-canwritetoboardtracker	Boolean	Determines if the program can write to the board tracker on the form for the user to view
-stalemate	Boolean	Determines is stalemate has occurred
Object	Sub/Function	Description
+New()	Sub	Creating a new game
+checkwinner()	Sub	Searches for a checkmate/stalemate
+show()	Sub	Display pieces on the board for the user
+selection(x, y)	Function	Selection of a picture box
+reback()	Sub	Clears the images of the picture boxes
+recanmove()	Sub	Resets the -canmove object
+recheckwhite()	Sub	Resets the -checkforwhite object
+recheckblack()	Sub	Resets the -checkforblack object
+fillcheckforblack()	Sub	See is black king is in check
+fillcheckforwhite()	Sub	See if white king is in check
+showcheck()	Sub	Display check on the board for the user
+showpossiblemove(selected, x, y, prex, prey)	Sub	Displays possible moves for the user on the form
+PawnPromotion()	Sub	Calculates pawn promotion for pawns
+CastlingW()	Sub	Calculates castling for white
+CastlingB()	Sub	Calculates castling for black
+KingsOnly()	Sub	Calculates stalemate if only kings are left on the board
+StoreGameReplay()	Sub	Storing game replay in database for the user
+AIMoving()	Sub	Easy AI
+AIMovingHard()	Sub	Hard AI
+SecondDepth()	Sub	Minimax algorithm
+LoadTempBoard()	Sub	Used in SecondDepth()
+Resetboard()	Sub	Used in the minimax algorithm
+LoadSecondDepthBoard()	Sub	Used in SecondDepth()
+ResetSecondDepthBoard()	Sub	Used in SecondDepth()
+CalculateBoardPoints()	Sub	Calculates board points(for minimax)
+AICheckmate	Sub	If AI can checkmate, it will move there

User defined structures

2 Dimensional Arrays are a main structure used throughout the program. An example of the use of this is the board. The board is an 8 by 8 2D array, which represents each 64 of the squares on the board:



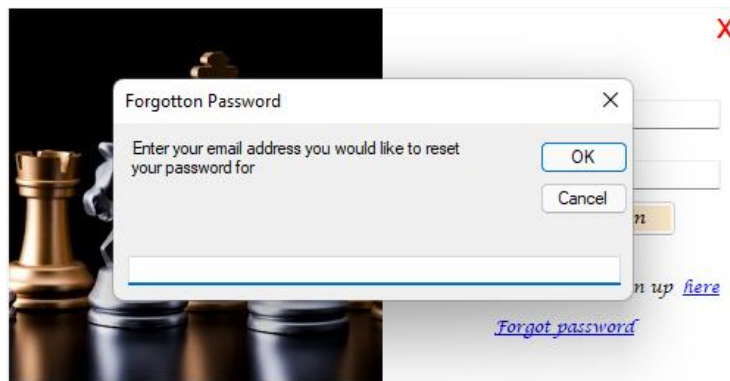
Each square on this chess board is individually stored within the array. For example, A8, is represented by `board(0, 0)`, as it is the first element within the array. C7, is represented by `board(1, 2)` as it is the second row and the column across. 2D arrays are useful when data is naturally organized in rows and column, which is why it is perfect to use for a Chess Game.

HCI

Login



1. Username textbox - the user will type in their username or email into this textbox for it to be checked with the database
2. Password textbox - the user will type in their password into this textbox for it to be checked with the database
3. Show password checkbox - when the user types in their password into the *password textbox*, it will display all the characters as *, to improve security within the program. Therefore, if the user ticks the checkbox, then it reveals the password they have entered (and vice versa when the user unchecks the textbox)
4. Login button - refer to the database, checking if the username/email and password match up with a player's profile. If this is the case then allow the user to be signed in, otherwise reject the user's request
5. Signing-up linked label - if the user does not have an account they will need to sign up. If they click on the linked label, it will hide the current form (*cannot close the base form*), and show the sign-up form
6. Forgot password linked label - if a user already has an account, however they have forgotten their password, the program should show the user a message box, where they can enter their email for which the account is signed up with:



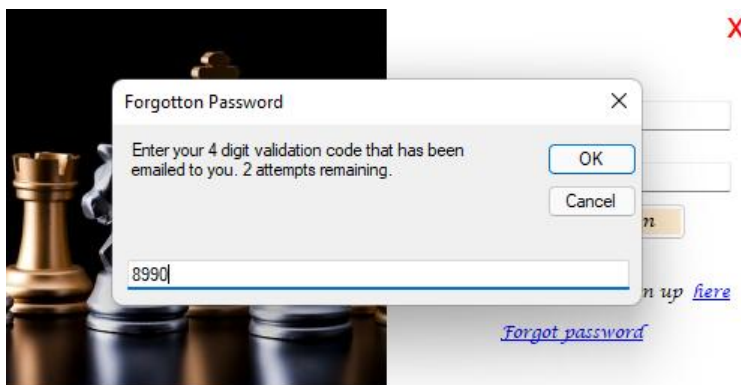
If the user selects the forgotten password linked label, this is the textbox that will appear. The cancel button returns the user back to the login form. If the user enters an email, it will check it against the database and if that email is found, a 4-digit verification code will be sent to that email, otherwise an “Invalid email” message box will appear.



This is an example of a random 4-digit code the computer generated, which was sent to an email address, trying to recover their password

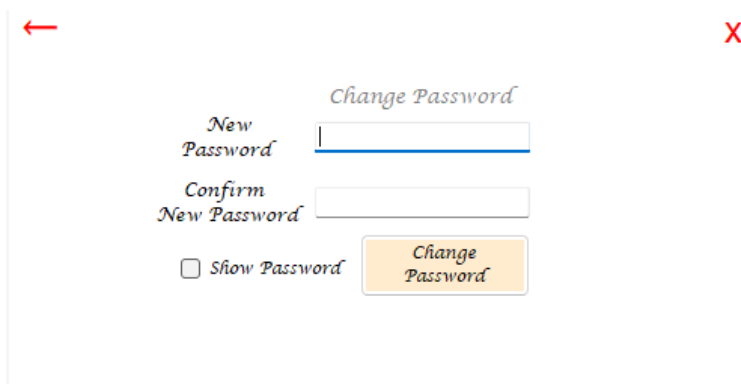
Reset Password

Your 4 digit reset code is : 8990



The user has 3 attempts to enter the correct code into the message box. If the user enters the 4 numbers correctly then they can successfully change their password for their account, otherwise if the user enters the numbers incorrectly in any way, then they will lose an attempt. If the attempts get to 0 then it will close the recovery process

If the user gets to this form, it means they have successfully recovered their account and is allowed to change their password.



The return button at the top left of this form takes the user back to the main menu if they changed their mind about changing their password. However, if the user clicks the change password button (if they have entered a valid password with all the requirements), then the program will display a message box confirming the password change and will send the user back to the login form.

- Exit button - if the user wants to exit the program and they click this button, a Yes/No message box will appear asking the user to confirm whether they want to close the program or not

Sign-Up

The image shows a sign-up form with the following elements and annotations:

- 1**: Points to the Username text box.
- 2**: Points to the Email text box.
- 3**: Points to the Password text box.
- 4**: Points to the Confirm Password text box.
- 5**: Points to the 'Show Password' checkbox.
- 6**: Points to the 'Sign Up' button.
- 7**: Points to the 'Return' button.
- 8**: Points to the 'Shortcuts' box containing email domain suggestions.
- 9**: Points to an 'X' icon in the top right corner.

- Username textbox - the user will enter their desired username into this textbox
- Email textbox - the user will enter their email into this textbox
- Password textbox - the user will enter their password into this textbox
- Confirm password textbox - the user will re-enter their password into their textbox to make sure the user did not enter their password incorrectly
- Show password checkbox - when the user types in their password into the *password textbox/confirm password textbox*, it will display all the characters as *, to improve security within the program. Therefore, if the user ticks the checkbox, then it reveals the password they have entered (and vice versa when the user unchecks the textbox)
- Sign-up button - Before the user can sign up the program will check that all the data the user has entered is valid:

Username (Valid if...)

- Username textbox is not empty
- The username entered is not "Guest" or "AI" (*used later in the program*)
- The username entered has a suitable length (*between 2 and 25 characters long*)
- The user entered is not already in use

Email (Valid if...)

- The email is in the correct format (e.g., testing123@hotmail.com)
- The email entered is not already in use

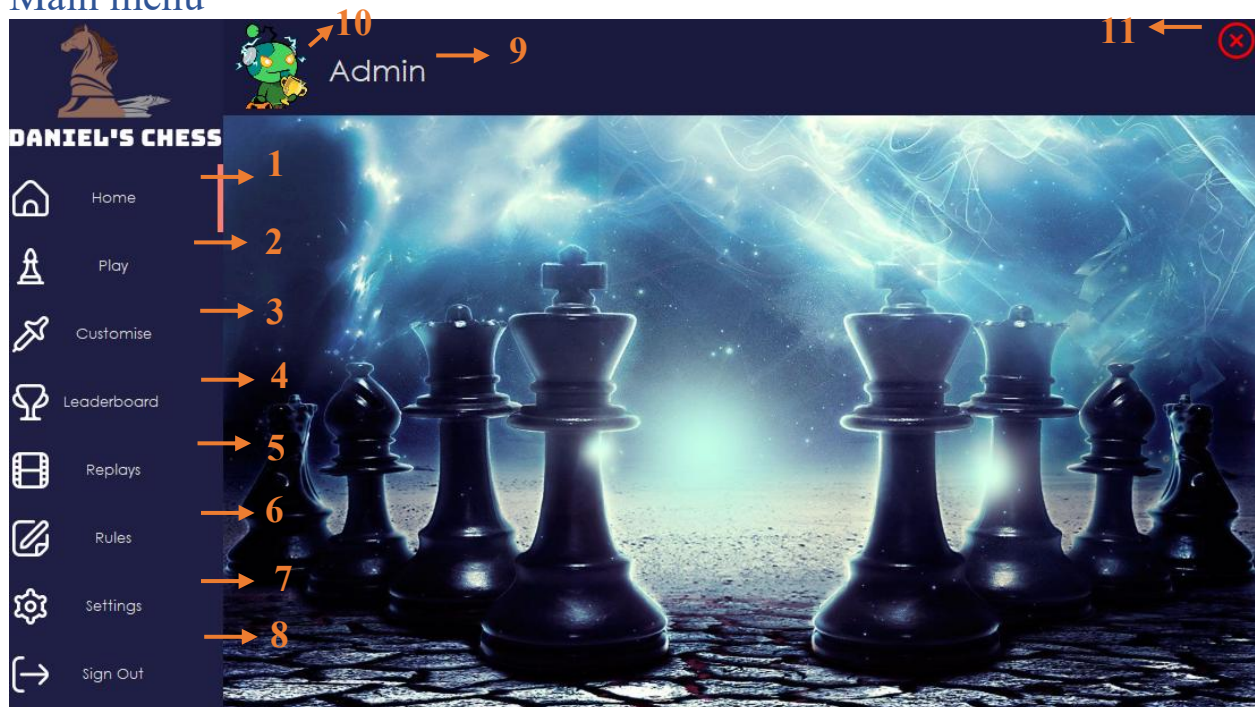
Password (Valid if...)

- Password textbox is not empty
- Confirm password textbox is not empty
- Confirm password textbox matches correctly
- The password entered has a suitable length (*between 5 and 30 characters long*) for secure password purposes

If the user's data is valid then their account will be added to the database, as well as a sign-up confirmation email being sent. After, the sign-up form will close, and the login form will open.

7. Return button - if the user activates the return button it will send them back to the login form. The only exception to this is being if the user is *already signed in* (referred to on the **play form**)
8. Shortcut's button - to help the user enter their email, these shortcut buttons add the text (corresponding to the button) into the email textbox.
9. Exit button - if the user wants to exit the program and they click this button, a Yes/No message box will appear asking the user to confirm whether they want to close the program or not

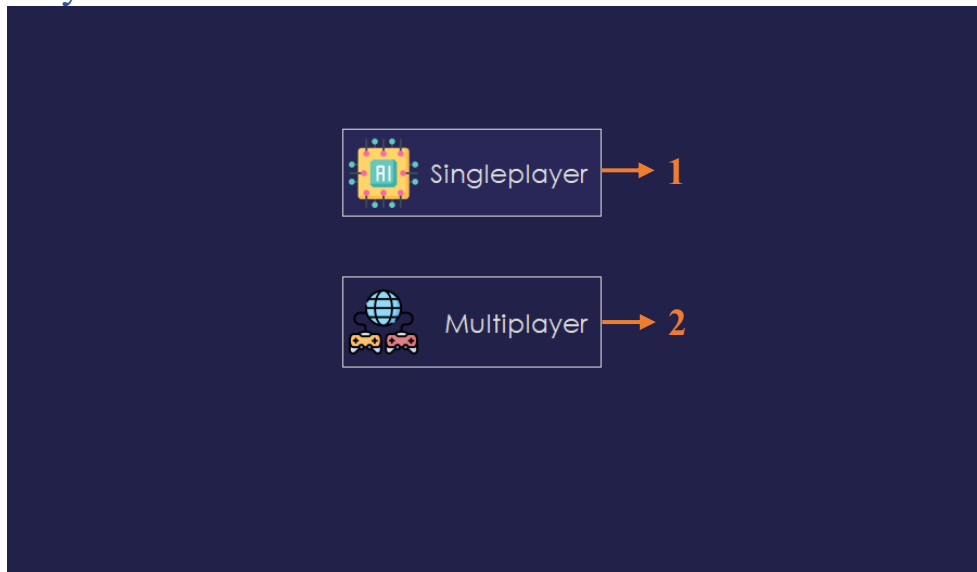
Main menu



1. Home button - if the user activates this button, then the home page form will be loaded onto the main menu (*note - when the main menu starts up, home page is automatically loaded first*)
2. Play button - if the user activates this button, then the play form will be loaded onto the main menu
3. Customise button - if the user activates this button, then the customisation form will be loaded onto the main menu
4. Leaderboard button - if the user activates this button, then the leaderboard form will be loaded onto the main menu
5. Replay button - if the user activates this button, then the replay form will be loaded onto the main menu
6. Rules button - if the user activates this button, then the rules form will be loaded onto the main menu
7. Settings button - if the user activates this button, then the settings form will be loaded onto the main menu
8. Sign out button - if the user activates this button, then a Yes/No message box will appear asking the user to confirm whether they would like to sign out of their account or not
9. Username label - after the user has logged in through the *login form*, their username will be displayed at the top of the main menu
10. Profile picture - after the user has logged in through the *login form*, their selected profile picture will be displayed at the top of the main menu (*new users get a default profile picture*)

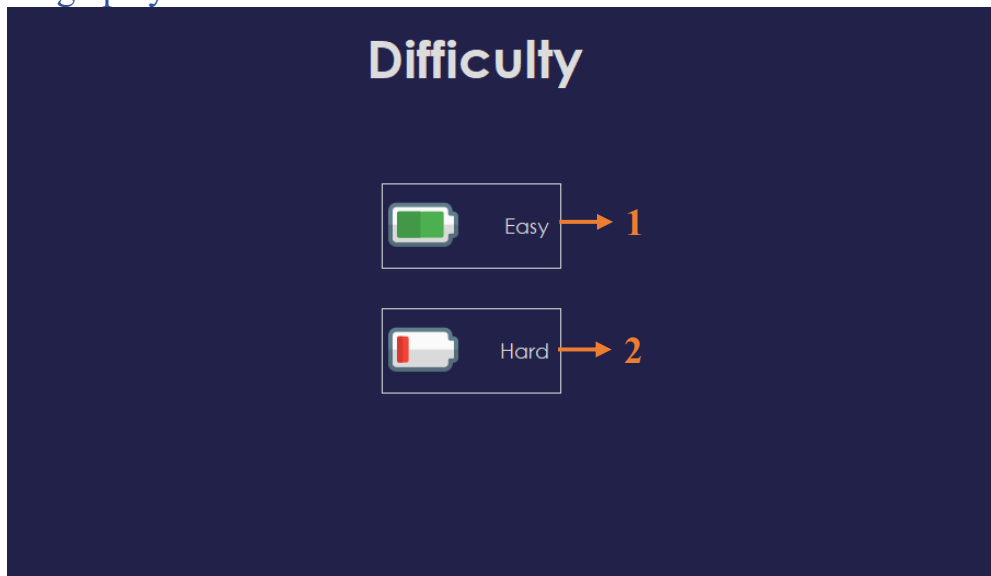
11. Exit button - if the user wants to exit the program and they click this button, a Yes/No message box will appear asking the user to confirm whether they want to close the program or not

Play



1. Singleplayer button - if the user activates this button, the singleplayer form will be shown and the play form will be closed
2. Multiplayer button - if the user activates this button, the multiplayer form will be shown and the play form will be closed

Singleplayer



1. Easy button - the game form is shown, and the singleplayer form is closed (*AI = easy mode*)
2. Hard button - the game form is shown, and the singleplayer form is closed (*AI = hard mode*)

Multiplayer

1. Player 1 label - loading the username of player one (*logged in user*)
2. Player 2 label - loading the username of player two (*N/A until updated*)
3. Username textbox - the user will type in their username or email into this textbox for it to be checked with the database
4. Password textbox - the user will type in password into this textbox for it to be checked with the database
5. Show password checkbox - when the user types in their password into the *password textbox*, it will display all the characters as *, to improve security within the program. Therefore, if the user ticks the checkbox, then it reveals the password they have entered (and vice versa when the user unchecks the textbox)
6. Login button - refer to the database, checking if the username/email and password match up with a player's profile. If this is the case then allow the user to be signed in, otherwise reject the user's request
7. Guest button - if the user does not have an account/doesn't want to sign in, they can play as a guest. Activating this button means the player 2 username will be "Guest" (*which is why the user cannot sign up with the username 'Guest'*)
8. Sign-up linked label - if the user does not have an account they can sign up. If the linked label is activated then it will show the sign-up form, and disable the main menu form (*not close it*):

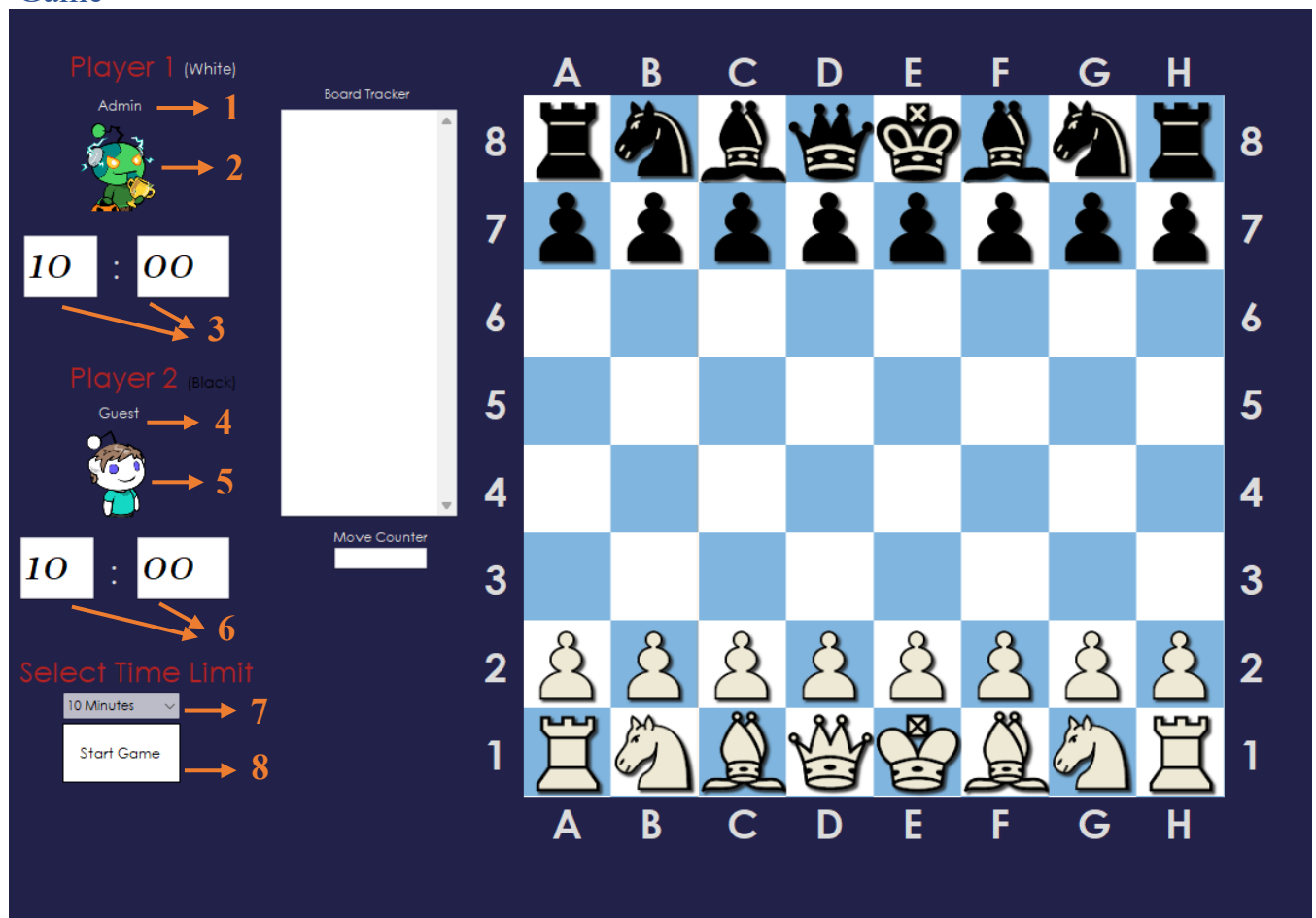
The screenshot shows the 'DANIEL'S CHESS' Admin interface. On the left is a sidebar with icons and labels for Home, Play, Customise, Leaderboard, Replays, Rules, Settings, and Sign Out. The main area features a chessboard image and a sign-up form overlay. The form includes fields for Username, Email, Password, and Confirm Password, along with a 'Show Password' checkbox, a 'Sign Up' button, and a 'Return' button. A 'Shortcuts' section lists email domains: @icloud.com, @hotmail.com, @mgs.kent.sch.uk, @gmail.com, and @yahoo.com. A red 'X' icon is in the top right corner of the form overlay.

The sign-up form is the same form referenced under the *sign-up* section. The difference is when the user selects the 'return' button or signs up successfully, it will return to the main menu form, instead of the login form.

After the user has either logged in with an account, or selected to play as a guest, the *player 2 label* will change to the correct username, and button to go to the game form will appear:

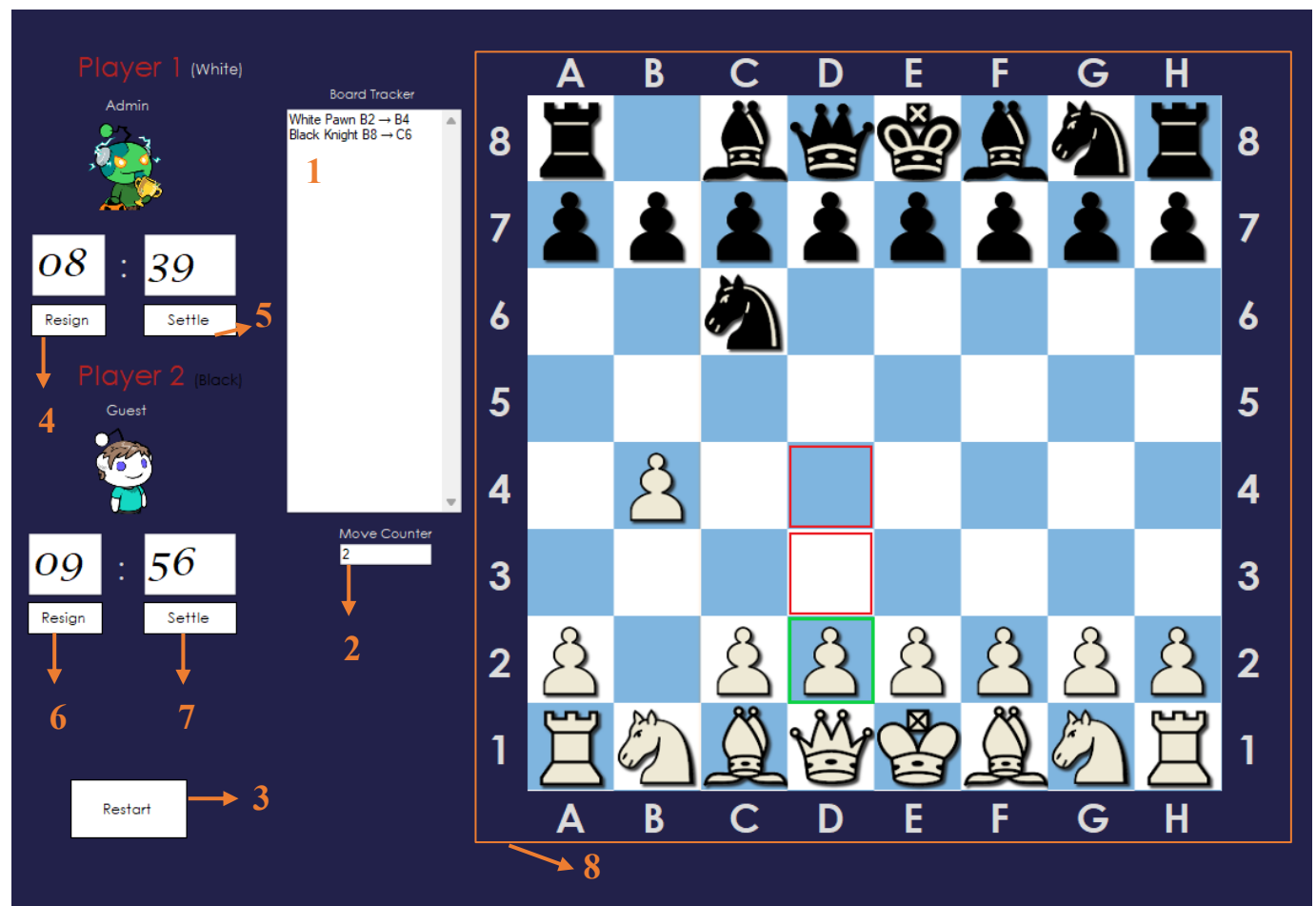
The screenshot shows the 'DANIEL'S CHESS' Admin interface with the multiplayer form. The sidebar is the same as in the previous image. The main area shows the multiplayer form with labels for Player 1 (Admin) and Player 2 (Guest). A 'Play' button is visible. Annotations with arrows point to the 'Updated player 2 label' and the 'Button to go to the game form, and close the multiplayer form'.

Game



Note - this is the interaction the user's get before the game is started

1. Player 1 label - loading the username of player one (*logged in user*)
2. Player 1 profile picture - loading the selected profile picture for player one (*logged in user*)
3. Player 1 minutes & seconds textbox - both textboxes start at 00. When the time combo box has been selected with a time, the textboxes will display that time and will act as timers for player 1
4. Player 2 label - loading the username of player two
5. Player 2 profile picture - loading the selected profile picture for player two
6. Player 2 minutes & seconds textbox - both textboxes start at 00. When the time combo box has been selected with a time, the textboxes will display that time and will act as timers for player 2
7. Time combo box - if the drop down is selected then show the available game times. When a user has selected their desired game time, it will update both minutes and seconds textboxes
8. Start game button - this button can only be activated when a time has been selected by the user. If this is the case then it will correctly start the game, hiding the *time combo box* during its execution



Note - this is the interaction the user's get after the game is started

1. Board tracker textbox - after a player has completed a move, the program copy's the moves details into the board tracker. It uses chess notation from the horizontal axis and the vertical axis, to calculate the movement, as well as the colour of the piece and the type of piece.
2. Move counter textbox - after a move has been completed the move counter will increase by 1
3. Restart button - if the user activates this button, it will restart the game. All the chess pieces will return to their original position, the timers would get reset and all textboxes will be cleared
4. Player 1 resignation button - if the user activates this button, the program will display a Yes/No message box promoting the user to either confirm their resignation or continuation of the game
5. Player 1 settle button (**Multiplayer only**)- if the user activates this button, the program will display settlement form, and disable the game form
6. Player 2 resignation button (**Multiplayer only**)- if the user activates this button, the program will display a Yes/No message box promoting the user to either confirm their resignation or continuation of the game
7. Player 2 settle button (**Multiplayer only**)- if the user activates this button, the program will display settlement form, and disable the game form:



If both players agree to settle, then the game will end, and the result will be a draw. In contrast, if any other outcome of the settlement form happens, the game will continue, and the settlement will be rejected

8. Board - the board contains 64 squares, meaning there are 64 interactive places on the board for the user:

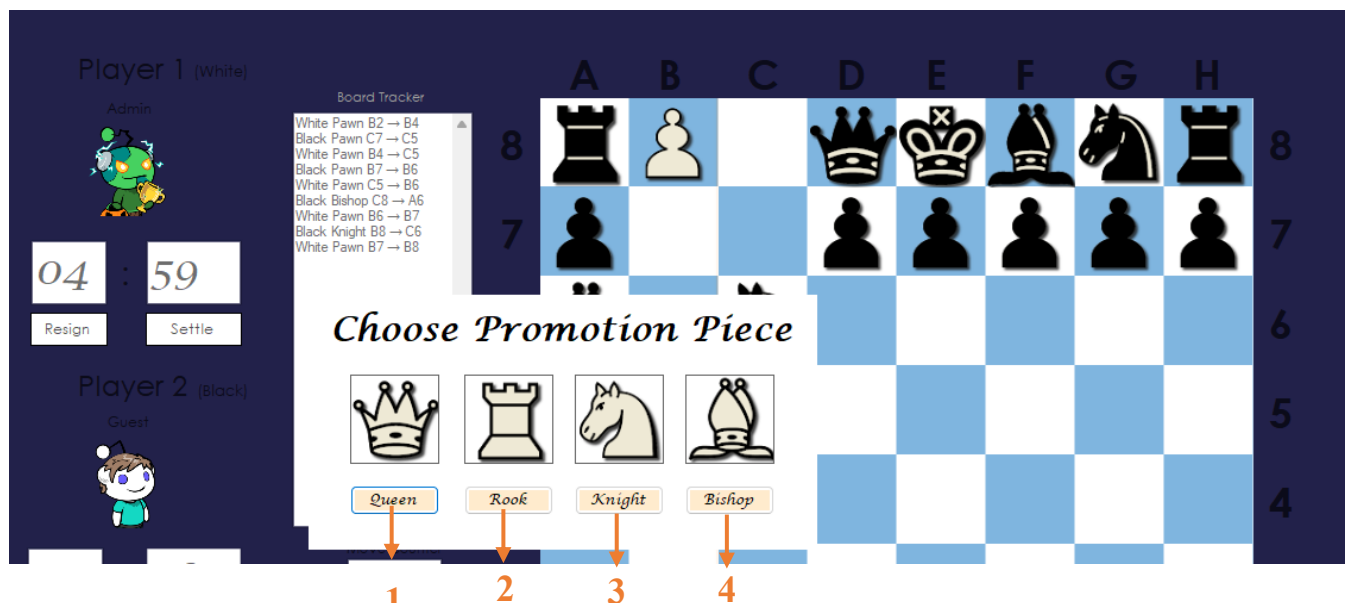
Blank square - if the user selects a blank square on the board, then nothing will happen

Square containing a piece - if a player a coloured piece on their turn, a green selection boarder will appear around the piece they have selected. Furthermore, the program will calculate every possible moves the piece the user has selected can do, and on each square where the piece can move, a red boarder will be around those square (*as seen in the image above*)

Check - if one of the kings is in check, a blue boarder around that king will appear on the board to make it clear to user that their king is in check. If the king is not in check after the next move, then the program will remove that boarder around the king

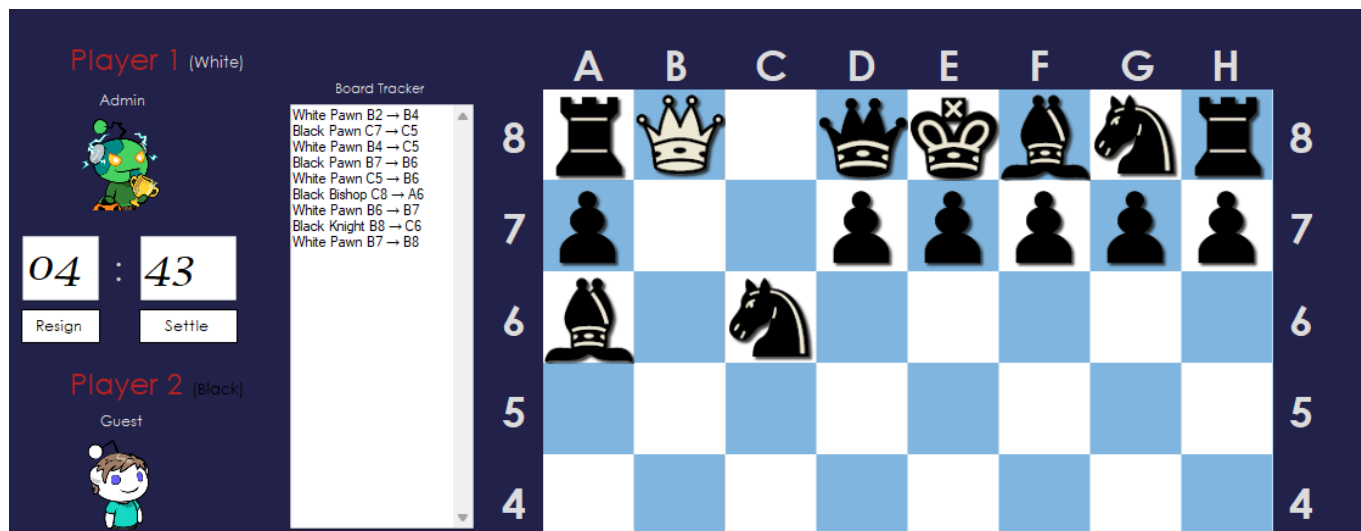
Pawn Promotion

Pawn promotion happens when a pawn has reached the opposite end of the board. If a player has done that within a game, then a form will show up for the user to select a piece for the pawn to turn into. The game form will be disabled, and the corresponding pawn promotion forms for the correct colour will be displayed.

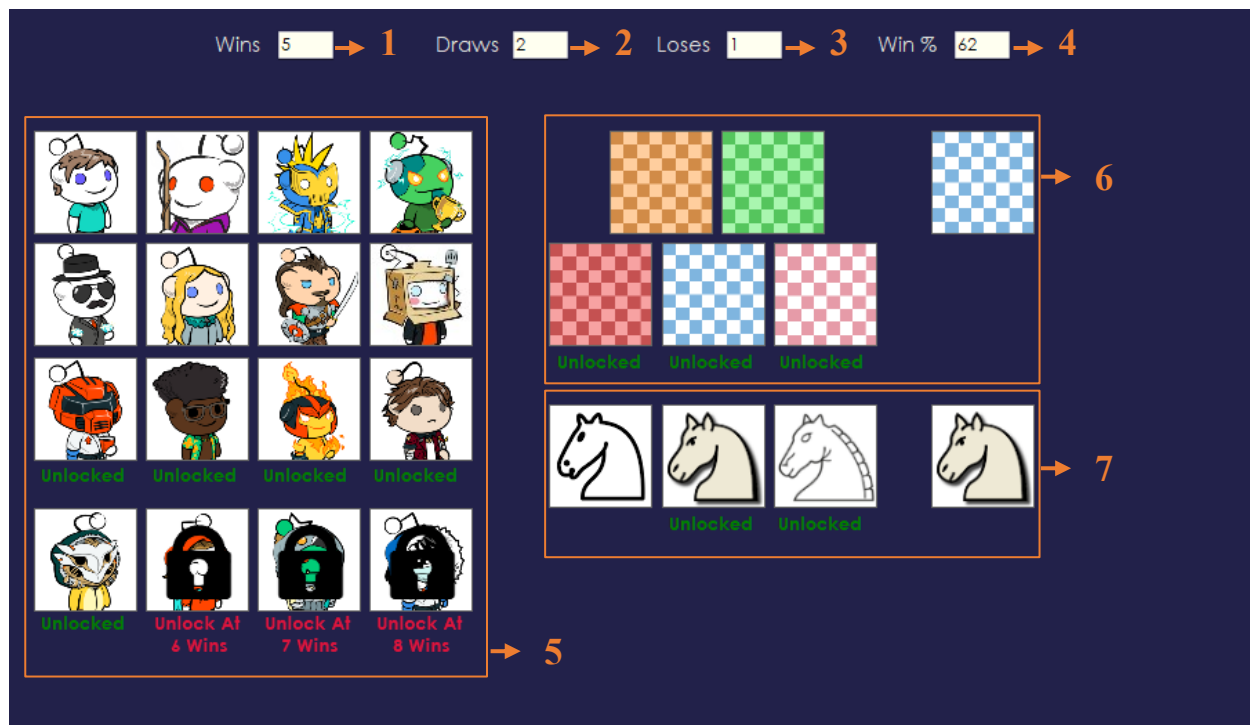


Note - this is the interaction happens for black too

1, 2, 3 & 4 - if the user activates this button, a Yes/No message box will appear to confirm the user's choice of piece. After the user has confirmed the piece, the pawn will be replaced with the corresponding piece they selected. The game form will be once again enabled, and the pawn promotion form will close:



Customise



1. Wins textbox - data is gathered from the database from the user that is logged in and displayed in this textbox
2. Draws textbox - data is gathered from the database from the user that is logged in and displayed in this textbox
3. Loses textbox - data is gathered from the database from the user that is logged in and displayed in this textbox
4. Win percentage textbox - after the data has been gathered for wins/draws/lost, the program will perform a calculation calculating the win percentage of the user
5. Profile picture selection - if the user selects an unlocked profile picture, then update the value of the profile picture in the database. New users should get 6 pictures to choose from, the ones after that they will have to unlock with a certain number of wins
6. Board colour selection - if the user selects an unlocked board colour, then update the value of the board colour in the database. New users should get 2 board colours to choose from, the ones after that they will have to unlock with a certain number of wins
7. Piece type selection - if the user selects an unlocked piece type, then update the value of the piece type in the database. New users should get the default piece type from the start. The ones after that they will have to unlock with a certain number of wins

Leaderboard

Leaderboard

Username	Wins	Draws	Loses	ProfilePicture
Daniel	13	0	3	Locked8
Admin	5	2	1	Skin3
Jenson	2	1	0	Default
Harry	2	0	1	Default
Molly	2	0	0	Skin5
Davidl	2	1	1	Default
Saranyan	1	0	1	Default
Ben	1	0	0	Default
James	0	0	0	Default
Brandon	0	0	0	Default

Daniel



1

Admin



2

Jenson



3

Arrows in the original image point from the table to the labels (2) and from the labels to the profile pictures (3).

1. Leaderboard grid - the database will select every user in it, with their wins, draws, and loses, and order them within the data grid from highest to lowest wins
2. 1st, 2nd, 3rd username labels - the top 3 users in the database will have their usernames loaded onto these 3 labels
3. 1st, 2nd, 3rd profile pictures - the top 3 users in the database will have their corresponding profile picture below their user label

Replays



Note - this is the interaction the user's get before the replay is selected

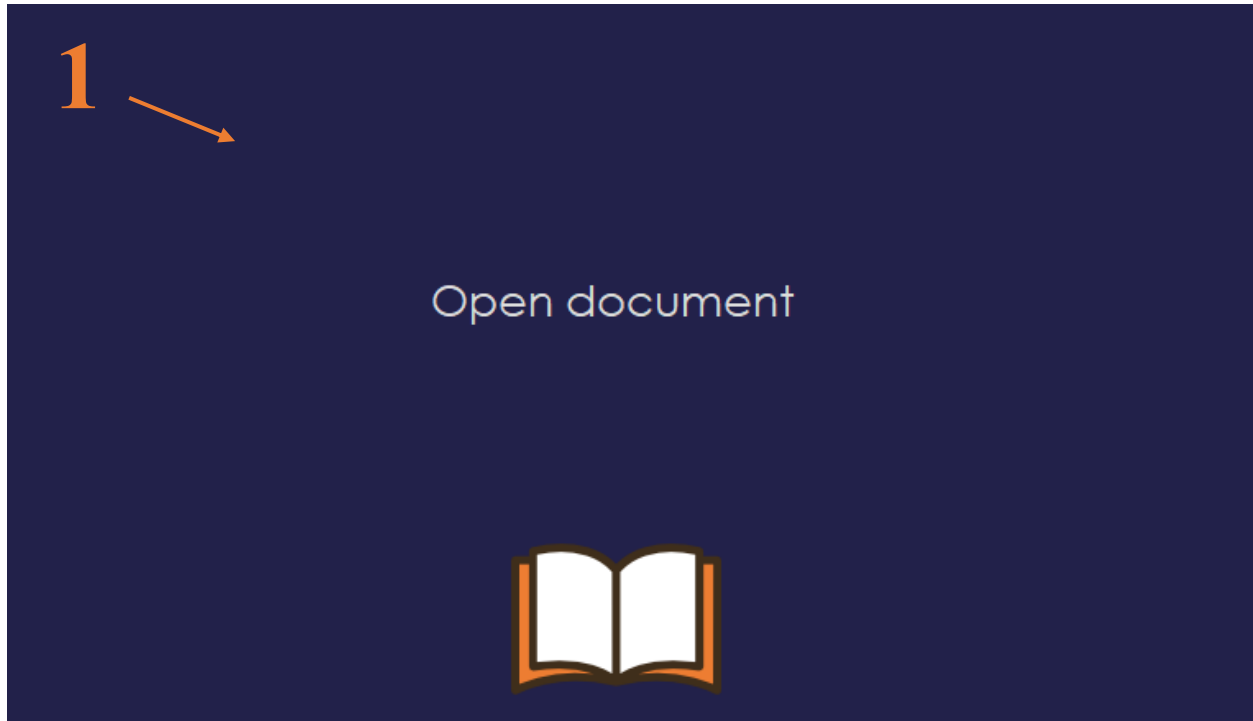
1. Replay selection textbox - every user has 5 slots to save a replay in it. At the end of a game, player 1 has an option to save a game replay. If the user selects a replay that has data in it then the user can view that replay, otherwise if the replay is empty then a message box will show up letting the user know that there is no replay saved under the selected section



1. Rewind button - this button is only shown if the replay is not on the first move. If this button is activated then every second it will automatically rewind one move at a time, shown on the board
2. Pause button - if this button is activated then it will pause the board, keeping all the pieces where they are

3. Play button - this button is only shown if the replay is not on the last move. If this button is activated then every second it will automatically play one move at a time, shown on the board
4. Fast forward button - this button is only shown if the replay is not on the last move. If this button is activated then every half-second it will automatically play one move at a time, shown on the board
5. Change replay button - if the user selects this button, it will return the replay form to the first state above, allowing the user to select the replay they would like to view

Rules



1. Rules button - if this button is activated by the user, the program will open a word document stored in the systems debug folder, which is filled with all the rules of chess for a player understand how to play chess. To make sure the user cannot change this word document, there is a password for any user trying to edit the document

Settings

The screenshot shows a dark-themed settings window. The top section is titled 'Change Password' and contains two white text input fields labeled 'New Password' and 'Confirm New Password'. Below these fields is a checkbox labeled 'Show Password' with an eye icon to its right. To the right of the checkbox is a button with a circular arrow icon and the text 'Change Password'. The bottom section is titled 'Delete Account' and contains a button with a trash can icon and the text 'Delete Account'. Five orange arrows with numbers 1 through 5 point to the following elements: 1 points to the 'New Password' field, 2 points to the 'Confirm New Password' field, 3 points to the 'Show Password' checkbox, 4 points to the 'Change Password' button, and 5 points to the 'Delete Account' button.

1. Password textbox - the user will enter their new password into this textbox
2. Confirm password textbox - the user will confirm their new password in this textbox
3. Show password checkbox - when the user types in their password into the *password textbox/confirm password textbox*, it will display all the characters as *, to improve security within the program. Therefore, if the user ticks the checkbox, then it reveals the password they have entered (and vice versa when the user unchecks the textbox)
4. Change password button - if the user activates this button, it will do the same password validation checks like the *sign-up* form. If the password is valid then a message box will confirm that the password has been successfully changed, otherwise there will be an error message box
5. Delete account button - If the user selects this button, the program will display a Yes/No message box, getting the user to confirm whether they would like to delete their account or not

Objective

1 - Signing up

Summary: if the user signs up with an account that has all the requirements (e.g., valid password, valid email, valid username), and the username/email is not already in use by another user, then it should allow the user to sign up, with their account being stored within a database.

Research

Document (<https://blog.hubspot.com/marketing/sign-up-form>)

Document (<https://www.abtasty.com/blog/best-practices-sign-up-forms/>)

Simplicity was mentioned in both documents regarding creating the sign-up form. The less fields on the form the easier it will be for the user to complete the form and gives a cleaner feel to the form. Both documents heavily focused on the design aspects of the form, giving advice on how it should look and what the user will respond to the best.

Document (https://www.w3schools.com/sql/sql_select.asp)

SQL statements is the way the program carry's out procedures with the access database. This website gives a full give to all the SQL statements, which will help me check if the user is valid to sign up, and if this is true, importing the new user into the database.

The SQL SELECT Statement

The **SELECT** statement is used to select data from a database.

The data returned is stored in a result table, called the result-set.

SQL Select Statement will be used when the program checks for existing account

SELECT Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

The SQL INSERT INTO Statement

The **INSERT INTO** statement is used to insert new records in a table.

INSERT INTO Syntax

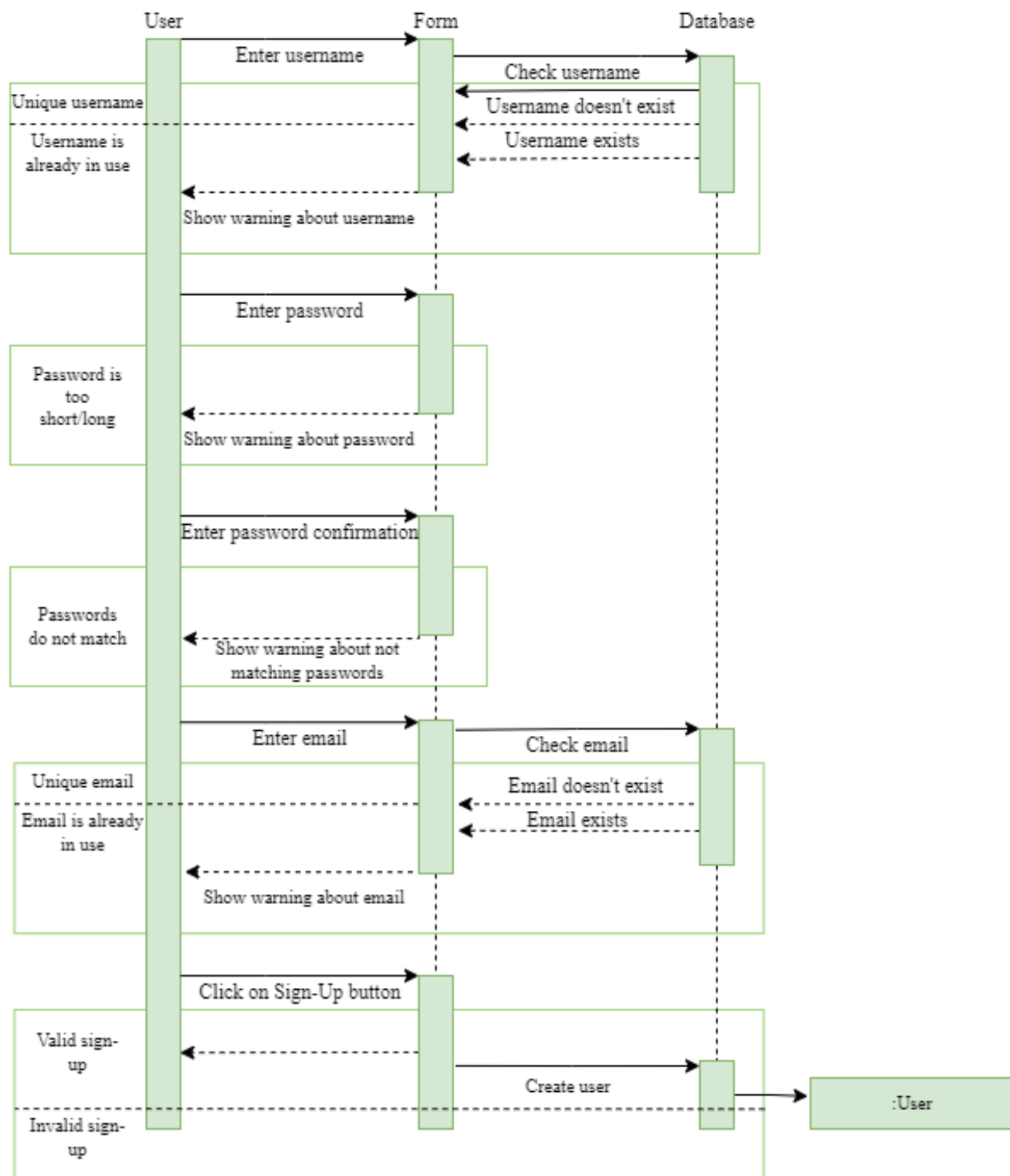
It is possible to write the **INSERT INTO** statement in two ways:

1. Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

SQL Insert Into will be used to insert the new user's account into the database

UML



Pseudo Code

```
username ← userinput
email ← userinput

Userisvalid ← False
If username = "" Then
    Userisvalid = False
ElseIf username = "Guest" Or "AI" Then
    Userisvalid = False
ElseIf Len(username) <= 25 And Len(username) >= 2 Then
    Userisvalid = True
Endif
password ← userinput
confirmpassword ← userinput

If Userisvalid = True Then
    If password = "" Then
        Userisvalid = False
    ElseIf confirmpassword <> password Then
        Userisvalid = False
    ElseIf Len(password) <= 30 And Len(password) >= 5 Then
        Userisvalid = True
    Endif
Endif

For count = 0 To Table1.rows.count
    If username = Table1.username Then
        Userisvalid = False
    ElseIf email = Table1.email Then
        Userisvalid = False
    Endif
Next

If Userisvalid = True Then
    Table1.username ← username
    Table1.email ← email
    Table1.password ← password
Endif
```

VB.Net Code

```
Imports System.Data
Imports System.Data.OleDb
Public Class SignUpForm
    Dim UserIsValid As Boolean = True 'Variable which determines whether or
not the user can sign up
    Public Sub CheckAvailability()
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename
        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Email] FROM [Table1]" 'SQL Statement
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1
            If UsernameTxt.Text = ds.Tables(0).Rows(count).Item(0) Then
                UserIsValid = False
                MessageBox.Show("Username already in use")
            ElseIf EmailTxt.Text = ds.Tables(0).Rows(count).Item(1) Then
                UserIsValid = False
                MessageBox.Show("Email already in use")
            End If
        Next
    End Sub

    Private Sub SignUpButton_Click(sender As Object, e As EventArgs) Handles
SignUpButton.Click
        'Making the conditions for the username
        If UsernameTxt.Text = "" Then 'Username being empty
            MessageBox.Show("Please enter a username")
            UserIsValid = False
        ElseIf UsernameTxt.Text = "Guest" Or UsernameTxt.Text = "AI" Then
            MessageBox.Show("Username is not available to use")
            UserIsValid = False
        ElseIf Len(UsernameTxt.Text) <= 25 And Len(UsernameTxt.Text) >= 2
Then 'Username to be checked against DB
            UserIsValid = True
        Else
            MessageBox.Show("Username is invalid. Please make it between 2
and 25 characters")
            UserIsValid = False
        End If

        'Making the conditions for the password
```

```

If UserIsValid = True Then
    If Passwordtxt.Text = "" Then
        MessageBox.Show("Please enter a password")
        UserIsValid = False
    ElseIf ConfirmPasswordtxt.Text = "" Then
        MessageBox.Show("Please confirm your password")
        UserIsValid = False
    ElseIf Len(Passwordtxt.Text) < 5 Then
        MessageBox.Show("Password is too short. Please make it
between 5 and 30 characters")
        UserIsValid = False
    ElseIf Len(Passwordtxt.Text) > 30 Then
        MessageBox.Show("Password is too long. Please make it between
5 and 30 characters")
        UserIsValid = False
    ElseIf Passwordtxt.Text <> ConfirmPasswordtxt.Text Then
        MessageBox.Show("Passwords do not match up")
        UserIsValid = False
    ElseIf Len(Passwordtxt.Text) >= 5 And Len(Passwordtxt.Text) <= 30
And Passwordtxt.Text = ConfirmPasswordtxt.Text Then 'Password is valid
        UserIsValid = True
    End If
End If

'Checking availability against db
If UserIsValid = True Then
    CheckAvailability()
End If

'If everything is valid then add users credentials to the database
If UserIsValid = True Then
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename
    Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
    cn.Open() 'open connection
    Dim sql As String
    sql = "INSERT INTO [Table1] ([Username], [Password], [Email],
[ProfilePicture], [BoardColour], [PieceType]) VALUES ('"
    sql = sql & Usernametxt.Text & "',''"
    sql = sql & Passwordtxt.Text & "',''"
    sql = sql & Emailtxt.Text & "',''"
    sql = sql & "Default" & "',''"
    sql = sql & "YellowBoard" & "',''"
    sql = sql & "Set1" & "');";
    'sends SQL command to OLEDB
    Dim cmd As New OleDbCommand(sql, cn)
    'Execute SQL command that doesn't return a query
    cmd.ExecuteNonQuery()
    'close connection
    cn.Close()
    MessageBox.Show("Sign up complete!")

```

```
'clearing text boxes
UsernameTxt.Text = ""
EmailTxt.Text = ""
PasswordTxt.Text = ""
ConfirmPasswordTxt.Text = ""

If MultiplayerForm.Player1AlreadySignedIn = True Then
    MultiplayerForm.Show()
    MainMenu.Enabled = True
    Me.Close()
Else
    LoginForm.Show()
    Me.Close()
End If
End Sub
End Class
```

Test Plan

Username

Description: testing the username the user has entered after they have clicked the sign-up button

Test Data	Type of Test Data	Expected Outcome
Preacher	Normal	Valid Username
Admin	Invalid	Username already taken
Guest/AI	Invalid	Username unavailable
A	Invalid	Invalid Username
UsernameIsWhichIsMoreThan CharactersLong	Invalid	Invalid Username
JJ	Borderline	Valid Username

Password

Description: testing the password the user has entered after they have clicked the sign-up button

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	JUJ24d	Normal	Valid Password
JUJ24d	1234567	Invalid	Invalid Password (Don't match)
1234	1234	Invalid	Invalid Password (Too short)
12345	12345	Borderline	Valid Password

Email

Description: testing the email the user has entered after they have clicked the sign-up button

Test Data	Type of Test Data	Expected Outcome
Testing123@hotmail.com	Normal	Valid Email
15dlawrence@mgs.kent.sch.uk	Invalid	Invalid Email (Already Taken)
Testing	Invalid	Invalid Email (Not in the correct form)
N/A	Borderline	N/A

2 - Logging in

Summary: if the user types in valid data into the username textbox and the password text box, check it against the database. If the username/email is found in the database, and the password matches up to it then allow the user to be logged in. Also have a working recovery password option for the user.

Research

Document (<https://www.uxforthemasses.com/login-page/>)

Login forms are very similar to the basis of sign-up forms. This website states to have a clear and simple form, however, to have a clear differentiation between the sign-up form and login form. Furthermore, it says to not only allow the user to login with a username, but also with their email. In addition, if CAPS lock is accidentally left on by the user, a reminder should help the user realise it. Finally, users who forget their password need an easy but secure way of recovering their account.

Document (https://www.w3schools.com/sql/sql_select.asp)

SQL statements is the way the program carry's out procedures with the access database. This website gives a full give to all the SQL statements, which will help me check if the user has an account to login with.

Log In

[Forgot password?](#)

Simple example of a Login Form

The SQL SELECT Statement

The **SELECT** statement is used to select data from a database.

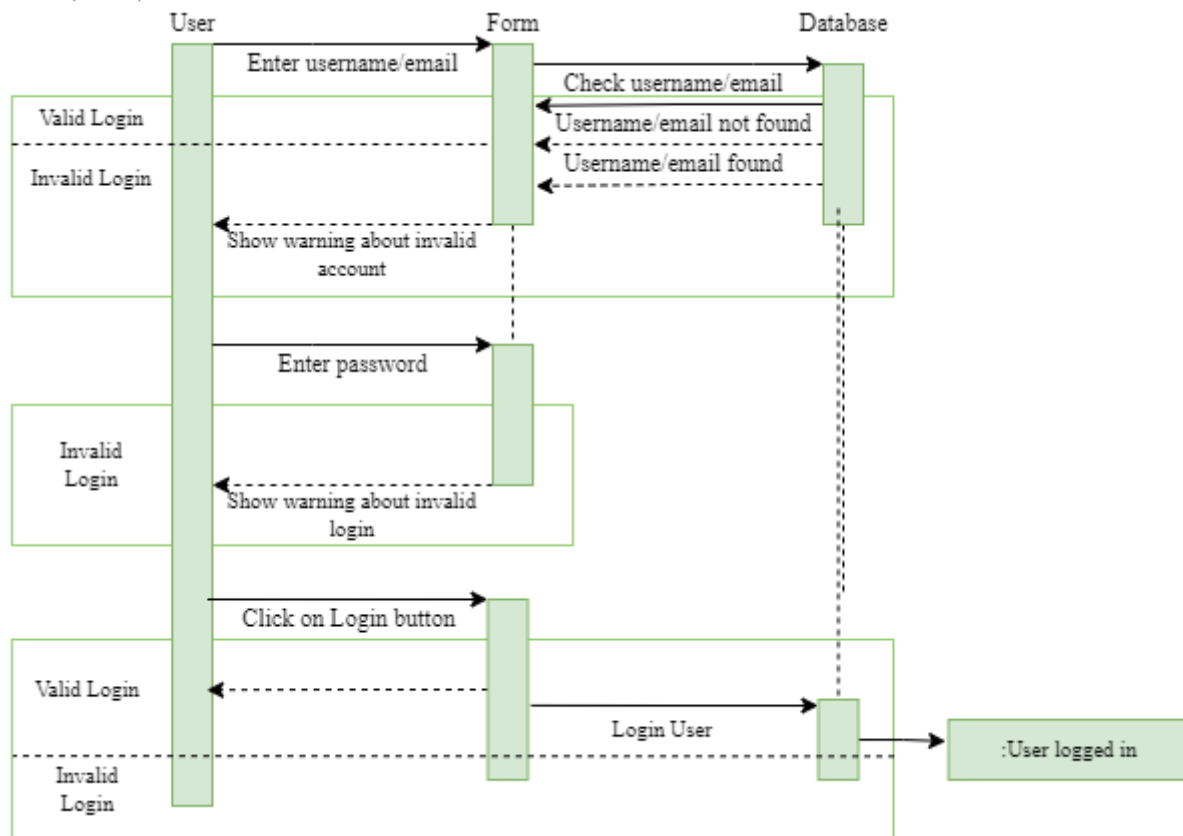
The data returned is stored in a result table, called the result-set.

SQL Select Statement will be used when allowing the user to login or not

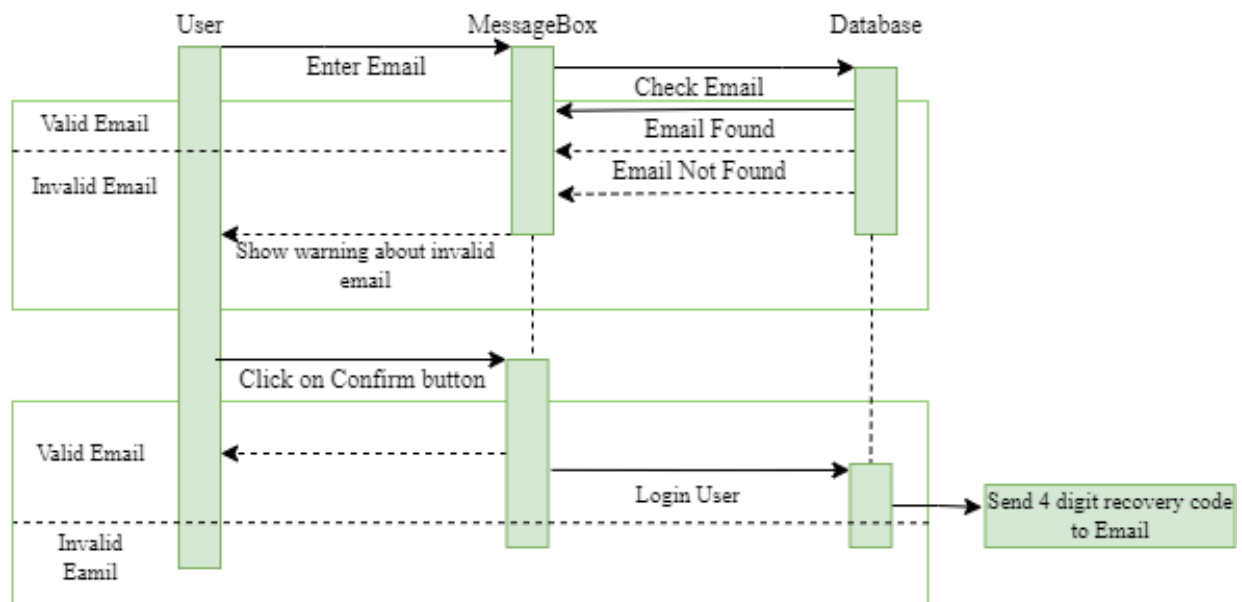
SELECT Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

UML(Main)



UML(Forgotten password)



Pseudo Code(Main)

```

username ← userinput
password ← userinput

For count = 0 To Table1.rows.count
    If username = Table1.username And password = Table1.password Then
        Loginisvalid = True
        Exit For
    ElseIf username = Table1.email And password = Table1.password
        Loginisvalid = True
        Exit For
    Else
        Loginisvalid = False
    Endif
Next

```

Pseudo Code(Forgotten password)

```

email ← userinput
Sub ForgottenPassword
For count = 0 To Table1.rows.count
    If email = Table1.email Then
        attempcount ← 3
        number(1) = RANDOM_INT
        number(2) = RANDOM_INT
        number(3) = RANDOM_INT
        number(4) = RANDOM_INT
        smtpserver ← smtpclient
        emailmessage ← message
        email.to.add(email)
        email.body = number(1) & number(2) & number(3) & number(4)
        smtpserver.send(email)
        Do
            Output("Enter your 4-digit validation code that has been emailed
to you " & attempcount & " attemps remaining")
            myresetcode ← userinput
            If myresetcode = number(1) & number(2) & number(3) & number(4) Then
                changepassword ← True
                Exit Sub
            Else
                attempcount = attempcount - 1
            Endif
        Loop Until attempcount = 0
        Output("Ran out of attemps")
        Exit Sub
    Endif
Next
Output("Email not found")
End Sub

```

VB.Net Code

```
Imports System.Data.OleDb
Imports System.Data
Imports System.Net.Mail
Public Class LoginForm
    Dim LoginValid As Boolean = True 'Variable for knowing if User
    Credentials match up
    Public Player1Username As String
    Public Player1ID As Integer
    Private Sub SignUpLink_Changed(sender As Object, e As
LinkLabelLinkClickedEventArgs) Handles SignUpLink.LinkClicked
        'Clear Username and Password text box
        UsernameTxt.Text = ""
        PasswordTxt.Text = ""
        'Show sign up form
        SignUpForm.Show()
        'Have to hide the form cannot close it as it is the startup form,
otherwise program will stop
        Me.Hide()

    End Sub

    Private Sub LoginButton_Click(sender As Object, e As EventArgs) Handles
LoginButton.Click
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT * FROM [Table1]" 'SQL Statement
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for number of
users in table, (-1) as starts from 0
            If UsernameTxt.Text = ds.Tables(0).Rows(count).Item(1) And
PasswordTxt.Text = ds.Tables(0).Rows(count).Item(2) Then
                LoginValid = True
                'Storing username for player 1
                Player1Username = ds.Tables(0).Rows(count).Item(1)
                Player1ID = ds.Tables(0).Rows(count).Item(0)
                'If login valid exit loop
                Exit For
            ElseIf UsernameTxt.Text = ds.Tables(0).Rows(count).Item(3) And
PasswordTxt.Text = ds.Tables(0).Rows(count).Item(2) Then
                LoginValid = True
            End If
        Next
```

```

        'Storing username for player 1
        Player1Username = ds.Tables(0).Rows(count).Item(1)
        Player1ID = ds.Tables(0).Rows(count).Item(0)
        'If login valid exit loop
        Exit For
    Else
        LoginValid = False
    End If
Next
cn.Close() 'Close connection

'Displaying logged in or not
If LoginValid = True Then
    MessageBox.Show("Welcome " & Player1Username)
    'Clearing text boxes
    UsernameTxt.Text = ""
    PasswordTxt.Text = ""
    MainMenu.Show() 'open main menu form
    Me.Hide() 'hide login form
Else
    MessageBox.Show("Incorrect details, please try again.")
    PasswordTxt.Text = ""
End If
End Sub
Dim Message, Title, MyValue, MyResetCode
Dim number(4) As Integer
Private Sub ForgottenPasswordLink_LinkClicked(sender As Object, e As
LinkLabelLinkClickedEventArgs) Handles ForgottenPasswordLink.LinkClicked
    Message = "Enter your email address you would like to reset your
password for" ' Set prompt.
    Title = "Forgotten Password" ' Set title.
    ' Display message, title, and default value.
    MyValue = InputBox(Message, Title)
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Email], [ID] FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer
    Dim attemptscount As Integer = 3

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0

```

```

        If MyValue = ds.Tables(0).Rows(count).Item(0) Then 'For player 1
stats
            SendCode()
            Player1ID = ds.Tables(0).Rows(count).Item(1)
            Do
                Message = "Enter your 4-digit validation code that has
been emailed to you. " & attemptscount & " attempts remaining." ' Set
prompt.
                Title = "Forgotten Password" ' Set title.
                ' Display message, title, and default value.
                MyResetCode = InputBox(Message, Title)
                If MyResetCode = number(1) & number(2) & number(3) &
number(4) Then
                    ForgottenPasswordForm.Show()
                    Exit Sub
                Else
                    attemptscount -= 1
                End If
            Loop Until attemptscount = 0
            MessageBox.Show("Ran out of attempts")
            Exit Sub
        End If
    Next
    cn.Close()
    MessageBox.Show("Email not found")

End Sub

Private Sub SendCode()
    Dim rnd As New Random
    'Produce 4 random numbers between 0 and 9
    number(1) = rnd.Next(0, 10)
    number(2) = rnd.Next(0, 10)
    number(3) = rnd.Next(0, 10)
    number(4) = rnd.Next(0, 10)
    Dim smtpserver As New SmtpClient
    Dim email As New MailMessage
    smtpserver.UseDefaultCredentials = False
    'Email being sent from
    smtpserver.Credentials = New
Net.NetworkCredential("15dlawrence@mgs.kent.sch.uk", "NeaChess123")
    smtpserver.Port = 587
    smtpserver.EnableSsl = True
    smtpserver.Host = "smtp-mail.outlook.com"
    email = New MailMessage
    email.From = New MailAddress("15dlawrence@mgs.kent.sch.uk")
    'Email being sent too...
    email.To.Add(MyValue)
    'Email Subject
    email.Subject = "Reset Password"
    email.IsBodyHtml = False
    'Sending message
    email.Body = "Your 4-digit reset code is : " & number(1) & number(2)
& number(3) & number(4)
    smtpserver.Send(email)
End Sub
End Class

```

Test Plan

Login

Description: testing the username/email and password the user has entered after they have clicked the login button

Test Data 1 - Username/Email	Test Data 2 - Password	Type of Test Data	Expected Outcome
Admin	Admin	Normal	Valid Login
Daniel42r@icloud.com	Admin	Normal	Valid Login
Admin	Tester	Invalid	Invalid Login (Incorrect Password)
Testing	Admin	Invalid	Invalid Login (Account not found)
N/A	N/A	Borderline	N/A

Forgotten password

Description: testing the email and the 4-digit code the user enters after they have clicked the forgotten password linked label (*testing purposes - code is 4983*)

Test Data 1 - Email	Test Data 2 - Code	Type of Test Data	Expected Outcome
Admin	N/A	Normal	Invalid Email
15dlawrence@mgs.kent.sch.uk	4983	Normal	Valid Login
15dlawrence@mgs.kent.sch.uk	5000	Invalid	Invalid Code
testing@mgs.kent.sch.uk	N/A	Invalid	Invalid Login (Email not found)
N/A	N/A	Borderline	N/A

3 - Game form

Summary: correctly loading the board/accounts into the game with the pieces in the correct starting positions. After the user has chosen a time, correctly allow the game to start. When the game has finished, display the result and store result (and replay if saved) within the database. Allow chess notation throughout the game.

Research

Document (<https://www.chess.com/lessons>)

The SQL SELECT Statement

The **SELECT** statement is used to select data from a database.

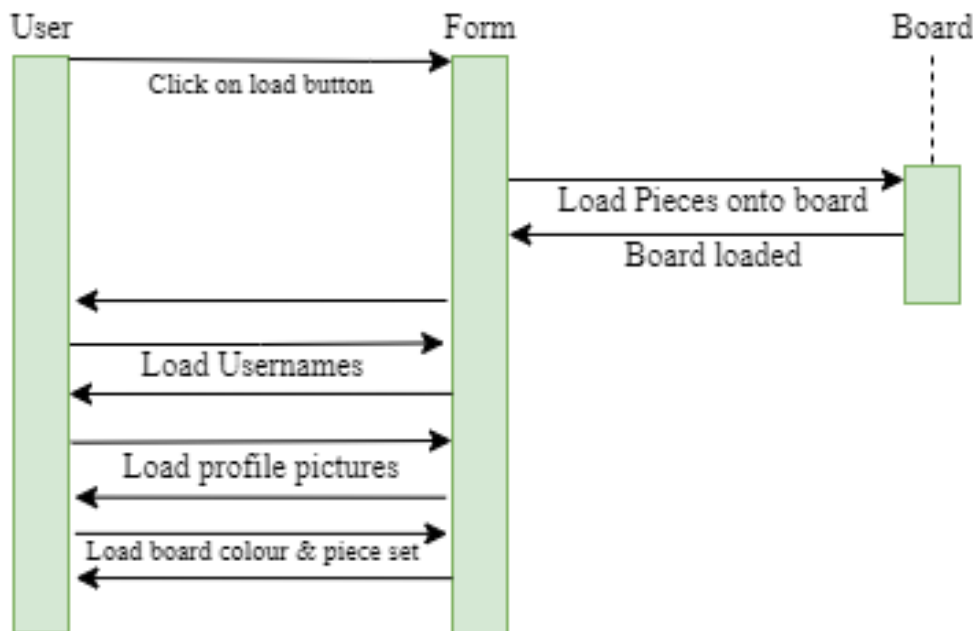
The data returned is stored in a result table, called the result-set.

SELECT Syntax

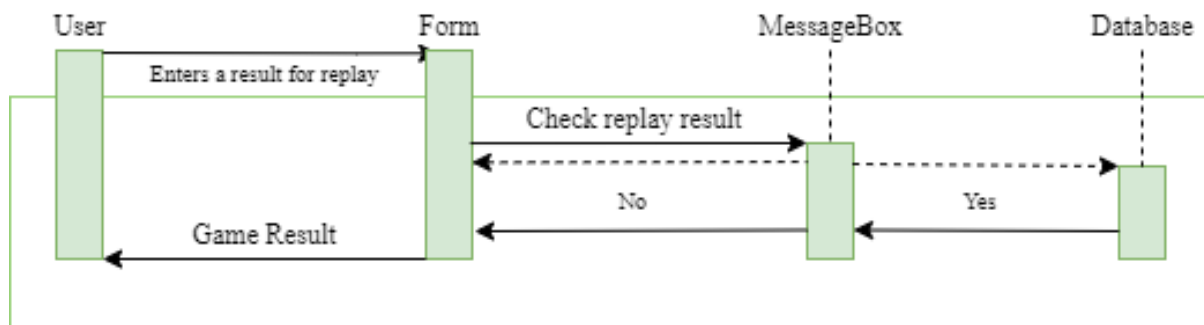
```
SELECT column1, column2, ...
FROM table_name;
```

SQL Select statement will be used when loading the account information of the players onto the page

UML(Account/Board Loading)



UML(End of game)



Pseudo Code

N/A - UI

VB.Net Code

```
Imports System.Data.OleDb
```

```
Imports System.Data
```

```
Public Class GameForm1
```

```
    'Chess pieces:
```

```
    '1) Pawn
```

```
    '2) Bishop
```

```
    '3) Rook
```

```
    '4) Knight
```

```
    '5) Queen
```

```
    '6) King
```

```
    Public pb(7, 7) As PictureBox
```

```
    Public whiteking As King
```

```
    Public whitequeen(8) As Queen '1 Queen to start, 8 other possible queens  
due to pawn promotions
```

```
    Public whitebishop(9) As Bishop '2 Bishops to start, 8 other possible  
bishops due to pawn promotions
```

```
    Public whiteknight(9) As Knight '2 Knights to start, 8 other possible  
knights due to pawn promotions
```

```
    Public whitepawn(7) As Pawn
```

```
    Public whiterook(9) As Rook '2 Rooks to start, 8 other possible rooks due  
to pawn promotions
```

```
    Public blackking As King
```

```
    Public blackqueen(8) As Queen '1 Queen to start, 8 other possible queens  
due to pawn promotions
```

```
    Public blackbishop(9) As Bishop '2 Bishops to start, 8 other possible  
bishops due to pawn promotions
```

```
    Public blackknight(9) As Knight '2 Knights to start, 8 other possible  
knights due to pawn promotions
```

```
    Public blackpawn(7) As Pawn
```

```
    Public blackrook(9) As Rook '2 Rooks to start, 8 other possible rooks due  
to pawn promotions
```

```
    Public wqueencount As Integer = 0
```

```
    Public wbishopcount As Integer = 0
```

```
    Public wknightcount As Integer = 0
```

```
    Public wrookcount As Integer = 0
```

```
    Public bqueencount As Integer = 0
```

```
    Public bbishopcount As Integer = 0
```

```
    Public bknightcount As Integer = 0
```

```
    Public brookcount As Integer = 0
```

```
    Public playerssettle As Boolean
```

```
    Public ga As Game
```

```
    Public x, y, selectionx, selectiony As Integer
```

```
    Public selected As Boolean = False
```

```
    Public allowgametostart As Boolean = False
```

```
    Public gamestarted As Boolean = False
```

```
    Public selectedpiece As String = Nothing
```

```
    Public Set1 As Boolean = False
```

```
    Public Set2 As Boolean = False
```

```
    Public Set3 As Boolean = False
```

```
    Private Sub GameForm1_Load(sender As Object, e As EventArgs) Handles  
MyBase.Load
```

```
        'Loading player 1 name
```

Daniel Lawrence

Candidate Number: 7300

Centre Number: 61715


```

Player1Label.Text = LoginForm.Player1Username
'Loading player 2 username
Player2Label.Text = MultiplayerForm.Player2Username
'Making sure buttons not needed aren't visible
RestartButton.Visible = False
'Give board tracker properties
BoardTracker.Multiline = True
BoardTracker.ScrollBars = ScrollBars.Vertical
'Setting times to 00
MinutesTextBox1.AppendText("00")
SecondsTextBox1.AppendText("00")
MinutesTextBox2.AppendText("00")
SecondsTextBox2.AppendText("00")
'Loading User 1 And User 2 profile pictures
'Setting database location
Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
'opening connection
Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
Dim DBsource As String = "Data Source=" & dbfilename

Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
Dim sql As String
sql = "SELECT [Username],[ProfilePicture],[BoardColour],[PieceType]
FROM [Table1]" 'SQL Statement
Dim cmd As New OleDbCommand(sql, cn)
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
Dim count As Integer

For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for number of
users in table, (-1) as starts from 0
If Player1Label.Text = ds.Tables(0).Rows(count).Item(0) Then 'For
player 1 profile picture (for a user that has signed in)
Select Case (ds.Tables(0).Rows(count).Item(1))
Case "Default"
P1ProfilePicture.Image = My.Resources.DefaultSkin
Case "Skin1"
P1ProfilePicture.Image = My.Resources.Skin1
Case "Skin2"
P1ProfilePicture.Image = My.Resources.Skin2
Case "Skin3"
P1ProfilePicture.Image = My.Resources.Skin3
Case "Skin4"
P1ProfilePicture.Image = My.Resources.Skin4
Case "Skin5"
P1ProfilePicture.Image = My.Resources.Skin5
Case "Skin6"
P1ProfilePicture.Image = My.Resources.Skin6
Case "Skin7"
P1ProfilePicture.Image = My.Resources.Skin7
Case "Locked1"

```

```

        P1ProfilePicture.Image = My.Resources.Locked1
    Case "Locked2"
        P1ProfilePicture.Image = My.Resources.Locked2
    Case "Locked3"
        P1ProfilePicture.Image = My.Resources.Locked3
    Case "Locked4"
        P1ProfilePicture.Image = My.Resources.Locked4
    Case "Locked5"
        P1ProfilePicture.Image = My.Resources.Locked5
    Case "Locked6"
        P1ProfilePicture.Image = My.Resources.Locked6
    Case "Locked7"
        P1ProfilePicture.Image = My.Resources.Locked7
    Case "Locked8"
        P1ProfilePicture.Image = My.Resources.Locked8
End Select
Select Case (ds.Tables(0).Rows(count).Item(2))
    Case "YellowBoard"
        Me.BackgroundImage = My.Resources.YellowBoard
    Case "GreenBoard"
        Me.BackgroundImage = My.Resources.GreenBoard
    Case "PinkBoard"
        Me.BackgroundImage = My.Resources.PinkBoard
    Case "RedBoard"
        Me.BackgroundImage = My.Resources.RedBoard
    Case "BlueBoard"
        Me.BackgroundImage = My.Resources.BlueBoard
End Select
Select Case (ds.Tables(0).Rows(count).Item(3))
    Case "Set1"
        Set1 = True
    Case "Set2"
        Set2 = True
    Case "Set3"
        Set3 = True
End Select
ElseIf Player2Label.Text = ds.Tables(0).Rows(count).Item(0) Then
'For player 2 profile picture (for a user that has signed in)
    Select Case (ds.Tables(0).Rows(count).Item(1))
        Case "Default"
            P2ProfilePicture.Image = My.Resources.DefaultSkin
        Case "Skin1"
            P2ProfilePicture.Image = My.Resources.Skin1
        Case "Skin2"
            P2ProfilePicture.Image = My.Resources.Skin2
        Case "Skin3"
            P2ProfilePicture.Image = My.Resources.Skin3
        Case "Skin4"
            P2ProfilePicture.Image = My.Resources.Skin4
        Case "Skin5"
            P2ProfilePicture.Image = My.Resources.Skin5
        Case "Skin6"
            P2ProfilePicture.Image = My.Resources.Skin6
        Case "Skin7"
            P2ProfilePicture.Image = My.Resources.Skin7
        Case "Locked1"
            P2ProfilePicture.Image = My.Resources.Locked1

```

```

        Case "Locked2"
            P2ProfilePicture.Image = My.Resources.Locked2
        Case "Locked3"
            P2ProfilePicture.Image = My.Resources.Locked3
        Case "Locked4"
            P2ProfilePicture.Image = My.Resources.Locked4
        Case "Locked5"
            P2ProfilePicture.Image = My.Resources.Locked5
        Case "Locked6"
            P2ProfilePicture.Image = My.Resources.Locked6
        Case "Locked7"
            P2ProfilePicture.Image = My.Resources.Locked7
        Case "Locked8"
            P2ProfilePicture.Image = My.Resources.Locked8
    End Select
Else
    P2ProfilePicture.Image = My.Resources.DefaultSkin 'Default
profile picture for a guest (Or AI)
End If
Next
cn.Close() 'Close connection
'Creating picture boxes as an array
pb = New PictureBox(7, 7) {{Me.p00, Me.p01, Me.p02, Me.p03, Me.p04,
Me.p05, Me.p06, Me.p07},
    {Me.p10, Me.p11, Me.p12, Me.p13, Me.p14, Me.p15, Me.p16, Me.p17},
    {Me.p20, Me.p21, Me.p22, Me.p23, Me.p24, Me.p25, Me.p26, Me.p27},
    {Me.p30, Me.p31, Me.p32, Me.p33, Me.p34, Me.p35, Me.p36, Me.p37},
    {Me.p40, Me.p41, Me.p42, Me.p43, Me.p44, Me.p45, Me.p46, Me.p47},
    {Me.p50, Me.p51, Me.p52, Me.p53, Me.p54, Me.p55, Me.p56, Me.p57},
    {Me.p60, Me.p61, Me.p62, Me.p63, Me.p64, Me.p65, Me.p66, Me.p67},
    {Me.p70, Me.p71, Me.p72, Me.p73, Me.p74, Me.p75, Me.p76, Me.p77}}
'Creating all objects
allobjects()
ga.show()
End Sub

Private Sub TimeComboBox_SelectedIndexChanged(sender As Object, e As
EventArgs) Handles TimeComboBox.SelectedIndexChanged
    'If a time has been selected, copy it to both players time textboxes,
    and allow the game to start
    If TimeComboBox.Text = "Unlimited" Then
        MinutesTextBox1.Text = "∞"
        MinutesTextBox2.Text = "∞"
        SecondsTextBox1.Text = "∞"
        SecondsTextBox2.Text = "∞"
        allowgametostart = True
    ElseIf TimeComboBox.Text = "2 Minutes" Then
        MinutesTextBox1.Text = "02"
        MinutesTextBox2.Text = "02"
        SecondsTextBox1.Text = "00"
        SecondsTextBox2.Text = "00"
        allowgametostart = True
    ElseIf TimeComboBox.Text = "5 Minutes" Then
        MinutesTextBox1.Text = "05"
        MinutesTextBox2.Text = "05"
        SecondsTextBox1.Text = "00"
        SecondsTextBox2.Text = "00"
    End If
End Sub

```

```

        allowgametostart = True
    ElseIf TimeComboBox.Text = "10 Minutes" Then
        MinutesTextBox1.Text = "10"
        MinutesTextBox2.Text = "10"
        SecondsTextBox1.Text = "00"
        SecondsTextBox2.Text = "00"
        allowgametostart = True
    ElseIf TimeComboBox.Text = "20 Minutes" Then
        MinutesTextBox1.Text = "20"
        MinutesTextBox2.Text = "20"
        SecondsTextBox1.Text = "00"
        SecondsTextBox2.Text = "00"
        allowgametostart = True
    ElseIf TimeComboBox.Text = "30 Minutes" Then
        MinutesTextBox1.Text = "30"
        MinutesTextBox2.Text = "30"
        SecondsTextBox1.Text = "00"
        SecondsTextBox2.Text = "00"
        allowgametostart = True
    End If
End Sub

Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles
P1Timer.Tick
    If SecondsTextBox1.Text = 0 And MinutesTextBox1.Text = 0 Then
        'If timer reaches 0 disable timers
        P1Timer.Enabled = False
        P2Timer.Enabled = False
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()
        'Display winner
        MsgBox("Ran out of time, " & Player2Label.Text & " is the
winner!")
        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Loses As Integer
        Dim Player2Wins As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String

```

```

        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter
        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Loses = ds.Tables(0).Rows(count).Item(2)
                ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                    Player2Wins = ds.Tables(0).Rows(count).Item(1)
                End If
            Next
            cn.Close() 'Close connection

            'Updating the database with the new P1 user stats
            Dim cmd As New OleDbCommand(sql, cn)
            sql = "update [Table1]"
            sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "' "
            sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "';"
            cmd = New OleDbCommand(sql, cn)
            cn.Open() 'Open connection
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection

            'Updating the database with the new P2 user stats
            sql = "update [Table1]"
            sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "' "
            sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID & "';"
            cmd = New OleDbCommand(sql, cn)
            cn.Open() 'Open connection
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection
            ga.StoreGameReplay()
        ElseIf SecondsTextBox1.Text = 0 Then
            MinutesTextBox1.Text -= 1
            For i = 0 To 9
                If MinutesTextBox1.Text = i Then
                    MinutesTextBox1.Text = "0" & i
                End If
            Next
        End If
        If P1Timer.Enabled = True Then
            If SecondsTextBox1.Text > 0 Then
                SecondsTextBox1.Text -= 1
                For i = 0 To 9
                    If SecondsTextBox1.Text = i Then
                        SecondsTextBox1.Text = "0" & i
                    End If
                Next
            ElseIf SecondsTextBox1.Text = 0 Then
                SecondsTextBox1.Text = 59
            End If
        End If
    End Sub

```

```

        End If
    End If
End Sub

Private Sub P2Timer_Tick(sender As Object, e As EventArgs) Handles
P2Timer.Tick
    If SecondsTextBox2.Text = 0 And MinutesTextBox2.Text = 0 Then
        'If timer reaches 0 disable timers
        P1Timer.Enabled = False
        P2Timer.Enabled = False
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()

        'Display winner
        MsgBox("Ran out of time, " & Player1Label.Text & " is the
winner!")
        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Wins As Integer
        Dim Player2Loses As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Wins = ds.Tables(0).Rows(count).Item(1)
            ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                Player2Loses = ds.Tables(0).Rows(count).Item(2)
            End If
        End For
    End If
End Sub

```

```

Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "' "
sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats
sql = "update [Table1]"
sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "' "
sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

ga.StoreGameReplay()

ElseIf SecondsTextBox2.Text = 0 Then
    MinutesTextBox2.Text -= 1
    For i = 0 To 9
        If MinutesTextBox2.Text = i Then
            MinutesTextBox2.Text = "0" & i
        End If
    Next
Else
End If

If P2Timer.Enabled = True Then
    If SecondsTextBox2.Text > 0 Then
        SecondsTextBox2.Text -= 1
        For i = 0 To 9
            If SecondsTextBox2.Text = i Then
                SecondsTextBox2.Text = "0" & i
            End If
        Next
    ElseIf SecondsTextBox2.Text = 0 Then
        SecondsTextBox2.Text = 59
    End If
End If
End Sub

Private Sub StartGameButton_Click(sender As Object, e As EventArgs)
Handles StartGameButton.Click
    gamestarted = True 'trigger starting of game

    If allowgametostart = False Then 'If no time limit has been selected
then you cannot start the game
        MsgBox("Please select a time limit")
    Else 'If time limit has been selected then run the program

```

```

        TimeLimitLabel.Visible = False
        TimeComboBox.Visible = False 'User cannot change the time during
the game running
        P1Timer.Start() 'Starting the P1 timer as white is always first
        RestartButton.Visible = True 'If the game is started user has an
option to restart the game whenever they want
        StartGameButton.Visible = False 'If the game is started hide the
button, after restart button is pressed it shall show up again

        allobjects() 'loading in all of the pieces into the array to be
used
        ga.recanmove()
        ga.show()
        ga.reback()
        ga.recheckblack()
        ga.recheckwhite()
    End If
End Sub
Private Sub RestartButton_Click(sender As Object, e As EventArgs) Handles
RestartButton.Click
    P1Timer.Stop()
    P2Timer.Stop()
    BoardTracker.Text = ""
    StartGameButton.Visible = True
    P1ResignButton.Visible = False
    P2ResignButton.Visible = False
    P1SettleButton.Visible = False
    P2SettleButton.Visible = False

    TimeComboBox.SelectedIndex = -1
    TimeComboBox.Visible = True
    TimeLimitLabel.Visible = True
    SecondsTextBox1.Text = "00"
    SecondsTextBox2.Text = "00"
    MinutesTextBox1.Text = "00"
    MinutesTextBox2.Text = "00"
    MoveCounterTextBox.Text = ""
    'Game has stopped
    gamestarted = False
    'Get rid of back
    ga.reback()
    'Create new pieces and game
    allobjects()
    'Display pieces
    ga.show()
    'Game is not allowed to start until time has been selected again
    allowgametostart = False
    gamestarted = False

    wqueencount = 0
    wbishopcount = 0
    wknightcount = 0
    wrookcount = 0
    bqueencount = 0
    bbishopcount = 0
    bknightcount = 0
    brookcount = 0

```



```

End Sub

Private Sub MoveCounterTextBox_TextChanged(sender As Object, e As
EventArgs) Handles MoveCounterTextBox.TextChanged
    If MoveCounterTextBox.Text = "1" Then
        P1ResignButton.Visible = True 'If the game is started the player
has the option to resign the win

        If gameistwooplayer = True Then
            P2ResignButton.Visible = True 'If the game is started the
player has the option to resign the win
            P2SettleButton.Visible = True 'Settle for player 2
            P1SettleButton.Visible = True 'If game started allow option
for players to settle
        End If
    End If
End Sub

Private Sub P1ResignButton_Click(sender As Object, e As EventArgs)
Handles P1ResignButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult

    'Stopping timers
    P1Timer.Stop()
    P2Timer.Stop()
    'Displaying messagebox
    result = MessageBox.Show("Are you sure you want to resign", "Player 1
Resigning", button)
    'Getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Game has ended, player 2 has one
        'Stop game
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()

        'Display winner
        MessageBox.Show("Resigned, " & Player2Label.Text & " is the
winner!")
        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Loses As Integer
        Dim Player2Wins As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access

```

```

        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Loses = ds.Tables(0).Rows(count).Item(2)
                ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                    Player2Wins = ds.Tables(0).Rows(count).Item(1)
                End If
            Next

            cn.Close() 'Close connection

            'Updating the database with the new P1 user stats
            Dim cmd As New OleDbCommand(sql, cn)
            sql = "update [Table1]"
            sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "' "
            sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
            cmd = New OleDbCommand(sql, cn)
            cn.Open() 'Open connection
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection

            'Updating the database with the new P2 user stats
            sql = "update [Table1]"
            sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "' "
            sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
            cmd = New OleDbCommand(sql, cn)
            cn.Open() 'Open connection
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection

            ga.StoreGameReplay()
        ElseIf result = System.Windows.Forms.DialogResult.No Then
            If ga.movecounter Mod 2 = 0 Then
                P1Timer.Start()
            Else
                P2Timer.Start()
            End If
        End If
    End If

```

```

End Sub
Private Sub P2ResignButton_Click(sender As Object, e As EventArgs)
Handles P2ResignButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult

    'stop timers
    P1Timer.Stop()
    P2Timer.Stop()
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want to resign", "Player 2
Resigning", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Game has ended, player 1 has one
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()

        'Display winner
        MessageBox.Show("Resigned, " & Player1Label.Text & " is the
winner!")
        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Wins As Integer
        Dim Player2Loses As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats

```

```

        Player1Wins = ds.Tables(0).Rows(count).Item(1)
    ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
        Player2Loses = ds.Tables(0).Rows(count).Item(2)
    End If
Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "' "
sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats
sql = "update [Table1]"
sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "' "
sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

ga.StoreGameReplay()

ElseIf result = System.Windows.Forms.DialogResult.No Then
    If ga.movecounter Mod 2 = 0 Then
        P1Timer.Start()
    Else
        P2Timer.Start()
    End If
End If
End Sub

Private Sub P1SettleButton_Click(sender As Object, e As EventArgs)
Handles P1SettleButton.Click, P2SettleButton.Click
    Settle.Show()
End Sub

```

```

Imports System.Data.OleDb
Imports System.Data
Public Class Game
    Public board(7, 7) As Integer '8 by 8 grid like the size of the board
    Public checkforwhite(7, 7) As Boolean '^
    Public checkforblack(7, 7) As Boolean '^
    Public gameflag As Boolean = True 'gameflag being true allows game to
continue, if not game is ended
    Public player As Integer = 1 'player always start of as 1
    Public blackindanger As Boolean = False
    Public whiteindanger As Boolean = False
    Public movecounter As Integer 'variable to count moves and be displayed
on the game form
    Public stalemate As Boolean

    Public Sub New()
        'creating an array for chess board, positive pieces are white and
negative pieces are black, 0 are also blanks
        board = {{-5, -4, -3, -2, -1, -3, -4, -5},
                {-6, -6, -6, -6, -6, -6, -6, -6},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {6, 6, 6, 6, 6, 6, 6, 6},
                {5, 4, 3, 2, 1, 3, 4, 5}}
        recheckblack()
        recheckwhite()
    End Sub
    Private Sub checkwinner()
        Dim checkmate(5) As Boolean
        If AICalculating = False Then
            show() 'show images
        End If
        If blackindanger = True Then
            gameflag = False
            checkmate(0) = True
            checkmate(1) = True
            checkmate(2) = True
            checkmate(3) = True
            checkmate(4) = True
            checkmate(5) = True
            checkforwhite(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True

            'Possible moves for the black king, if there is a move possible
then checkmate = false
            GameForm1.blackking.possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    If GameForm1.blackking.flags(i, j) = True Then
                        checkmate(0) = False
                    End If
                Next
            Next
        End Sub
    End Sub

```

```

        'Possible moves for the black pawns, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 7
        GameForm1.blackpawn(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackpawn(k).flags(i, j) = True Then
                    checkmate(1) = False
                End If
            Next
        Next
    Next

    'Possible moves for the black rook, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.brookcount
        GameForm1.blackrook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackrook(k).flags(i, j) = True Then
                    checkmate(2) = False
                End If
            Next
        Next
    Next

    'Possible moves for the black bishop, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.bbishopcount
        GameForm1.blackbishop(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackbishop(k).flags(i, j) = True Then
                    checkmate(3) = False
                End If
            Next
        Next
    Next

    'Possible moves for the black knight, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.bknightcount
        GameForm1.blackknight(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackknight(k).flags(i, j) = True Then
                    checkmate(4) = False
                End If
            Next
        Next
    Next

```

```

        Next
    Next
Next

    'Possible moves for the black queen, if there is a move possible
then checkmate = false
    For k As Integer = 0 To GameForm1.bqueencount
        GameForm1.blackqueen(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackqueen(k).flags(i, j) = True Then
                    checkmate(5) = False
                End If
            Next
        Next
    Next
Next

    If checkmate(0) = True And checkmate(1) = True And checkmate(2) =
True And checkmate(3) = True And checkmate(4) = True And checkmate(5) = True
Then
        If AICalculating = False Then
            'Stop game
            gameflag = False
            GameForm1.gamestarted = False
            'Stop timers
            GameForm1.P1Timer.Stop()
            GameForm1.P2Timer.Stop()
            'Hide buttons
            GameForm1.P1ResignButton.Hide()
            GameForm1.P1SettleButton.Hide()
            GameForm1.P2ResignButton.Hide()
            GameForm1.P2SettleButton.Hide()

            'Display winner
            MsgBox("Checkmate, " & GameForm1.Player1Label.Text & " is
the winner!")

            'Getting both users stats and storing them temporarily
            (only win and lose, as settling form has the drawn stats changed)
            Dim Player1Wins As Integer
            Dim Player2Loses As Integer

            'Setting database location
            Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
            'Opening connection
            Dim DBconn As String =
"Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE provider for MS Access
            Dim DBsource As String = "Data Source=" & dbfilename

            Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
            Dim da As OleDbDataAdapter 'adapter
            Dim ds As New DataSet 'set used with adapter
            Dim sql As String

```

```

sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]"

'SQL Statement
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered
by adapter

Dim count As Integer

For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop
for amount of users in table, (-1) as starts from 0
    If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
        Player1Wins = ds.Tables(0).Rows(count).Item(1)
    ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
        Player2Loses = ds.Tables(0).Rows(count).Item(2)
    End If
Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "'"
sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "'"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats
sql = "update [Table1]"
sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "'"
sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID & "'"

";"

cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

StoreGameReplay()
End If
Else
    'Checkmate is not true, therefore continue game
    gameflag = True
End If
End If

If gameflag = True Then
    stalemate = True
    'Possible moves for the black king, if there is a move possible
then stalemate = false
    GameForm1.blackking.possiblemoves()
    For i As Integer = 0 To 7

```



```

        For j As Integer = 0 To 7
            If GameForm1.blackking.flags(i, j) = True Then
                stalemate = False
            End If
        Next
    Next

    'Possible moves for the black pawns, if there is a move possible
    then stalemate = false
    For k As Integer = 0 To 7
        GameForm1.blackpawn(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackpawn(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

    'Possible moves for the black rook, if there is a move possible
    then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.brookcount
        GameForm1.blackrook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackrook(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

    'Possible moves for the black bishop, if there is a move possible
    then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.bbishopcount
        GameForm1.blackbishop(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackbishop(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

    'Possible moves for the black knight, if there is a move possible
    then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.bknightcount
        GameForm1.blackknight(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackknight(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

```

```

        Next

        'Possible moves for the black queen, if there is a move possible
    then stalemate = false
        For k As Integer = 0 To GameForm1.bqueencount
            GameForm1.blackqueen(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    If GameForm1.blackqueen(k).flags(i, j) = True Then
                        stalemate = False
                    End If
                Next
            Next
        Next

        If stalemate = True Then
            Settle.SettleAgreed()
        End If
        'Check if it is only kings left on the board
        KingsOnly()
        If stalemate = True Then
            Settle.SettleAgreed()
        End If
    End If

    If whiteindanger = True Then
        show()
        gameflag = False
        checkmate(0) = True
        checkmate(1) = True
        checkmate(2) = True
        checkmate(3) = True
        checkmate(4) = True
        checkmate(5) = True
        checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        'Possible moves for the white king, if there is a move possible
    then checkmate = false
        GameForm1.whiteking.possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whiteking.flags(i, j) = True Then
                    checkmate(0) = False
                End If
            Next
        Next

        'Possible moves for the white pawn, if there is a move possible
    then checkmate = false
        For k As Integer = 0 To 7
            GameForm1.whitepawn(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True

```

```

        If GameForm1.whitepawn(k).flags(i, j) = True Then
            checkmate(1) = False
        End If
    Next
Next
Next
    'Possible moves for the white rook, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.wrookcount
        GameForm1.whiterook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whiterook(k).flags(i, j) = True Then
                    checkmate(2) = False
                End If
            Next
        Next
    Next
Next
    'Possible moves for the white bishop, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.wbishopcount
        GameForm1.whitebishop(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whitebishop(k).flags(i, j) = True Then
                    checkmate(3) = False
                End If
            Next
        Next
    Next
Next
    'Possible moves for the white knight, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.wknightcount
        GameForm1.whiteknight(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whiteknight(k).flags(i, j) = True Then
                    checkmate(4) = False
                End If
            Next
        Next
    Next
Next
    'Possible moves for the white queen, if there is a move possible
then checkmate = false
    For k As Integer = 0 To GameForm1.wqueencount
        GameForm1.whitequeen(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whitequeen(k).flags(i, j) = True Then

```

```

        End If
    Next
Next

    If checkmate(0) = True And checkmate(1) = True And checkmate(2) =
True And checkmate(3) = True And checkmate(4) = True And checkmate(5) = True
Then
    If AICalculating = False Then
        'Stop game
        gameflag = False
        GameForm1.gamestarted = False
        'Stop timers
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()
        'Hide buttons
        GameForm1.P1ResignButton.Hide()
        GameForm1.P1SettleButton.Hide()
        GameForm1.P2ResignButton.Hide()
        GameForm1.P2SettleButton.Hide()

        'Display winner
        MsgBox("Checkmate, " & GameForm1.Player2Label.Text & " is
the winner!")

        'Getting both users stats and storing them temporarily
        (only win and lose, as settling form has the drawn stats changed)
        Dim Player1Loses As Integer
        Dim Player2Wins As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String =
"Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]"
        'SQL Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered
        by adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop
for amount of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Loses = ds.Tables(0).Rows(count).Item(2)

```

```

ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
    Player2Wins = ds.Tables(0).Rows(count).Item(1)
End If
Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "' "
sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats
sql = "update [Table1]"
sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "' "
sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"

cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

StoreGameReplay()
End If
Else
    gameflag = True
End If
End If

If gameflag = True Then
    stalemate = True
    'Possible moves for the white king, if there is a move possible
then stalemate = false
    GameForm1.whiteking.possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If GameForm1.whiteking.flags(i, j) = True Then
                stalemate = False
            End If
        Next
    Next
    'Possible moves for the white pawn, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 7
        GameForm1.whitepawn(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whitepawn(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

```

```

        Next
    Next
Next
    'Possible moves for the white rook, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.wrookcount
        GameForm1.whiterook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whiterook(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
    'Possible moves for the white bishop, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.wbishopcount
        GameForm1.whitebishop(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whitebishop(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
    'Possible moves for the white knight, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.wknightcount
        GameForm1.whiteknight(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whiteknight(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
    'Possible moves for the white queen, if there is a move possible
then stalemate = false
    For k As Integer = 0 To GameForm1.wqueencount
        GameForm1.whitequeen(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whitequeen(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

    If stalemate = True And AICalculating = False Then
        Settle.SettleAgreed()
    End If
    'Check if it is only kings left on the board
    KingsOnly()

```

```

        If stalemate = True And AICalculating = False Then
            Settle.SettleAgreed()
        End If
    End If
End Sub

'showing images of pieces
Public Sub show()
    If GameForm1.Set1 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)
                    Case 1
                        GameForm1.pb(i, j).Image = My.Resources.Set1KingW
                    Case 2
                        GameForm1.pb(i, j).Image =
My.Resources.Set1QueenW
                    Case 3
                        GameForm1.pb(i, j).Image =
My.Resources.Set1BishopW
                    Case 4
                        GameForm1.pb(i, j).Image =
My.Resources.Set1KnightW
                    Case 5
                        GameForm1.pb(i, j).Image = My.Resources.Set1RookW
                    Case 6
                        GameForm1.pb(i, j).Image = My.Resources.Set1PawnW
                    Case -1
                        GameForm1.pb(i, j).Image = My.Resources.Set1KingB
                    Case -2
                        GameForm1.pb(i, j).Image =
My.Resources.Set1QueenB
                    Case -3
                        GameForm1.pb(i, j).Image =
My.Resources.Set1BishopB
                    Case -4
                        GameForm1.pb(i, j).Image =
My.Resources.Set1KnightB
                    Case -5
                        GameForm1.pb(i, j).Image = My.Resources.Set1RookB
                    Case -6
                        GameForm1.pb(i, j).Image = My.Resources.Set1PawnB
                    Case Else
                        GameForm1.pb(i, j).Image = Nothing
                End Select
            Next j
            showcheck()
        Next i
    ElseIf GameForm1.Set2 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)
                    Case 1
                        GameForm1.pb(i, j).Image = My.Resources.Set2KingW
                    Case 2
                        GameForm1.pb(i, j).Image =
My.Resources.Set2QueenW

```

```

        Case 3
            GameForm1.pb(i, j).Image =
My.Resources.Set2BishopW
        Case 4
            GameForm1.pb(i, j).Image =
My.Resources.Set2KnightW
        Case 5
            GameForm1.pb(i, j).Image = My.Resources.Set2RookW
        Case 6
            GameForm1.pb(i, j).Image = My.Resources.Set2PawnW
        Case -1
            GameForm1.pb(i, j).Image = My.Resources.Set2KingB
        Case -2
            GameForm1.pb(i, j).Image =
My.Resources.Set2QueenB
        Case -3
            GameForm1.pb(i, j).Image =
My.Resources.Set2BishopB
        Case -4
            GameForm1.pb(i, j).Image =
My.Resources.Set2KnightB
        Case -5
            GameForm1.pb(i, j).Image = My.Resources.Set2RookB
        Case -6
            GameForm1.pb(i, j).Image = My.Resources.Set2PawnB
        Case Else
            GameForm1.pb(i, j).Image = Nothing
    End Select
    showcheck()
Next
Next
ElseIf GameForm1.Set3 = True Then
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            Select Case board(i, j)
                Case 1
                    GameForm1.pb(i, j).Image = My.Resources.Set3KingW
                Case 2
                    GameForm1.pb(i, j).Image =
My.Resources.Set3QueenW
                Case 3
                    GameForm1.pb(i, j).Image =
My.Resources.Set3BishopW
                Case 4
                    GameForm1.pb(i, j).Image =
My.Resources.Set3KnightW
                Case 5
                    GameForm1.pb(i, j).Image = My.Resources.Set3RookW
                Case 6
                    GameForm1.pb(i, j).Image = My.Resources.Set3PawnW
                Case -1
                    GameForm1.pb(i, j).Image = My.Resources.Set3KingB
                Case -2
                    GameForm1.pb(i, j).Image =
My.Resources.Set3QueenB
                Case -3

```



```

        GameForm1.pb(i, j).Image =
My.Resources.Set3BishopB
        Case -4
            GameForm1.pb(i, j).Image =
My.Resources.Set3KnightB
        Case -5
            GameForm1.pb(i, j).Image = My.Resources.Set3RookB
        Case -6
            GameForm1.pb(i, j).Image = My.Resources.Set3PawnB
        Case Else
            GameForm1.pb(i, j).Image = Nothing
        End Select
        showcheck()
    Next
Next
End If
End Sub

Public Sub StoreGameReplay()
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show(LoginForm.Player1Username & ", would you
like to store this replay?", "Storing Replay", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.No Then
        Exit Sub
    End If

    Dim mystring As String
    Dim premove As String
    Dim premoveintotable As String = ""
    Dim move As String
    Dim moveintotable As String = ""

    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim sql As String
    Dim lines As Integer

    For lines = 0 To GameForm1.BoardTracker.Lines.Length - 1
        mystring = (Right(GameForm1.BoardTracker.Lines(lines).ToString,
7))
        premove = Left(mystring, 2)
        premoveintotable = premoveintotable & premove

        move = Right(mystring, 2)
        moveintotable = moveintotable & move
    
```

```

Next

Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
sql = "SELECT [Username],[PreMove1], [Move1], [PreMove2], [Move2],
[PreMove3], [Move3], [PreMove4], [Move4], [PreMove5], [Move5] FROM [Table1]"
'SQL Statement
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
Dim count As Integer

Dim checkforemptyset As String
Dim emptysetnumber As Integer
For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
    If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats

        For i As Integer = 2 To 10 Step 2
            Try
                checkforemptyset = ds.Tables(0).Rows(count).Item(i)
            Catch ex As Exception
                emptysetnumber = Int(i / 2)
            Exit For
            End Try
        Next
    End If
Next

cn.Close() 'Close connection

If emptysetnumber = 0 Then
    Dim Message, Title, MyValue
    Do
        'Set prompt.
        Message = "Enter a value between 1 and 5 (Replace Replay)"
        Title = "Replay Replace" ' Set title.
        ' Display message, title, and default value.
        MyValue = InputBox(Message, Title, "1")
        Try
            MyValue = CInt(MyValue)
        Catch ex As Exception
            MyValue = "10"
        End Try
        Loop Until MyValue >= 1 And MyValue <= 5
        emptysetnumber = MyValue
    End If

    sql = "update [Table1]"
    sql = sql & "SET [PreMove" & emptysetnumber & "] = '" &
premoveintotable & "' "
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
    Dim cmd As New OleDbCommand(sql, cn)
    cmd = New OleDbCommand(sql, cn)
    cn.Open() 'Open connection
    cmd.ExecuteNonQuery()

```

```

        cn.Close() 'Close connection

        sql = "update [Table1]"
        sql = sql & "SET [Move" & emptysetnumber & "]" = '" & moveintotable &
"'"

        sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

    End Sub

Imports System.Threading
Imports System.Data.OleDb
Imports System.Data
Public Class Settle

    Public Sub SettleAgreed()
        GameForm1.ga.gameflag = False
        If GameForm1.ga.stalemate = True Then
            MessageBox.Show("Stalemate - Draw")
        Else
            MessageBox.Show("Settle Agreed - Draw")
        End If
        'Hiding Buttons
        GameForm1.P1SettleButton.Visible = False
        GameForm1.P2SettleButton.Visible = False
        GameForm1.P1ResignButton.Visible = False
        GameForm1.P2ResignButton.Visible = False

        'Getting both users stats for draws and storing them temporarily
        Dim Player1Draws As Integer
        Dim Player2Draws As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Draws] FROM [Table1]" 'SQL Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        Dim count As Integer

```

```

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
            If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats
                Player1Draws = ds.Tables(0).Rows(count).Item(1)
            ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                Player2Draws = ds.Tables(0).Rows(count).Item(1)
            End If
        Next

        cn.Close() 'Close connection

        'Updating the database with the new P1 user stats
        Dim cmd As New OleDbCommand(sql, cn)
        sql = "update [Table1]"
        sql = sql & "SET [Draws] = '" & Player1Draws + 1 & "'"
        sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "'"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Draws] = '" & Player2Draws + 1 & "'"
        sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID & "'"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection
        'Disable game
        GameForm1.ga.gameflag = False
        GameForm1.gamestarted = False
        'Hide buttons
        GameForm1.P1ResignButton.Hide()
        GameForm1.P1SettleButton.Hide()
        GameForm1.P2ResignButton.Hide()
        GameForm1.P2SettleButton.Hide()

        GameForm1.ga.StoreGameReplay()
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()
    End Sub
End Class

```

Test Plan

Due to the objectives in this section mostly being without any user input, the number of test plans are very limited.

Loading account

Description: testing the game form loads correctly with all the user's information

Username	Profile Picture	Board Colour	Piece Type	Data Type	Expected Outcome
Admin	Skin3	GreenBoard	Set2	Normal	Game successfully loaded
Admin	Picture	YellowBoard	Set1	Invalid	Profile picture not found
Admin	Skin2	BrownBoard	Set1	Invalid	Board colour not found
Admin	Locked6	YellowBoard	Set9	Invalid	Piece set not found
Admin	Default	YellowBoard	Set1	Borderline	Game successfully loaded

Note - the invalid data types should never actually occur whilst the program is running, as they are never mentioned being inserted into the database.

Loading pieces onto the board

Description: testing the game form loads the chess board with the correct pieces

Key:

6 ← white pawn, 5 ← white rook, 4 ← white knight, 3 ← white bishop, 2 ← white queen, 1 ← white king
 -6 ← black pawn, -5 ← black rook, -4 ← black knight, -3 ← black bishop, -2 ← black queen, -1 ← black king

	A	B	C	D	E	F	G	H	Data Type	Expected Outcome
8	-5	-4	-3	-2	-1	-3	-4	-5	Normal	Pieces loaded successfully
7	-6	-6	-6	-6	-6	-6	-6	-6		
6										
5										
4										
3										
2	6	6	6	6	6	6	6	6		
1	5	4	3	2	1	3	4	5		

Saving replays

Description: testing the user's input when saving a replay

Replay number	Data Type	Expected Outcome
2	Normal	Replay Saved
9	Invalid	Invalid replay number
1	Borderline	Replay Saved

Note - you can only save up to 5 replays at a time

4 - Game logic

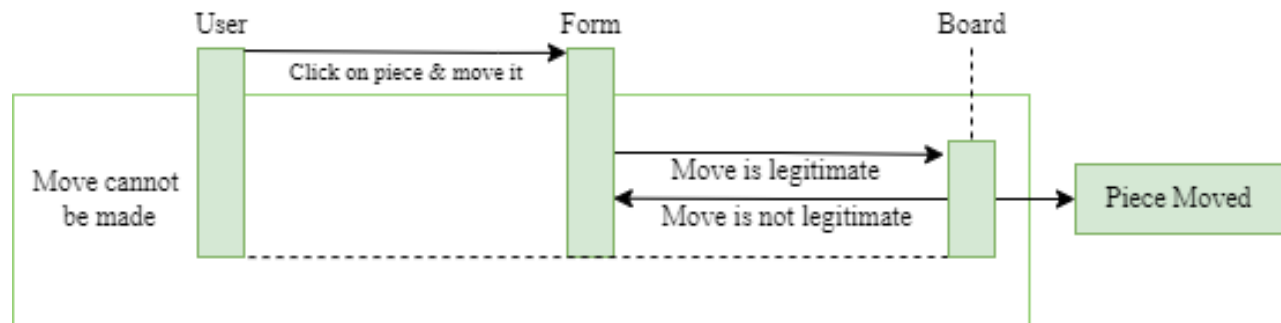
Summary: whilst the game is being played correctly implement the logic of chess in the game, this includes the movement of the pieces (including special moves, pawn promotion and castling), and checking the king.

Research

Document (<https://www.ichess.net/blog/chess-pieces-moves/>)

The research in this section was about the correct movement of the pieces, therefore the analysis was previously stated in the [Walkthrough:](#) section

UML



Pseudo code

N/A - when calculating the logic of the game, it depends on the user interface for the coordinates of the chess pieces, therefore I chose to not include the pseudo code for this section, just the VB.NET.

VB.Net Code

Note - as the code for this section is very long, the links for full code for the sections are:

Full Code

[Bishop](#)

[Game](#)

[King](#)

[Knight](#)

[Pawn](#)

[Queen](#)

[Rook](#)

Bishop Snippet

```
Public Class Bishop
    Public xposition, yposition As Integer 'Coordinates for bishop
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the bishop
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
bishop, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)
    Public Sub New(ByVal x, ByVal y, ByVal pvalue) 'giving the new piece its
coordinates, and piece value
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags() 'setting every possible move to false
    End Sub
    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function possiblemoves() As Boolean(,) 'returning all possible
moves
        reflags() 'setting every possible move to false
        'only calculate moves of the piece if it is still on the board
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags
    End Function
    Public Sub whitemove() 'all moves for white
```



```

    'Storing position of bishop
    Dim x As Integer = xposition
    Dim y As Integer = yposition
    allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

    'diagonal left top to bottom
    While (x < 7 And y > 0)
        x = x + 1
        y = y - 1

        'Code to make sure black king cannot move into a check position
        If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
            flags(x, y) = True
            allcheckmoves = False
        End If

        If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
            GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
bishop was not there, would the king be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(x, y) = True 'allow movement
            ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = 3
            If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
white piece, exit loop (cannot travel through a piece)
                Exit While
            End If
            ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
                If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                    GameForm1.ga.recurrisionstop = False

```

```

        GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(x, y) = True
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = 0
        GameForm1.ga.board(xposition, yposition) = 3
    End If
    If GameForm1.ga.board(x, y) < 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
            GameForm1.ga.board(x, y) = 10 'change desired space
to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the bishop was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = 3
        End If
        Exit While
    End If
Else
        Exit While
    End If
End While

```

Game Snippet

```

Public Sub mov(ByVal selected, ByVal x, ByVal y, ByVal prex, ByVal prey)
    Select Case selected

```

```

        Case "whiteking"
            movecounter += 1
            GameForm1.MoveCounterTextBox.Text = movecounter
            If x = 7 And y = 2 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(0).AllowedToCastle = True And castleincheck(0) =
False Then

                Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
                Dim result As DialogResult
                'displaying MessageBox
                result = MessageBox.Show("Are you sure you want castle",
"Castle", button)

                'getting result of the messagebox
                If result = System.Windows.Forms.DialogResult.Yes Then
                    GameForm1.whiteking.changeposition(x, y, prex, prey)
                    GameForm1.whiterook(0).changeposition(7, 3, 7, 0)
                    GameForm1.whiteking.AllowedToCastle = False
                    GameForm1.whiterook(0).AllowedToCastle = False
                    GameForm1.whiterook(1).AllowedToCastle = False
                    GameForm1.P1Timer.Stop()
                    If GameForm1.MinutesTextBox1.Text = "00" Then
                        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                            GameForm1.SecondsTextBox1.Text += 2
                            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
                                End If
                            End If
                            GameForm1.P2Timer.Start()
                            player = 2
                            recheckwhite() 'White cannot be in check if king has
been moved
                                recheckblack() 'Black cannot be in check by the king,
therefore set all squares to not in check
                                    If GameForm1.gameisoneplayer = True And gameflag =
True Then

                                        If GameForm1.EasyMode = True Then
                                            AIMoving()
                                        ElseIf GameForm1.HardMode = True Then
                                            AIMovingHard()
                                        End If
                                    End If
                                End If
                            Exit Sub
                        Else
                            movecounter -= 1
                            Exit Sub
                        End If
                    End If
                If x = 7 And y = 6 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(1).AllowedToCastle = True And castleincheck(1) =
False Then

                    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
                    Dim result As DialogResult
                    'displaying MessageBox
                    result = MessageBox.Show("Are you sure you want castle",
"Castle", button)

                    'getting result of the messagebox
                    If result = System.Windows.Forms.DialogResult.Yes Then
                        GameForm1.whiteking.changeposition(x, y, prex, prey)

```

```

GameForm1.whiterook(1).changeposition(7, 5, 7, 7)
GameForm1.whiteking.AllowedToCastle = False
GameForm1.whiterook(1).AllowedToCastle = False
GameForm1.whiterook(0).AllowedToCastle = False
GameForm1.P1Timer.Stop()
If GameForm1.MinutesTextBox1.Text = "00" Then
    If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
        GameForm1.SecondsTextBox1.Text += 2
        GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
    End If
End If
GameForm1.P2Timer.Start()
player = 2

recheckwhite() 'White cannot be in check if king has
been moved
recheckblack() 'Black cannot be in check by the king,
therefore set all squares to not in check
If GameForm1.gameisoneplayer = True And gameflag =
True Then

    If GameForm1.EasyMode = True Then
        AIMoving()
    ElseIf GameForm1.HardMode = True Then
        AIMovingHard()
    End If
End If
Exit Sub
Else
    movecounter -= 1
    Exit Sub
End If
End If
GameForm1.whiteking.changeposition(x, y, prex, prey)
GameForm1.whiteking.AllowedToCastle = False
'After a white king has moved, if it has taken a black piece
set its x and y values to 0

For L As Integer = 0 To GameForm1.bqueencount
    If GameForm1.whiteking.xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackqueen(L).yposition Then
        GameForm1.blackqueen(L).stillonboard = False
    End If
Next

For L As Integer = 0 To 1 + GameForm1.bknightcount
    If GameForm1.whiteking.xposition =
GameForm1.blackknight(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackknight(L).yposition Then
        GameForm1.blackknight(L).stillonboard = False
    End If
Next

For L As Integer = 0 To 1 + GameForm1.bbishopcount

```

```

        If GameForm1.whiteking.xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackbishop(L).yposition Then
            GameForm1.blackbishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.brookcount
        If GameForm1.whiteking.xposition =
GameForm1.blackrook(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackrook(L).yposition Then
            GameForm1.blackrook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.whiteking.xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackpawn(L).yposition Then
            GameForm1.blackpawn(L).stillonboard = False
        End If
    Next

    GameForm1.P1Timer.Stop()
    If GameForm1.MinutesTextBox1.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
            GameForm1.SecondsTextBox1.Text += 2
            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
        End If
    End If
    GameForm1.P2Timer.Start()

    recheckwhite() 'White cannot be in check if the pawn has
moved

    GameForm1.ga.recurrisionstop = False

    player = 2
    fillcheckforblack()
    showcheck() 'Show the check if there is one

    checkwinner()

    If GameForm1.gameisoneplayer = True And gameflag = True Then

        If GameForm1.EasyMode = True Then
            AIMoving()
        ElseIf GameForm1.HardMode = True Then
            AIMovingHard()
        End If
    End If

```

King Snippet

Public Class King

Daniel Lawrence

Candidate Number: 7300

Centre Number: 61715

```

Public xposition, yposition As Integer 'Coordinates for king
Public piecevalue As Integer 'negative means black, positive means white
Public flags(7, 7) As Boolean '2D array - for the movement of the king
Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
Public AllowedToCastle As Boolean = True

'positive colour means white and negative black
Public Sub New(ByVal x, ByVal y, ByVal pval)
    xposition = x
    yposition = y
    piecevalue = pval
    reflags()
End Sub

Public Function returnx() As Integer
    Return xposition
End Function
Public Function returny() As Integer
    Return yposition
End Function
Public Function returnvalue()
    Return piecevalue
End Function
Public Function possiblemoves() As Boolean(,) 'showing possible moves
    reflags()
    If piecevalue > 0 Then
        whitemove()
    End If
    If piecevalue < 0 Then
        blackmove()
    End If
    Return flags
End Function
Public Sub whitemove()

    GameForm1.ga.CastlingW()

    If xposition + 1 <= 7 Then
        'one space vertically downwards
        If GameForm1.ga.board(xposition + 1, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite()
            If GameForm1.ga.checkforwhite(xposition + 1, yposition) =
False Then
                flags(xposition + 1, yposition) = True 'allow movement
            End If
        End If
    End If

```

```

        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 1

        ElseIf GameForm1.ga.board(xposition + 1, yposition) <= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition + 1, yposition) =
False Then
                flags(xposition + 1, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
            (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

```

Knight Snippet

```

Public Class Knight
    Public xposition, yposition As Integer 'Coordinates for knight
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the knight
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
knight, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)
    'Positive colour is white and negative is black
    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer

```

```

    Return yposition
End Function
Public Function returnvalue() As Integer
    Return piecevalue
End Function
Public Function possiblemoves() As Boolean(,) 'showing possible moves
    reflags()
    If piecevalue > 0 And stillonboard = True Then
        whitemove()
    End If
    If piecevalue < 0 And stillonboard = True Then
        blackmove()
    End If
    Return flags
End Function

Public Sub whitemove()
    '
    'Down 2 and left 1
    '
    If xposition + 2 <= 7 And yposition - 1 >= 0 Then
        '
        'Code to make sure black king cannot move into a check position
        allcheckmoves = True
        If GameForm1.ga.board(xposition + 2, yposition - 1) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
            flags(xposition + 2, yposition - 1) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(xposition + 2, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 2, yposition - 1)
            'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
            GameForm1.ga.board(xposition + 2, yposition - 1) = 4 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition + 2, yposition - 1) = True 'allow
movement
            ElseIf xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
                flags(xposition + 2, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)

```



```

        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 2, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition + 2, yposition - 1) <= 0 Then
'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition + 2, yposition - 1) = 0 Then
'If the space is empty
            GameForm1.ga.board(xposition + 2, yposition - 1) = 4
'Change space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition + 2, yposition - 1) = True
            End If
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition + 2, yposition - 1) = 0
'Return the random number back to a blank space
            End If
            If GameForm1.ga.board(xposition + 2, yposition - 1) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
                If xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition + 2, yposition -
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(xposition + 2, yposition - 1) = 4
'change desired space to a random number
                    GameForm1.ga.recurrsionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(xposition + 2, yposition - 1) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    GameForm1.ga.board(xposition + 2, yposition - 1) =
temp
                    GameForm1.ga.board(xposition, yposition) = 4
                End If
            End If
        End If
    End If

```

End If

Pawn Snippet

```
Public Class Pawn
    Public xposition, yposition As Integer 'Coordinates for pawn
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the pawn
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
pawn, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)

    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue 'Positive P value is White, Negative P value is
Black
        reflags()
        recheckking()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'Calculation of the
possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags 'Returning all possible positions the pawn can move to
    End Function
    Public Sub whitemove()

        If xposition = 6 Then 'If pawn is at starting position
            '
            'one square vertically down
            '
            '
            If xposition - 1 >= 0 Then
                If GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
```

```

        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
        GameForm1.ga.board(xposition - 1, yposition) = 6 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition) = True 'allow
movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition) = 0
        GameForm1.ga.board(xposition, yposition) = 6
        ElseIf GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition - 1, yposition) = 6 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn moved to the desired space, would the king still be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 1, yposition) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = 6
        End If
    End If

```

Queen Snippet

```

Public Class Queen
    Public xposition, yposition As Integer
    Public piecevalue As Integer
    Public flags(7, 7) As Boolean
    Public checktheking(7, 7) As Boolean
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean

    'positive colour is white and negative is black

```

```

Public Sub New(ByVal x, ByVal y, ByVal pvalue)
    xposition = x
    yposition = y
    piecevalue = pvalue
    reflags()
End Sub

Public Function returnx() As Integer
    Return xposition
End Function
Public Function returny() As Integer
    Return yposition
End Function
Public Function returnvalue() As Integer
    Return piecevalue
End Function
Public Function possiblemoves() As Boolean(,) 'showing possible moves
    reflags()
    If piecevalue > 0 And stillonboard = True Then
        whitemove()
    End If
    If piecevalue < 0 And stillonboard = True Then
        blackmove()
    End If
    Return flags
End Function
Public Sub whitemove()
    Dim x As Integer = xposition
    Dim y As Integer = yposition
    allcheckmoves = True
    'diagonal left top to bottom
    While (x < 7 And y > 0)
        x = x + 1
        y = y - 1
        '
        'Code to make sure black king cannot move into a check position
        If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
            flags(x, y) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
            GameForm1.ga.board(x, y) = -2 'change desired space to a
random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check

```

```

        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(x, y) = True
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(x, y) = 0
            GameForm1.ga.board(xposition, yposition) = 2
        End If
        If GameForm1.ga.board(x, y) < 0 Then 'If the queen can take a
black piece it can move there to get rid of check
            If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(x, y) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check

```

```

        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
        flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit While
End If
Else
    Exit While
End If
End While

```

Rook Snippet

```

Public Class Rook
    Public xposition, yposition As Integer 'Coordinates for rook
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the rook
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public AllowedToCastle As Boolean = True
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
rook, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)

    'positive colour is white and negative is black
    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'showing possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
    End Function

```

```

    If piecevalue < 0 And stillonboard = True Then
        blackmove()
    End If
    Return flags
End Function
Public Sub whitemove()
    If stillonboard = True Then
        GameForm1.ga.CastlingW()
    End If

    If xposition > 0 Then
        allcheckmoves = True
        'vertically downwards
        For i As Integer = xposition - 1 To 0 Step -1
            'Code to make sure black king cannot move into a check
            position
            If GameForm1.ga.board(i, yposition) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
                flags(i, yposition) = True
                allcheckmoves = False
            End If
            '
            '
            If GameForm1.ga.board(i, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
                Dim temp As Integer 'declaring variable to store value
from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
                GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see that if
the rook was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(i, yposition) = True 'allow movement
                ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                    flags(i, yposition) = True 'allow movement
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
                GameForm1.ga.board(i, yposition) = temp
                GameForm1.ga.board(xposition, yposition) = 5
                If GameForm1.ga.board(i, yposition) < 0 Then
                    Exit For
                End If
                ElseIf GameForm1.ga.board(i, yposition) <= 0 Then 'If king is
in check, check for available spaces

```

```

        If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
            GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(i, yposition) = True
            End If

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
            End If
            If GameForm1.ga.board(i, yposition) < 0 Then 'If the rook
can take a black piece it can move there to get rid of check
                If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
                    GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the rook was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(i, yposition) = True 'allow movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    GameForm1.ga.board(i, yposition) = temp
                    GameForm1.ga.board(xposition, yposition) = 5
                End If
                Exit For
            End If
        Else
            Exit For
        End If
    Next
End If

```


Test Plan

Description: testing the correct movement of the pieces. White pieces are used in the test plan as the black pieces are the exact same, however board flipped around

Pawn(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
B2	B3/B4	No	Normal/ Borderline	Can Move (Only if B3/B4 doesn't result in a white check & doesn't travel through any piece)
B4	B5	No	Normal	Can Move (Only if B5 doesn't result in a white check & doesn't travel through any piece)
C4	B5/D5	No	Normal	Can Move (Only if B5/D5 doesn't result in a white check & doesn't travel through any piece)
B2	B3/B4	Yes	Normal/ Borderline	Can Move (Only if B3/B4 doesn't result in a white check & doesn't travel through any piece)
B4	B5	No	Normal	Can Move (Only if B5 doesn't result in a white check & doesn't travel through any piece)
C4	B5/D5	Yes	Normal	Can Move (Only if B5/D5 doesn't result in a white check & doesn't travel through any piece)

Rook(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D3	D1/D2//D4/D5/D6/D7/D8	No	Normal/Borderline	Can Move (Only if D1/D2/D4/D5/D6/D7/D8 doesn't result in a white check & doesn't travel through any piece)
D4	A4/B4/C4/E4/F4/G4/H4	No	Normal/Borderline	Can Move (Only if A4/B4/C4/E4/F4/G4/H4 doesn't result in a white check & doesn't travel through any piece)
D3	D1/D2//D4/D5/D6/D7/D8	Yes	Normal/Borderline	Can Move (Only if D1/D2/D4/D5/D6/D7/D8 doesn't result in a white check & doesn't travel through any piece)
D4	A4/B4/C4/E4/F4/G4/H4	Yes	Normal/Borderline	Can Move (Only if A4/B4/C4/E4/F4/G4/H4 doesn't result in a white check & doesn't travel through any piece)

Knight(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D5	C7/E7/C3/E3/B6/F6/B4/F4	No	Normal/ Borderline	Can Move (Only if C7/E7/C3/E3/B6/F6/B4/F4 doesn't result in a white check)
D5	C7/E7/C3/E3/B6/F6/B4/F4	Yes	Normal/ Borderline	Can Move (Only if C7/E7/C3/E3/B6/F6/B4/F4 doesn't result in a white check)

Bishop(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D2	C1/E3/F4/G5/H6	No	Normal/ Borderline	Can Move (Only if C2/E3/F4/G5/H6 doesn't result in a white check)
D2	A5/B4/C3/E1	No	Normal/ Borderline	Can Move (Only if A5/B4/C3/E1 doesn't result in a white check)
C1	D2/E3/F4/G5/H6	Yes	Normal/ Borderline	Can Move (Only if D2/E3/F4/G5/H6 doesn't result in a white check)
D2	A5/B4/C3/E1	Yes	Normal/ Borderline	Can Move (Only if A5/B4/C3/E1 doesn't result in a white check)

Queen(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	D1/E2/F3/H5	No	Normal/ Borderline	Can Move (Only if D1/E2/F3/H5 doesn't result in a white check)
G4	C8/D7/E6/F5/H3	No	Normal/ Borderline	Can Move (Only if C8/D7/E6/F5/H3 doesn't result in a white check)
G4	G1/G2/G3/G5/G6/G7/G8	No	Normal/ Borderline	Can Move (Only if G1/G2/G3/G5/G6/G7/G8 doesn't result in a white check)
G4	A4/B4/C4/D4/E4/F4/H4	No	Normal/ Borderline	Can Move (Only if A4/B4/C4/D4/E4/F4/H4 doesn't result in a white check)
G4	D1/E2/F3/H5	Yes	Normal/ Borderline	Can Move (Only if D1/E2/F3/H5 doesn't result in a white check)
G4	C8/D7/E6/F5/H3	Yes	Normal/ Borderline	Can Move (Only if C8/D7/E6/F5/H3 doesn't result in a white check)

G4	G1/G2/G3/G5/G6/G7/G8	Yes	Normal/ Borderline	Can Move (Only if G1/G2/G3/G5/G6/G7/G8 doesn't result in a white check)
G4	A4/B4/C4/D4/E4/F4/H4	Yes	Normal/ Borderline	Can Move (Only if A4/B4/C4/D4/E4/F4/H4 doesn't result in a white check)

King(White)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
E3	D2/E2/F2/F3/F4/E4/D4/D3	No	Normal/ Borderline	Can Move (Only if D2/E2/F2/F3/F4/E4/D4/D3 doesn't result in a white check)
E3	D2/E2/F2/F3/F4/E4/D4/D3	Yes	Normal/ Borderline	Can Move (Only if D2/E2/F2/F3/F4/E4/D4/D3 doesn't result in a white check)

5 - AI opponent

Summary: Easy mode is when the AI randomly moves on the board with no strategy. Hard mode is when the AI uses a minimax algorithm, so the program has some strategy.

Research

Document (<https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977>)

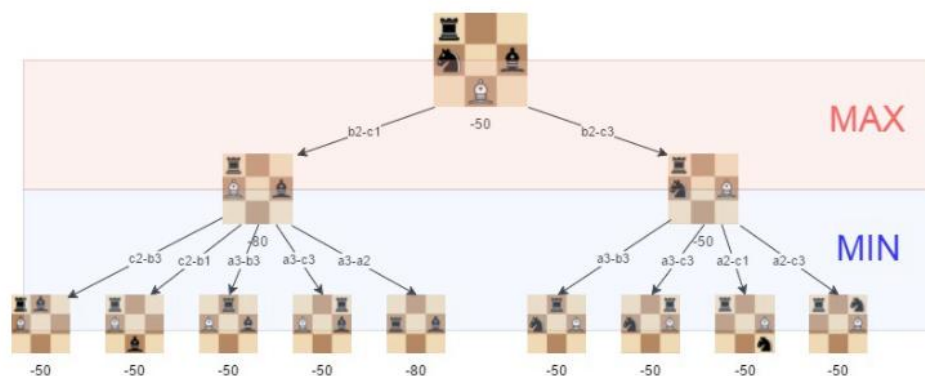
This website gives an explanation on how to create a chess AI using the minimax algorithm. It essentially gives a step-by-step guide on how to do so, making it relatively easy to comprehend (*more detailed analysis on this document within the formal analysis section*).

Step 3: Search tree using Minimax

Next we're going to create a search tree from which the algorithm can choose the best move. This is done by using the Minimax algorithm.

In this algorithm, the recursive tree of all possible moves is explored to a given depth, and the position is evaluated at the ending "leaves" of the tree.

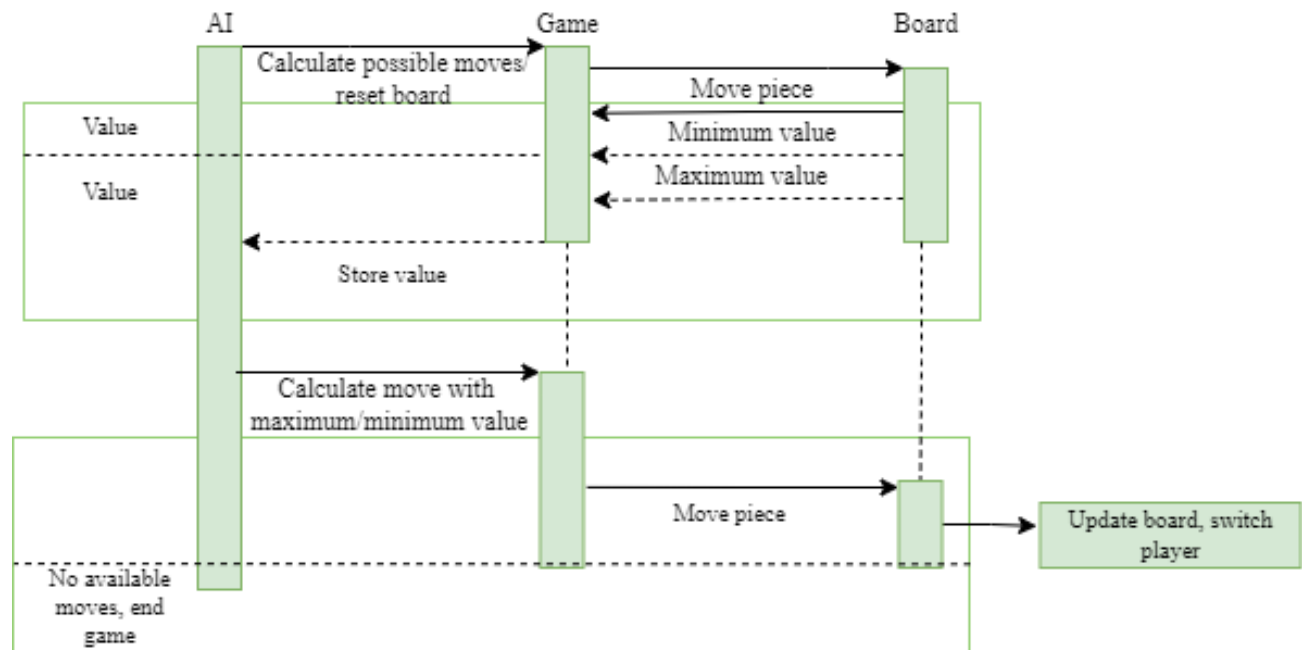
After that, we return either the smallest or the largest value of the child to the parent node, depending on whether it's a white or black to move. (That is, we try to either minimize or maximize the outcome at each level.)



A visualization of the minimax algorithm in an artificial position. The best move for white is b2-c3, because we can guarantee that we can get to a position where the evaluation is -50

The diagram above summarizes the concept of the minimax algorithm. The algorithm calculates every possible move for a given depth (the depth is how many moves ahead it looks) and returns a value for the minimum/maximum number (depending on colour).

UML



Pseudo Code

The code for the minimax algorithm is difficult to write in pseudo code, as the VB.Net code refers to many subroutines created in the program. Therefore, a brief part of the algorithm has been written.

```

maximumpoints ← 0
Subroutine calculateboardscore
  points ← 0
  For i as integer = 0 To 7
    For j as integer = 0 to 7
      Select Case Board
        Case 6
          points = points + 10
        Case 5
          points = points + 50
        Case 4
          points = points + 30
        Case 3
          points = points + 30
        Case 2
          points = points + 90
        Case 1
          points = points + 900
        Case -6
          points = points - 10
        Case -5
          points = points - 50
        Case -4
          points = points - 30
        Case -3
          points = points - 30
        Case -2
          points = points - 90
        Case -1

```

```

        points = points - 900
    Case End
Next
Next

If points > maximumpoints Then
    maximumpoints = points
Endif
End subroutine

Subroutine minimax
    For i as integer = 0 To 7
        blackpawn(i).calcpossiblemoves()
        calculateboardscore()
    Next

    For i as integer = 0 to rookcount
        blackrook(i).calcpossiblemoves
        calculateboardscore()
    Next

    For i as integer = 0 to knightcount
        blackknight(i).calcpossiblemoves
        calculateboardscore()
    Next

    For i as integer = 0 to bishopcount
        blackbishop(i).calcpossiblemoves
        calculateboardscore()
    Next

    For i as integer = 0 to queencount
        blackqueen(i).calcpossiblemoves
        calculateboardscore()
    Next

    blackking.calcpossiblemoves
    calculateboardscore()

    If piece.calculateboardscore = maximumpoints Then
        move(piece)
    Endif
End subroutine

```

VB.Net Code(Easy AI)

```

Private Sub AIMoving()
    Dim AIx(1) As Integer
    Dim AIy(1) As Integer
    Dim rnd As New Random
    'Easy mode, getting the computer to select a piece and random and
    randomly move it
    Do
        Do
            recanmove()
            AIx(0) = rnd.Next(0, 8)
            AIy(0) = rnd.Next(0, 8)
            selection(AIx(0), AIy(0))
        Loop Until board(AIx(0), AIy(0)) < 0

        Select Case board(AIx(0), AIy(0))
            Case -1
                GameForm1.selectedpiece = ("blackking")
                For i As Integer = 0 To 7
                    For j As Integer = 0 To 7
                        If GameForm1.blackking.flags(i, j) = True Then
                            Exit Do
                        End If
                    Next
                Next
            Case -2
                GameForm1.selectedpiece = ("blackqueen")
                For i As Integer = 0 To 7
                    For j As Integer = 0 To 7
                        If GameForm1.blackqueen(z).flags(i, j) = True
Then
                            Exit Do
                        End If
                    Next
                Next
            Case -3
                GameForm1.selectedpiece = ("blackbishop")
                For i As Integer = 0 To 7
                    For j As Integer = 0 To 7
                        If GameForm1.blackbishop(z).flags(i, j) = True
Then
                            Exit Do
                        End If
                    Next
                Next
            Case -4
                GameForm1.selectedpiece = "blackknight"
                For i As Integer = 0 To 7
                    For j As Integer = 0 To 7
                        If GameForm1.blackknight(z).flags(i, j) = True
Then
                            Exit Do
                        End If
                    Next
                Next
        End Select
    End Do
End Sub

```

```

        Next
    Case -5
        GameForm1.selectedpiece = ("blackrook")
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackrook(z).flags(i, j) = True Then
                    Exit Do
                End If
            Next
        Next
    Case -6
        GameForm1.selectedpiece = ("blackpawn")
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackpawn(z).flags(i, j) = True Then
                    Exit Do
                End If
            Next
        Next
    End Select
    recanmove()
    reback()
    GameForm1.selected = False
Loop
reback()
Do
    AIx(1) = rnd.Next(0, 8)
    AIy(1) = rnd.Next(0, 8)
    If canmove(AIx(1), AIy(1)) = True Then
        mov(GameForm1.selectedpiece, AIx(1), AIy(1), AIx(0), AIy(0))
    End If
Loop Until player = 1
GameForm1.selected = False
show()
End Sub

```

VB.Net Code(Hard AI)

```

Dim six(5) As Integer
Dim x(5) As Integer
Dim y(5) As Integer
Dim prex(5) As Integer
Dim prey(5) As Integer
Dim selected(5) As String
Dim AICalculating As Boolean = False
Private Sub AIMovingHard()
    AICalculating = True

    Dim DontRepeat As Boolean
    Dim PieceCanMove As Boolean

    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            GameForm1.pb(i, j).Enabled = False
        Next
    Next

```



```

Next
show()
'Storing current board
LoadTempBoard()
'Changing it so when pieces move the board tracker does not add text
canwritetoboardtracker = False

'Black Pawn
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 7
    '
    '
    Dim xpos As Integer = GameForm1.blackpawn(i).xposition
    Dim ypos As Integer = GameForm1.blackpawn(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recursionstop = False
            fillcheckforblack()
            GameForm1.blackpawn(i).possiblemoves()
            'If the pawn can move
            If GameForm1.blackpawn(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the pawn
                GameForm1.blackpawn(i).changeposition(j, k,
GameForm1.blackpawn(i).xposition, GameForm1.blackpawn(i).yposition)
                'Change the x and y position of the pawn (as write to
board = false)

                GameForm1.blackpawn(i).xposition = j
                GameForm1.blackpawn(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                ' Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                            If
canmove(GameForm1.blackpawn(i).xposition, GameForm1.blackpawn(i).yposition) =
False Then
                                ' If black can check the white king,
-5 off Of q, (most post effective way, will check king unless the pawn can
take a piece)

                                q = q - 5
                                AICheckmate()
                            End If

```

```

        End If
    Next
Next

'Change player to 2
player = 2

'Change the x and y position of the pawn back to
original
GameForm1.blackpawn(i).xposition = xpos
GameForm1.blackpawn(i).yposition = ypos

'Store point value if pawn moves there
If DontRepeat = True Then
    six(0) = q
    x(0) = j
    y(0) = k
    prex(0) = GameForm1.blackpawn(i).xposition
    prey(0) = GameForm1.blackpawn(i).yposition
    selected(0) = "blackpawn"
    DontRepeat = False
Else
    'Calculate random number (either a 0 or a 1)
    Dim samemove As Integer
    Dim rnd As New Random
    samemove = rnd.Next(0, 2)
    If q < six(0) Then
        six(0) = q
        x(0) = j
        y(0) = k
        prex(0) = GameForm1.blackpawn(i).xposition
        prey(0) = GameForm1.blackpawn(i).yposition
        selected(0) = "blackpawn"
    End If
    'If the maximum move is the same value as the one
already stored, randomly choose a move
    If q = six(0) And samemove = 1 Then
        six(0) = q
        x(0) = j
        y(0) = k
        prex(0) = GameForm1.blackpawn(i).xposition
        prey(0) = GameForm1.blackpawn(i).yposition
        selected(0) = "blackpawn"
        DontRepeat = False
    End If
End If

'Reset board back to original
Resetboard()
End If
Next
Next
Next

If PieceCanMove = False Then
    six(0) = 1000
End If

```

```

'Black Knight
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 1 + GameForm1.bknightcount
    '
    '
    Dim xpos As Integer = GameForm1.blackknight(i).xposition
    Dim ypos As Integer = GameForm1.blackknight(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recurrisionstop = False
            fillcheckforblack()
            GameForm1.blackknight(i).possiblemoves()
            'If the knight can move
            If GameForm1.blackknight(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the knight
                GameForm1.blackknight(i).changeposition(j, k,
GameForm1.blackknight(i).xposition, GameForm1.blackknight(i).yposition)
                'Change the x and y position of the knight (as write
to board = false)

                GameForm1.blackknight(i).xposition = j
                GameForm1.blackknight(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                ' Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                            If
canmove(GameForm1.blackknight(i).xposition,
GameForm1.blackknight(i).yposition) = False Then
                                'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the knight can
take a piece)

                                    q = q - 5
                                    AICheckmate()
                                End If
                            End If
                        Next
                    Next
                Next

                'Change player to 2
                player = 2
            End If
        Next
    Next

```

```

original
    'Change the x and y position of the knight back to
    GameForm1.blackknight(i).xposition = xpos
    GameForm1.blackknight(i).yposition = ypos

    'Store point value if knight moves there
    If DontRepeat = True Then
        six(1) = q
        x(1) = j
        y(1) = k
        prex(1) = GameForm1.blackknight(i).xposition
        prey(1) = GameForm1.blackknight(i).yposition
        selected(1) = "blackknight"
        DontRepeat = False
    Else
        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(1) Then
            six(1) = q
            x(1) = j
            y(1) = k
            prex(1) = GameForm1.blackknight(i).xposition
            prey(1) = GameForm1.blackknight(i).yposition
            selected(1) = "blackknight"
        End If
        'If the maximum move is the same value as the one
        already stored, randomly choose a move
        If q = six(1) And samemove = 1 Then
            six(1) = q
            x(1) = j
            y(1) = k
            prex(1) = GameForm1.blackknight(i).xposition
            prey(1) = GameForm1.blackknight(i).yposition
            selected(1) = "blackknight"
            DontRepeat = False
        End If
    End If

    'Reset board back to original
    Resetboard()
End If
Next
Next
Next

If PieceCanMove = False Then
    six(1) = 1000
End If

'Black Queen
DontRepeat = True
PieceCanMove = False

```

```

For i As Integer = 0 To GameForm1.bqueencount
    '
    '
    Dim xpos As Integer = GameForm1.blackqueen(i).xposition
    Dim ypos As Integer = GameForm1.blackqueen(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recursionstop = False
            fillcheckforblack()
            GameForm1.blackqueen(i).possiblemoves()
            'If the queen can move
            If GameForm1.blackqueen(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the queen
                GameForm1.blackqueen(i).changeposition(j, k,
GameForm1.blackqueen(i).xposition, GameForm1.blackqueen(i).yposition)
                'Change the x and y position of the queen (as write
to board = false)

                GameForm1.blackqueen(i).xposition = j
                GameForm1.blackqueen(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                'Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackqueen(i).xposition, GameForm1.blackqueen(i).yposition)
= False Then
                                    '
                                    'If black can check
the white king, - 5 off of q, (most post effective way, will check king
unless the queen can take a piece)
                                        q = q - 5
                                        AICheckmate()
                                    End If
                                End If
                            Next
                        Next

                        'Change player to 2
                        player = 2

                        'Change the x and y position of the queen back to
original

                        GameForm1.blackqueen(i).xposition = xpos
                        GameForm1.blackqueen(i).yposition = ypos

```

```

        'Store point value if queen moves there
    If DontRepeat = True Then
        six(2) = q
        x(2) = j
        y(2) = k
        prex(2) = GameForm1.blackqueen(i).xposition
        prey(2) = GameForm1.blackqueen(i).yposition
        selected(2) = "blackqueen"
        DontRepeat = False
    Else
        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(2) Then
            six(2) = q
            x(2) = j
            y(2) = k
            prex(2) = GameForm1.blackqueen(i).xposition
            prey(2) = GameForm1.blackqueen(i).yposition
            selected(2) = "blackqueen"
        End If
        'If the maximum move is the same value as the one
already stored, randomly choose a move
        If q = six(2) And samemove = 1 Then
            six(2) = q
            x(2) = j
            y(2) = k
            prex(2) = GameForm1.blackqueen(i).xposition
            prey(2) = GameForm1.blackqueen(i).yposition
            selected(2) = "blackqueen"
            DontRepeat = False
        End If
    End If

    'Reset board back to original
    Resetboard()
End If

Next
Next
Next

If PieceCanMove = False Then
    six(2) = 1000
End If

'Black Bishop
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 1 + GameForm1.bbishopcount
    '
    '
    Dim xpos As Integer = GameForm1.blackbishop(i).xposition
    Dim ypos As Integer = GameForm1.blackbishop(i).yposition
    '

```

```

'
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        recursionstop = False
        fillcheckforblack()
        GameForm1.blackbishop(i).possiblemoves()
        'If the bishop can move
        If GameForm1.blackbishop(i).flags(j, k) = True Then
            PieceCanMove = True
            'Changing position of the bishop
            GameForm1.blackbishop(i).changeposition(j, k,
GameForm1.blackbishop(i).xposition, GameForm1.blackbishop(i).yposition)
            'Change the x and y position of the bishop (as write
to board = false)

            GameForm1.blackbishop(i).xposition = j
            GameForm1.blackbishop(i).yposition = k

            'Change player to 1
            player = 1
            'Second Depth
            SecondDepth()

            'See if white king is in check
            fillcheckforwhite()

            'Code will always check the king
            For a As Integer = 0 To 7
                For b As Integer = 0 To 7
                    If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                        If
canmove(GameForm1.blackbishop(i).xposition,
GameForm1.blackbishop(i).yposition) = False Then
                            'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the bishop can
take a piece)

                                q = q - 5
                                AICheckmate()
                                End If
                            End If
                        Next
                    Next

                    'Change player to 2
                    player = 2

                    'Change the x and y position of the bishop back to
original

                    GameForm1.blackbishop(i).xposition = xpos
                    GameForm1.blackbishop(i).yposition = ypos

                    'Store point value if bishop moves there
                    If DontRepeat = True Then
                        six(3) = q
                        x(3) = j
                        y(3) = k

```

```

        prex(3) = GameForm1.blackbishop(i).xposition
        prey(3) = GameForm1.blackbishop(i).yposition
        selected(3) = "blackbishop"
        DontRepeat = False
    Else
        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(3) Then
            six(3) = q
            x(3) = j
            y(3) = k
            prex(3) = GameForm1.blackbishop(i).xposition
            prey(3) = GameForm1.blackbishop(i).yposition
            selected(3) = "blackbishop"
        End If
        'If the maximum move is the same value as the one
already stored, randomly choose a move
        If q = six(3) And samemove = 1 Then
            six(3) = q
            x(3) = j
            y(3) = k
            prex(3) = GameForm1.blackbishop(i).xposition
            prey(3) = GameForm1.blackbishop(i).yposition
            selected(3) = "blackbishop"
            DontRepeat = False
        End If
    End If

    'Reset board back to original
    Resetboard()
End If
Next
Next

Next

If PieceCanMove = False Then
    six(3) = 1000
End If

'Black Rook
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 1 + GameForm1.brookcount
    '
    '
    Dim xpos As Integer = GameForm1.blackrook(i).xposition
    Dim ypos As Integer = GameForm1.blackrook(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recurrisionstop = False

```



```

fillcheckforblack()
GameForm1.blackrook(i).possiblemoves()
'If the rook can move
If GameForm1.blackrook(i).flags(j, k) = True Then
    PieceCanMove = True
    'Changing position of the rook
    GameForm1.blackrook(i).changeposition(j, k,
GameForm1.blackrook(i).xposition, GameForm1.blackrook(i).yposition)
    'Change the x and y position of the rook (as write to
board = false)

    GameForm1.blackrook(i).xposition = j
    GameForm1.blackrook(i).yposition = k

    'Change player to 1
    player = 1
    'Second Depth
    SecondDepth()

    'See if white king is in check
    fillcheckforwhite()

    'Code will always check the king
    For a As Integer = 0 To 7
        For b As Integer = 0 To 7
            If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                If
canmove(GameForm1.blackrook(i).xposition, GameForm1.blackrook(i).yposition) =
False Then
                    'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the rook can
take a piece)
                        q = q - 5
                        AICheckmate()
                    End If
                End If
            Next
        Next
    Next

    'Change player to 2
    player = 2

    'Change the x and y position of the rook back to
original
    GameForm1.blackrook(i).xposition = xpos
    GameForm1.blackrook(i).yposition = ypos
    'Store point value if rook moves there
    If DontRepeat = True Then
        six(4) = q
        x(4) = j
        y(4) = k
        prex(4) = GameForm1.blackrook(i).xposition
        prey(4) = GameForm1.blackrook(i).yposition
        selected(4) = "blackrook"
        DontRepeat = False
    Else

```

```

        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(4) Then
            six(4) = q
            x(4) = j
            y(4) = k
            prex(4) = GameForm1.blackrook(i).xposition
            prey(4) = GameForm1.blackrook(i).yposition
            selected(4) = "blackrook"
        End If
        'If the maximum move is the same value as the one
already stored, randomly choose a move
        If q = six(4) And samemove = 1 Then
            six(4) = q
            x(4) = j
            y(4) = k
            prex(4) = GameForm1.blackrook(i).xposition
            prey(4) = GameForm1.blackrook(i).yposition
            selected(4) = "blackrook"
            DontRepeat = False
        End If
    End If
End If

'Reset board back to original
Resetboard()

End If
Next
Next
Next

If PieceCanMove = False Then
    six(4) = 1000
End If

'Black King
DontRepeat = True
PieceCanMove = False
'
'
Dim xposking As Integer = GameForm1.blackking.xposition
Dim yposking As Integer = GameForm1.blackking.yposition
'
'
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        recursionstop = False
        fillcheckforblack()
        GameForm1.blackking.possiblemoves()
        'If the king can move
        If GameForm1.blackking.flags(j, k) = True Then
            PieceCanMove = True
            'Changing position of the king
            GameForm1.blackking.changeposition(j, k,
GameForm1.blackking.xposition, GameForm1.blackking.yposition)

```

```

board = false)

'Change the x and y position of the king (as write to
GameForm1.blackking.xposition = j
GameForm1.blackking.yposition = k

'Change player to 1
player = 1
'Second Depth
SecondDepth()

'Change player to 2
player = 2

'Change the x and y position of the king back to original
GameForm1.blackking.xposition = xposking
GameForm1.blackking.yposition = yposking

'Store point value if king moves there
If DontRepeat = True Then
    six(5) = q
    x(5) = j
    y(5) = k
    prex(5) = GameForm1.blackking.xposition
    prey(5) = GameForm1.blackking.yposition
    selected(5) = "blackking"
    DontRepeat = False
Else
    'Calculate random number (either a 0 or a 1)
    Dim samemove As Integer
    Dim rnd As New Random
    samemove = rnd.Next(0, 2)
    If q < six(5) Then
        six(5) = q
        x(5) = j
        y(5) = k
        prex(5) = GameForm1.blackking.xposition
        prey(5) = GameForm1.blackking.yposition
        selected(5) = "blackking"
    End If
    'If the maximum move is the same value as the one
already stored, randomly choose a move
    If q = six(5) And samemove = 1 Then
        six(5) = q
        x(5) = j
        y(5) = k
        prex(5) = GameForm1.blackking.xposition
        prey(5) = GameForm1.blackking.yposition
        selected(5) = "blackking"
        DontRepeat = False
    End If
End If

'Reset board back to original
Resetboard()
End If
Next
Next

```

```

If PieceCanMove = False Then
    six(5) = 1000
End If

canwritetoboardtracker = True
For i As Integer = 0 To 7
    For j As Integer = 0 To 7
        GameForm1.pb(i, j).Enabled = True
    Next
Next
Resetboard()
show()
reback()
'Bubble sort algorithm for deciding best move
Dim iTemp As Integer
Dim iTemp1 As String
For iPass = 1 To 4
    For i As Integer = 0 To 4
        If six(i) < six(i + 1) Then
            iTemp = six(i)
            six(i) = six(i + 1)
            six(i + 1) = iTemp
            iTemp = x(i)
            x(i) = x(i + 1)
            x(i + 1) = iTemp
            iTemp = y(i)
            y(i) = y(i + 1)
            y(i + 1) = iTemp
            iTemp = prex(i)
            prex(i) = prex(i + 1)
            prex(i + 1) = iTemp
            iTemp = prey(i)
            prey(i) = prey(i + 1)
            prey(i + 1) = iTemp
            iTemp1 = selected(i)
            selected(i) = selected(i + 1)
            selected(i + 1) = iTemp1
        End If
    Next
Next

Dim random As Integer = -1
Dim rndd As New Random
'if multiple moves have the same minimum losing value, randomly
select one
'ignore king due to chess logic
If six(5) = six(4) Then
    random = 4
    If six(4) = six(3) Then
        random = rndd.Next(3, 5)
        If six(3) = six(2) Then
            random = rndd.Next(2, 5)
            If six(2) = six(1) Then
                random = rndd.Next(1, 5)
                If six(1) = six(0) Then
                    random = rndd.Next(0, 5)
                End If
            End If
        End If
    End If
End If

```

```

        End If
    End If
End If
End If
End If
AICalculating = False

If random = -1 Then
    mov(selected(5), x(5), y(5), prex(5), prey(5))
Else
    mov(selected(random), x(random), y(random), prex(random),
prey(random))
End If
End Sub

Dim tempboard(7, 7) As Integer
Dim points As Integer
Dim count As Integer = 0

Private Sub SecondDepth()
    LoadSecondDepthBoard()

    recurrisionstop = False
    fillcheckforwhite()
    recanmove()
    Application.DoEvents()
    'Reset count
    count = 0

    'Move all the white pawns into all possible positions
    For i As Integer = 0 To 7
        If GameForm1.whitepawn(i).stillonboard = False Then
            GameForm1.whitepawn(i).reflags()
        Else
            If board(GameForm1.whitepawn(i).xposition,
GameForm1.whitepawn(i).yposition) = 6 Then
                GameForm1.whitepawn(i).possiblemoves()
                For j As Integer = 0 To 7
                    For k As Integer = 0 To 7
                        'If the pawn can move
                        If GameForm1.whitepawn(i).flags(j, k) = True Then
                            'Move to that square

                            GameForm1.whitepawn(i).changeposition(j, k,
GameForm1.whitepawn(i).xposition, GameForm1.whitepawn(i).yposition)
                            'Reset board back to before 2nd depth move
was made

                            CalculateBoardPoints()

                            ResetSecondDepthBoard()
                        End If
                    Next
                Next

                GameForm1.whitepawn(i).fillcheckking()
            End If
        End If
    Next
End Sub

```

```

        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                'If the pawn can move
                If GameForm1.whitepawn(i).flags(j, k) = True Then
                    canmove(j, k) = True
                End If
            Next
        Next
    End If
End If
Next

'Move all the white knights into all possible positions
For i As Integer = 0 To 1 + GameForm1.wknightcount
    If GameForm1.whiteknight(i).stillonboard = False Then
        GameForm1.whiteknight(i).reflags()
    Else
        If board(GameForm1.whiteknight(i).xposition,
GameForm1.whiteknight(i).yposition) = 4 Then
            GameForm1.whiteknight(i).possiblemoves()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    'If the knight can move
                    If GameForm1.whiteknight(i).flags(j, k) = True
Then
                        'Move to that square
                        GameForm1.whiteknight(i).changeposition(j, k,
GameForm1.whiteknight(i).xposition, GameForm1.whiteknight(i).yposition)
                        'Reset board back to before 2nd depth move
was made

                        CalculateBoardPoints()

                        If GameForm1.whiteknight(i).flags(j, k) =
True Then
                            canmove(j, k) = True
                        End If
                        ResetSecondDepthBoard()
                        recursionstop = False
                        fillcheckforwhite()
                    End If
                Next
            Next
        End If
    End If
Next

'Move all the white queens into all possible positions
For i As Integer = 0 To GameForm1.wqueencount
    If GameForm1.whitequeen(i).stillonboard = False Then
        GameForm1.whitequeen(i).reflags()
    Else
        If board(GameForm1.whitequeen(i).xposition,
GameForm1.whitequeen(i).yposition) = 2 Then
            GameForm1.whitequeen(i).possiblemoves()
            For j As Integer = 0 To 7

```

```

        For k As Integer = 0 To 7
            'If the queen can move
            If GameForm1.whitequeen(i).flags(j, k) = True
Then
                'Move to that square
                GameForm1.whitequeen(i).changeposition(j, k,
GameForm1.whitequeen(i).xposition, GameForm1.whitequeen(i).yposition)
                'Reset board back to before 2nd depth move
was made
                CalculateBoardPoints()
                'TESTING
                If GameForm1.whitequeen(i).flags(j, k) = True
Then
                    canmove(j, k) = True
                End If
                ResetSecondDepthBoard()
                recurrssionstop = False
                fillcheckforwhite()
            End If
        Next
    Next
End If
Next

'Move all the white bishops into all possible positions

For i As Integer = 0 To 1 + GameForm1.wbishopcount
    If GameForm1.whitebishop(i).stillonboard = False Then
        GameForm1.whitebishop(i).reflags()
    Else
        If board(GameForm1.whitebishop(i).xposition,
GameForm1.whitebishop(i).yposition) = 3 Then
            GameForm1.whitebishop(i).possiblemoves()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    'If the bishop can move
                    If GameForm1.whitebishop(i).flags(j, k) = True
Then
                        'Move to that square
                        GameForm1.whitebishop(i).changeposition(j, k,
GameForm1.whitebishop(i).xposition, GameForm1.whitebishop(i).yposition)
                        'Reset board back to before 2nd depth move
was made
                        CalculateBoardPoints()

                        If GameForm1.whitebishop(i).flags(j, k) =
True Then
                            canmove(j, k) = True
                        End If

                        ResetSecondDepthBoard()
                        recurrssionstop = False
                        fillcheckforwhite()
                    End If
                Next
            Next
        Next
    Next

```

```

        End If
    End If
Next

'Move all the white rook into all possible positions

For i As Integer = 0 To 1 + GameForm1.wrookcount
    If GameForm1.whiterook(i).stillonboard = False Then
        GameForm1.whiterook(i).reflags()
    Else
        If board(GameForm1.whiterook(i).xposition,
GameForm1.whiterook(i).yposition) = 5 Then
            GameForm1.whiterook(i).possiblemoves()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    'If the rook can move
                    If GameForm1.whiterook(i).flags(j, k) = True Then
                        'Move to that square
                        GameForm1.whiterook(i).changeposition(j, k,
GameForm1.whiterook(i).xposition, GameForm1.whiterook(i).yposition)
                        'Reset board back to before 2nd depth move
was made

                        CalculateBoardPoints()
                        If GameForm1.whiterook(i).flags(j, k) = True

                            canmove(j, k) = True
                        End If
                        ResetSecondDepthBoard()
                        recursionstop = False
                        fillcheckforwhite()
                    End If
                Next
            Next
        End If
    End If
Next

'Move all the white king into all possible positions

GameForm1.whiteking.possiblemoves()
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        'If the king can move
        If GameForm1.whiteking.flags(j, k) = True Then

            'Move to that square
            GameForm1.whiteking.changeposition(j, k,
GameForm1.whiteking.xposition, GameForm1.whiteking.yposition)
            'Reset board back to before 2nd depth move was made
            CalculateBoardPoints()
            ResetSecondDepthBoard()
            recursionstop = False
            fillcheckforwhite()
        End If
    Next
Next

```



```

Next

GameForm1.whiteking.fillcheckking()

For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        'If the pawn can move
        If GameForm1.whiteking.flags(j, k) = True Then
            canmove(j, k) = True
        End If
    Next
Next
End Sub

Private Sub LoadTempBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            tempboard(i, j) = board(i, j)
        Next
    Next
End Sub

Private Sub Resetboard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            board(i, j) = tempboard(i, j)
        Next
    Next
End Sub

Dim tempboard2(7, 7) As Integer
Private Sub LoadSecondDepthBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            tempboard2(i, j) = board(i, j)
        Next
    Next
End Sub

Private Sub ResetSecondDepthBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            board(i, j) = tempboard2(i, j)
        Next
    Next
End Sub

Dim q As Integer 'Used in minmax algorithm, minimum loss move

Private Sub CalculateBoardPoints()
    count = count + 1
    points = 0
    'Calculation
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            Select Case board(i, j)
                Case 1
                    'King = 900 points
                    points += 900
                Case 2
                    'Queen = 90 points

```

```

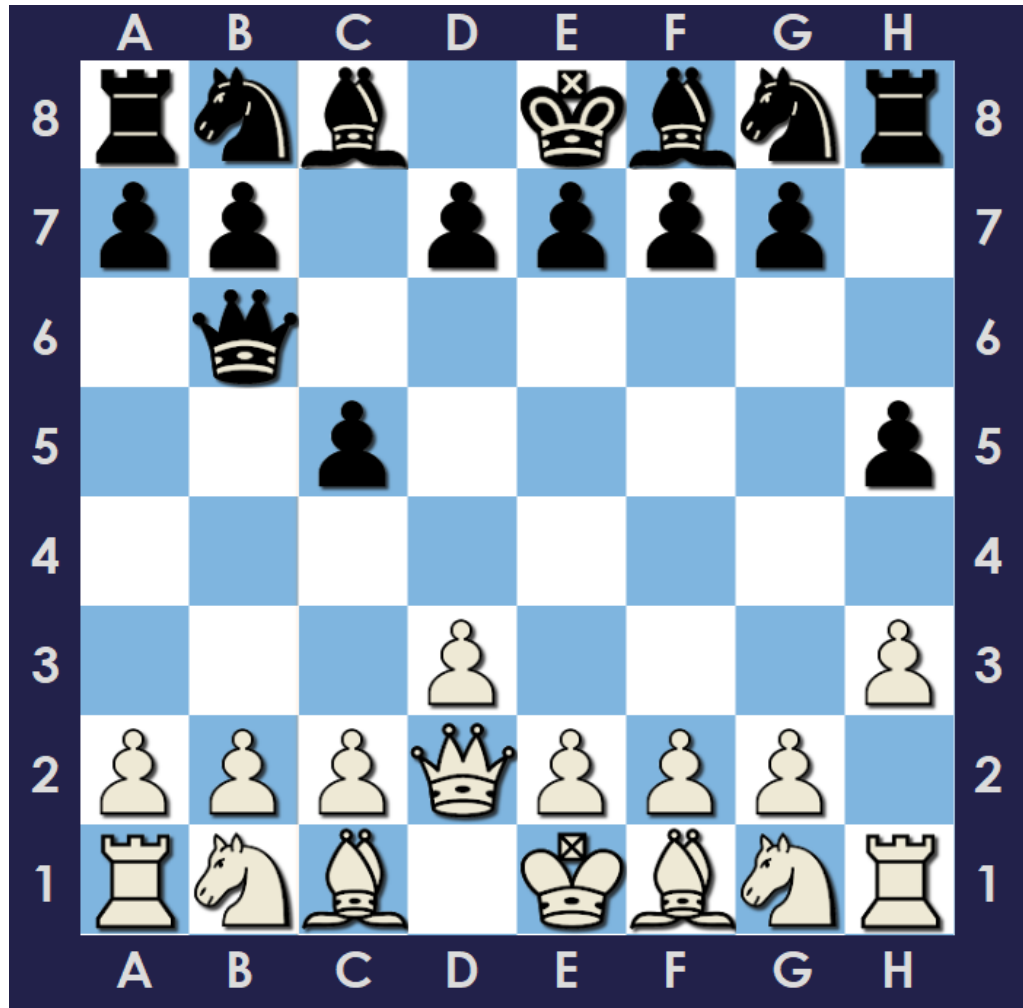
        points += 90
    Case 3
        'Bishop = 30 points
        points += 30
    Case 4
        'Knight = 30 points
        points += 30
    Case 5
        'Rook = 50 points
        points += 50
    Case 6
        'Pawn = 10 point
        points += 10
    Case -1
        'King = -900 points
        points += -900
    Case -2
        'Queen = -90 points
        points += -90
    Case -3
        'Bishop = -30 points
        points += -30
    Case -4
        'Knight = -30 points
        points += -30
    Case -5
        'Rook = -50 points
        points += -50
    Case -6
        'Pawn = -10 point
        points += -10
    End Select
Next
Next
If count = 1 Then
    q = points
End If
If points > q Then
    q = points
End If
End Sub
Private Sub AICheckmate()
    Dim AICheckmate As Boolean = False
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If canmove(i, j) = False Then
                AICheckmate = True
            Else
                AICheckmate = False
            End If
        Next
    Next
    If AICheckmate = True Then
        q = q - 100
    End If
End Sub

```

Test Plan

Due to the objectives in this section mostly being without any user input, the number of test plans are very limited.

Minimax



Description: testing the minimax algorithm makes the correct move in a specific situation

Test Data	Data Type	Expected Outcome
A4 → C6	Normal	B8/B7/D7 → C6
C2 → H7	Borderline	H8 → H7

6 - Replays

Summary: correctly load the pieces and the board onto the form. Allow the user to select the replay they would like to view and load the movements on the board. The user will be able to move rewind, pause, and play the replays.

Research

Document (<https://www.quora.com/Why-do-we-replay-games-Does-the-game-have-to-be-good-for-us-to-play-it-again-and-have-the-qualities-to-be-replayable-Does-it-depend-on-the-person>)

There are different reasons to why implementation of replays is a good idea. Some of them include reliving the game experience, analysing mistakes, and checking for game errors. In most games replays are recorded at the end of the game, with the user manually asking the program to do so. With my program, I chose to store the game replays within the access database, therefore it is easily accessible under the username of the player.

For detailed analysis on the replay system, go to the *analysis* section of the document.



Example of an existing chess replay system

The SQL SELECT Statement

The **SELECT** statement is used to select data from a database.

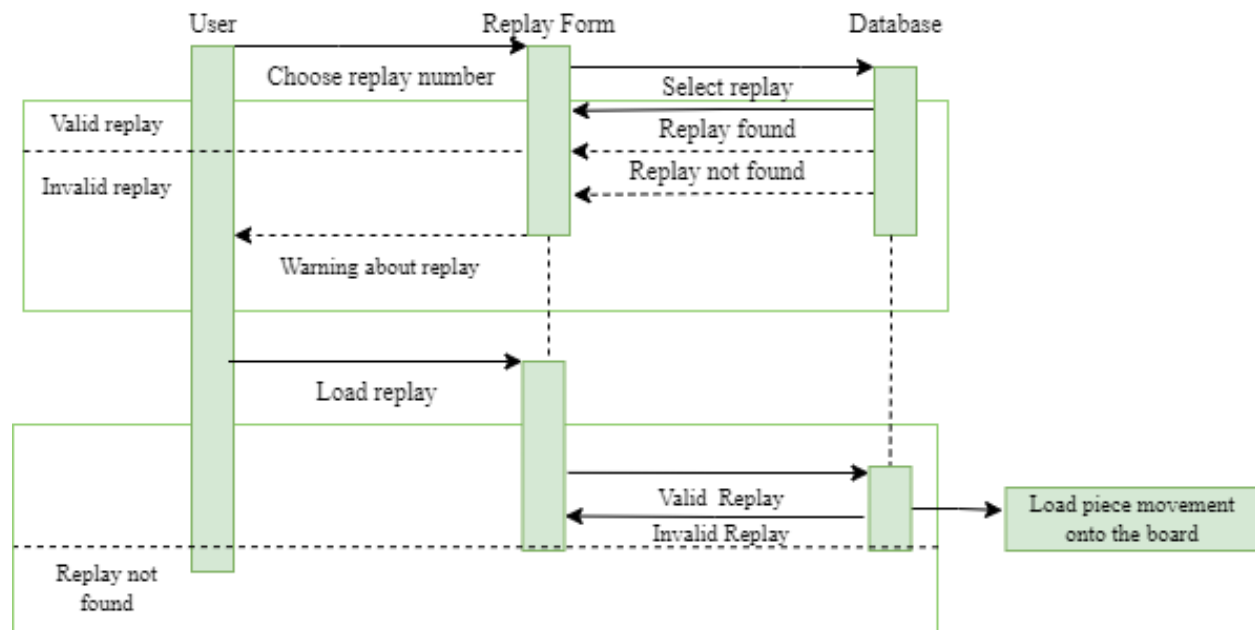
The data returned is stored in a result table, called the result-set.

SELECT Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

SQL Select statement will be used when loading the user's replay onto the replay form

UML



Pseudo Code

```

Sub Main
    replaynumber <-- userInput
    count <-- 1
    If Table1.Move(replaynumber) & Table1.Premove(replaynumber) <> Nothing
    Then
        PostMove <-- Table1.Move(replaynumber)
        PreMove <-- Table1.Premove(replaynumber)
    Endif

    If Premove = Nothing Then
        Output("Replay not found")
    Else
        Output("Would you like to move forward or backward")
        result <-- userInput
        If result = forward Then
            Movepiece()
        ElseIf result = backward Then
            Movepieceback()
        Endif
    Endif
End sub

Sub Movepiece
    x <-- x + 1
    If x <= 0 Then
        x <-- 1
    Endif

    Currentposition = Substring(x, x)
    Nextposition = SubString(x, x)

    If Currentposition = "A" Then

```

```

        yposition(0) <-- 0
    ElseIf Currentposition = "B" Then
        yposition(0) <-- 1
    ElseIf Currentposition = "C" Then
        yposition(0) <-- 2
    ElseIf Currentposition = "D" Then
        yposition(0) <-- 3
    ElseIf Currentposition = "E" Then
        yposition(0) <-- 4
    ElseIf Currentposition = "F" Then
        yposition(0) <-- 5
    ElseIf Currentposition = "G" Then
        yposition(0) <-- 6
    ElseIf Currentposition = "H" Then
        yposition(0) <-- 7
    Endif

    x <-- x + 1
    Currentposition = Substring(x, x)
    Nextposition = SubString(x, x)

    If Nextposition = "A" Then
        Nextposition(0) <-- 0
    ElseIf Nextposition = "B" Then
        xposition(0) <-- 1
    ElseIf Nextposition = "C" Then
        xposition(0) <-- 2
    ElseIf Nextposition = "D" Then
        xposition(0) <-- 3
    ElseIf Nextposition = "E" Then
        xposition(0) <-- 4
    ElseIf Nextposition = "F" Then
        xposition(0) <-- 5
    ElseIf Nextposition = "G" Then
        xposition(0) <-- 6
    ElseIf Nextposition = "H" Then
        xposition(0) <-- 7
    Endif

    y <-- x

    If board(xposition(0), yposition(0)) <> Nothing Then
        Premovepiecevalue(count) <-- board(xposition(1), yposition(1))
        board(xposition(1), yposition(1)) <-- board(xposition(0),
yposition(0))
        board(xposition(0), yposition(0)) -- 0
        count <-- count + 1
    Else
        Movepiece()
    Endif

    If x >= Len(Table1.Move(replaynumber)) Then
        x <-- Len(Table1.Move(replaynumber)) - 2
        Exit Sub
    Endif
End Sub

```

```
Sub Movepieceback
```

```
  If y <= 0 Then
    Exit Sub
  Endif
```

```
  y <-- y - 1
  Currentposition = Substring(y, y)
  Nextposition = SubString(y, y)
```

```
  If Currentposition = "A" Then
    yposition(0) <-- 0
  ElseIf Currentposition = "B" Then
    yposition(0) <-- 1
  ElseIf Currentposition = "C" Then
    yposition(0) <-- 2
  ElseIf Currentposition = "D" Then
    yposition(0) <-- 3
  ElseIf Currentposition = "E" Then
    yposition(0) <-- 4
  ElseIf Currentposition = "F" Then
    yposition(0) <-- 5
  ElseIf Currentposition = "G" Then
    yposition(0) <-- 6
  ElseIf Currentposition = "H" Then
    yposition(0) <-- 7
  Endif
```

```
  y <-- y + 1
  Currentposition = Substring(y, y)
  Nextposition = SubString(y, y)
```

```
  If Nextposition = "A" Then
    Nextposistion(0) <-- 0
  ElseIf Nextposition = "B" Then
    xposition(0) <-- 1
  ElseIf Nextposition = "C" Then
    xposition(0) <-- 2
  ElseIf Nextposition = "D" Then
    xposition(0) <-- 3
  ElseIf Nextposition = "E" Then
    xposition(0) <-- 4
  ElseIf Nextposition = "F" Then
    xposition(0) <-- 5
  ElseIf Nextposition = "G" Then
    xposition(0) <-- 6
  ElseIf Nextposition = "H" Then
    xposition(0) <-- 7
  Endif
```

```
  board(xposition(0), yposition(0)) <-- board(xposition(1),
yposition(1))
  count <-- count - 1
  board(xposition(1), yposition(1)) <-- Premovepiecevalue(count)
  y <-- y - 2
  x <-- y - 2
```

```
End Sub
```

VB.Net Code

```

Imports System.Data.OleDb
Imports System.Data
Imports System.Threading
Public Class ReplayForm
    Dim pb(7, 7) As PictureBox
    Dim board(7, 7) As Integer
    Dim Set1 As Boolean = False
    Dim Set2 As Boolean = False
    Dim Set3 As Boolean = False
    Dim PreMove As String
    Dim PostMove As String
    Dim gamelength As Integer
    Dim ReplayNumber As Integer = 0
    Private Sub ReplayForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load

        'Creating picture boxes as an array
        pb = New PictureBox(7, 7) {{Me.p00, Me.p01, Me.p02, Me.p03, Me.p04,
Me.p05, Me.p06, Me.p07},
            {Me.p10, Me.p11, Me.p12, Me.p13, Me.p14, Me.p15, Me.p16, Me.p17},
            {Me.p20, Me.p21, Me.p22, Me.p23, Me.p24, Me.p25, Me.p26, Me.p27},
            {Me.p30, Me.p31, Me.p32, Me.p33, Me.p34, Me.p35, Me.p36, Me.p37},
            {Me.p40, Me.p41, Me.p42, Me.p43, Me.p44, Me.p45, Me.p46, Me.p47},
            {Me.p50, Me.p51, Me.p52, Me.p53, Me.p54, Me.p55, Me.p56, Me.p57},
            {Me.p60, Me.p61, Me.p62, Me.p63, Me.p64, Me.p65, Me.p66, Me.p67},
            {Me.p70, Me.p71, Me.p72, Me.p73, Me.p74, Me.p75, Me.p76, Me.p77}}

        board = {{-5, -4, -3, -2, -1, -3, -4, -5},
            {-6, -6, -6, -6, -6, -6, -6, -6},
            {0, 0, 0, 0, 0, 0, 0, 0},
            {0, 0, 0, 0, 0, 0, 0, 0},
            {0, 0, 0, 0, 0, 0, 0, 0},
            {0, 0, 0, 0, 0, 0, 0, 0},
            {6, 6, 6, 6, 6, 6, 6, 6},
            {5, 4, 3, 2, 1, 3, 4, 5}}

        LoadReplay()
        showpiece()
    End Sub

    Private Sub showpiece()
        'Loading pieces onto board for user
        For i As Integer = 0 To 7
            If board(7, i) = -6 Then
                board(7, i) = -2
            End If
        Next

        If Set1 = True Then
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    Select Case board(i, j)
                        Case 1
                            pb(i, j).Image = My.Resources.Set1KingW
                        Case 2
                            pb(i, j).Image = My.Resources.Set1QueenW
                    End Select
                Next j
            Next i
        End If
    End Sub
End Class

```



```

        Case 3
            pb(i, j).Image = My.Resources.Set1BishopW
        Case 4
            pb(i, j).Image = My.Resources.Set1KnightW
        Case 5
            pb(i, j).Image = My.Resources.Set1RookW
        Case 6
            pb(i, j).Image = My.Resources.Set1PawnW
        Case -1
            pb(i, j).Image = My.Resources.Set1KingB
        Case -2
            pb(i, j).Image = My.Resources.Set1QueenB
        Case -3
            pb(i, j).Image = My.Resources.Set1BishopB
        Case -4
            pb(i, j).Image = My.Resources.Set1KnightB
        Case -5
            pb(i, j).Image = My.Resources.Set1RookB
        Case -6
            pb(i, j).Image = My.Resources.Set1PawnB
        Case Else
            pb(i, j).Image = Nothing
    End Select
Next
Next
ElseIf Set2 = True Then
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            Select Case board(i, j)
                Case 1
                    pb(i, j).Image = My.Resources.Set2KingW
                Case 2
                    pb(i, j).Image = My.Resources.Set2QueenW
                Case 3
                    pb(i, j).Image = My.Resources.Set2BishopW
                Case 4
                    pb(i, j).Image = My.Resources.Set2KnightW
                Case 5
                    pb(i, j).Image = My.Resources.Set2RookW
                Case 6
                    pb(i, j).Image = My.Resources.Set2PawnW
                Case -1
                    pb(i, j).Image = My.Resources.Set2KingB
                Case -2
                    pb(i, j).Image = My.Resources.Set2QueenB
                Case -3
                    pb(i, j).Image = My.Resources.Set2BishopB
                Case -4
                    pb(i, j).Image = My.Resources.Set2KnightB
                Case -5
                    pb(i, j).Image = My.Resources.Set2RookB
                Case -6
                    pb(i, j).Image = My.Resources.Set2PawnB
                Case Else
                    pb(i, j).Image = Nothing
            End Select
        Next
    Next

```

```

        Next
    ElseIf Set3 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)
                    Case 1
                        pb(i, j).Image = My.Resources.Set3KingW
                    Case 2
                        pb(i, j).Image = My.Resources.Set3QueenW
                    Case 3
                        pb(i, j).Image = My.Resources.Set3BishopW
                    Case 4
                        pb(i, j).Image = My.Resources.Set3KnightW
                    Case 5
                        pb(i, j).Image = My.Resources.Set3RookW
                    Case 6
                        pb(i, j).Image = My.Resources.Set3PawnW
                    Case -1
                        pb(i, j).Image = My.Resources.Set3KingB
                    Case -2
                        pb(i, j).Image = My.Resources.Set3QueenB
                    Case -3
                        pb(i, j).Image = My.Resources.Set3BishopB
                    Case -4
                        pb(i, j).Image = My.Resources.Set3KnightB
                    Case -5
                        pb(i, j).Image = My.Resources.Set3RookB
                    Case -6
                        pb(i, j).Image = My.Resources.Set3PawnB
                    Case Else
                        pb(i, j).Image = Nothing
                End Select
            Next
        Next
    End If
End Sub

Private Sub redoback()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            pb(i, j).BackgroundImage = Nothing
        Next
    Next
End Sub

Private Sub LoadReplay()
    'Loading User 1 profile details
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
    'connection

```

```

Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
Dim sql As String
sql = "SELECT [Username],[BoardColour],[PieceType], [PreMove1],
[Move1], [PreMove2], [Move2], [PreMove3], [Move3], [PreMove4], [Move4],
[PreMove5], [Move5] FROM [Table1]" 'SQL Statement
Dim cmd As New OleDbCommand(sql, cn)
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
Dim count As Integer

For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
    If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 profile picture (for a user that has signed in)
        Select Case (ds.Tables(0).Rows(count).Item(1))
            Case "YellowBoard"
                Me.BackgroundImage = My.Resources.YellowBoard
            Case "GreenBoard"
                Me.BackgroundImage = My.Resources.GreenBoard
            Case "PinkBoard"
                Me.BackgroundImage = My.Resources.PinkBoard
            Case "RedBoard"
                Me.BackgroundImage = My.Resources.RedBoard
            Case "BlueBoard"
                Me.BackgroundImage = My.Resources.BlueBoard
        End Select
        Select Case (ds.Tables(0).Rows(count).Item(2))
            Case "Set1"
                Set1 = True
            Case "Set2"
                Set2 = True
            Case "Set3"
                Set3 = True
        End Select
        Try
            If ReplayNumber = 1 Then
                PreMove = ds.Tables(0).Rows(count).Item(3)
                PostMove = ds.Tables(0).Rows(count).Item(4)
            ElseIf ReplayNumber = 2 Then
                PreMove = ds.Tables(0).Rows(count).Item(5)
                PostMove = ds.Tables(0).Rows(count).Item(6)
            ElseIf ReplayNumber = 3 Then
                PreMove = ds.Tables(0).Rows(count).Item(7)
                PostMove = ds.Tables(0).Rows(count).Item(8)
            ElseIf ReplayNumber = 4 Then
                PreMove = ds.Tables(0).Rows(count).Item(9)
                PostMove = ds.Tables(0).Rows(count).Item(10)
            ElseIf ReplayNumber = 5 Then
                PreMove = ds.Tables(0).Rows(count).Item(11)
                PostMove = ds.Tables(0).Rows(count).Item(12)
            End If
            gamelength = Len(PreMove)
        Catch ex As Exception
            PreMove = ""
        End Try
    End If
End For

```

```

        PostMove = ""
    End Try
End If
Next
cn.Close() 'Close connection
End Sub

Dim CurrentPos As String = "Nothing"
Dim NextPos As String
Dim x As Integer = 0
Dim count As Integer = 1
Dim xposition(1) As Integer
Dim yposition(1) As Integer
Dim Premovepiecevalue(1000) As Integer
Dim Postmovepiecevalue As Integer
Private Sub MovePiece()
    RewindButton.Show()

    x = x + 1
    If x <= 0 Then
        x = 1
    End If
    CurrentPos = Mid(PreMove, x, 1)
    NextPos = Mid(PostMove, x, 1)
    Select Case CurrentPos
        Case "A"
            yposition(0) = 0
        Case "B"
            yposition(0) = 1
        Case "C"
            yposition(0) = 2
        Case "D"
            yposition(0) = 3
        Case "E"
            yposition(0) = 4
        Case "F"
            yposition(0) = 5
        Case "G"
            yposition(0) = 6
        Case "H"
            yposition(0) = 7
    End Select
    Select Case NextPos
        Case "A"
            yposition(1) = 0
        Case "B"
            yposition(1) = 1
        Case "C"
            yposition(1) = 2
        Case "D"
            yposition(1) = 3
        Case "E"
            yposition(1) = 4
        Case "F"
            yposition(1) = 5
        Case "G"
            yposition(1) = 6
    End Select

```

```

        Case "H"
            yposition(1) = 7
        End Select

        x = x + 1

        CurrentPos = Mid(PreMove, x, 1)
        NextPos = Mid(PostMove, x, 1)

        Select Case CurrentPos
            Case "1"
                xposition(0) = 7
            Case "2"
                xposition(0) = 6
            Case "3"
                xposition(0) = 5
            Case "4"
                xposition(0) = 4
            Case "5"
                xposition(0) = 3
            Case "6"
                xposition(0) = 2
            Case "7"
                xposition(0) = 1
            Case "8"
                xposition(0) = 0
        End Select
        Select Case NextPos
            Case "1"
                xposition(1) = 7
            Case "2"
                xposition(1) = 6
            Case "3"
                xposition(1) = 5
            Case "4"
                xposition(1) = 4
            Case "5"
                xposition(1) = 3
            Case "6"
                xposition(1) = 2
            Case "7"
                xposition(1) = 1
            Case "8"
                xposition(1) = 0
        End Select

        y = x

        If board(xposition(0), yposition(0)) <> Nothing Then
            Premovepiecevalue(count) = board(xposition(1), yposition(1))
            board(xposition(1), yposition(1)) = board(xposition(0),
yposition(0))
            board(xposition(0), yposition(0)) = 0
            count += 1

            RewindButton.Show()

```

```

        showpiece()
    ElseIf board(xposition(0), yposition(0)) = Nothing Then
        MovePiece()
    End If
    If x >= gamelength Then 'End of Game

        FastFowardTimer.Stop()
        AutoPlayTimer.Stop()

        PauseButton.Hide()
        PlayButton.Hide()
        FastFowardButton.Hide()

        x = gamelength - 2
        Exit Sub
    End If
End Sub
Private Sub MovePieceBack()

    If y <= 0 Then
        RewindButton.Hide()
        Exit Sub
    End If
    y = y - 1
    CurrentPos = Mid(PreMove, y, 1)
    NextPos = Mid(PostMove, y, 1)
    Select Case CurrentPos
        Case "A"
            yposition(0) = 0
        Case "B"
            yposition(0) = 1
        Case "C"
            yposition(0) = 2
        Case "D"
            yposition(0) = 3
        Case "E"
            yposition(0) = 4
        Case "F"
            yposition(0) = 5
        Case "G"
            yposition(0) = 6
        Case "H"
            yposition(0) = 7
    End Select
    Select Case NextPos
        Case "A"
            yposition(1) = 0
        Case "B"
            yposition(1) = 1
        Case "C"
            yposition(1) = 2
        Case "D"
            yposition(1) = 3
        Case "E"
            yposition(1) = 4
        Case "F"
            yposition(1) = 5
    End Select

```

```

        Case "G"
            yposition(1) = 6
        Case "H"
            yposition(1) = 7
    End Select

    y = y + 1
    CurrentPos = Mid(PreMove, y, 1)
    NextPos = Mid(PostMove, y, 1)

    Select Case CurrentPos
        Case "1"
            xposition(0) = 7
        Case "2"
            xposition(0) = 6
        Case "3"
            xposition(0) = 5
        Case "4"
            xposition(0) = 4
        Case "5"
            xposition(0) = 3
        Case "6"
            xposition(0) = 2
        Case "7"
            xposition(0) = 1
        Case "8"
            xposition(0) = 0
    End Select
    Select Case NextPos
        Case "1"
            xposition(1) = 7
        Case "2"
            xposition(1) = 6
        Case "3"
            xposition(1) = 5
        Case "4"
            xposition(1) = 4
        Case "5"
            xposition(1) = 3
        Case "6"
            xposition(1) = 2
        Case "7"
            xposition(1) = 1
        Case "8"
            xposition(1) = 0
    End Select
    board(xposition(0), yposition(0)) = board(xposition(1), yposition(1))
    count = count - 1
    board(xposition(1), yposition(1)) = Premovepiecevalue(count)

    showpiece()
    y = y - 2
    x = y - 2
End Sub
Private Sub Button1_Click(sender As Object, e As EventArgs)
    MovePiece()
End Sub

```

```

Dim y As Integer
Private Sub BackMoveButton_Click(sender As Object, e As EventArgs)
    MovePieceBack()
End Sub

Private Sub ReplaySelectBox_SelectedIndexChanged(sender As Object, e As
EventArgs) Handles ReplaySelectBox.SelectedIndexChanged
    Select Case ReplaySelectBox.SelectedItem
        Case "Replay 1"
            ReplayNumber = 1
            LoadReplay()
        Case "Replay 2"
            ReplayNumber = 2
            LoadReplay()
        Case "Replay 3"
            ReplayNumber = 3
            LoadReplay()
        Case "Replay 4"
            ReplayNumber = 4
            LoadReplay()
        Case "Replay 5"
            ReplayNumber = 5
            LoadReplay()
    End Select

    If PreMove <> "" And PostMove <> "" Then
        ReplaySelectBox.Hide()
        ChangeReplayButton.Show()
        PlayButton.Show()
        FastFowardButton.Show()
        PauseButton.Show()
    Else
        MessageBox.Show("Replay not found, please try a different one")
    End If
End Sub

Private Sub ChangeReplayButton_Click(sender As Object, e As EventArgs)
Handles ChangeReplayButton.Click
    'Hiding buttons
    ChangeReplayButton.Hide()
    PlayButton.Hide()
    FastFowardButton.Hide()
    PauseButton.Hide()
    RewindButton.Hide()
    AutoPlayTimer.Stop()
    FastFowardTimer.Stop()
    ReverseTimer.Stop()
    ReplaySelectBox.Show()
    'Make sure board is back to start
    board = {{-5, -4, -3, -2, -1, -3, -4, -5},
             {-6, -6, -6, -6, -6, -6, -6, -6},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {6, 6, 6, 6, 6, 6, 6, 6}}

```



```

        {5, 4, 3, 2, 1, 3, 4, 5}}
    'Displaying pieces on board
    showpiece()
    'Reset x and y
    x = 0
    y = 0
End Sub

Private Sub ReturnLabel_Click(sender As Object, e As EventArgs)
    'Show main menu form
    MainMenu.Show()
    'Close current form
    Me.Close()
End Sub

Private Sub CloseProgramLabel_Click(sender As Object, e As EventArgs)
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'end program
    End
    End If
End Sub

Private Sub PlayButton_Click(sender As Object, e As EventArgs) Handles
PlayButton.Click
    AutoPlayTimer.Start()
    ReverseTimer.Stop()
    FastFowardTimer.Stop()
End Sub

Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles
AutoPlayTimer.Tick
    If x < gamelength Then
        MovePiece()
    ElseIf x >= gamelength Then
        AutoPlayTimer.Stop()
    End If
End Sub

Private Sub PauseButton_Click(sender As Object, e As EventArgs) Handles
PauseButton.Click
    AutoPlayTimer.Stop()
    ReverseTimer.Stop()
    FastFowardTimer.Stop()
End Sub

Private Sub RewindButton_Click(sender As Object, e As EventArgs) Handles
RewindButton.Click
    AutoPlayTimer.Stop()
    FastFowardTimer.Stop()
    ReverseTimer.Start()

```

```

        PauseButton.Show()
        PlayButton.Show()
        FastFowardButton.Show()
    End Sub

    Private Sub Reverse_Tick(sender As Object, e As EventArgs) Handles
ReverseTimer.Tick
        If y > 0 Then
            MovePieceBack()
        ElseIf y <= 0 Then
            ReverseTimer.Stop()
            RewindButton.Hide()

        End If
    End Sub

    Private Sub FastFowardButton_Click(sender As Object, e As EventArgs)
Handles FastFowardButton.Click
        AutoPlayTimer.Stop()
        ReverseTimer.Stop()
        FastFowardTimer.Start()
    End Sub

    Private Sub FastFowardTimer_Tick(sender As Object, e As EventArgs)
Handles FastFowardTimer.Tick
        If x < gamelength Then
            MovePiece()
        ElseIf x >= gamelength Then
            FastFowardTimer.Stop()
        End If
    End Sub
End Class

```

Test Plan

Loading Replay

Description: testing the form loading the correct moves onto the form depending on the users replay selection.

Replay number	Moves inside replay number	Data Type	Expected Outcome
2	A2A4	Normal	Replay Loaded
9	Nothing	Invalid	Invalid replay number
1	B1D3	Borderline	Replay Loaded

7 - Leaderboard

Summary: correctly load and display an ordered leaderboard.

Research

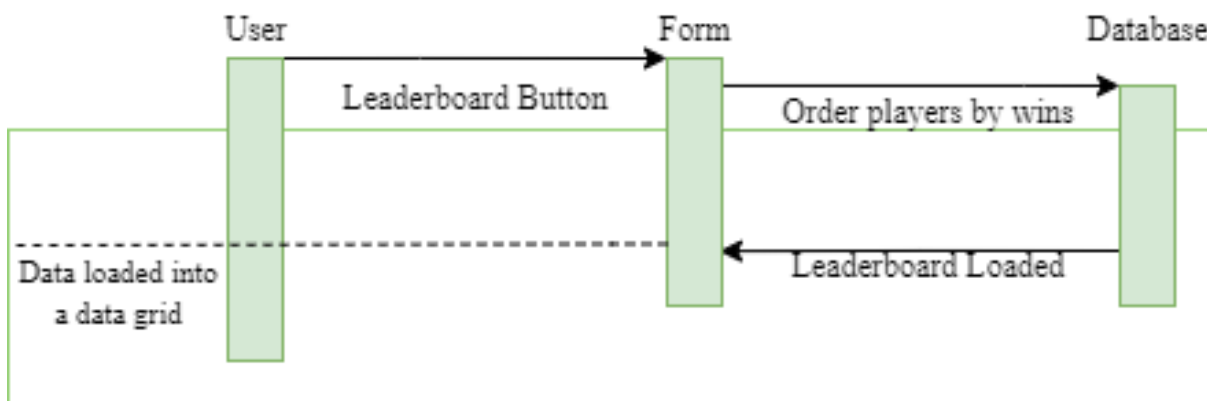
Leaderboards are a way to give any game a competitive aspect to any game. I have referenced numerous times to why leaderboards give benefits to the program in the analysis section. They are relatively simple to implement and give players a work towards within the game, with the focus on giving motivation to beat your opponent.



Rank	Player Name	Score
1.	AliceFox	342.134
2.	ZikZak	342.134
3.	Cherry	342.134
4.	RUDI	342.134
5.	AliceFox	342.134
6.	ZikZak	342.134
7.	Cherry	342.134
8.	RUDI	342.134
9.	AliceFox	342.134
10.	ZikZak	342.134

Example of a Leader board within an existing game

UML



Pseudo Code

N/A - the reason why there is no pseudo code written for this section is because to gather the data from the leaderboard and order it, the program uses power of access and SQL to do both commands in one statement, therefore there is not much pseudo code available to do for this section.

```
sql = "SELECT [Username],[Wins],[Draws],[Loses], [ProfilePicture] FROM
[Table1] ORDER BY [Wins] DESC" 'SQL Statement, instead of using bubble sort,
use order by function within the database
```

VB.Net Code

```
Imports System.Data
Imports System.Data.OleDb
Public Class LeaderboardForm
    Private Sub LeaderboardForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins],[Draws],[Loses], [ProfilePicture]
FROM [Table1] ORDER BY [Wins] DESC" 'SQL Statement, instead of using bubble
sort, use order by function within the database
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection

        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        'Population of datagrid
        LeaderboardGrid.DataSource = ds.Tables("Table1")
        'Identifying top 3 places
        FirstLabel.Text = ds.Tables(0).Rows(0).Item(0)
        SecondLabel.Text = ds.Tables(0).Rows(1).Item(0)
        ThirdLabel.Text = ds.Tables(0).Rows(2).Item(0)
        'Profile pictures for top 3
        Dim SkinType(2) As String
        SkinType(0) = ds.Tables(0).Rows(0).Item(4)
        SkinType(1) = ds.Tables(0).Rows(1).Item(4)
        SkinType(2) = ds.Tables(0).Rows(2).Item(4)

        Select Case SkinType(0)
            Case "Default"
                FirstPlaceBox.Image = My.Resources.DefaultSkin
            Case "Skin1"
                FirstPlaceBox.Image = My.Resources.Skin1
```

```

Case "Skin2"
    FirstPlaceBox.Image = My.Resources.Skin2
Case "Skin3"
    FirstPlaceBox.Image = My.Resources.Skin3
Case "Skin4"
    FirstPlaceBox.Image = My.Resources.Skin4
Case "Skin5"
    FirstPlaceBox.Image = My.Resources.Skin5
Case "Skin6"
    FirstPlaceBox.Image = My.Resources.Skin6
Case "Skin7"
    FirstPlaceBox.Image = My.Resources.Skin7
Case "Locked1"
    FirstPlaceBox.Image = My.Resources.Locked1
Case "Locked2"
    FirstPlaceBox.Image = My.Resources.Locked2
Case "Locked3"
    FirstPlaceBox.Image = My.Resources.Locked3
Case "Locked4"
    FirstPlaceBox.Image = My.Resources.Locked4
Case "Locked5"
    FirstPlaceBox.Image = My.Resources.Locked5
Case "Locked6"
    FirstPlaceBox.Image = My.Resources.Locked6
Case "Locked7"
    FirstPlaceBox.Image = My.Resources.Locked7
Case "Locked8"
    FirstPlaceBox.Image = My.Resources.Locked8
End Select
Select Case SkinType (1)
Case "Default"
    SecondPlaceBox.Image = My.Resources.DefaultSkin
Case "Skin1"
    SecondPlaceBox.Image = My.Resources.Skin1
Case "Skin2"
    SecondPlaceBox.Image = My.Resources.Skin2
Case "Skin3"
    SecondPlaceBox.Image = My.Resources.Skin3
Case "Skin4"
    SecondPlaceBox.Image = My.Resources.Skin4
Case "Skin5"
    SecondPlaceBox.Image = My.Resources.Skin5
Case "Skin6"
    SecondPlaceBox.Image = My.Resources.Skin6
Case "Skin7"
    SecondPlaceBox.Image = My.Resources.Skin7
Case "Locked1"
    SecondPlaceBox.Image = My.Resources.Locked1
Case "Locked2"
    SecondPlaceBox.Image = My.Resources.Locked2
Case "Locked3"
    SecondPlaceBox.Image = My.Resources.Locked3
Case "Locked4"
    SecondPlaceBox.Image = My.Resources.Locked4
Case "Locked5"
    SecondPlaceBox.Image = My.Resources.Locked5
Case "Locked6"

```

```

        SecondPlaceBox.Image = My.Resources.Locked6
    Case "Locked7"
        SecondPlaceBox.Image = My.Resources.Locked7
    Case "Locked8"
        SecondPlaceBox.Image = My.Resources.Locked8
End Select
Select Case SkinType (2)
    Case "Default"
        ThirdPlaceBox.Image = My.Resources.DefaultSkin
    Case "Skin1"
        ThirdPlaceBox.Image = My.Resources.Skin1
    Case "Skin2"
        ThirdPlaceBox.Image = My.Resources.Skin2
    Case "Skin3"
        ThirdPlaceBox.Image = My.Resources.Skin3
    Case "Skin4"
        ThirdPlaceBox.Image = My.Resources.Skin4
    Case "Skin5"
        ThirdPlaceBox.Image = My.Resources.Skin5
    Case "Skin6"
        ThirdPlaceBox.Image = My.Resources.Skin6
    Case "Skin7"
        ThirdPlaceBox.Image = My.Resources.Skin7
    Case "Locked1"
        ThirdPlaceBox.Image = My.Resources.Locked1
    Case "Locked2"
        ThirdPlaceBox.Image = My.Resources.Locked2
    Case "Locked3"
        ThirdPlaceBox.Image = My.Resources.Locked3
    Case "Locked4"
        ThirdPlaceBox.Image = My.Resources.Locked4
    Case "Locked5"
        ThirdPlaceBox.Image = My.Resources.Locked5
    Case "Locked6"
        ThirdPlaceBox.Image = My.Resources.Locked6
    Case "Locked7"
        ThirdPlaceBox.Image = My.Resources.Locked7
    Case "Locked8"
        ThirdPlaceBox.Image = My.Resources.Locked8
End Select
cn.Close()
End Sub
End Class

```

Test Plan

Description: testing to see whether the database works correctly when ordering the users

Username	Wins	Data Type	Expected Outcome
Admin, Daniel, Tester	3,5,9	Normal	Tester, Daniel, Admin
N/A	N/A	Invalid	N/A
N/A	N/A	Borderline	N/A

8 - Customisation

Summary: load in the current users' customisations from the database. Allow the user to change the customisable items on the form, each time it is change update it in the database.

Research

Document (<https://www.gamedeveloper.com/design/how-personalization-improves-video-games>)

Customisation is added into games for the user-friendly side of the project. It is used for a player to create their own stamp on the game. In my project the features the user can change is their profile picture, board colour and piece type.

Document (https://www.w3schools.com/sql/sql_update.asp)

SQL UPDATE Statement

[< Previous](#)

The SQL UPDATE Statement

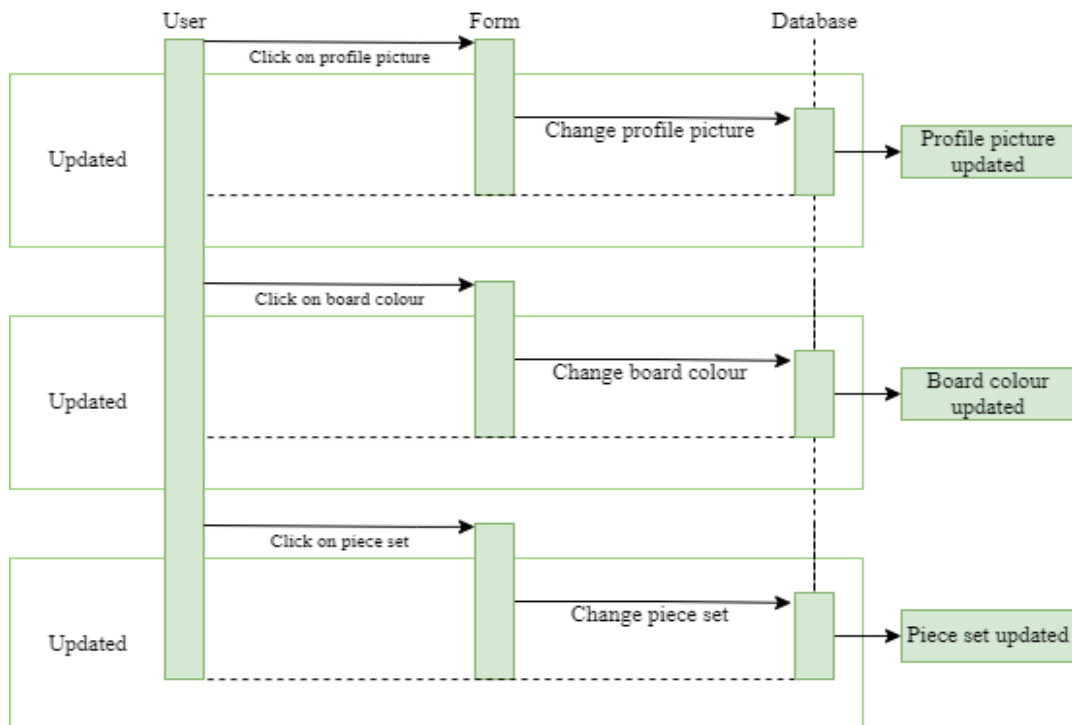
The **UPDATE** statement is used to modify the existing records in a table.

UPDATE Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

SQL Update statement will be used whenever the user is updating their customisation choices into the database.

UML



Pseudo Code

N/A

VB.Net Code

```
Imports System.Data.OleDb
Imports System.Data
Public Class CustomisationForm
    Dim Skinname As String
    Dim BoardColour As String
    Dim PieceType As String
    Private Sub CustomisationForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        StatsCalculator()
        'As stats have been calculated, check if any unlockable profile
pictures can be chosen
        If Winstxt.Text >= 1 Then
            'Making image available
            Locked1Box.Enabled = True
            Locked1Box.Image = Nothing
            'Editing the text
            U1WinLabel.ForeColor = Color.Green
            U1WinLabel.Text = "Unlocked"
        End If

        If Winstxt.Text >= 2 Then
            'Making images available
            Locked2Box.Enabled = True
            Locked2Box.Image = Nothing
            'Editing the text
            U2WinLabel.ForeColor = Color.Green
            U2WinLabel.Text = "Unlocked"
        End If

        If Winstxt.Text >= 3 Then
            'Making images available
            Locked3Box.Enabled = True
            Locked3Box.Image = Nothing
            'Editing the text
            U3WinLabel.ForeColor = Color.Green
            U3WinLabel.Text = "Unlocked"
        End If

        If Winstxt.Text >= 4 Then
            'Making images available
            Locked4Box.Enabled = True
            Locked4Box.Image = Nothing
            'Editing the text
            U4WinLabel.ForeColor = Color.Green
            U4WinLabel.Text = "Unlocked"
        End If

        If Winstxt.Text >= 5 Then
            'Making images available
            Locked5Box.Enabled = True
            Locked5Box.Image = Nothing
            'Editing the text
            U5WinLabel.ForeColor = Color.Green
        End If
    End Sub
End Class
```



```

        U5WinLabel.Text = "Unlocked"
    End If

    If Winstxt.Text >= 6 Then
        'Making images available
        Locked6Box.Enabled = True
        Locked6Box.Image = Nothing
        'Editing the text
        U6WinLabel.ForeColor = Color.Green
        U6WinLabel.Text = "Unlocked"
    End If

    If Winstxt.Text >= 7 Then
        'Making images available
        Locked7Box.Enabled = True
        Locked7Box.Image = Nothing
        'Editing the text
        U7WinLabel.ForeColor = Color.Green
        U7WinLabel.Text = "Unlocked"
    End If

    If Winstxt.Text >= 8 Then
        'Making images available
        Locked8Box.Enabled = True
        Locked8Box.Image = Nothing
        'Editing the text
        U8WinLabel.ForeColor = Color.Green
        U8WinLabel.Text = "Unlocked"
    End If

    'Load current board and piece selection
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[BoardColour], [PieceType], [ProfilePicture]
FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 profile picture (for a user that has signed in)

```

```

BoardColour = ds.Tables(0).Rows(count).Item(1)
PieceType = ds.Tables(0).Rows(count).Item(2)
Select Case (ds.Tables(0).Rows(count).Item(1))
    Case "YellowBoard"
        SelectedBoardBox.Image =
My.Resources.YellowBoardPreview
    Case "GreenBoard"
        SelectedBoardBox.Image =
My.Resources.GreenBoardPreview
    Case "PinkBoard"
        SelectedBoardBox.Image =
My.Resources.PinkBoardPreview
    Case "RedBoard"
        SelectedBoardBox.Image = My.Resources.RedBoardPreview
    Case "BlueBoard"
        SelectedBoardBox.Image =
My.Resources.BlueBoardPreview
End Select
Select Case (ds.Tables(0).Rows(count).Item(2))
    Case "Set1"
        SelectedPieceBox.Image = My.Resources.Set1KnightW
    Case "Set2"
        SelectedPieceBox.Image = My.Resources.Set2KnightW
    Case "Set3"
        SelectedPieceBox.Image = My.Resources.Set3KnightW
End Select
Select Case (ds.Tables(0).Rows(count).Item(3))
    Case "Default"
        MainMenu.ProfilePicture.Image =
My.Resources.DefaultSkin
    Case "Skin1"
        MainMenu.ProfilePicture.Image = My.Resources.Skin1
    Case "Skin2"
        MainMenu.ProfilePicture.Image = My.Resources.Skin2
    Case "Skin3"
        MainMenu.ProfilePicture.Image = My.Resources.Skin3
    Case "Skin4"
        MainMenu.ProfilePicture.Image = My.Resources.Skin4
    Case "Skin5"
        MainMenu.ProfilePicture.Image = My.Resources.Skin5
    Case "Skin6"
        MainMenu.ProfilePicture.Image = My.Resources.Skin6
    Case "Skin7"
        MainMenu.ProfilePicture.Image = My.Resources.Skin7
    Case "Locked1"
        MainMenu.ProfilePicture.Image = My.Resources.Locked1
    Case "Locked2"
        MainMenu.ProfilePicture.Image = My.Resources.Locked2
    Case "Locked3"
        MainMenu.ProfilePicture.Image = My.Resources.Locked3
    Case "Locked4"
        MainMenu.ProfilePicture.Image = My.Resources.Locked4
    Case "Locked5"
        MainMenu.ProfilePicture.Image = My.Resources.Locked5
    Case "Locked6"
        MainMenu.ProfilePicture.Image = My.Resources.Locked6
    Case "Locked7"

```

```

        MainMenu.ProfilePicture.Image = My.Resources.Locked7
    Case "Locked8"
        MainMenu.ProfilePicture.Image = My.Resources.Locked8
    End Select
End If
Next
cn.Close() 'Close connection
'As stats have been calculated, check if any unlockable boards or
piece sets are available
If Winstxt.Text >= 1 Then
    RedBoardBox.Enabled = True
    RedBoardBox.Image = Nothing
    'Editing the text
    U1WinLabel2.ForeColor = Color.Green
    U1WinLabel2.Text = "Unlocked"
    Set2Box.Enabled = True
    Set2Box.Image = Nothing
    'Editing the text
    U1WinLabel3.ForeColor = Color.Green
    U1WinLabel3.Text = "Unlocked"
End If

If Winstxt.Text >= 2 Then
    BlueBoardBox.Enabled = True
    BlueBoardBox.Image = Nothing
    'Editing the text
    U2WinLabel2.ForeColor = Color.Green
    U2WinLabel2.Text = "Unlocked"
    Set3Box.Enabled = True
    Set3Box.Image = Nothing
    'Editing the text
    U2WinLabel3.ForeColor = Color.Green
    U2WinLabel3.Text = "Unlocked"
End If

If Winstxt.Text >= 3 Then
    PinkBoardBox.Enabled = True
    PinkBoardBox.Image = Nothing
    'Editing the text
    U3WinLabel2.ForeColor = Color.Green
    U3WinLabel2.Text = "Unlocked"
End If
End Sub
Private Sub LoadIntoDB()

    'Updating profile picture in database
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    cn.Open() 'Open connection

```

```

    Dim sql As String
    sql = "update [Table1]"
    sql = sql & "SET [ProfilePicture] = '" & Skinname & "', "
    sql = sql & "[BoardColour] = '" & BoardColour & "', "
    sql = sql & "[PieceType] = '" & PieceType & "' "
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"

    Dim cmd As New OleDbCommand(sql, cn)

    cmd.ExecuteNonQuery()
    cn.Close() 'Close connection
End Sub
Private Sub StatsCalculator()
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[Wins],[Draws],[Loses] FROM [Table1]" 'SQL
Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats
            Winstxt.Text = ds.Tables(0).Rows(count).Item(1)
            Drawstxt.Text = ds.Tables(0).Rows(count).Item(2)
            Losestxt.Text = ds.Tables(0).Rows(count).Item(3)
        End If
    Next
    cn.Close()
    'Creating the Win %
    WinPercentagetxt.Text = (Int((CInt(Winstxt.Text) /
(CInt(Winstxt.Text) + CInt(Drawstxt.Text) + CInt(Losestxt.Text)) * 100)))
'Used append text to make form more user friendly and not highlight text from
the get-go
End Sub
Private Sub BoxClicked(sender As Object, e As EventArgs) Handles
DefaultSkinBox.Click
    MainMenu.ProfilePicture.Image = DefaultSkinBox.Image
    Skinname = "Default"
    LoadIntoDB()
End Sub

```

```

    Private Sub Skin1Box_Click(sender As Object, e As EventArgs) Handles
Skin1Box.Click
        MainMenu.ProfilePicture.Image = Skin1Box.Image
        Skinname = "Skin1"
        LoadIntoDB ()
    End Sub
    Private Sub Skin2Box_Click(sender As Object, e As EventArgs) Handles
Skin2Box.Click
        MainMenu.ProfilePicture.Image = Skin2Box.Image
        Skinname = "Skin2"
        LoadIntoDB ()
    End Sub
    Private Sub Skin3Box_Click(sender As Object, e As EventArgs) Handles
Skin3Box.Click
        MainMenu.ProfilePicture.Image = Skin3Box.Image
        Skinname = "Skin3"
        LoadIntoDB ()
    End Sub
    Private Sub Skin4Box_Click(sender As Object, e As EventArgs) Handles
Skin4Box.Click
        MainMenu.ProfilePicture.Image = Skin4Box.Image

        Skinname = "Skin4"
        LoadIntoDB ()
    End Sub
    Private Sub Skin5Box_Click(sender As Object, e As EventArgs) Handles
Skin5Box.Click
        MainMenu.ProfilePicture.Image = Skin5Box.Image
        Skinname = "Skin5"
        LoadIntoDB ()
    End Sub
    Private Sub Skin6Box_Click(sender As Object, e As EventArgs) Handles
Skin6Box.Click
        MainMenu.ProfilePicture.Image = Skin6Box.Image
        Skinname = "Skin6"
        LoadIntoDB ()
    End Sub
    Private Sub Skin7Box_Click(sender As Object, e As EventArgs) Handles
Skin7Box.Click
        MainMenu.ProfilePicture.Image = Skin7Box.Image
        Skinname = "Skin7"
        LoadIntoDB ()
    End Sub
    Private Sub Locked1Box_Click(sender As Object, e As EventArgs) Handles
Locked1Box.Click
        MainMenu.ProfilePicture.Image = Locked1Box.BackgroundImage
        Skinname = "Locked1"
        LoadIntoDB ()
    End Sub

    Private Sub Locked2Box_Click(sender As Object, e As EventArgs) Handles
Locked2Box.Click
        MainMenu.ProfilePicture.Image = Locked2Box.BackgroundImage
        Skinname = "Locked2"
        LoadIntoDB ()
    End Sub

```

```

    Private Sub Locked3Box_Click(sender As Object, e As EventArgs) Handles
Locked3Box.Click
        MainMenu.ProfilePicture.Image = Locked3Box.BackgroundImage
        Skinname = "Locked3"
        LoadIntoDB ()
    End Sub
    Private Sub Locked4Box_Click(sender As Object, e As EventArgs) Handles
Locked4Box.Click
        MainMenu.ProfilePicture.Image = Locked4Box.BackgroundImage
        Skinname = "Locked4"
        LoadIntoDB ()
    End Sub
    Private Sub Locked5Box_Click(sender As Object, e As EventArgs) Handles
Locked5Box.Click
        MainMenu.ProfilePicture.Image = Locked5Box.BackgroundImage
        Skinname = "Locked5"
        LoadIntoDB ()
    End Sub
    Private Sub Locked6Box_Click(sender As Object, e As EventArgs) Handles
Locked6Box.Click
        MainMenu.ProfilePicture.Image = Locked6Box.BackgroundImage
        Skinname = "Locked6"
        LoadIntoDB ()
    End Sub
    Private Sub Locked7Box_Click(sender As Object, e As EventArgs) Handles
Locked7Box.Click
        MainMenu.ProfilePicture.Image = Locked7Box.BackgroundImage
        Skinname = "Locked7"
        LoadIntoDB ()
    End Sub
    Private Sub Locked8Box_Click(sender As Object, e As EventArgs) Handles
Locked8Box.Click
        MainMenu.ProfilePicture.Image = Locked8Box.BackgroundImage
        Skinname = "Locked8"
        LoadIntoDB ()
    End Sub
    Private Sub YellowBoardBox_Click(sender As Object, e As EventArgs)
Handles YellowBoardBox.Click
        SelectedBoardBox.Image = My.Resources.YellowBoardPreview
        BoardColour = "YellowBoard"
        LoadIntoDB ()
    End Sub
    Private Sub GreenBoardBox_Click(sender As Object, e As EventArgs) Handles
GreenBoardBox.Click
        SelectedBoardBox.Image = My.Resources.GreenBoardPreview
        BoardColour = "GreenBoard"
        LoadIntoDB ()
    End Sub
    Private Sub PinkBoardBox_Click(sender As Object, e As EventArgs) Handles
PinkBoardBox.Click
        SelectedBoardBox.Image = My.Resources.PinkBoardPreview
        BoardColour = "PinkBoard"
        LoadIntoDB ()
    End Sub
    Private Sub RedBoardBox_Click(sender As Object, e As EventArgs) Handles
RedBoardBox.Click
        SelectedBoardBox.Image = My.Resources.RedBoardPreview

```

```

        BoardColour = "RedBoard"
        LoadIntoDB ()
    End Sub
    Private Sub BlueBoardBox_Click(sender As Object, e As EventArgs) Handles
BlueBoardBox.Click
        SelectedBoardBox.Image = My.Resources.BlueBoardPreview
        BoardColour = "BlueBoard"
        LoadIntoDB ()
    End Sub
    Private Sub Set1Box_Click(sender As Object, e As EventArgs) Handles
Set1Box.Click
        SelectedPieceBox.Image = My.Resources.Set1KnightW
        PieceType = "Set1"
        LoadIntoDB ()
    End Sub
    Private Sub Set2Box_Click(sender As Object, e As EventArgs) Handles
Set2Box.Click
        SelectedPieceBox.Image = My.Resources.Set2KnightW
        PieceType = "Set2"
        LoadIntoDB ()
    End Sub
    Private Sub Set3Box_Click(sender As Object, e As EventArgs) Handles
Set3Box.Click
        SelectedPieceBox.Image = My.Resources.Set3KnightW
        PieceType = "Set3"
        LoadIntoDB ()
    End Sub
End Class

```

Test Plan

Description: testing whether or not the user's customisations update on the form and in the database

Profile picture (user has 6 wins)

Test Data	Data Type	Expected Outcome
Locked3	Normal	Profile picture changed
Locked6	Borderline	Profile picture changed
Locked7	Invalid	Profile picture unchanged (Not unlocked)

Board colour (user has 2 wins)

Test Data	Data Type	Expected Outcome
Red board	Normal	Board colour changed
Blue board	Borderline	Board colour changed
Pink board	Invalid	Board colour unchanged (Not unlocked)

Piece set (user has 1 win)

Test Data	Data Type	Expected Outcome
Set1	Normal	Piece set changed
Set2	Borderline	Piece set changed
Set3	Invalid	Piece set unchanged (Not unlocked)

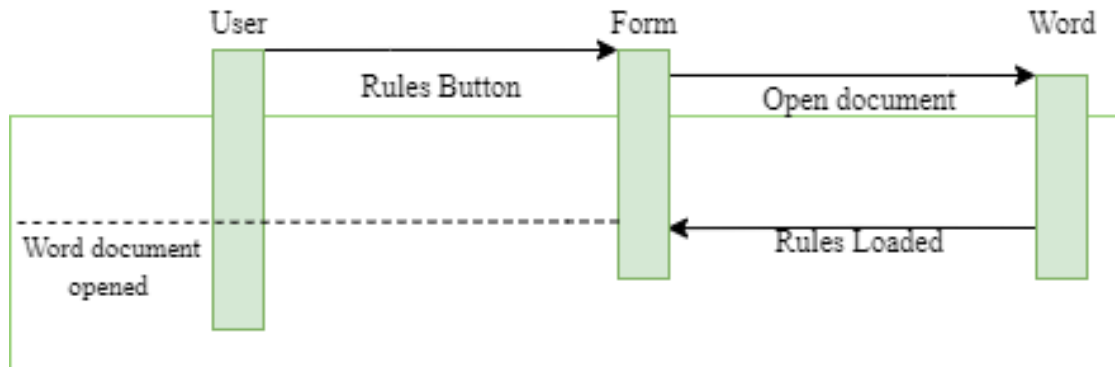
9 - Rules

Summary: correctly open a word document that includes the correct rules of chess.

Research

Rules are easy to implement into the game. If a user wants to view the rules within my program, the program will open a pre-written word document (which the user cannot edit). The website used to refer to when writing the rules was (https://en.wikipedia.org/wiki/Rules_of_chess).

UML



Pseudo Code

N/A

VB.Net Code

```

Imports Microsoft.Office.Interop
Public Class RulesForm
    Public wordapp As Word.Application

    Private Sub RulesButton_Click(sender As Object, e As EventArgs) Handles
RulesButton.Click
        'Don't want more than 1 form open at a time
        If wordapp IsNot Nothing Then
            wordapp.Quit()
        End If
        'Open rules document
        'creating rules document again as error would otherwise occur when
opening the rules document, if once closed.
        wordapp = CreateObject("word.application")
        'making word document visible
        wordapp.Visible = True
        'opening already existing document, as then can make it read only so
user cannot edit it

        wordapp.Documents.Open(System.Windows.Forms.Application.StartupPath &
"\RulesDocument.docx")
    End Sub
End Class

```

Test Plan

N/A

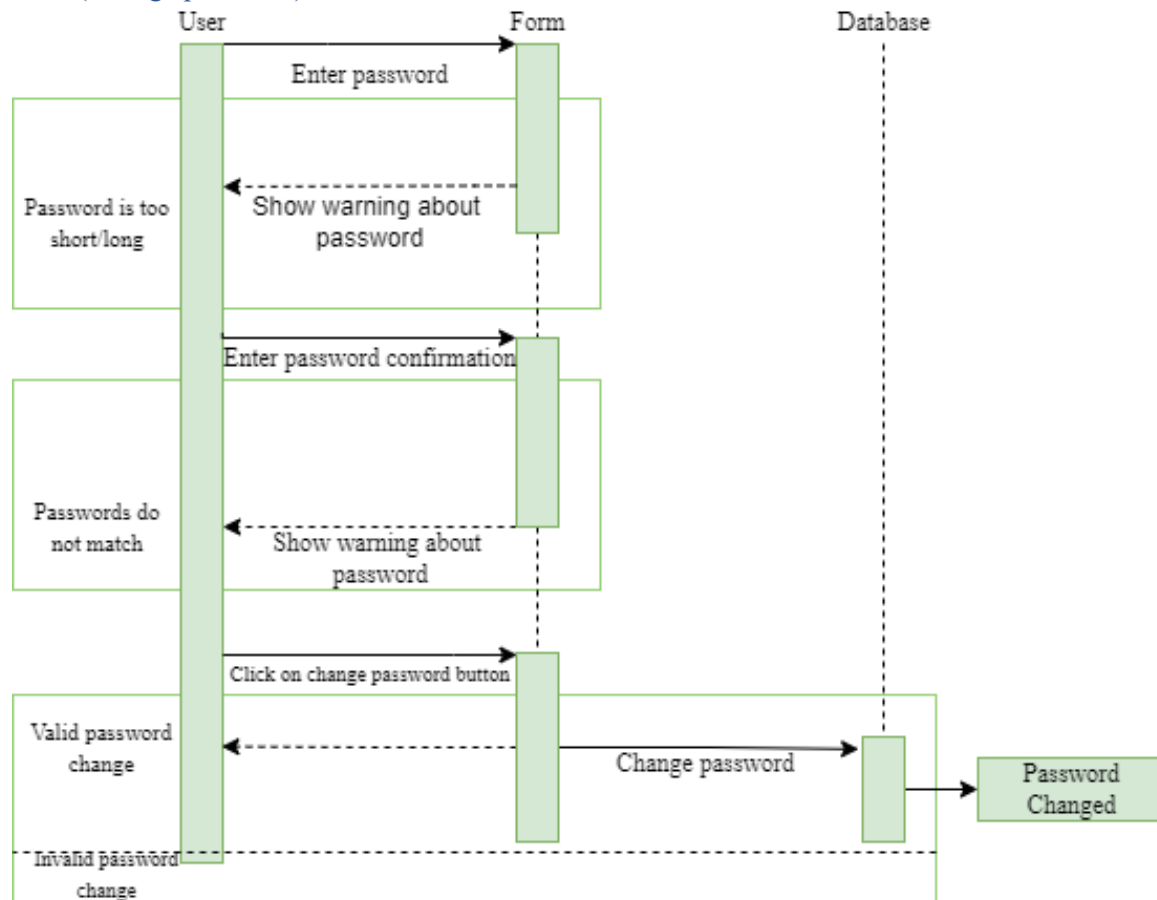
10 - Settings

Summary: allows the user to change their password and delete their account.

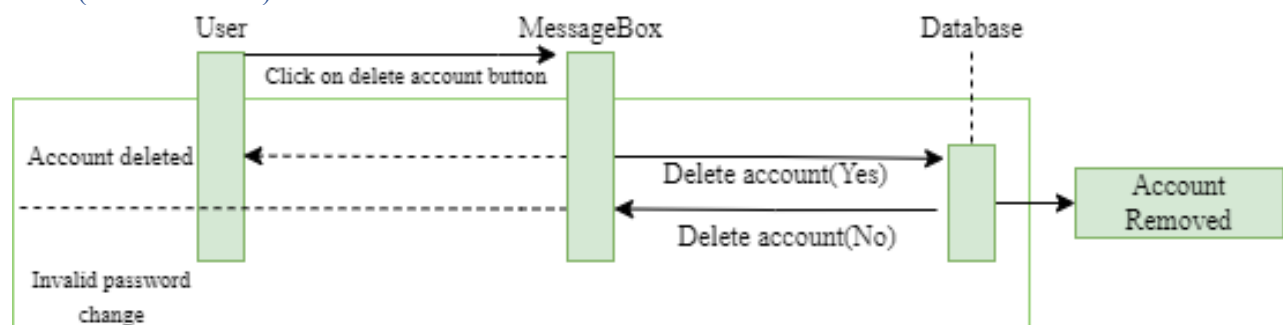
Research

Having a settings section in all games is a necessity. Every game I researched about had a section dedicated to having user settings. The only reason my program has a settings option is for the user, not the performance of the game like most existing games. Consequently, there is not much research available for this section because of the limitations of my project.

UML(Change password)



UML(Delete account)



Pseudo Code

```
password <-- userinput
confirmpassword <-- userinput

If password = "" Then
    Passwordvalid = False
ElseIf confirmpassword <> password Then
    Passwordvalid = False
ElseIf Len(password) <= 30 And Len(password) >= 5 Then
    Passwordvalid = True
Endif

If Passwordvalid = True Then
    Table1.password <-- password
    Output("Password changed successfully")
Else
    Output("Invalid password")
Endif
```

Pseudo Code(Delete account)

N/A

VB.Net Code

```
Imports System.Data.OleDb 'Importing to allow vb to connect to access
Imports System.Data
Public Class SettingsForm
    Dim PasswordIsValid As Boolean = True 'Variable which determines whether
or not the user can change password
    Private Sub SettingsForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        'Setting text box input
        Me.Passwordtxt.PasswordChar = "*"
        Me.ConfirmPasswordtxt.PasswordChar = "*"
    End Sub
    Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
Handles ShowPassword_CheckedChanged
        If Me.ShowPassword.Checked = True Then
            Me.Passwordtxt.PasswordChar = "" 'If user ticks the show password
box, show password
            Me.ConfirmPasswordtxt.PasswordChar = ""
        Else
            Me.Passwordtxt.PasswordChar = "*" 'If box is unticked turn
password into *
            Me.ConfirmPasswordtxt.PasswordChar = "*"
        End If
    End Sub
    Private Sub ChangePasswordButton_Click(sender As Object, e As EventArgs)
Handles ChangePasswordButton.Click
        If Passwordtxt.Text = "" Then 'Invalid
            MessageBox.Show("Please enter a new password")
            PasswordIsValid = False
        ElseIf ConfirmPasswordtxt.Text = "" Then 'Invalid
            MessageBox.Show("Please confirm your new password")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) < 5 Then 'Invalid
```

```

        MessageBox.Show("Password is too short. Please make it between 5
and 30 characters")
        PasswordIsValid = False
    ElseIf Len(Passwordtxt.Text) > 30 Then 'Invalid
        MessageBox.Show("Password is too long. Please make it between 5
and 30 characters")
        PasswordIsValid = False
    ElseIf Passwordtxt.Text <> ConfirmPasswordtxt.Text Then 'Invalid
        MessageBox.Show("Passwords do not match up")
        PasswordIsValid = False
    ElseIf Len(Passwordtxt.Text) >= 5 And Len(Passwordtxt.Text) <= 30 And
Passwordtxt.Text = ConfirmPasswordtxt.Text Then 'Valid
        PasswordIsValid = True
    End If

    If PasswordIsValid = True Then
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        'displaying messagebox
        result = MessageBox.Show("Are you sure you want to change your
password?", "Change Password Confirmation", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            'Update password
            'Setting database location
            Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
            'Opening connection
            Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;"
'OLE provider for MS Access
            Dim DBsource As String = "Data Source=" & dbfilename

            Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
            cn.Open() 'Open connection
            Dim sql As String
            sql = "update [Table1]"
            sql = sql & "SET [Password] = '" & Passwordtxt.Text & "'"
            sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
            Dim cmd As New OleDbCommand(sql, cn)
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection
            MessageBox.Show("Password Changed")
            'Clearing text boxes
            Passwordtxt.Text = ""
            ConfirmPasswordtxt.Text = ""
        Else
            PasswordIsValid = False
        End If
    End If
End Sub

Private Sub DeleteAccountButton_Click(sender As Object, e As EventArgs)
Handles DeleteAccountButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox

```

```

        result = MessageBox.Show("Are you sure you want to delete your
account?", "Delete Account", button)
'getting result of the messagebox
If result = System.Windows.Forms.DialogResult.Yes Then
    'Update password
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
    cn.Open() 'Open connection
    Dim sql As String
    sql = "DELETE * FROM [Table1]"
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"

    Dim cmd As New OleDbCommand(sql, cn)
    cmd.ExecuteNonQuery()
    cn.Close() 'Close connection
    MessageBox.Show("Account Deleted")
    'Show Login form
    LoginForm.Show()
    'Close Current form
    MainMenu.Close()
End If
End Sub
End Class

```

Test Plan(Change Password)

Description: testing whether the user types in a valid password, and update it in the database

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	JUJ24d	Normal	Changed Password
JUJ24d	1234567	Invalid	Invalid Password (Don't match)
1234	1234	Invalid	Invalid Password (Too short)
12345	12345	Borderline	Changed Password

Test Plan(Delete Account)

Description: testing whether the user can delete their account

Test Data 1 - Confirm/Deny	Type of Test Data	Expected Outcome
Confirm	Normal	Account deleted
Deny	Normal	Account not deleted
N/A	Invalid	N/A
N/A	Borderline	N/A

Full VB.Net Source Code

Note - the only reference used in the project is the Microsoft Word 16.0 Object Library.

Forms:

- AccountSettings
- BPawnChange *(used in pawn promotion)*
- WPawnChange *(used in pawn promotion)*
- Customisation
- ForgottenPassword
- GameForm1 *(where the game is played)*
- Home
- Leaderboard
- MainMenu
- Multiplayer
- Play
- Replay
- Settle
- Sign-Up
- Singleplayer

Classes:

- Bishop
- Game
- King
- Knight
- Pawn
- Queen
- Rook

Forms

Account Settings

```
Imports System.Data.OleDb 'Importing to allow vb to connect to access
Imports System.Data
Public Class SettingsForm

    Dim PasswordIsValid As Boolean = True 'Variable which determines whether
    or not the user can change password

    Private Sub SettingsForm_Load(sender As Object, e As EventArgs) Handles
    MyBase.Load
        'Setting text box input
        Me.Passwordtxt.PasswordChar = "*"
        Me.ConfirmPasswordtxt.PasswordChar = "*"
    End Sub

    Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
    Handles ShowPassword.CheckedChanged
        If Me.ShowPassword.Checked = True Then
            Me.Passwordtxt.PasswordChar = "" 'If user ticks the show password
            box, show password
            Me.ConfirmPasswordtxt.PasswordChar = ""
        Else
            Me.Passwordtxt.PasswordChar = "*" 'If box is unticked turn
            password into *
            Me.ConfirmPasswordtxt.PasswordChar = "*"
        End If
    End Sub

    Private Sub ChangePasswordButton_Click(sender As Object, e As EventArgs)
    Handles ChangePasswordButton.Click
        If Passwordtxt.Text = "" Then 'Invalid
            MessageBox.Show("Please enter a new password")
            PasswordIsValid = False
        ElseIf ConfirmPasswordtxt.Text = "" Then 'Invalid
            MessageBox.Show("Please confirm your new password")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) < 5 Then 'Invalid
            MessageBox.Show("Password is too short. Please make it between 5
            and 30 characters")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) > 30 Then 'Invalid
            MessageBox.Show("Password is too long. Please make it between 5
            and 30 characters")
            PasswordIsValid = False
        ElseIf Passwordtxt.Text <> ConfirmPasswordtxt.Text Then 'Invalid
            MessageBox.Show("Passwords do not match up")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) >= 5 And Len(Passwordtxt.Text) <= 30 And
            Passwordtxt.Text = ConfirmPasswordtxt.Text Then 'Valid
            PasswordIsValid = True
        End If

        If PasswordIsValid = True Then
            Dim button As MessageBoxButtons = MessageBoxButtons.YesNo

```

```

        Dim result As DialogResult
        'displaying messagebox
        result = MessageBox.Show("Are you sure you want to change your
password?", "Change Password Confirmation", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            'Update password
            'Setting database location
            Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
            'Opening connection
            Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;"
'OLE provider for MS Access
            Dim DBsource As String = "Data Source=" & dbfilename

            Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
            cn.Open() 'Open connection
            Dim sql As String
            sql = "update [Table1]"
            sql = sql & "SET [Password] = '" & Passwordtxt.Text & "' "
            sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
            Dim cmd As New OleDbCommand(sql, cn)
            cmd.ExecuteNonQuery()
            cn.Close() 'Close connection
            MessageBox.Show("Password Changed")
            'Clearing text boxes
            Passwordtxt.Text = ""
            ConfirmPasswordtxt.Text = ""
        Else
            PasswordIsValid = False
        End If
    End If
End Sub

Private Sub DeleteAccountButton_Click(sender As Object, e As EventArgs)
Handles DeleteAccountButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want to delete your
account?", "Delete Account", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Update password
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename
        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        cn.Open() 'Open connection
        Dim sql As String
        sql = "DELETE * FROM [Table1]"

```

```

        sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";";

        Dim cmd As New OleDbCommand(sql, cn)
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection
        MessageBox.Show("Account Deleted")
        'Show Login form
        LoginForm.Show()
        'Close Current form
        MainMenu.Close()
    End If
End Sub
End Class

```

BPawnChange

```

Public Class BPawnChangeForm
    Private Sub BPawnChangeForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        'Pausing timers
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()
        'Displaying piece types
        If GameForm1.Set1 = True Then
            Me.QueenBox.Image = My.Resources.Set1QueenB
            Me.RookBox.Image = My.Resources.Set1RookB
            Me.KnightBox.Image = My.Resources.Set1KnightB
            Me.BishopBox.Image = My.Resources.Set1BishopB
        ElseIf GameForm1.Set2 = True Then
            Me.QueenBox.Image = My.Resources.Set2QueenB
            Me.RookBox.Image = My.Resources.Set2RookB
            Me.KnightBox.Image = My.Resources.Set2KnightB
            Me.BishopBox.Image = My.Resources.Set2BishopB
        ElseIf GameForm1.Set3 = True Then
            Me.QueenBox.Image = My.Resources.Set3QueenB
            Me.RookBox.Image = My.Resources.Set3RookB
            Me.KnightBox.Image = My.Resources.Set3KnightB
            Me.BishopBox.Image = My.Resources.Set3BishopB
        End If
    End Sub

    Private Sub QueenButton_Click(sender As Object, e As EventArgs) Handles
QueenButton.Click
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        result = MessageBox.Show("Are you sure you want to choose Queen",
"Queen", Button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            GameForm1.bqueencount += 1
            For i As Integer = 0 To 7
                If GameForm1.ga.board(7, i) = -6 Then
                    GameForm1.ga.board(7, i) = -2
                    GameForm1.blackqueen(GameForm1.bqueencount) = New
Queen(7, i, -2)
                End If
            Next i
            GameForm1.ga.show()
        End If
    End Sub
End Class

```



```

        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite()
    End If
Next
'Enabling gameform and timers
GameForm1.Enabled = True
If GameForm1.ga.movecounter Mod 2 = 0 Then
    GameForm1.P1Timer.Start()
Else
    GameForm1.P2Timer.Start()
End If
Me.Close()
End If
End Sub

Private Sub RookButton_Click(sender As Object, e As EventArgs) Handles
RookButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    result = MessageBox.Show("Are you sure you want to choose Rook",
"Rook", Button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.brookcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(7, i) = -6 Then
                GameForm1.ga.board(7, i) = -5
                GameForm1.blackrook(1 + GameForm1.brookcount) = New
Rook(7, i, -5)
                GameForm1.ga.show()
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite()
            End If
        Next
        'Enabling gameform and timers
        GameForm1.Enabled = True
        If GameForm1.ga.movecounter Mod 2 = 0 Then
            GameForm1.P1Timer.Start()
        Else
            GameForm1.P2Timer.Start()
        End If
        Me.Close()
    End If
End Sub

Private Sub KnightButton_Click(sender As Object, e As EventArgs) Handles
KnightButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    result = MessageBox.Show("Are you sure you want to choose Knight",
"Knight", Button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.bknightcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(7, i) = -6 Then
                GameForm1.ga.board(7, i) = -4

```

```

        GameForm1.blackknight(1 + GameForm1.bknightcount) = New
Knight(7, i, -4)
        GameForm1.ga.show()
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite()
    End If
Next
'Enabling gameform and timers
GameForm1.Enabled = True
If GameForm1.ga.movecounter Mod 2 = 0 Then
    GameForm1.P1Timer.Start()
Else
    GameForm1.P2Timer.Start()
End If
Me.Close()
End If
End Sub

Private Sub BishopButton_Click(sender As Object, e As EventArgs) Handles
BishopButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    result = MessageBox.Show("Are you sure you want to choose Bishop",
"Bishop", button)
    'getting result of the messagebox

    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.bbishopcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(7, i) = -6 Then
                GameForm1.ga.board(7, i) = -3
                GameForm1.blackbishop(1 + GameForm1.bbishopcount) = New
Bishop(7, i, -3)
                GameForm1.ga.show()
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite()
            End If
        Next
        'Enabling gameform and timers
        GameForm1.Enabled = True
        If GameForm1.ga.movecounter Mod 2 = 0 Then
            GameForm1.P1Timer.Start()
        Else
            GameForm1.P2Timer.Start()
        End If
        Me.Close()
    End If
End Sub
End Sub
End Class

```

WPawnChange

```

Public Class WPawnChangeForm
    Private Sub WPawnChangeForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        'Pausing timers
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()

        If GameForm1.Set1 = True Then
            Me.QueenBox.Image = My.Resources.Set1QueenW
            Me.RookBox.Image = My.Resources.Set1RookW
            Me.KnightBox.Image = My.Resources.Set1KnightW
            Me.BishopBox.Image = My.Resources.Set1BishopW
        ElseIf GameForm1.Set2 = True Then
            Me.QueenBox.Image = My.Resources.Set2QueenW
            Me.RookBox.Image = My.Resources.Set2RookW
            Me.KnightBox.Image = My.Resources.Set2KnightW
            Me.BishopBox.Image = My.Resources.Set2BishopW
        ElseIf GameForm1.Set3 = True Then
            Me.QueenBox.Image = My.Resources.Set3QueenW
            Me.RookBox.Image = My.Resources.Set3RookW
            Me.KnightBox.Image = My.Resources.Set3KnightW
            Me.BishopBox.Image = My.Resources.Set3BishopW
        End If
    End Sub

    Private Sub QueenButton_Click(sender As Object, e As EventArgs) Handles
QueenButton.Click
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        result = MessageBox.Show("Are you sure you want to choose Queen",
"Queen", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            GameForm1.wqueencount += 1
            For i As Integer = 0 To 7
                If GameForm1.ga.board(0, i) = 6 Then
                    GameForm1.ga.board(0, i) = 2
                    GameForm1.whitequeen(GameForm1.wqueencount) = New
Queen(0, i, 2)
                    GameForm1.ga.show()

                End If
            Next
            GameForm1.Enabled = True
            If GameForm1.ga.movecounter Mod 2 = 0 Then
                GameForm1.P1Timer.Start()
            Else
                GameForm1.P2Timer.Start()
            End If
            Me.Close()
        End If
    End Sub

    Private Sub RookButton_Click(sender As Object, e As EventArgs) Handles
RookButton.Click

```

```

    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    result = MessageBox.Show("Are you sure you want to choose Rook",
"Rook", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.wrookcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(0, i) = 6 Then
                GameForm1.ga.board(0, i) = 5
                GameForm1.whiterook(1 + GameForm1.wrookcount) = New
Rook(0, i, 5)
                GameForm1.ga.show()
            End If
        Next
        GameForm1.Enabled = True
        If GameForm1.ga.movecounter Mod 2 = 0 Then
            GameForm1.P1Timer.Start()
        Else
            GameForm1.P2Timer.Start()
        End If
        Me.Close()
    End If
End Sub

Private Sub KnightButton_Click(sender As Object, e As EventArgs) Handles
KnightButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    result = MessageBox.Show("Are you sure you want to choose Knight",
"Knight", Button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.wknightcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(0, i) = 6 Then
                GameForm1.ga.board(0, i) = 4
                GameForm1.whiteknight(1 + GameForm1.wknightcount) = New
Knight(0, i, 4)
                GameForm1.ga.show()
            End If
        Next
        GameForm1.Enabled = True
        If GameForm1.ga.movecounter Mod 2 = 0 Then
            GameForm1.P1Timer.Start()
        Else
            GameForm1.P2Timer.Start()
        End If
        Me.Close()
    End If
End Sub

Private Sub BishopButton_Click(sender As Object, e As EventArgs) Handles
BishopButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult

```

```

        result = MessageBox.Show("Are you sure you want to choose Bishop",
        "Bishop", Button)
        'getting result of the messagebox

    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.wbishopcount += 1
        For i As Integer = 0 To 7
            If GameForm1.ga.board(0, i) = 6 Then
                GameForm1.ga.board(0, i) = 3
                GameForm1.whitebishop(1 + GameForm1.wbishopcount) = New
                Bishop(0, i, 3)
                GameForm1.ga.show()
            End If
        Next
        GameForm1.Enabled = True
        If GameForm1.ga.movecounter Mod 2 = 0 Then
            GameForm1.P1Timer.Start()
        Else
            GameForm1.P2Timer.Start()
        End If
        Me.Close()
    End If
End Sub
End Class

```

Customisation

```

Imports System.Data.OleDb
Imports System.Data
Public Class CustomisationForm
    Dim Skinname As String
    Dim BoardColour As String
    Dim PieceType As String
    Private Sub CustomisationForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        StatsCalculator()
        'As stats have been calculated, check if any unlockable profile
pictures can be chosen
        If Winstxt.Text >= 1 Then
            'Making image available
            Locked1Box.Enabled = True
            Locked1Box.Image = Nothing
            'Editing the text
            U1WinLabel.ForeColor = Color.Green
            U1WinLabel.Text = "Unlocked"
        End If

        If Winstxt.Text >= 2 Then
            'Making images available
            Locked2Box.Enabled = True
            Locked2Box.Image = Nothing
            'Editing the text
            U2WinLabel.ForeColor = Color.Green
            U2WinLabel.Text = "Unlocked"
        End If
    End Sub
End Class

```

```

If Winstxt.Text >= 3 Then
    'Making images available
    Locked3Box.Enabled = True
    Locked3Box.Image = Nothing
    'Editing the text
    U3WinLabel.ForeColor = Color.Green
    U3WinLabel.Text = "Unlocked"
End If

If Winstxt.Text >= 4 Then
    'Making images available
    Locked4Box.Enabled = True
    Locked4Box.Image = Nothing
    'Editing the text
    U4WinLabel.ForeColor = Color.Green
    U4WinLabel.Text = "Unlocked"
End If

If Winstxt.Text >= 5 Then
    'Making images available
    Locked5Box.Enabled = True
    Locked5Box.Image = Nothing
    'Editing the text
    U5WinLabel.ForeColor = Color.Green
    U5WinLabel.Text = "Unlocked"
End If

If Winstxt.Text >= 6 Then
    'Making images available
    Locked6Box.Enabled = True
    Locked6Box.Image = Nothing
    'Editing the text
    U6WinLabel.ForeColor = Color.Green
    U6WinLabel.Text = "Unlocked"
End If

If Winstxt.Text >= 7 Then
    'Making images available
    Locked7Box.Enabled = True
    Locked7Box.Image = Nothing
    'Editing the text
    U7WinLabel.ForeColor = Color.Green
    U7WinLabel.Text = "Unlocked"
End If

If Winstxt.Text >= 8 Then
    'Making images available
    Locked8Box.Enabled = True
    Locked8Box.Image = Nothing
    'Editing the text
    U8WinLabel.ForeColor = Color.Green
    U8WinLabel.Text = "Unlocked"
End If

'Load current board and piece selection
'Setting database location

```

```

    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[BoardColour], [PieceType], [ProfilePicture]
FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 profile picture (for a user that has signed in)
            BoardColour = ds.Tables(0).Rows(count).Item(1)
            PieceType = ds.Tables(0).Rows(count).Item(2)
            Select Case (ds.Tables(0).Rows(count).Item(1))
                Case "YellowBoard"
                    SelectedBoardBox.Image =
My.Resources.YellowBoardPreview
                Case "GreenBoard"
                    SelectedBoardBox.Image =
My.Resources.GreenBoardPreview
                Case "PinkBoard"
                    SelectedBoardBox.Image =
My.Resources.PinkBoardPreview
                Case "RedBoard"
                    SelectedBoardBox.Image = My.Resources.RedBoardPreview
                Case "BlueBoard"
                    SelectedBoardBox.Image =
My.Resources.BlueBoardPreview
            End Select
            Select Case (ds.Tables(0).Rows(count).Item(2))
                Case "Set1"
                    SelectedPieceBox.Image = My.Resources.Set1KnightW
                Case "Set2"
                    SelectedPieceBox.Image = My.Resources.Set2KnightW
                Case "Set3"
                    SelectedPieceBox.Image = My.Resources.Set3KnightW
            End Select
            Select Case (ds.Tables(0).Rows(count).Item(3))
                Case "Default"
                    MainMenu.ProfilePicture.Image =
My.Resources.DefaultSkin
                Case "Skin1"
                    MainMenu.ProfilePicture.Image = My.Resources.Skin1

```

```

        Case "Skin2"
            MainMenu.ProfilePicture.Image = My.Resources.Skin2
        Case "Skin3"
            MainMenu.ProfilePicture.Image = My.Resources.Skin3
        Case "Skin4"
            MainMenu.ProfilePicture.Image = My.Resources.Skin4
        Case "Skin5"
            MainMenu.ProfilePicture.Image = My.Resources.Skin5
        Case "Skin6"
            MainMenu.ProfilePicture.Image = My.Resources.Skin6
        Case "Skin7"
            MainMenu.ProfilePicture.Image = My.Resources.Skin7
        Case "Locked1"
            MainMenu.ProfilePicture.Image = My.Resources.Locked1
        Case "Locked2"
            MainMenu.ProfilePicture.Image = My.Resources.Locked2
        Case "Locked3"
            MainMenu.ProfilePicture.Image = My.Resources.Locked3
        Case "Locked4"
            MainMenu.ProfilePicture.Image = My.Resources.Locked4
        Case "Locked5"
            MainMenu.ProfilePicture.Image = My.Resources.Locked5
        Case "Locked6"
            MainMenu.ProfilePicture.Image = My.Resources.Locked6
        Case "Locked7"
            MainMenu.ProfilePicture.Image = My.Resources.Locked7
        Case "Locked8"
            MainMenu.ProfilePicture.Image = My.Resources.Locked8
    End Select
End If
Next
cn.Close() 'Close connection
'As stats have been calculated, check if any unlockable boards or
piece sets are available
If Winstxt.Text >= 1 Then
    RedBoardBox.Enabled = True
    RedBoardBox.Image = Nothing
    'Editing the text
    U1WinLabel2.ForeColor = Color.Green
    U1WinLabel2.Text = "Unlocked"
    Set2Box.Enabled = True
    Set2Box.Image = Nothing
    'Editing the text
    U1WinLabel3.ForeColor = Color.Green
    U1WinLabel3.Text = "Unlocked"
End If

If Winstxt.Text >= 2 Then
    BlueBoardBox.Enabled = True
    BlueBoardBox.Image = Nothing
    'Editing the text
    U2WinLabel2.ForeColor = Color.Green
    U2WinLabel2.Text = "Unlocked"
    Set3Box.Enabled = True
    Set3Box.Image = Nothing
    'Editing the text
    U2WinLabel3.ForeColor = Color.Green

```



```

        U2WinLabel3.Text = "Unlocked"
    End If

    If Winstxt.Text >= 3 Then
        PinkBoardBox.Enabled = True
        PinkBoardBox.Image = Nothing
        'Editing the text
        U3WinLabel2.ForeColor = Color.Green
        U3WinLabel2.Text = "Unlocked"
    End If
End Sub
Private Sub LoadIntoDB()

    'Updating profile picture in database
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    cn.Open() 'Open connection
    Dim sql As String
    sql = "update [Table1]"
    sql = sql & "SET [ProfilePicture] = '" & Skinname & "', "
    sql = sql & "[BoardColour] = '" & BoardColour & "', "
    sql = sql & "[PieceType] = '" & PieceType & "' "
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"

    Dim cmd As New OleDbCommand(sql, cn)

    cmd.ExecuteNonQuery()
    cn.Close() 'Close connection
End Sub
Private Sub StatsCalculator()
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[Wins],[Draws],[Loses] FROM [Table1]" 'SQL
Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter

```

```

    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats
            Winstxt.Text = ds.Tables(0).Rows(count).Item(1)
            Drawstxt.Text = ds.Tables(0).Rows(count).Item(2)
            Losestxt.Text = ds.Tables(0).Rows(count).Item(3)
        End If
    Next
    cn.Close()
    'Creating the Win %
    WinPercentagetxt.Text = (Int((CInt(Winstxt.Text) /
(CInt(Winstxt.Text) + CInt(Drawstxt.Text) + CInt(Losestxt.Text)) * 100)))
'Used append text to make form more user friendly and not highlight text from
the get-go
End Sub
Private Sub BoxClicked(sender As Object, e As EventArgs) Handles
DefaultSkinBox.Click
    MainMenu.ProfilePicture.Image = DefaultSkinBox.Image
    Skinname = "Default"
    LoadIntoDB()
End Sub

Private Sub Skin1Box_Click(sender As Object, e As EventArgs) Handles
Skin1Box.Click
    MainMenu.ProfilePicture.Image = Skin1Box.Image
    Skinname = "Skin1"
    LoadIntoDB()
End Sub

Private Sub Skin2Box_Click(sender As Object, e As EventArgs) Handles
Skin2Box.Click
    MainMenu.ProfilePicture.Image = Skin2Box.Image
    Skinname = "Skin2"
    LoadIntoDB()
End Sub

Private Sub Skin3Box_Click(sender As Object, e As EventArgs) Handles
Skin3Box.Click
    MainMenu.ProfilePicture.Image = Skin3Box.Image
    Skinname = "Skin3"
    LoadIntoDB()
End Sub

Private Sub Skin4Box_Click(sender As Object, e As EventArgs) Handles
Skin4Box.Click
    MainMenu.ProfilePicture.Image = Skin4Box.Image

    Skinname = "Skin4"
    LoadIntoDB()
End Sub

Private Sub Skin5Box_Click(sender As Object, e As EventArgs) Handles
Skin5Box.Click

```

```
        MainMenu.ProfilePicture.Image = Skin5Box.Image
        Skinname = "Skin5"
        LoadIntoDB ()
    End Sub

    Private Sub Skin6Box_Click(sender As Object, e As EventArgs) Handles
Skin6Box.Click
        MainMenu.ProfilePicture.Image = Skin6Box.Image
        Skinname = "Skin6"
        LoadIntoDB ()
    End Sub

    Private Sub Skin7Box_Click(sender As Object, e As EventArgs) Handles
Skin7Box.Click
        MainMenu.ProfilePicture.Image = Skin7Box.Image
        Skinname = "Skin7"
        LoadIntoDB ()
    End Sub
    Private Sub Locked1Box_Click(sender As Object, e As EventArgs) Handles
Locked1Box.Click
        MainMenu.ProfilePicture.Image = Locked1Box.BackgroundImage
        Skinname = "Locked1"
        LoadIntoDB ()
    End Sub

    Private Sub Locked2Box_Click(sender As Object, e As EventArgs) Handles
Locked2Box.Click
        MainMenu.ProfilePicture.Image = Locked2Box.BackgroundImage
        Skinname = "Locked2"
        LoadIntoDB ()
    End Sub

    Private Sub Locked3Box_Click(sender As Object, e As EventArgs) Handles
Locked3Box.Click
        MainMenu.ProfilePicture.Image = Locked3Box.BackgroundImage
        Skinname = "Locked3"
        LoadIntoDB ()
    End Sub

    Private Sub Locked4Box_Click(sender As Object, e As EventArgs) Handles
Locked4Box.Click
        MainMenu.ProfilePicture.Image = Locked4Box.BackgroundImage
        Skinname = "Locked4"
        LoadIntoDB ()
    End Sub

    Private Sub Locked5Box_Click(sender As Object, e As EventArgs) Handles
Locked5Box.Click
        MainMenu.ProfilePicture.Image = Locked5Box.BackgroundImage
        Skinname = "Locked5"
        LoadIntoDB ()
    End Sub

    Private Sub Locked6Box_Click(sender As Object, e As EventArgs) Handles
Locked6Box.Click
        MainMenu.ProfilePicture.Image = Locked6Box.BackgroundImage
        Skinname = "Locked6"
```

```

        LoadIntoDB ()
    End Sub

    Private Sub Locked7Box_Click(sender As Object, e As EventArgs) Handles
Locked7Box.Click
        MainMenu.ProfilePicture.Image = Locked7Box.BackgroundImage
        Skinname = "Locked7"
        LoadIntoDB ()
    End Sub

    Private Sub Locked8Box_Click(sender As Object, e As EventArgs) Handles
Locked8Box.Click
        MainMenu.ProfilePicture.Image = Locked8Box.BackgroundImage
        Skinname = "Locked8"
        LoadIntoDB ()
    End Sub

    Private Sub YellowBoardBox_Click(sender As Object, e As EventArgs)
Handles YellowBoardBox.Click
        SelectedBoardBox.Image = My.Resources.YellowBoardPreview
        BoardColour = "YellowBoard"
        LoadIntoDB ()
    End Sub

    Private Sub GreenBoardBox_Click(sender As Object, e As EventArgs) Handles
GreenBoardBox.Click
        SelectedBoardBox.Image = My.Resources.GreenBoardPreview
        BoardColour = "GreenBoard"
        LoadIntoDB ()
    End Sub

    Private Sub PinkBoardBox_Click(sender As Object, e As EventArgs) Handles
PinkBoardBox.Click
        SelectedBoardBox.Image = My.Resources.PinkBoardPreview
        BoardColour = "PinkBoard"
        LoadIntoDB ()
    End Sub

    Private Sub RedBoardBox_Click(sender As Object, e As EventArgs) Handles
RedBoardBox.Click
        SelectedBoardBox.Image = My.Resources.RedBoardPreview
        BoardColour = "RedBoard"
        LoadIntoDB ()
    End Sub

    Private Sub BlueBoardBox_Click(sender As Object, e As EventArgs) Handles
BlueBoardBox.Click
        SelectedBoardBox.Image = My.Resources.BlueBoardPreview
        BoardColour = "BlueBoard"
        LoadIntoDB ()
    End Sub

    Private Sub Set1Box_Click(sender As Object, e As EventArgs) Handles
Set1Box.Click
        SelectedPieceBox.Image = My.Resources.Set1KnightW
        PieceType = "Set1"
        LoadIntoDB ()
    End Sub

```

```

    Private Sub Set2Box_Click(sender As Object, e As EventArgs) Handles
Set2Box.Click
        SelectedPictureBox.Image = My.Resources.Set2KnightW
        PieceType = "Set2"
        LoadIntoDB()
    End Sub
    Private Sub Set3Box_Click(sender As Object, e As EventArgs) Handles
Set3Box.Click
        SelectedPictureBox.Image = My.Resources.Set3KnightW
        PieceType = "Set3"
        LoadIntoDB()
    End Sub
End Class

```

Forgotten Password

```

Imports System.Data.OleDb
Imports System.Data

```

```

Public Class ForgottenPasswordForm
    Dim PasswordIsValid As Boolean = True 'Variable which determines whether
or not the user can sign up
    Private Sub ChangePasswordButton_Click(sender As Object, e As EventArgs)
Handles ChangePasswordButton.Click
        If Passwordtxt.Text = "" Then
            MessageBox.Show("Please enter a new password")
            PasswordIsValid = False
        ElseIf ConfirmPasswordtxt.Text = "" Then
            MessageBox.Show("Please confirm your new password")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) < 5 Then
            MessageBox.Show("Password is too short. Please make it between 5
and 30 characters")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) > 30 Then
            MessageBox.Show("Password is too long. Please make it between 5
and 30 characters")
            PasswordIsValid = False
        ElseIf Passwordtxt.Text <> ConfirmPasswordtxt.Text Then
            MessageBox.Show("Passwords do not match up")
            PasswordIsValid = False
        ElseIf Len(Passwordtxt.Text) >= 5 And Len(Passwordtxt.Text) <= 30 And
Passwordtxt.Text = ConfirmPasswordtxt.Text Then 'Password is valid
            PasswordIsValid = True
        End If

        If PasswordIsValid = True Then
            Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
            Dim result As DialogResult
            'displaying messagebox
            result = MessageBox.Show("Are you sure you want to change your
password?", "Change Password Confirmation", button)
            'getting result of the messagebox
            If result = System.Windows.Forms.DialogResult.Yes Then
                'Update password
                'Setting database location
            End If
        End If
    End Sub
End Class

```

```

        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;"
'OLE provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        cn.Open() 'Open connection
        Dim sql As String
        sql = "update [Table1]"
        sql = sql & "SET [Password] = '" & Passwordtxt.Text & "' "
        sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
        Dim cmd As New OleDbCommand(sql, cn)
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection
        MessageBox.Show("Password Changed")
        'Close form
        Me.Close()

    Else
        PasswordIsValid = False
    End If
End If
End Sub

Private Sub ForgottenPasswordForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
    'Setting text box input
    Me.Passwordtxt.PasswordChar = "*"
    Me.ConfirmPasswordtxt.PasswordChar = "*"
End Sub

Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
Handles ShowPassword.CheckedChanged
    If Me.ShowPassword.Checked = True Then
        Me.Passwordtxt.PasswordChar = "" 'If user ticks the show password
box, show password
        Me.ConfirmPasswordtxt.PasswordChar = ""
    Else
        Me.Passwordtxt.PasswordChar = "*" 'If box is unticked turn
password into *
        Me.ConfirmPasswordtxt.PasswordChar = "*"
    End If
End Sub

Private Sub CloseProgramLabel_Click(sender As Object, e As EventArgs)
Handles CloseProgramLabel.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'end program

```

```

        End
    End If
End Sub

Private Sub ReturnLabel_Click(sender As Object, e As EventArgs) Handles
ReturnLabel.Click
    'Show login form
    LoginForm.Show()
    'Close current form
    Me.Close()
End Sub
End Class

```

GameForm1

```

Imports System.Data.OleDb
Imports System.Data

```

```

Public Class GameForm1
    'Chess pieces:
    '1) Pawn
    '2) Bishop
    '3) Rook
    '4) Knight
    '5) Queen
    '6) King

    Public pb(7, 7) As PictureBox
    Public whiteking As King
    Public whitequeen(8) As Queen '1 Queen to start, 8 other possible queens
due to pawn promotions
    Public whitebishop(9) As Bishop '2 Bishops to start, 8 other possible
bishops due to pawn promotions
    Public whiteknight(9) As Knight '2 Knights to start, 8 other possible
knights due to pawn promotions
    Public whitepawn(7) As Pawn
    Public whiterook(9) As Rook '2 Rooks to start, 8 other possible rooks due
to pawn promotions
    Public blackking As King
    Public blackqueen(8) As Queen '1 Queen to start, 8 other possible queens
due to pawn promotions
    Public blackbishop(9) As Bishop '2 Bishops to start, 8 other possible
bishops due to pawn promotions
    Public blackknight(9) As Knight '2 Knights to start, 8 other possible
knights due to pawn promotions
    Public blackpawn(7) As Pawn
    Public blackrook(9) As Rook '2 Rooks to start, 8 other possible rooks due
to pawn promotions
    'Count in case of pawn promotion
    Public wqueencount As Integer = 0
    Public wbishopcount As Integer = 0
    Public wknightcount As Integer = 0
    Public wrookcount As Integer = 0
    Public bqueencount As Integer = 0
    Public bbishopcount As Integer = 0
    Public bknightcount As Integer = 0

```

```

Public brookcount As Integer = 0

Public playerssettle As Boolean

Public ga As Game
Public x, y, selectionx, selectiony As Integer
Public selected As Boolean = False
Public allowgametostart As Boolean = False
Public gamestarted As Boolean = False
Public selectedpiece As String = Nothing
'Code for piece sets
Public Set1 As Boolean = False
Public Set2 As Boolean = False
Public Set3 As Boolean = False
'Code for AI
Public gameistwooplayer As Boolean = False
Public gameisoneoplayer As Boolean = False
Public EasyMode As Boolean = False
Public HardMode As Boolean = False

Private Sub GameForm1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    'Loading player 1 name
    Player1Label.Text = LoginForm.Player1Username
    'Loading player 2 username
    Player2Label.Text = MultiplayerForm.Player2Username
    'Making sure buttons not needed aren't visible
    RestartButton.Visible = False
    'Give board tracker properties
    BoardTracker.Multiline = True
    BoardTracker.ScrollBars = ScrollBars.Vertical
    'Setting times to 00
    MinutesTextBox1.AppendText("00")
    SecondsTextBox1.AppendText("00")
    MinutesTextBox2.AppendText("00")
    SecondsTextBox2.AppendText("00")
    'Loading User 1 And User 2 profile pictures
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[ProfilePicture],[BoardColour],[PieceType]
FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

```



```

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
    If Player1Label.Text = ds.Tables(0).Rows(count).Item(0) Then 'For
player 1 profile picture (for a user that has signed in)
        Select Case (ds.Tables(0).Rows(count).Item(1))
            Case "Default"
                P1ProfilePicture.Image = My.Resources.DefaultSkin
            Case "Skin1"
                P1ProfilePicture.Image = My.Resources.Skin1
            Case "Skin2"
                P1ProfilePicture.Image = My.Resources.Skin2
            Case "Skin3"
                P1ProfilePicture.Image = My.Resources.Skin3
            Case "Skin4"
                P1ProfilePicture.Image = My.Resources.Skin4
            Case "Skin5"
                P1ProfilePicture.Image = My.Resources.Skin5
            Case "Skin6"
                P1ProfilePicture.Image = My.Resources.Skin6
            Case "Skin7"
                P1ProfilePicture.Image = My.Resources.Skin7
            Case "Locked1"
                P1ProfilePicture.Image = My.Resources.Locked1
            Case "Locked2"
                P1ProfilePicture.Image = My.Resources.Locked2
            Case "Locked3"
                P1ProfilePicture.Image = My.Resources.Locked3
            Case "Locked4"
                P1ProfilePicture.Image = My.Resources.Locked4
            Case "Locked5"
                P1ProfilePicture.Image = My.Resources.Locked5
            Case "Locked6"
                P1ProfilePicture.Image = My.Resources.Locked6
            Case "Locked7"
                P1ProfilePicture.Image = My.Resources.Locked7
            Case "Locked8"
                P1ProfilePicture.Image = My.Resources.Locked8
        End Select
        Select Case (ds.Tables(0).Rows(count).Item(2))
            Case "YellowBoard"
                Me.BackgroundImage = My.Resources.YellowBoard
            Case "GreenBoard"
                Me.BackgroundImage = My.Resources.GreenBoard
            Case "PinkBoard"
                Me.BackgroundImage = My.Resources.PinkBoard
            Case "RedBoard"
                Me.BackgroundImage = My.Resources.RedBoard
            Case "BlueBoard"
                Me.BackgroundImage = My.Resources.BlueBoard
        End Select
        Select Case (ds.Tables(0).Rows(count).Item(3))
            Case "Set1"
                Set1 = True
            Case "Set2"
                Set2 = True

```

```

        Case "Set3"
            Set3 = True
        End Select
    ElseIf Player2Label.Text = ds.Tables(0).Rows(count).Item(0) Then
'For player 2 profile picture (for a user that has signed in)
        Select Case (ds.Tables(0).Rows(count).Item(1))
            Case "Default"
                P2ProfilePicture.Image = My.Resources.DefaultSkin
            Case "Skin1"
                P2ProfilePicture.Image = My.Resources.Skin1
            Case "Skin2"
                P2ProfilePicture.Image = My.Resources.Skin2
            Case "Skin3"
                P2ProfilePicture.Image = My.Resources.Skin3
            Case "Skin4"
                P2ProfilePicture.Image = My.Resources.Skin4
            Case "Skin5"
                P2ProfilePicture.Image = My.Resources.Skin5
            Case "Skin6"
                P2ProfilePicture.Image = My.Resources.Skin6
            Case "Skin7"
                P2ProfilePicture.Image = My.Resources.Skin7
            Case "Locked1"
                P2ProfilePicture.Image = My.Resources.Locked1
            Case "Locked2"
                P2ProfilePicture.Image = My.Resources.Locked2
            Case "Locked3"
                P2ProfilePicture.Image = My.Resources.Locked3
            Case "Locked4"
                P2ProfilePicture.Image = My.Resources.Locked4
            Case "Locked5"
                P2ProfilePicture.Image = My.Resources.Locked5
            Case "Locked6"
                P2ProfilePicture.Image = My.Resources.Locked6
            Case "Locked7"
                P2ProfilePicture.Image = My.Resources.Locked7
            Case "Locked8"
                P2ProfilePicture.Image = My.Resources.Locked8
        End Select
    Else
        P2ProfilePicture.Image = My.Resources.DefaultSkin 'Default
profile picture for a guest (Or AI)
    End If
Next
cn.Close() 'Close connection

'Creating picture boxes as an array
pb = New PictureBox(7, 7) {{Me.p00, Me.p01, Me.p02, Me.p03, Me.p04,
Me.p05, Me.p06, Me.p07},
    {Me.p10, Me.p11, Me.p12, Me.p13, Me.p14, Me.p15, Me.p16, Me.p17},
    {Me.p20, Me.p21, Me.p22, Me.p23, Me.p24, Me.p25, Me.p26, Me.p27},
    {Me.p30, Me.p31, Me.p32, Me.p33, Me.p34, Me.p35, Me.p36, Me.p37},
    {Me.p40, Me.p41, Me.p42, Me.p43, Me.p44, Me.p45, Me.p46, Me.p47},
    {Me.p50, Me.p51, Me.p52, Me.p53, Me.p54, Me.p55, Me.p56, Me.p57},
    {Me.p60, Me.p61, Me.p62, Me.p63, Me.p64, Me.p65, Me.p66, Me.p67},
    {Me.p70, Me.p71, Me.p72, Me.p73, Me.p74, Me.p75, Me.p76, Me.p77}}
'Creating all objects

```

```

        allobjects()
        ga.show()
    End Sub

    Private Sub TimeComboBox_SelectedIndexChanged(sender As Object, e As
EventArgs) Handles TimeComboBox.SelectedIndexChanged
        'If a time has been selected, copy it to both players time textboxes,
        and allow the game to start
        If TimeComboBox.Text = "Unlimited" Then
            MinutesTextBox1.Text = "∞"
            MinutesTextBox2.Text = "∞"
            SecondsTextBox1.Text = "∞"
            SecondsTextBox2.Text = "∞"
            allowgametostart = True
        ElseIf TimeComboBox.Text = "2 Minutes" Then
            MinutesTextBox1.Text = "02"
            MinutesTextBox2.Text = "02"
            SecondsTextBox1.Text = "00"
            SecondsTextBox2.Text = "00"
            allowgametostart = True
        ElseIf TimeComboBox.Text = "5 Minutes" Then
            MinutesTextBox1.Text = "05"
            MinutesTextBox2.Text = "05"
            SecondsTextBox1.Text = "00"
            SecondsTextBox2.Text = "00"
            allowgametostart = True
        ElseIf TimeComboBox.Text = "10 Minutes" Then
            MinutesTextBox1.Text = "10"
            MinutesTextBox2.Text = "10"
            SecondsTextBox1.Text = "00"
            SecondsTextBox2.Text = "00"
            allowgametostart = True
        ElseIf TimeComboBox.Text = "20 Minutes" Then
            MinutesTextBox1.Text = "20"
            MinutesTextBox2.Text = "20"
            SecondsTextBox1.Text = "00"
            SecondsTextBox2.Text = "00"
            allowgametostart = True
        ElseIf TimeComboBox.Text = "30 Minutes" Then
            MinutesTextBox1.Text = "30"
            MinutesTextBox2.Text = "30"
            SecondsTextBox1.Text = "00"
            SecondsTextBox2.Text = "00"
            allowgametostart = True
        End If
    End Sub

    Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles
P1Timer.Tick
        If SecondsTextBox1.Text = 0 And MinutesTextBox1.Text = 0 Then
            'If timer reaches 0 disable timers
            P1Timer.Enabled = False
            P2Timer.Enabled = False
            ga.gameflag = False
            gamestarted = False
            'Hide buttons

```

```

P1ResignButton.Hide()
P1SettleButton.Hide()
P2ResignButton.Hide()
P2SettleButton.Hide()

'Display winner
MsgBox("Ran out of time, " & Player2Label.Text & " is the
winner!")

'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
Dim Player1Loses As Integer
Dim Player2Wins As Integer

'Setting database location
Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
'Opening connection
Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
Dim DBsource As String = "Data Source=" & dbfilename

Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
Dim sql As String
sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter
Dim count As Integer

For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
Player1Loses = ds.Tables(0).Rows(count).Item(2)
ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
Player2Wins = ds.Tables(0).Rows(count).Item(1)
End If
Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "'"
sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

```

```

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "' "
        sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        ga.StoreGameReplay()

    ElseIf SecondsTextBox1.Text = 0 Then
        MinutesTextBox1.Text -= 1
        For i = 0 To 9
            If MinutesTextBox1.Text = i Then
                MinutesTextBox1.Text = "0" & i
            End If
        Next
    End If
    If P1Timer.Enabled = True Then
        If SecondsTextBox1.Text > 0 Then
            SecondsTextBox1.Text -= 1
            For i = 0 To 9
                If SecondsTextBox1.Text = i Then
                    SecondsTextBox1.Text = "0" & i
                End If
            Next
        ElseIf SecondsTextBox1.Text = 0 Then
            SecondsTextBox1.Text = 59
        End If
    End If
End Sub

Private Sub P2Timer_Tick(sender As Object, e As EventArgs) Handles
P2Timer.Tick
    If SecondsTextBox2.Text = 0 And MinutesTextBox2.Text = 0 Then
        'If timer reaches 0 disable timers
        P1Timer.Enabled = False
        P2Timer.Enabled = False
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()

        'Display winner
        MsgBox("Ran out of time, " & Player1Label.Text & " is the
winner!")
        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Wins As Integer
        Dim Player2Loses As Integer

```

```

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Wins = ds.Tables(0).Rows(count).Item(1)
            ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                Player2Loses = ds.Tables(0).Rows(count).Item(2)
            End If
        Next

        cn.Close() 'Close connection

        'Updating the database with the new P1 user stats
        Dim cmd As New OleDbCommand(sql, cn)
        sql = "update [Table1]"
        sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "' "
        sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "';"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "' "
        sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID & "';"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        ga.StoreGameReplay()

```

```

ElseIf SecondsTextBox2.Text = 0 Then
    MinutesTextBox2.Text -= 1
    For i = 0 To 9
        If MinutesTextBox2.Text = i Then
            MinutesTextBox2.Text = "0" & i
        End If
    Next
Else
End If
If P2Timer.Enabled = True Then
    If SecondsTextBox2.Text > 0 Then
        SecondsTextBox2.Text -= 1
        For i = 0 To 9
            If SecondsTextBox2.Text = i Then
                SecondsTextBox2.Text = "0" & i
            End If
        Next
    ElseIf SecondsTextBox2.Text = 0 Then
        SecondsTextBox2.Text = 59
    End If
End If
End Sub
Private Sub StartGameButton_Click(sender As Object, e As EventArgs)
Handles StartGameButton.Click
    gamestarted = True 'trigger starting of game

    If allowgametostart = False Then 'If no time limit has been selected
then you cannot start the game
        MsgBox("Please select a time limit")
    Else 'If time limit has been selected then run the program
        TimeLimitLabel.Visible = False
        TimeComboBox.Visible = False 'User cannot change the time during
the game running
        P1Timer.Start() 'Starting the P1 timer as white is always first
        RestartButton.Visible = True 'If the game is started user has an
option to restart the game whenever they want
        StartGameButton.Visible = False 'If the game is started hide the
button, after restart button is pressed it shall show up again

        allobjects() 'loading in all of the pieces into the array to be
used

        ga.recanmove()
        ga.show()
        ga.reback()
        ga.recheckblack()
        ga.recheckwhite()
    End If
End Sub
Private Sub RestartButton_Click(sender As Object, e As EventArgs) Handles
RestartButton.Click
    P1Timer.Stop()
    P2Timer.Stop()
    BoardTracker.Text = ""
    StartGameButton.Visible = True
    P1ResignButton.Visible = False
    P2ResignButton.Visible = False

```

```

P1SettleButton.Visible = False
P2SettleButton.Visible = False

TimeComboBox.SelectedIndex = -1
TimeComboBox.Visible = True
TimeLimitLabel.Visible = True
SecondsTextBox1.Text = "00"
SecondsTextBox2.Text = "00"
MinutesTextBox1.Text = "00"
MinutesTextBox2.Text = "00"
MoveCounterTextBox.Text = ""
'Game has stopped
gamestarted = False
'Get rid of back
ga.reback()
'Create new pieces and game
allobjects()
'Display pieces
ga.show()
'Game is not allowed to start until time has been selected again
allowgametostart = False
gamestarted = False
'Reset count of pieces
wqueencount = 0
wbishopcount = 0
wknightcount = 0
wrookcount = 0
bqueencount = 0
bbishopcount = 0
bknightcount = 0
brookcount = 0

End Sub

Private Sub MoveCounterTextBox_TextChanged(sender As Object, e As
EventArgs) Handles MoveCounterTextBox.TextChanged
    If MoveCounterTextBox.Text = "1" Then
        P1ResignButton.Visible = True 'If the game is started the player
has the option to resign the win
        If gameistwooplayer = True Then
            P2ResignButton.Visible = True 'If the game is started the
player has the option to resign the win
            P2SettleButton.Visible = True 'Settle for player 2
            P1SettleButton.Visible = True 'If game started allow option
for players to settle
        End If
    End If
End Sub

Private Sub P1ResignButton_Click(sender As Object, e As EventArgs)
Handles P1ResignButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult

    'stopping timers
    P1Timer.Stop()
    P2Timer.Stop()

```



```

        'displaying messagebox
        result = MessageBox.Show("Are you sure you want to resign", "Player 1
Resigning", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            'Game has ended, player 2 has one
            'Stop game
            ga.gameflag = False
            gamestarted = False
            'Hide buttons
            P1ResignButton.Hide()
            P1SettleButton.Hide()
            P2ResignButton.Hide()
            P2SettleButton.Hide()

            'Display winner
            MessageBox.Show("Resigned, " & Player2Label.Text & " is the
winner!")

            'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
            Dim Player1Loses As Integer
            Dim Player2Wins As Integer

            'Setting database location
            Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
            'Opening connection
            Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
            Dim DBsource As String = "Data Source=" & dbfilename

            Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
            Dim da As OleDbDataAdapter 'adapter
            Dim ds As New DataSet 'set used with adapter
            Dim sql As String
            sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
            da = New OleDbDataAdapter(sql, cn)
            cn.Open() 'Open connection
            da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

            Dim count As Integer

            For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
                If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                    Player1Loses = ds.Tables(0).Rows(count).Item(2)
                ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                    Player2Wins = ds.Tables(0).Rows(count).Item(1)
                End If
            Next

            cn.Close() 'Close connection

```

```

        'Updating the database with the new P1 user stats
        Dim cmd As New OleDbCommand(sql, cn)
        sql = "update [Table1]"
        sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "'"
        sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "'"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "'"
        sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID & "'"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        ga.StoreGameReplay()
    ElseIf result = System.Windows.Forms.DialogResult.No Then
        If ga.movecounter Mod 2 = 0 Then
            P1Timer.Start()
        Else
            P2Timer.Start()
        End If
    End If
End Sub

Private Sub P2ResignButton_Click(sender As Object, e As EventArgs)
Handles P2ResignButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult

    'stop timers
    P1Timer.Stop()
    P2Timer.Stop()
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want to resign", "Player 2
Resigning", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Game has ended, player 1 has one
        ga.gameflag = False
        gamestarted = False
        'Hide buttons
        P1ResignButton.Hide()
        P1SettleButton.Hide()
        P2ResignButton.Hide()
        P2SettleButton.Hide()

        'Display winner
        MessageBox.Show("Resigned, " & Player1Label.Text & " is the
winner!")
    End If
End Sub

```

```

        'Getting both users stats and storing them temporarily (only win
and lose, as settling form has the drawn stats changed)
        Dim Player1Wins As Integer
        Dim Player2Loses As Integer

        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]" 'SQL
Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by
adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount
of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Wins = ds.Tables(0).Rows(count).Item(1)
            ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                Player2Loses = ds.Tables(0).Rows(count).Item(2)
            End If
        Next

        cn.Close() 'Close connection

        'Updating the database with the new P1 user stats
        Dim cmd As New OleDbCommand(sql, cn)
        sql = "update [Table1]"
        sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "' "
        sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "' "
        sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection

```

```

cmd.ExecuteNonQuery()
cn.Close() 'Close connection

ga.StoreGameReplay()

ElseIf result = System.Windows.Forms.DialogResult.No Then
    If ga.movecounter Mod 2 = 0 Then
        P1Timer.Start()
    Else
        P2Timer.Start()
    End If
End If
End Sub

Private Sub P1SettleButton_Click(sender As Object, e As EventArgs)
Handles P1SettleButton.Click, P2SettleButton.Click
    Settle.Show()
End Sub

Private Sub ReturnLabel_Click(sender As Object, e As EventArgs)
    Application.DoEvents()
    'Pause timers
    P1Timer.Stop()
    P2Timer.Stop()

    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want return to the main
menu?", "Main Menu", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Show main menu form
        MainMenu.Show()
        'Close current form
        Me.Close()
        'Close Multiplayer Menu Form (as we don't need the data from it
anymore)
        MultiplayerForm.Close()
    Else
        'Resume timers if game is started
        If gamestarted = True Then
            If ga.movecounter Mod 2 = 0 Then
                P1Timer.Start()
            Else
                P2Timer.Start()
            End If
        End If
    End If
End Sub

Public Sub allobjects()
    blackpawn(0) = New Pawn(1, 0, -6)
    blackpawn(1) = New Pawn(1, 1, -6)
    blackpawn(2) = New Pawn(1, 2, -6)
    blackpawn(3) = New Pawn(1, 3, -6)

```

```

blackpawn(4) = New Pawn(1, 4, -6)
blackpawn(5) = New Pawn(1, 5, -6)
blackpawn(6) = New Pawn(1, 6, -6)
blackpawn(7) = New Pawn(1, 7, -6)
blackrook(0) = New Rook(0, 0, -5)
blackrook(1) = New Rook(0, 7, -5)
blackknight(0) = New Knight(0, 1, -4)
blackknight(1) = New Knight(0, 6, -4)
blackbishop(0) = New Bishop(0, 2, -3)
blackbishop(1) = New Bishop(0, 5, -3)
blackking = New King(0, 4, -1)
blackqueen(0) = New Queen(0, 3, -2)

whitepawn(0) = New Pawn(6, 0, 6)
whitepawn(1) = New Pawn(6, 1, 6)
whitepawn(2) = New Pawn(6, 2, 6)
whitepawn(3) = New Pawn(6, 3, 6)
whitepawn(4) = New Pawn(6, 4, 6)
whitepawn(5) = New Pawn(6, 5, 6)
whitepawn(6) = New Pawn(6, 6, 6)
whitepawn(7) = New Pawn(6, 7, 6)
whiterook(0) = New Rook(7, 0, 5)
whiterook(1) = New Rook(7, 7, 5)
whiteknight(0) = New Knight(7, 1, 4)
whiteknight(1) = New Knight(7, 6, 4)
whitebishop(0) = New Bishop(7, 2, 3)
whitebishop(1) = New Bishop(7, 5, 3)
whiteking = New King(7, 4, 1)
whitequeen(0) = New Queen(7, 3, 2)
ga = New Game
End Sub

Private Sub p00_Click(sender As Object, e As EventArgs) Handles
p00.Click, P2ProfilePicture.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 0
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then 'if selected is true and
piece can move there then
                ga.mov(selectedpiece, x, y, selectionx, selectiony) 'move
selected piece, from x y to selection x, y
            End If
            selected = False

            ga.show()
        End If
    ElseIf gamestarted = False Then

```

```

        MessageBox.Show("Game needs to be started before using the
board")
    End If

End Sub

Private Sub p01_Click(sender As Object, e As EventArgs) Handles p01.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 1
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p02_Click(sender As Object, e As EventArgs) Handles p02.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p03_Click(sender As Object, e As EventArgs) Handles p03.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p04_Click(sender As Object, e As EventArgs) Handles p04.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p05_Click(sender As Object, e As EventArgs) Handles p05.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 5
        If selected <> True Then

```

```

        ga.recanmove()
        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p06_Click(sender As Object, e As EventArgs) Handles p06.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p07_Click(sender As Object, e As EventArgs) Handles p07.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 0
        y = 7
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then

```



```

        ga.mov(selectedpiece, x, y, selectionx, selectiony)
    End If
    selected = False
    ga.show()
End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p10_Click(sender As Object, e As EventArgs) Handles p10.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 0

        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If

            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If

End Sub

Private Sub p11_Click(sender As Object, e As EventArgs) Handles p11.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 1
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    End If
End Sub

```

```

        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p12_Click(sender As Object, e As EventArgs) Handles p12.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p13_Click(sender As Object, e As EventArgs) Handles p13.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p14_Click(sender As Object, e As EventArgs) Handles p14.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p15_Click(sender As Object, e As EventArgs) Handles p15.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p16_Click(sender As Object, e As EventArgs) Handles p16.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 6
        If selected <> True Then

```

```

        ga.recanmove()
        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p17_Click(sender As Object, e As EventArgs) Handles p17.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 1
        y = 7
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True

        Else

            If ga.canmove(x, y) = True Then

                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If

            selected = False
            ga.show()
        End If

    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p20_Click(sender As Object, e As EventArgs) Handles p20.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 0
        If selected <> True Then
            ga.recanmove()

```

```

        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p21_Click(sender As Object, e As EventArgs) Handles p21.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 1
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p22_Click(sender As Object, e As EventArgs) Handles p22.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then

```

```

        ga.mov(selectedpiece, x, y, selectionx, selectiony)
    End If
    selected = False
    ga.show()
End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p23_Click(sender As Object, e As EventArgs) Handles p23.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p24_Click(sender As Object, e As EventArgs) Handles p24.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then

```

```

        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p25_Click(sender As Object, e As EventArgs) Handles p25.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p26_Click(sender As Object, e As EventArgs) Handles p26.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 2
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p27_Click(sender As Object, e As EventArgs) Handles p27.Click

```

```

        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 2
            y = 7
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p30_Click(sender As Object, e As EventArgs) Handles p30.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 3
            y = 0
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p31_Click(sender As Object, e As EventArgs) Handles p31.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 3
            y = 1
            If selected <> True Then
                ga.recanmove()

```



```

        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p32_Click(sender As Object, e As EventArgs) Handles p32.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 3
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p33_Click(sender As Object, e As EventArgs) Handles p33.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 3
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)

```

```

        End If
        selected = False
        ga.show()
    End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p34_Click(sender As Object, e As EventArgs) Handles p34.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 3
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p35_Click(sender As Object, e As EventArgs) Handles p35.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 3
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

        End If
    End Sub

    Private Sub p36_Click(sender As Object, e As EventArgs) Handles p36.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 3
            y = 6
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p37_Click(sender As Object, e As EventArgs) Handles p37.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 3
            y = 7
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p40_Click(sender As Object, e As EventArgs) Handles p40.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()

```

```

x = 4
y = 0
If selected <> True Then
    ga.recanmove()
    selectedpiece = ga.selection(x, y)
    selectionx = x
    selectiony = y
    selected = True
Else
    If ga.canmove(x, y) = True Then
        ga.mov(selectedpiece, x, y, selectionx, selectiony)
    End If
    selected = False
    ga.show()
End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p41_Click(sender As Object, e As EventArgs) Handles p41.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 1
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p42_Click(sender As Object, e As EventArgs) Handles p42.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y

```

```

        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p43_Click(sender As Object, e As EventArgs) Handles p43.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p44_Click(sender As Object, e As EventArgs) Handles p44.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    End If
End Sub

```

```

        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p45_Click(sender As Object, e As EventArgs) Handles p45.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p46_Click(sender As Object, e As EventArgs) Handles p46.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p47_Click(sender As Object, e As EventArgs) Handles p47.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 4
        y = 7
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p50_Click(sender As Object, e As EventArgs) Handles p50.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 0
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

Private Sub p51_Click(sender As Object, e As EventArgs) Handles p51.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 1
        If selected <> True Then

```

```

        ga.recanmove()
        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p52_Click(sender As Object, e As EventArgs) Handles p52.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p53_Click(sender As Object, e As EventArgs) Handles p53.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then

```



```

        ga.mov(selectedpiece, x, y, selectionx, selectiony)
    End If
    selected = False
    ga.show()
End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p54_Click(sender As Object, e As EventArgs) Handles p54.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p55_Click(sender As Object, e As EventArgs) Handles p55.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then

```

```

        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p56_Click(sender As Object, e As EventArgs) Handles p56.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p57_Click(sender As Object, e As EventArgs) Handles p57.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 5
        y = 7
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p60_Click(sender As Object, e As EventArgs) Handles p60.Click

```

```

        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 6
            y = 0
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p61_Click(sender As Object, e As EventArgs) Handles p61.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 6
            y = 1
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p62_Click(sender As Object, e As EventArgs) Handles p62.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 6
            y = 2
            If selected <> True Then
                ga.recanmove()

```

```

        selectedpiece = ga.selection(x, y)
        selectionx = x
        selectiony = y
        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p63_Click(sender As Object, e As EventArgs) Handles p63.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 6
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p64_Click(sender As Object, e As EventArgs) Handles p64.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 6
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)

```

```

        End If
        selected = False
        ga.show()
    End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p65_Click(sender As Object, e As EventArgs) Handles p65.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 6
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p66_Click(sender As Object, e As EventArgs) Handles p66.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 6
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

```

```

        End If
    End Sub

    Private Sub p67_Click(sender As Object, e As EventArgs) Handles p67.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 6
            y = 7
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p70_Click(sender As Object, e As EventArgs) Handles p70.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()
            x = 7
            y = 0
            If selected <> True Then
                ga.recanmove()
                selectedpiece = ga.selection(x, y)
                selectionx = x
                selectiony = y
                selected = True
            Else
                If ga.canmove(x, y) = True Then
                    ga.mov(selectedpiece, x, y, selectionx, selectiony)
                End If
                selected = False
                ga.show()
            End If
        ElseIf gamestarted = False Then
            MessageBox.Show("Game needs to be started before using the
board")
        End If
    End Sub

    Private Sub p71_Click(sender As Object, e As EventArgs) Handles p71.Click
        If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
            ga.reback()

```

```

x = 7
y = 1
If selected <> True Then
    ga.recanmove()
    selectedpiece = ga.selection(x, y)
    selectionx = x
    selectiony = y
    selected = True
Else
    If ga.canmove(x, y) = True Then
        ga.mov(selectedpiece, x, y, selectionx, selectiony)
    End If
    selected = False
    ga.show()
End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p72_Click(sender As Object, e As EventArgs) Handles p72.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 2
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p73_Click(sender As Object, e As EventArgs) Handles p73.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 3
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y

```

```

        selected = True
    Else
        If ga.canmove(x, y) = True Then
            ga.mov(selectedpiece, x, y, selectionx, selectiony)
        End If
        selected = False
        ga.show()
    End If
ElseIf gamestarted = False Then
    MessageBox.Show("Game needs to be started before using the
board")
End If
End Sub

Private Sub p74_Click(sender As Object, e As EventArgs) Handles p74.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 4
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p75_Click(sender As Object, e As EventArgs) Handles p75.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 5
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    End If
End Sub

```



```

        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p76_Click(sender As Object, e As EventArgs) Handles p76.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 6
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub

Private Sub p77_Click(sender As Object, e As EventArgs) Handles p77.Click
    If (gamestarted = True And gameistwooplayer = True) Or (gamestarted =
True And ga.player = 1) Then
        ga.reback()
        x = 7
        y = 7
        If selected <> True Then
            ga.recanmove()
            selectedpiece = ga.selection(x, y)
            selectionx = x
            selectiony = y
            selected = True
        Else
            If ga.canmove(x, y) = True Then
                ga.mov(selectedpiece, x, y, selectionx, selectiony)
            End If
            selected = False
            ga.show()
        End If
    ElseIf gamestarted = False Then
        MessageBox.Show("Game needs to be started before using the
board")
    End If
End Sub
End Class

```

Home

```
Public Class HomeForm
    Private Sub HomeForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        Dim random As Integer
        Dim rnd As New Random

        random = rnd.Next(0, 2)

        Select Case random
            Case 0
                Me.BackgroundImage = My.Resources.Background1
            Case 1
                Me.BackgroundImage = My.Resources.Background2
        End Select
    End Sub
End Class
```

Leaderboard

```
Imports System.Data
Imports System.Data.OleDb
Public Class LeaderboardForm
    Private Sub LeaderboardForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins],[Draws],[Loses], [ProfilePicture]
FROM [Table1] ORDER BY [Wins] DESC" 'SQL Statement, instead of using bubble
sort, use order by function within the database
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection

        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        'Population of datagrid
        LeaderboardGrid.DataSource = ds.Tables("Table1")
        'Identifying top 3 places
        FirstLabel.Text = ds.Tables(0).Rows(0).Item(0)
        SecondLabel.Text = ds.Tables(0).Rows(1).Item(0)
        ThirdLabel.Text = ds.Tables(0).Rows(2).Item(0)
        'Profile pictures for top 3
        Dim SkinType(2) As String
        SkinType(0) = ds.Tables(0).Rows(0).Item(4)
        SkinType(1) = ds.Tables(0).Rows(1).Item(4)
```

```

SkinType(2) = ds.Tables(0).Rows(2).Item(4)

Select Case SkinType(0)
    Case "Default"
        FirstPlaceBox.Image = My.Resources.DefaultSkin
    Case "Skin1"
        FirstPlaceBox.Image = My.Resources.Skin1
    Case "Skin2"
        FirstPlaceBox.Image = My.Resources.Skin2
    Case "Skin3"
        FirstPlaceBox.Image = My.Resources.Skin3
    Case "Skin4"
        FirstPlaceBox.Image = My.Resources.Skin4
    Case "Skin5"
        FirstPlaceBox.Image = My.Resources.Skin5
    Case "Skin6"
        FirstPlaceBox.Image = My.Resources.Skin6
    Case "Skin7"
        FirstPlaceBox.Image = My.Resources.Skin7
    Case "Locked1"
        FirstPlaceBox.Image = My.Resources.Locked1
    Case "Locked2"
        FirstPlaceBox.Image = My.Resources.Locked2
    Case "Locked3"
        FirstPlaceBox.Image = My.Resources.Locked3
    Case "Locked4"
        FirstPlaceBox.Image = My.Resources.Locked4
    Case "Locked5"
        FirstPlaceBox.Image = My.Resources.Locked5
    Case "Locked6"
        FirstPlaceBox.Image = My.Resources.Locked6
    Case "Locked7"
        FirstPlaceBox.Image = My.Resources.Locked7
    Case "Locked8"
        FirstPlaceBox.Image = My.Resources.Locked8
End Select
Select Case SkinType(1)
    Case "Default"
        SecondPlaceBox.Image = My.Resources.DefaultSkin
    Case "Skin1"
        SecondPlaceBox.Image = My.Resources.Skin1
    Case "Skin2"
        SecondPlaceBox.Image = My.Resources.Skin2
    Case "Skin3"
        SecondPlaceBox.Image = My.Resources.Skin3
    Case "Skin4"
        SecondPlaceBox.Image = My.Resources.Skin4
    Case "Skin5"
        SecondPlaceBox.Image = My.Resources.Skin5
    Case "Skin6"
        SecondPlaceBox.Image = My.Resources.Skin6
    Case "Skin7"
        SecondPlaceBox.Image = My.Resources.Skin7
    Case "Locked1"
        SecondPlaceBox.Image = My.Resources.Locked1
    Case "Locked2"
        SecondPlaceBox.Image = My.Resources.Locked2

```

```

        Case "Locked3"
            SecondPlaceBox.Image = My.Resources.Locked3
        Case "Locked4"
            SecondPlaceBox.Image = My.Resources.Locked4
        Case "Locked5"
            SecondPlaceBox.Image = My.Resources.Locked5
        Case "Locked6"
            SecondPlaceBox.Image = My.Resources.Locked6
        Case "Locked7"
            SecondPlaceBox.Image = My.Resources.Locked7
        Case "Locked8"
            SecondPlaceBox.Image = My.Resources.Locked8
    End Select
    Select Case SkinType (2)
        Case "Default"
            ThirdPlaceBox.Image = My.Resources.DefaultSkin
        Case "Skin1"
            ThirdPlaceBox.Image = My.Resources.Skin1
        Case "Skin2"
            ThirdPlaceBox.Image = My.Resources.Skin2
        Case "Skin3"
            ThirdPlaceBox.Image = My.Resources.Skin3
        Case "Skin4"
            ThirdPlaceBox.Image = My.Resources.Skin4
        Case "Skin5"
            ThirdPlaceBox.Image = My.Resources.Skin5
        Case "Skin6"
            ThirdPlaceBox.Image = My.Resources.Skin6
        Case "Skin7"
            ThirdPlaceBox.Image = My.Resources.Skin7
        Case "Locked1"
            ThirdPlaceBox.Image = My.Resources.Locked1
        Case "Locked2"
            ThirdPlaceBox.Image = My.Resources.Locked2
        Case "Locked3"
            ThirdPlaceBox.Image = My.Resources.Locked3
        Case "Locked4"
            ThirdPlaceBox.Image = My.Resources.Locked4
        Case "Locked5"
            ThirdPlaceBox.Image = My.Resources.Locked5
        Case "Locked6"
            ThirdPlaceBox.Image = My.Resources.Locked6
        Case "Locked7"
            ThirdPlaceBox.Image = My.Resources.Locked7
        Case "Locked8"
            ThirdPlaceBox.Image = My.Resources.Locked8
    End Select
    cn.Close()
End Sub
End Class

```

Leaderboard

```
Imports System.Data.OleDb
Imports System.Data
Imports System.Net.Mail
Public Class LoginForm
    Dim LoginValid As Boolean = True 'Variable for knowing if User
    Credentials match up
    Public Player1Username As String
    Public Player1ID As Integer
    Private Sub SignUpLink_Changed(sender As Object, e As
LinkLabelLinkClickedEventArgs) Handles SignUpLink.LinkClicked
        'Clear Username and Password text box
        Usenametxt.Text = ""
        Passwordtxt.Text = ""
        'Show sign up form
        SignUpForm.Show()
        'Have to hide the form cannot close it as it is the startup form,
otherwise program will stop
        Me.Hide()

    End Sub

    Private Sub LoginButton_Click(sender As Object, e As EventArgs) Handles
LoginButton.Click
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT * FROM [Table1]" 'SQL Statement
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
            If Usenametxt.Text = ds.Tables(0).Rows(count).Item(1) And
Passwordtxt.Text = ds.Tables(0).Rows(count).Item(2) Then
                LoginValid = True
                'Storing username for player 1
                Player1Username = ds.Tables(0).Rows(count).Item(1)
                Player1ID = ds.Tables(0).Rows(count).Item(0)
                'If login valid exit loop
                Exit For
            End If
        Next
    End Sub
End Class
```

```

        ElseIf Usenametxt.Text = ds.Tables(0).Rows(count).Item(3) And
Passwordtxt.Text = ds.Tables(0).Rows(count).Item(2) Then
            LoginValid = True
            'Storing username for player 1
            Player1Username = ds.Tables(0).Rows(count).Item(1)
            Player1ID = ds.Tables(0).Rows(count).Item(0)
            'If login valid exit loop
            Exit For
        Else
            LoginValid = False
        End If
    Next
    cn.Close() 'Close connection

    'Displaying logged in or not
    If LoginValid = True Then
        MessageBox.Show("Welcome " & Player1Username)
        'Clearing text boxes
        Usenametxt.Text = ""
        Passwordtxt.Text = ""
        MainMenu.Show() 'open main menu form
        Me.Hide() 'hide login form
    Else
        MessageBox.Show("Incorrect details, please try again.")
        Passwordtxt.Text = ""
    End If

End Sub

Private Sub LoginForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
    'Setting text box input
    Me.Passwordtxt.PasswordChar = "*"
    Me.AcceptButton = LoginButton
End Sub

Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
Handles ShowPassword.CheckedChanged

    If Me.ShowPassword.Checked = True Then
        Me.Passwordtxt.PasswordChar = "" 'If user ticks the show password
box, show password
    Else
        Me.Passwordtxt.PasswordChar = "*" 'If box is unticked turn
password into *
    End If
End Sub

Private Sub CloseProgramLabel_Click(sender As Object, e As EventArgs)
Handles CloseProgramLabel.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox

```

```

        If result = System.Windows.Forms.DialogResult.Yes Then
            'end program
        End
    End If
End Sub

Dim Message, Title, MyValue, MyResetCode
Dim number(4) As Integer
Private Sub ForgottenPasswordLink_LinkClicked(sender As Object, e As
LinkLabelLinkClickedEventArgs) Handles ForgottenPasswordLink.LinkClicked
    Message = "Enter your email address you would like to reset your
password for" ' Set prompt.
    Title = "Forgotten Password" ' Set title.
    ' Display message, title, and default value.
    MyValue = InputBox(Message, Title)
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Email], [ID] FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer
    Dim attemptscount As Integer = 3

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If MyValue = ds.Tables(0).Rows(count).Item(0) Then 'For player 1
stats
            SendCode()
            Player1ID = ds.Tables(0).Rows(count).Item(1)
            Do
                Message = "Enter your 4-digit validation code that has
been emailed to you. " & attemptscount & " attempts remaining." ' Set
prompt.
                Title = "Forgotten Password" ' Set title.
                ' Display message, title, and default value.
                MyResetCode = InputBox(Message, Title)
                If MyResetCode = number(1) & number(2) & number(3) &
number(4) Then
                    ForgottenPasswordForm.Show()
                    Exit Sub
                Else
                    attemptscount -= 1
                End If
            Loop Until attemptscount = 0
        End If
    Next count
End Sub

```

```

        MessageBox.Show("Ran out of attempts")
    Exit Sub
End If
Next
cn.Close()
MessageBox.Show("Email not found")

End Sub

Private Sub SendCode()
    Dim rnd As New Random
    'Produce 4 random numbers between 0 and 9
    number(1) = rnd.Next(0, 10)
    number(2) = rnd.Next(0, 10)
    number(3) = rnd.Next(0, 10)
    number(4) = rnd.Next(0, 10)
    Dim smtpserver As New SmtpClient
    Dim email As New MailMessage
    smtpserver.UseDefaultCredentials = False
    'Email being sent from
    smtpserver.Credentials = New
Net.NetworkCredential("15dlawrence@mgs.kent.sch.uk", "NeaChess123")
    smtpserver.Port = 587
    smtpserver.EnableSsl = True
    smtpserver.Host = "smtp-mail.outlook.com"
    email = New MailMessage
    email.From = New MailAddress("15dlawrence@mgs.kent.sch.uk")
    'Email being sent too...
    email.To.Add(MyValue)
    'Email Subject
    email.Subject = "Reset Password"
    email.IsBodyHtml = False
    'Sending message
    email.Body = "Your 4-digit reset code is : " & number(1) & number(2)
& number(3) & number(4)
    smtpserver.Send(email)
End Sub
End Class

```

Main Menu

```

Imports System.Data.OleDb
Imports System.Data

Public Class MainMenu
    Private currentform As Form

    Private Sub MainMenuTest_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        'Attaching Tool Tips
        HoverTool.SetToolTip(ExitButton, "Close")
        'Word document for rules
        'Set Default selection
        SelectedHomeTab.Show()
        'Load in user's username
        UserLabel.Text = LoginForm.Player1Username
    End Sub
End Class

```



```

        'Load in user's profile picture
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[ProfilePicture] FROM [Table1]" 'SQL
Statement
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
        Dim count As Integer
        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
            If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 profile picture (for a user that has signed in)
                Select Case (ds.Tables(0).Rows(count).Item(1))
                    Case "Default"
                        ProfilePicture.Image = My.Resources.DefaultSkin
                    Case "Skin1"
                        ProfilePicture.Image = My.Resources.Skin1
                    Case "Skin2"
                        ProfilePicture.Image = My.Resources.Skin2
                    Case "Skin3"
                        ProfilePicture.Image = My.Resources.Skin3
                    Case "Skin4"
                        ProfilePicture.Image = My.Resources.Skin4
                    Case "Skin5"
                        ProfilePicture.Image = My.Resources.Skin5
                    Case "Skin6"
                        ProfilePicture.Image = My.Resources.Skin6
                    Case "Skin7"
                        ProfilePicture.Image = My.Resources.Skin7
                    Case "Locked1"
                        ProfilePicture.Image = My.Resources.Locked1
                    Case "Locked2"
                        ProfilePicture.Image = My.Resources.Locked2
                    Case "Locked3"
                        ProfilePicture.Image = My.Resources.Locked3
                    Case "Locked4"
                        ProfilePicture.Image = My.Resources.Locked4
                    Case "Locked5"
                        ProfilePicture.Image = My.Resources.Locked5
                    Case "Locked6"
                        ProfilePicture.Image = My.Resources.Locked6
                    Case "Locked7"
                        ProfilePicture.Image = My.Resources.Locked7
                    Case "Locked8"

```

```

        ProfilePicture.Image = My.Resources.Locked8
    End Select
End If
Next
cn.Close() 'Close connection
'opening home form
OpenHomeForm(HomeForm)
'enable timer
Timer1.Enabled = True
End Sub

Private Sub ExitButton_Click(sender As Object, e As EventArgs) Handles
ExitButton.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'attempting to quit word document
        Try
            If RulesForm.wordapp IsNot Nothing Then
                RulesForm.wordapp.Quit()
            End If
        Catch ex As Exception
        End Try
        'end program
    End
End If
End Sub

Private Sub HomeButton_Click(sender As Object, e As EventArgs) Handles
HomeButton.Click
    'Show selection
    SelectedHomeTab.Show()
    'Hide selections
    SelectedPlayTab.Hide()
    SelectedCustomiseTab.Hide()
    SelectedLeaderBoardTab.Hide()
    SelectedReplaysTab.Hide()
    SelectedRulesTab.Hide()
    SelectedSettingsTab.Hide()
    SelectedSignOutTab.Hide()
    'Open form
    OpenHomeForm(HomeForm)
End Sub

Private Sub PlayButton_Click(sender As Object, e As EventArgs) Handles
PlayButton.Click
    'Show selection
    SelectedPlayTab.Show()
    'Hide selections
    SelectedHomeTab.Hide()
    SelectedCustomiseTab.Hide()
    SelectedLeaderBoardTab.Hide()
    SelectedReplaysTab.Hide()

```

```

        SelectedRulesTab.Hide()
        SelectedSettingsTab.Hide()
        SelectedSignOutTab.Hide()
        'Open form
        OpenPlayForm(PlayForm)
        'Close gameform
        GameForm1.Close()
    End Sub

    Private Sub CustomiseButton_Click(sender As Object, e As EventArgs)
Handles CustomiseButton.Click
        'Show selection
        SelectedCustomiseTab.Show()
        'Hide selections
        SelectedHomeTab.Hide()
        SelectedPlayTab.Hide()
        SelectedLeaderBoardTab.Hide()
        SelectedReplaysTab.Hide()
        SelectedRulesTab.Hide()
        SelectedSettingsTab.Hide()
        SelectedSignOutTab.Hide()
        'Open form
        OpenCustomisationForm(CustomisationForm)
    End Sub

    Private Sub LeaderboardButton_Click(sender As Object, e As EventArgs)
Handles LeaderboardButton.Click
        'Show selection
        SelectedLeaderBoardTab.Show()
        'Hide selections
        SelectedHomeTab.Hide()
        SelectedCustomiseTab.Hide()
        SelectedPlayTab.Hide()
        SelectedReplaysTab.Hide()
        SelectedRulesTab.Hide()
        SelectedSettingsTab.Hide()
        SelectedSignOutTab.Hide()
        'Open form
        OpenLeaderboardForm(LeaderboardForm)
    End Sub

    Private Sub ReplaysButton_Click(sender As Object, e As EventArgs) Handles
ReplaysButton.Click
        'Show selection
        SelectedReplaysTab.Show()
        'Hide selections
        SelectedHomeTab.Hide()
        SelectedCustomiseTab.Hide()
        SelectedLeaderBoardTab.Hide()
        SelectedPlayTab.Hide()
        SelectedRulesTab.Hide()
        SelectedSettingsTab.Hide()
        SelectedSignOutTab.Hide()
        'Open form
        OpenReplayForm(ReplayForm)
    End Sub

```

```

Private Sub RulesButton_Click(sender As Object, e As EventArgs) Handles
RulesButton.Click
    'Show selection
    SelectedRulesTab.Show()
    'Hide selections
    SelectedHomeTab.Hide()
    SelectedCustomiseTab.Hide()
    SelectedLeaderBoardTab.Hide()
    SelectedReplaysTab.Hide()
    SelectedPlayTab.Hide()
    SelectedSettingsTab.Hide()
    SelectedSignOutTab.Hide()
    'Open form
    OpenRulesForm(RulesForm)
End Sub

Private Sub SettingsButton_Click(sender As Object, e As EventArgs)
Handles SettingsButton.Click
    'Show selection
    SelectedSettingsTab.Show()
    'Hide selections
    SelectedHomeTab.Hide()
    SelectedCustomiseTab.Hide()
    SelectedLeaderBoardTab.Hide()
    SelectedReplaysTab.Hide()
    SelectedRulesTab.Hide()
    SelectedPlayTab.Hide()
    SelectedSignOutTab.Hide()
    'open form
    OpenSettingsForm(SettingsForm)
End Sub

Private Sub SignOutButton_Click(sender As Object, e As EventArgs) Handles
SignOutButton.Click
    'Show selection
    SelectedSignOutTab.Show()
    'Hide selections
    SelectedHomeTab.Hide()
    SelectedCustomiseTab.Hide()
    SelectedLeaderBoardTab.Hide()
    SelectedReplaysTab.Hide()
    SelectedRulesTab.Hide()
    SelectedPlayTab.Hide()
    SelectedSettingsTab.Hide()

    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want to sign out?", "Sign
out", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'Show Login form
        LoginForm.Show()
        'Close current form
        Me.Close()
    End If

```

```

End Sub
Private Sub OpenHomeForm(currentform As Form)
    If currentform IsNot Nothing Then
        currentform.Close()
        'Change size back to normal
        Size = New Drawing.Size(1103, 613)
    End If
    currentform = HomeForm
    currentform.Dock = DockStyle.Fill
    currentform.TopLevel = False
    MainPanel.Controls.Add(currentform)
    MainPanel.Tag = currentform
    currentform.BringToFront()
    'Recentre screen
    Me.CenterToScreen()
    currentform.Show()
End Sub

Private Sub OpenPlayForm(currentform As Form)
    If currentform IsNot Nothing Then
        currentform.Close()
        'Change size back to normal
        Size = New Drawing.Size(1103, 613)
    End If
    currentform = PlayForm
    currentform.Dock = DockStyle.Fill
    currentform.TopLevel = False
    MainPanel.Controls.Add(currentform)
    MainPanel.Tag = currentform
    currentform.BringToFront()
    'Recentre screen
    Me.CenterToScreen()
    currentform.Show()
End Sub

Private Sub OpenCustomisationForm(currentform As Form)
    If currentform IsNot Nothing Then
        currentform.Close()
        'Change size back to normal
        Size = New Drawing.Size(1103, 613)
    End If
    currentform = CustomisationForm
    currentform.Dock = DockStyle.Fill
    currentform.TopLevel = False
    MainPanel.Controls.Add(currentform)
    MainPanel.Tag = currentform
    currentform.BringToFront()
    'Recentre screen
    Me.CenterToScreen()
    currentform.Show()
End Sub

Private Sub OpenReplayForm(currentform As Form)
    If currentform IsNot Nothing Then
        currentform.Close()
    End If

```

```

        currentform = ReplayForm
        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainPanel.Controls.Add(currentform)
        MainPanel.Tag = currentform
        currentform.BringToFront()
        'Change size to be able to fit form in
        Size = New Drawing.Size(1103, 830)
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub

    Private Sub OpenLeaderboardForm(currentform As Form)

        If currentform IsNot Nothing Then
            currentform.Close()
            'Change size back to normal
            Size = New Drawing.Size(1103, 613)
        End If
        currentform = LeaderboardForm
        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainPanel.Controls.Add(currentform)
        MainPanel.Tag = currentform
        currentform.BringToFront()
        'Change size to be able to fit form in
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub

    Private Sub OpenRulesForm(currentform As Form)
        If currentform IsNot Nothing Then
            currentform.Close()
            'Change size back to normal
            Size = New Drawing.Size(1103, 613)
        End If
        currentform = RulesForm
        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainPanel.Controls.Add(currentform)
        MainPanel.Tag = currentform
        currentform.BringToFront()
        'Change size to be able to fit form in
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub

    Private Sub OpenSettingsForm(currentform As Form)
        If currentform IsNot Nothing Then
            currentform.Close()
            'Change size back to normal
            Size = New Drawing.Size(1103, 613)
        End If
        currentform = SettingsForm
    
```

```

        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainPanel.Controls.Add(currentform)
        MainPanel.Tag = currentform
        currentform.BringToFront()
        'Change size to be able to fit form in
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub

    Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles
Timer1.Tick
        Datelabel.Text = Date.Now.ToString("dd MMM yyyy  hh:mm:ss")
    End Sub

    Public Sub CentresScreen()
        Me.CenterToScreen()
    End Sub
End Class

```

Multiplayer

```

Imports System.Data.OleDb
Imports System.Data
Public Class MultiplayerForm
    Private currentform As Form
    Dim LoginValid As Boolean = True 'Variable for knowing if User
Credentials match up
    Public Player2Username As String
    Public Player2ID As Integer
    Public Player1AlreadySignedIn As Boolean = False 'Variable for if a user
signs up on this page, after they will return to this form
    Private Sub MultiplayerForm_Load(sender As Object, e As EventArgs)
Handles MyBase.Load
        'Loading in logged in players name
        User1Label.Text = LoginForm.Player1Username
        'Setting text box input
        Me.Passwordtxt.PasswordChar = "*"
    End Sub

    Private Sub GuestButton_Click(sender As Object, e As EventArgs) Handles
GuestButton.Click
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        'displaying messagebox
        result = MessageBox.Show("Are you sure you want to continue as a
Guest?", "Guest conformation", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            'User 2 is guest
            Player2Username = "Guest"
            User2Label.Text = Player2Username
            'Allow them to play by adding play button
            PlayButton.Visible = True
            'Hiding all the login components

```

```

        LoginButton.Visible = False
        GuestButton.Visible = False
        Label5.Visible = False
        Label4.Visible = False
        Label3.Visible = False
        SignUpLink.Visible = False
        UsernameTxt.Visible = False
        PasswordTxt.Visible = False
        ShowPassword.Visible = False
    End If
End Sub

Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
Handles ShowPassword.CheckedChanged
    If Me.ShowPassword.Checked = True Then
        Me.PasswordTxt.PasswordChar = "" 'If user ticks the show password
box, show password
    Else
        Me.PasswordTxt.PasswordChar = "*" 'If box is unticked turn
password into *
    End If
End Sub

Private Sub LoginButton_Click(sender As Object, e As EventArgs) Handles
LoginButton.Click
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[Password],[Email],[ID] FROM [Table1]"
'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If UsernameTxt.Text = ds.Tables(0).Rows(count).Item(0) And
PasswordTxt.Text = ds.Tables(0).Rows(count).Item(1) Then
            'Storing username for player 2
            Player2Username = ds.Tables(0).Rows(count).Item(0)
            Player2ID = ds.Tables(0).Rows(count).Item(3)

            If Player2Username = LoginForm.Player1Username Then 'If user
already signed in log in is not valid

```



```

        LoginValid = False
        MessageBox.Show("Error - User already signed in")
    Else
        LoginValid = True
    End If
    'If login valid exit loop
    Exit For
ElseIf Usernametxt.Text = ds.Tables(0).Rows(count).Item(2) And
Passwordtxt.Text = ds.Tables(0).Rows(count).Item(1) Then
    'Storing username for player 2
    Player2Username = ds.Tables(0).Rows(count).Item(0)
    If Player2Username = LoginForm.Player1Username Then 'If user
already signed in log in is not valid
        LoginValid = False
        MessageBox.Show("Error - User already signed in")
    Else
        LoginValid = True
    End If
    'If login valid exit loop
    Exit For
Else
    LoginValid = False
End If
Next
cn.Close() 'Close connection

'Displaying logged in or not
If LoginValid = True Then
    MessageBox.Show("Welcome " & Player2Username)
    User2Label.Text = Player2Username
    'Allow them to play by adding play button
    PlayButton.Visible = True
    'Clearing text boxes
    Usernametxt.Text = ""
    Passwordtxt.Text = ""
    'Hiding all the login components
    LoginButton.Visible = False
    GuestButton.Visible = False
    Label5.Visible = False
    Label4.Visible = False
    Label3.Visible = False
    SignUpLink.Visible = False
    Usernametxt.Visible = False
    Passwordtxt.Visible = False
    ShowPassword.Visible = False
Else
    MessageBox.Show("Incorrect details, please try again.")
    Passwordtxt.Text = ""
End If
End Sub

Private Sub SignUpLink_LinkClicked(sender As Object, e As
LinkLabelLinkClickedEventArgs) Handles SignUpLink.LinkClicked
    'Set variable as true so after user signs up they return to this form
    Player1AlreadySignedIn = True
    'Show sign up form
    SignUpForm.Show()

```

```

        'Disable Main Menu
        MainMenu.Enabled = False
        'Hide current form
        Me.Hide()
    End Sub

    Private Sub PlayButton_Click(sender As Object, e As EventArgs) Handles
PlayButton.Click
        GameForm1.gameistwooplayer = True
        'Open game form
        OpenGameForm(GameForm1)
    End Sub

    Private Sub OpenGameForm(currentform As Form)
        If currentform IsNot Nothing Then
            'Hide not close form, to allow form to give the difficulty level
to the gameform
            currentform.Hide()
        End If
        currentform = GameForm1
        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainMenu.MainPanel.Controls.Add(currentform)
        MainMenu.MainPanel.Tag = currentform
        currentform.BringToFront()
        'Change size to be able to fit form in
        MainMenu.Size = New Drawing.Size(1320, 870)
        'Recentre screen
        MainMenu.CentresScreen()
        currentform.Show()
    End Sub
End Class

```

Play

```

Public Class PlayForm
    Private currentform As Form

    Private Sub SinglePlayerButton_Click(sender As Object, e As EventArgs)
Handles SinglePlayerButton.Click
        OpenSinglePlayerForm(SingleplayerForm)
    End Sub

    Private Sub MultiplayerButton_Click(sender As Object, e As EventArgs)
Handles MultiplayerButton.Click
        OpenMultiplayerForm(MultiplayerForm)
    End Sub

    Private Sub OpenSinglePlayerForm(currentform As Form)
        If currentform IsNot Nothing Then
            currentform.Close()
            'Change size back to normal
            Size = New Drawing.Size(1103, 613)
        End If
        currentform = SingleplayerForm
        currentform.Dock = DockStyle.Fill
    End Sub
End Class

```

```

        currentform.TopLevel = False
        MainMenu.MainPanel.Controls.Add(currentform)
        MainMenu.MainPanel.Tag = currentform
        currentform.BringToFront()
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub

    Private Sub OpenMultiplayerForm(currentform As Form)
        If currentform IsNot Nothing Then
            currentform.Close()
            'Change size back to normal
            Size = New Drawing.Size(1103, 613)
        End If
        currentform = MultiplayerForm
        currentform.Dock = DockStyle.Fill
        currentform.TopLevel = False
        MainMenu.MainPanel.Controls.Add(currentform)
        MainMenu.MainPanel.Tag = currentform
        currentform.BringToFront()
        'Recentre screen
        Me.CenterToScreen()
        currentform.Show()
    End Sub
End Class

```

Replay

```

Imports System.Data.OleDb
Imports System.Data
Imports System.Threading
Public Class ReplayForm
    Dim pb(7, 7) As PictureBox
    Dim board(7, 7) As Integer
    Dim Set1 As Boolean = False
    Dim Set2 As Boolean = False
    Dim Set3 As Boolean = False
    Dim PreMove As String
    Dim PostMove As String
    Dim gamelength As Integer
    Dim ReplayNumber As Integer = 0
    Private Sub ReplayForm_Load(sender As Object, e As EventArgs) Handles
        MyBase.Load
            'Creating picture boxes as an array
            pb = New PictureBox(7, 7) {{Me.p00, Me.p01, Me.p02, Me.p03, Me.p04,
Me.p05, Me.p06, Me.p07},
                {Me.p10, Me.p11, Me.p12, Me.p13, Me.p14, Me.p15, Me.p16, Me.p17},
                {Me.p20, Me.p21, Me.p22, Me.p23, Me.p24, Me.p25, Me.p26, Me.p27},
                {Me.p30, Me.p31, Me.p32, Me.p33, Me.p34, Me.p35, Me.p36, Me.p37},
                {Me.p40, Me.p41, Me.p42, Me.p43, Me.p44, Me.p45, Me.p46, Me.p47},
                {Me.p50, Me.p51, Me.p52, Me.p53, Me.p54, Me.p55, Me.p56, Me.p57},
                {Me.p60, Me.p61, Me.p62, Me.p63, Me.p64, Me.p65, Me.p66, Me.p67},
                {Me.p70, Me.p71, Me.p72, Me.p73, Me.p74, Me.p75, Me.p76, Me.p77}}

```

```

board = {{-5, -4, -3, -2, -1, -3, -4, -5},
        {-6, -6, -6, -6, -6, -6, -6, -6},
        {0, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 0},
        {6, 6, 6, 6, 6, 6, 6, 6},
        {5, 4, 3, 2, 1, 3, 4, 5}}

LoadReplay()
showpiece()
End Sub

Private Sub showpiece()
    'Loading pieces onto board for user
    For i As Integer = 0 To 7
        If board(7, i) = -6 Then
            board(7, i) = -2
        End If
    Next
    If Set1 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)
                    Case 1
                        pb(i, j).Image = My.Resources.Set1KingW
                    Case 2
                        pb(i, j).Image = My.Resources.Set1QueenW
                    Case 3
                        pb(i, j).Image = My.Resources.Set1BishopW
                    Case 4
                        pb(i, j).Image = My.Resources.Set1KnightW
                    Case 5
                        pb(i, j).Image = My.Resources.Set1RookW
                    Case 6
                        pb(i, j).Image = My.Resources.Set1PawnW
                    Case -1
                        pb(i, j).Image = My.Resources.Set1KingB
                    Case -2
                        pb(i, j).Image = My.Resources.Set1QueenB
                    Case -3
                        pb(i, j).Image = My.Resources.Set1BishopB
                    Case -4
                        pb(i, j).Image = My.Resources.Set1KnightB
                    Case -5
                        pb(i, j).Image = My.Resources.Set1RookB
                    Case -6
                        pb(i, j).Image = My.Resources.Set1PawnB
                    Case Else
                        pb(i, j).Image = Nothing
                End Select
            Next
        Next
    ElseIf Set2 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)

```

```

Case 1
    pb(i, j).Image = My.Resources.Set2KingW
Case 2
    pb(i, j).Image = My.Resources.Set2QueenW
Case 3
    pb(i, j).Image = My.Resources.Set2BishopW
Case 4
    pb(i, j).Image = My.Resources.Set2KnightW
Case 5
    pb(i, j).Image = My.Resources.Set2RookW
Case 6
    pb(i, j).Image = My.Resources.Set2PawnW
Case -1
    pb(i, j).Image = My.Resources.Set2KingB
Case -2
    pb(i, j).Image = My.Resources.Set2QueenB
Case -3
    pb(i, j).Image = My.Resources.Set2BishopB
Case -4
    pb(i, j).Image = My.Resources.Set2KnightB
Case -5
    pb(i, j).Image = My.Resources.Set2RookB
Case -6
    pb(i, j).Image = My.Resources.Set2PawnB
Case Else
    pb(i, j).Image = Nothing
End Select
Next
Next
ElseIf Set3 = True Then
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            Select Case board(i, j)
            Case 1
                pb(i, j).Image = My.Resources.Set3KingW
            Case 2
                pb(i, j).Image = My.Resources.Set3QueenW
            Case 3
                pb(i, j).Image = My.Resources.Set3BishopW
            Case 4
                pb(i, j).Image = My.Resources.Set3KnightW
            Case 5
                pb(i, j).Image = My.Resources.Set3RookW
            Case 6
                pb(i, j).Image = My.Resources.Set3PawnW
            Case -1
                pb(i, j).Image = My.Resources.Set3KingB
            Case -2
                pb(i, j).Image = My.Resources.Set3QueenB
            Case -3
                pb(i, j).Image = My.Resources.Set3BishopB
            Case -4
                pb(i, j).Image = My.Resources.Set3KnightB
            Case -5
                pb(i, j).Image = My.Resources.Set3RookB
            Case -6
                pb(i, j).Image = My.Resources.Set3PawnB

```

```

        Case Else
            pb(i, j).Image = Nothing
        End Select
    Next
Next
End If
End Sub

Private Sub redoback()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            pb(i, j).BackgroundImage = Nothing
        Next
    Next
End Sub

Private Sub LoadReplay()
    'Loading User 1 profile details
    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[BoardColour],[PieceType], [PreMove1],
[Move1], [PreMove2], [Move2], [PreMove3], [Move3], [PreMove4], [Move4],
[PreMove5], [Move5] FROM [Table1]" 'SQL Statement
    Dim cmd As New OleDbCommand(sql, cn)
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 profile picture (for a user that has signed in)
            Select Case (ds.Tables(0).Rows(count).Item(1))
                Case "YellowBoard"
                    Me.BackgroundImage = My.Resources.YellowBoard
                Case "GreenBoard"
                    Me.BackgroundImage = My.Resources.GreenBoard
                Case "PinkBoard"
                    Me.BackgroundImage = My.Resources.PinkBoard
                Case "RedBoard"
                    Me.BackgroundImage = My.Resources.RedBoard
                Case "BlueBoard"
                    Me.BackgroundImage = My.Resources.BlueBoard
            End Select
            Select Case (ds.Tables(0).Rows(count).Item(2))

```

```

        Case "Set1"
            Set1 = True
        Case "Set2"
            Set2 = True
        Case "Set3"
            Set3 = True
    End Select
    Try
        If ReplayNumber = 1 Then
            PreMove = ds.Tables(0).Rows(count).Item(3)
            PostMove = ds.Tables(0).Rows(count).Item(4)
        ElseIf ReplayNumber = 2 Then
            PreMove = ds.Tables(0).Rows(count).Item(5)
            PostMove = ds.Tables(0).Rows(count).Item(6)
        ElseIf ReplayNumber = 3 Then
            PreMove = ds.Tables(0).Rows(count).Item(7)
            PostMove = ds.Tables(0).Rows(count).Item(8)
        ElseIf ReplayNumber = 4 Then
            PreMove = ds.Tables(0).Rows(count).Item(9)
            PostMove = ds.Tables(0).Rows(count).Item(10)
        ElseIf ReplayNumber = 5 Then
            PreMove = ds.Tables(0).Rows(count).Item(11)
            PostMove = ds.Tables(0).Rows(count).Item(12)
        End If
        gamelength = Len(PreMove)
    Catch ex As Exception
        PreMove = ""
        PostMove = ""
    End Try
End If
Next
cn.Close() 'Close connection
End Sub

Dim CurrentPos As String = "Nothing"
Dim NextPos As String
Dim x As Integer = 0
Dim count As Integer = 1
Dim xposition(1) As Integer
Dim yposition(1) As Integer
Dim Premovepiecevalue(1000) As Integer
Dim Postmovepiecevalue As Integer
Private Sub MovePiece()
    RewindButton.Show()

    x = x + 1
    If x <= 0 Then
        x = 1
    End If
    CurrentPos = Mid(PreMove, x, 1)
    NextPos = Mid(PostMove, x, 1)
    Select Case CurrentPos
        Case "A"
            yposition(0) = 0
        Case "B"
            yposition(0) = 1
        Case "C"

```

```

        yposition(0) = 2
    Case "D"
        yposition(0) = 3
    Case "E"
        yposition(0) = 4
    Case "F"
        yposition(0) = 5
    Case "G"
        yposition(0) = 6
    Case "H"
        yposition(0) = 7
End Select
Select Case NextPos
    Case "A"
        yposition(1) = 0
    Case "B"
        yposition(1) = 1
    Case "C"
        yposition(1) = 2
    Case "D"
        yposition(1) = 3
    Case "E"
        yposition(1) = 4
    Case "F"
        yposition(1) = 5
    Case "G"
        yposition(1) = 6
    Case "H"
        yposition(1) = 7
End Select

x = x + 1

CurrentPos = Mid(PreMove, x, 1)
NextPos = Mid(PostMove, x, 1)

Select Case CurrentPos
    Case "1"
        xposition(0) = 7
    Case "2"
        xposition(0) = 6
    Case "3"
        xposition(0) = 5
    Case "4"
        xposition(0) = 4
    Case "5"
        xposition(0) = 3
    Case "6"
        xposition(0) = 2
    Case "7"
        xposition(0) = 1
    Case "8"
        xposition(0) = 0
End Select
Select Case NextPos
    Case "1"
        xposition(1) = 7

```



```

        Case "2"
            xposition(1) = 6
        Case "3"
            xposition(1) = 5
        Case "4"
            xposition(1) = 4
        Case "5"
            xposition(1) = 3
        Case "6"
            xposition(1) = 2
        Case "7"
            xposition(1) = 1
        Case "8"
            xposition(1) = 0
    End Select

    y = x

    If board(xposition(0), yposition(0)) <> Nothing Then
        Premovepiecevalue(count) = board(xposition(1), yposition(1))
        board(xposition(1), yposition(1)) = board(xposition(0),
yposition(0))
        board(xposition(0), yposition(0)) = 0
        count += 1

        RewindButton.Show()
        showpiece()
    ElseIf board(xposition(0), yposition(0)) = Nothing Then
        MovePiece()
    End If
    If x >= gamelength Then 'End of Game

        FastFowardTimer.Stop()
        AutoPlayTimer.Stop()

        PauseButton.Hide()
        PlayButton.Hide()
        FastFowardButton.Hide()

        x = gamelength - 2
    Exit Sub
End If
End Sub
Private Sub MovePieceBack()

    If y <= 0 Then
        RewindButton.Hide()
    Exit Sub
    End If
    y = y - 1
    CurrentPos = Mid(PreMove, y, 1)
    NextPos = Mid(PostMove, y, 1)
    Select Case CurrentPos
        Case "A"
            yposition(0) = 0
        Case "B"

```

```

        yposition(0) = 1
    Case "C"
        yposition(0) = 2
    Case "D"
        yposition(0) = 3
    Case "E"
        yposition(0) = 4
    Case "F"
        yposition(0) = 5
    Case "G"
        yposition(0) = 6
    Case "H"
        yposition(0) = 7
End Select
Select Case NextPos
    Case "A"
        yposition(1) = 0
    Case "B"
        yposition(1) = 1
    Case "C"
        yposition(1) = 2
    Case "D"
        yposition(1) = 3
    Case "E"
        yposition(1) = 4
    Case "F"
        yposition(1) = 5
    Case "G"
        yposition(1) = 6
    Case "H"
        yposition(1) = 7
End Select

y = y + 1
CurrentPos = Mid(PreMove, y, 1)
NextPos = Mid(PostMove, y, 1)

Select Case CurrentPos
    Case "1"
        xposition(0) = 7
    Case "2"
        xposition(0) = 6
    Case "3"
        xposition(0) = 5
    Case "4"
        xposition(0) = 4
    Case "5"
        xposition(0) = 3
    Case "6"
        xposition(0) = 2
    Case "7"
        xposition(0) = 1
    Case "8"
        xposition(0) = 0
End Select
Select Case NextPos
    Case "1"

```

```

        xposition(1) = 7
    Case "2"
        xposition(1) = 6
    Case "3"
        xposition(1) = 5
    Case "4"
        xposition(1) = 4
    Case "5"
        xposition(1) = 3
    Case "6"
        xposition(1) = 2
    Case "7"
        xposition(1) = 1
    Case "8"
        xposition(1) = 0
End Select
board(xposition(0), yposition(0)) = board(xposition(1), yposition(1))
count = count - 1
board(xposition(1), yposition(1)) = Premovepiecevalue(count)

showpiece()
y = y - 2
x = y - 2
End Sub
Private Sub Button1_Click(sender As Object, e As EventArgs)
    MovePiece()
End Sub

Dim y As Integer
Private Sub BackMoveButton_Click(sender As Object, e As EventArgs)
    MovePieceBack()
End Sub

Private Sub ReplaySelectBox_SelectedIndexChanged(sender As Object, e As
EventArgs) Handles ReplaySelectBox.SelectedIndexChanged
    Select Case ReplaySelectBox.SelectedItem
        Case "Replay 1"
            ReplayNumber = 1
            LoadReplay()
        Case "Replay 2"
            ReplayNumber = 2
            LoadReplay()
        Case "Replay 3"
            ReplayNumber = 3
            LoadReplay()
        Case "Replay 4"
            ReplayNumber = 4
            LoadReplay()
        Case "Replay 5"
            ReplayNumber = 5
            LoadReplay()
    End Select

    If PreMove <> "" And PostMove <> "" Then
        ReplaySelectBox.Hide()
        ChangeReplayButton.Show()
        PlayButton.Show()
    End If
End Sub

```

```

        FastFowardButton.Show()
        PauseButton.Show()
    Else
        MessageBox.Show("Replay not found, please try a different one")
    End If
End Sub

Private Sub ChangeReplayButton_Click(sender As Object, e As EventArgs)
Handles ChangeReplayButton.Click
    'Hiding buttons
    ChangeReplayButton.Hide()
    PlayButton.Hide()
    FastFowardButton.Hide()
    PauseButton.Hide()
    RewindButton.Hide()
    AutoPlayTimer.Stop()
    FastFowardTimer.Stop()
    ReverseTimer.Stop()
    ReplaySelectBox.Show()
    'Make sure board is back to start
    board = {{-5, -4, -3, -2, -1, -3, -4, -5},
             {-6, -6, -6, -6, -6, -6, -6, -6},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {0, 0, 0, 0, 0, 0, 0, 0},
             {6, 6, 6, 6, 6, 6, 6, 6},
             {5, 4, 3, 2, 1, 3, 4, 5}}
    'Displaying pieces on board
    showpiece()
    'Reset x and y
    x = 0
    y = 0
End Sub

Private Sub ReturnLabel_Click(sender As Object, e As EventArgs)
    'Show main menu form
    MainMenu.Show()
    'Close current form
    Me.Close()
End Sub

Private Sub CloseProgramLabel_Click(sender As Object, e As EventArgs)
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'end program
    End If
End Sub

Private Sub PlayButton_Click(sender As Object, e As EventArgs) Handles
PlayButton.Click

```

```

        AutoPlayTimer.Start()
        ReverseTimer.Stop()
        FastFowardTimer.Stop()
    End Sub

    Private Sub Timer1_Tick(sender As Object, e As EventArgs) Handles
AutoPlayTimer.Tick
        If x < gamelength Then
            MovePiece()
        ElseIf x >= gamelength Then
            AutoPlayTimer.Stop()
        End If
    End Sub

    Private Sub PauseButton_Click(sender As Object, e As EventArgs) Handles
PauseButton.Click
        AutoPlayTimer.Stop()
        ReverseTimer.Stop()
        FastFowardTimer.Stop()
    End Sub

    Private Sub RewindButton_Click(sender As Object, e As EventArgs) Handles
RewindButton.Click
        AutoPlayTimer.Stop()
        FastFowardTimer.Stop()
        ReverseTimer.Start()

        PauseButton.Show()
        PlayButton.Show()
        FastFowardButton.Show()
    End Sub

    Private Sub Reverse_Tick(sender As Object, e As EventArgs) Handles
ReverseTimer.Tick
        If y > 0 Then
            MovePieceBack()
        ElseIf y <= 0 Then
            ReverseTimer.Stop()
            RewindButton.Hide()
        End If
    End Sub

    Private Sub FastFowardButton_Click(sender As Object, e As EventArgs)
Handles FastFowardButton.Click
        AutoPlayTimer.Stop()
        ReverseTimer.Stop()
        FastFowardTimer.Start()
    End Sub

    Private Sub FastFowardTimer_Tick(sender As Object, e As EventArgs)
Handles FastFowardTimer.Tick
        If x < gamelength Then
            MovePiece()
        ElseIf x >= gamelength Then
            FastFowardTimer.Stop()
        End If
    End Sub
End Class

```

Rules

```
Imports Microsoft.Office.Interop
Public Class RulesForm
    Public wordapp As Word.Application

    Private Sub RulesButton_Click(sender As Object, e As EventArgs) Handles
RulesButton.Click
        'Don't want more than 1 form open at a time
        If wordapp IsNot Nothing Then
            wordapp.Quit()
        End If
        'Open rules document
        'creating rules document again as error would otherwise occur when
opening the rules document, if once closed.
        wordapp = CreateObject("word.application")
        'making word document visible
        wordapp.Visible = True
        'opening already existing document, as then can make it read only so
user cannot edit it

        wordapp.Documents.Open(System.Windows.Forms.Application.StartupPath &
"\RulesDocument.docx")
    End Sub
End Class
```

Settle

```
Imports System.Threading
Imports System.Data.OleDb
Imports System.Data
Public Class Settle
    Dim P1Choice As Boolean = False
    Dim P2Choice As Boolean = False
    Dim WantToSettle(1) As Boolean
    Dim WantToNotSettle(1) As Boolean
    Private Sub Settle_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
        'Disable form
        GameForm1.Enabled = False
        'Make timers stop
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()
    End Sub

    Private Sub P1Yes_Click(sender As Object, e As EventArgs) Handles
P1Yes.Click
        'Make picture visible and display Tick
        Player1Box.BackgroundImage = Nothing
        Player1Box.Visible = True
        P1Choice = True
        WantToSettle(0) = True
        P1Yes.Hide()
        P1No.Hide()
        DecideSettle()
    End Sub
```

```

Private Sub P1No_Click(sender As Object, e As EventArgs) Handles
P1No.Click
    'Make picture visible and display Cross
    Player1Box.Image = Nothing
    Player1Box.Visible = True
    P1Choice = True
    WantToNotSettle(0) = True
    P1Yes.Hide()
    P1No.Hide()
    DecideSettle()
End Sub

Private Sub P2Yes_Click(sender As Object, e As EventArgs) Handles
P2Yes.Click
    'Make picture visible and display Tick
    Player2Box.BackgroundImage = Nothing
    Player2Box.Visible = True
    P2Choice = True
    WantToSettle(1) = True
    P2Yes.Hide()
    P2No.Hide()
    DecideSettle()
End Sub

Private Sub P2No_Click(sender As Object, e As EventArgs) Handles
P2No.Click
    'Make picture visible and display Cross
    Player2Box.Image = Nothing
    Player2Box.Visible = True
    P2Choice = True
    WantToNotSettle(1) = True
    P2Yes.Hide()
    P2No.Hide()
    DecideSettle()
End Sub

Private Sub DecideSettle()
    If P1Choice = True And P2Choice = True Then
        'Sleeping system so the users can see result of settle
        My.Application.DoEvents()
        Thread.Sleep(1000)
        If WantToSettle(0) = True And WantToSettle(1) = True Then
            SettleAgreed()
        Else
            MessageBox.Show("Settle Denied")
            If GameForm1.ga.movecounter Mod 2 = 0 Then
                GameForm1.P1Timer.Start()
            Else
                GameForm1.P2Timer.Start()
            End If
        End If
        'Show form
        GameForm1.Enabled = True
        GameForm1.Show()

        Me.Close()
    End If
End Sub

```

```

End Sub

Public Sub SettleAgreed()
    GameForm1.ga.gameflag = False
    If GameForm1.ga.stalemate = True Then
        MessageBox.Show("Stalemate - Draw")
    Else
        MessageBox.Show("Settle Agreed - Draw")
    End If
    'Hiding Buttons
    GameForm1.P1SettleButton.Visible = False
    GameForm1.P2SettleButton.Visible = False
    GameForm1.P1ResignButton.Visible = False
    GameForm1.P2ResignButton.Visible = False

    'Getting both users stats for draws and storing them temporarily
    Dim Player1Draws As Integer
    Dim Player2Draws As Integer

    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim da As OleDbDataAdapter 'adapter
    Dim ds As New DataSet 'set used with adapter
    Dim sql As String
    sql = "SELECT [Username],[Draws] FROM [Table1]" 'SQL Statement
    da = New OleDbDataAdapter(sql, cn)
    cn.Open() 'Open connection
    da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
        If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats
            Player1Draws = ds.Tables(0).Rows(count).Item(1)
        ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
            Player2Draws = ds.Tables(0).Rows(count).Item(1)
        End If
    Next

    cn.Close() 'Close connection

    'Updating the database with the new P1 user stats
    Dim cmd As New OleDbCommand(sql, cn)
    sql = "update [Table1]"
    sql = sql & "SET [Draws] = '" & Player1Draws + 1 & "'"
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"

```



```

cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats
sql = "update [Table1]"
sql = sql & "SET [Draws] = '" & Player2Draws + 1 & "'"
sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection
'Disable game
GameForm1.ga.gameflag = False
GameForm1.gamestarted = False
'Hide buttons
GameForm1.P1ResignButton.Hide()
GameForm1.P1SettleButton.Hide()
GameForm1.P2ResignButton.Hide()
GameForm1.P2SettleButton.Hide()

GameForm1.ga.StoreGameReplay()
GameForm1.P1Timer.Stop()
GameForm1.P2Timer.Stop()
End Sub
End Class

```

Sign-up

```

Imports System.Net.Mail
Imports System.Data
Imports System.Data.OleDb
Public Class SignUpForm
    Dim UserIsValid As Boolean = True 'Variable which determines whether or
not the user can sign up
    Public Sub CheckAvailability()
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename
        Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Email] FROM [Table1]" 'SQL Statement
        Dim cmd As New OleDbCommand(sql, cn)
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
    End Sub
End Class

```

```

    Dim count As Integer

    For count = 0 To ds.Tables(0).Rows.Count - 1
        If Usernameetxt.Text = ds.Tables(0).Rows(count).Item(0) Then
            UserIsValid = False
            MessageBox.Show("Username already in use")
        ElseIf Emailtxt.Text = ds.Tables(0).Rows(count).Item(1) Then
            UserIsValid = False
            MessageBox.Show("Email already in use")
        End If
    Next
End Sub

Private Sub SignUpButton_Click(sender As Object, e As EventArgs) Handles
    SignUpButton.Click
        'Making the conditions for the username
        If Usernameetxt.Text = "" Then 'Username being empty
            MessageBox.Show("Please enter a username")
            UserIsValid = False
        ElseIf Usernameetxt.Text = "Guest" Or Usernameetxt.Text = "AI" Then
            MessageBox.Show("Username is not available to use")
            UserIsValid = False
        ElseIf Len(Usernameetxt.Text) <= 25 And Len(Usernameetxt.Text) >= 2
Then 'Username to be checked against DB
            UserIsValid = True
        Else
            MessageBox.Show("Username is invalid. Please make it between 2
and 25 characters")
            UserIsValid = False
        End If

        'Making the conditions for the password
        If UserIsValid = True Then
            If Passwordtxt.Text = "" Then
                MessageBox.Show("Please enter a password")
                UserIsValid = False
            ElseIf ConfirmPasswordtxt.Text = "" Then
                MessageBox.Show("Please confirm your password")
                UserIsValid = False
            ElseIf Len(Passwordtxt.Text) < 5 Then
                MessageBox.Show("Password is too short. Please make it
between 5 and 30 characters")
                UserIsValid = False
            ElseIf Len(Passwordtxt.Text) > 30 Then
                MessageBox.Show("Password is too long. Please make it between
5 and 30 characters")
                UserIsValid = False
            ElseIf Passwordtxt.Text <> ConfirmPasswordtxt.Text Then
                MessageBox.Show("Passwords do not match up")
                UserIsValid = False
            ElseIf Len(Passwordtxt.Text) >= 5 And Len(Passwordtxt.Text) <= 30
And Passwordtxt.Text = ConfirmPasswordtxt.Text Then 'Password is valid
                UserIsValid = True
            End If
        End If

        'Checking availability against db

```

```

    If UserIsValid = True Then
        CheckAvailability()
    End If
    'Attempting to send email, if doesn't work the email the user entered
was not valid
    If UserIsValid = True Then
        Try
            Dim smtpserver As New SmtpClient
            Dim email As New MailMessage

            smtpserver.UseDefaultCredentials = False
            smtpserver.Credentials = New
Net.NetworkCredential("15dlawrence@mgs.kent.sch.uk", "NeaChess123")

            smtpserver.Port = 587
            smtpserver.EnableSsl = True

            smtpserver.Host = "smtp-mail.outlook.com"

            email = New MailMessage
            email.From = New MailAddress("15dlawrence@mgs.kent.sch.uk")
            email.To.Add(Emailtxt.Text)
            email.Subject = "Sign up confirmation"
            email.IsBodyHtml = False
            email.Body = "Thank you for signing up to my Chess Game " &
Usernametxt.Text & "!"
            smtpserver.Send(email)

            UserIsValid = True
        Catch ex As Exception
            UserIsValid = False
            MessageBox.Show("Please enter a valid email")
        End Try
    End If
    'If everything is valid then add users credentials to the database
    If UserIsValid = True Then
        'Setting database location
        Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
        'Opening connection
        Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename
        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        cn.Open() 'open connection

        Dim sql As String
        sql = "INSERT INTO [Table1] ([Username], [Password], [Email],
[ProfilePicture], [BoardColour], [PieceType]) VALUES ("
        sql = sql & Usernametxt.Text & ", "
        sql = sql & Passwordtxt.Text & ", "
        sql = sql & Emailtxt.Text & ", "
        sql = sql & "Default" & ", "
        sql = sql & "YellowBoard" & ", "
        sql = sql & "Set1" & ");"
        'sends SQL command to OLEBD

```

```

Dim cmd As New OleDbCommand(sql, cn)
'Execute SQL command that doesn't return a query
cmd.ExecuteNonQuery()
'close connection
cn.Close()
MessageBox.Show("Sign up complete!")
'clearing text boxes
UsernameTxt.Text = ""
EmailTxt.Text = ""
PasswordTxt.Text = ""
ConfirmPasswordTxt.Text = ""

If MultiplayerForm.Player1AlreadySignedIn = True Then
    MultiplayerForm.Show()
    MainMenu.Enabled = True
    Me.Close()
Else
    LoginForm.Show()
    Me.Close()
End If
End If
End Sub

Private Sub iCloudButton_Click(sender As Object, e As EventArgs) Handles
iCloudButton.Click
    EmailTxt.AppendText("@icloud.com")
End Sub

Private Sub HotmailButton_Click(sender As Object, e As EventArgs) Handles
HotmailButton.Click
    EmailTxt.AppendText("@hotmail.com")
End Sub

Private Sub MGSButton_Click(sender As Object, e As EventArgs) Handles
MGSButton.Click
    EmailTxt.AppendText("@mgs.kent.sch.uk")
End Sub

Private Sub GmailButton_Click(sender As Object, e As EventArgs) Handles
GmailButton.Click
    EmailTxt.AppendText("@gmail.com")
End Sub

Private Sub YahooButton_Click(sender As Object, e As EventArgs) Handles
YahooButton.Click
    EmailTxt.AppendText("@yahoo.com")
End Sub

Private Sub SignUpForm_Load(sender As Object, e As EventArgs) Handles
MyBase.Load
    'Setting text box input
    Me.PasswordTxt.PasswordChar = "*"
    Me.ConfirmPasswordTxt.PasswordChar = "*"
    Me.AcceptButton = SignUpButton
End Sub

```

```

    Private Sub ShowPassword_CheckedChanged(sender As Object, e As EventArgs)
Handles ShowPassword.CheckedChanged
    If Me.ShowPassword.Checked = True Then
        Me.Passwordtxt.PasswordChar = "" 'If user ticks the show password
box, show password
        Me.ConfirmPasswordtxt.PasswordChar = ""
    Else
        Me.Passwordtxt.PasswordChar = "*" 'If box is unticked turn
password into *
        Me.ConfirmPasswordtxt.PasswordChar = "*"
    End If
End Sub

    Private Sub ReturnButton_Click(sender As Object, e As EventArgs) Handles
ReturnButton.Click
    If MultiplayerForm.Player1AlreadySignedIn = True Then
        MultiplayerForm.Show()
        MainMenu.Enabled = True
        Me.Close()
    Else
        LoginForm.Show()
        Me.Close()
    End If

End Sub

    Private Sub CloseProgramLabel_Click(sender As Object, e As EventArgs)
Handles CloseProgramLabel.Click
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show("Are you sure you want close the program?",
"Exit Program", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        'end program
    End
End If
End Sub
End Class

```

Singleplayer

```

Public Class SingleplayerForm
    Private currentform As Form

    Private Sub EasyButton_Click(sender As Object, e As EventArgs) Handles
EasyButton.Click
        GameForm1.EasyMode = True
        GameForm1.gameisoneplayer = True
        'Setting player 2 name as AI
        MultiplayerForm.Player2Username = "AI"
        'Show game form
        OpenGameForm(GameForm1)
    End Sub

```

```
Private Sub HardButton_Click(sender As Object, e As EventArgs) Handles
HardButton.Click
    GameForm1.HardMode = True

    GameForm1.gameisoneplayer = True
    'Setting player 2 name as AI
    MultiplayerForm.Player2Username = "AI"
    'Show game form
    OpenGameForm(GameForm1)
End Sub

Private Sub OpenGameForm(currentform As Form)
    If currentform IsNot Nothing Then
        'Hide not close form, to allow form to give the difficulty level
to the gameform
        currentform.Hide()
    End If
    currentform = GameForm1
    currentform.Dock = DockStyle.Fill
    currentform.TopLevel = False
    MainMenu.MainPanel.Controls.Add(currentform)
    MainMenu.MainPanel.Tag = currentform
    currentform.BringToFront()
    'Change size to be able to fit form in
    MainMenu.Size = New Drawing.Size(1320, 870)
    'Recentre screen
    MainMenu.CentresScreen()
    currentform.Show()
End Sub
End Class
```

Classes

Bishop

```

Public Class Bishop
    Public xposition, yposition As Integer 'Coordinates for bishop
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the bishop
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
bishop, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)
    Public Sub New(ByVal x, ByVal y, ByVal pvalue) 'giving the new piece its
coordinates, and piece value
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags() 'setting every possible move to false
    End Sub
    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function possiblemoves() As Boolean(,) 'returning all possible
moves
        reflags() 'setting every possible move to false
        'only calculate moves of the piece if it is still on the board
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags
    End Function
    Public Sub whitemove() 'all moves for white
        'Storing position of bishop
        Dim x As Integer = xposition
        Dim y As Integer = yposition
        allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

        'diagonal left top to bottom
        While (x < 7 And y > 0)
            x = x + 1
            y = y - 1

            'Code to make sure black king cannot move into a check position
            If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
                flags(x, y) = True
                allcheckmoves = False
            End If
        End While
    End Sub
    Public Sub blackmove() 'all moves for black
        'Storing position of bishop
        Dim x As Integer = xposition
        Dim y As Integer = yposition
        allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

        'diagonal right top to bottom
        While (x > 0 And y > 0)
            x = x - 1
            y = y - 1

            'Code to make sure black king cannot move into a check position
            If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
                flags(x, y) = True
                allcheckmoves = False
            End If
        End While
    End Sub
End Class

```

```

        End If

        If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
            GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
bishop was not there, would the king be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(x, y) = True 'allow movement
            ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = 3
            If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
white piece, exit loop (cannot travel through a piece)
                Exit While
            End If
            ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
                If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                        flags(x, y) = True
                    End If
                    'Returning the value of the board back to the original
values (before the temp changes)
                    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    GameForm1.ga.board(x, y) = 0
                    GameForm1.ga.board(xposition, yposition) = 3
                End If
            End If

```



```

        If GameForm1.ga.board(x, y) < 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the bishop was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
    End If
    Exit While
End If
Else
    Exit While
End If
End While

'Storing position of the bishop
x = xposition
y = yposition
allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

'diagonal right bottom to top
While (x > 0 And y < 7)
    x = x - 1
    y = y + 1

    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
        flags(x, y) = True
        allcheckmoves = False
    End If

    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move

```

```

        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
bishop was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement

        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
        If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
white piece, exit loop (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = 3
            End If
            If GameForm1.ga.board(x, y) < 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there

```

```

        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the bishop was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
        End If
    Exit While
End If
Else
    Exit While
End If
End While

    'Storing position of bishop
    x = xposition
    y = yposition
    allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

    'diagonal right top to bottom
    While (x < 7 And y < 7)
        x = x + 1
        y = y + 1

        'Code to make sure black king cannot move into a check position
        If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
            flags(x, y) = True
            allcheckmoves = False
        End If

        If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space

```

```

        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
bishop was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
        If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
white piece, exit loop (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = 3
            End If
            If GameForm1.ga.board(x, y) < 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = 10 'change desired space
to a random number

```

```

        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the bishop was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
    End If
Exit While
End If
Else
    Exit While
End If
End While

'Storing positions of bishop
x = xposition
y = yposition
allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

'diagonal left bottom to top
While (x > 0 And y > 0)
    x = x - 1
    y = y - 1

    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
        flags(x, y) = True
        allcheckmoves = False
    End If

    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
bishop was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then

```

```

        flags(x, y) = True 'allow movement
    ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
        flags(x, y) = True 'allow movement
    End If
    'Returning the value of the board back to the original values
    (before the temp changes)
    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
    GameForm1.ga.board(x, y) = temp
    GameForm1.ga.board(xposition, yposition) = 3
    If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
white piece, exit loop (cannot travel through a piece)
        Exit While
    End If
    ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
        If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
            GameForm1.ga.board(x, y) = -10 'Change space to a random
number

            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(x, y) = True
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(x, y) = 0
            GameForm1.ga.board(xposition, yposition) = 3
        End If
        If GameForm1.ga.board(x, y) < 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
            If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board

                temp = GameForm1.ga.board(x, y) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                GameForm1.ga.board(x, y) = 10 'change desired space
to a random number

                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the bishop was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(x, y) = True 'allow movement

```

```

        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 3
    End If
    Exit While
End If
Else
    Exit While
End If
End While
End Sub
Public Sub blackmove()
    'Storing position of bishop
    Dim x As Integer = xposition
    Dim y As Integer = yposition
    allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

    'diagonal left top to bottom
    While (x < 7 And y > 0)
        x = x + 1
        y = y - 1

        'Code to make sure white king cannot move into a check position
        If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
            flags(x, y) = True
            allcheckmoves = False
        End If

        If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
bishop was not there, would the king be in check
            If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(x, y) = True 'allow movement
            ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
                flags(x, y) = True 'allow movement
            End If

```

```

        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -3
        If GameForm1.ga.board(x, y) > 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                GameForm1.ga.board(x, y) = 10 'Change space to a random
number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = -3
            End If
            If GameForm1.ga.board(x, y) > 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
                    GameForm1.ga.recurrsionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the bishop was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(x, y) = True 'allow movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

```



```

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
    GameForm1.ga.board(x, y) = temp
    GameForm1.ga.board(xposition, yposition) = -3
    End If
    Exit While
End If
Else
    Exit While
End If
End While

'Storing position of bishop
x = xposition
y = yposition
allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

'diagonal right bottom to top
While (x > 0 And y < 7)
    x = x - 1
    y = y + 1

    'Code to make sure white king cannot move into a check position
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True
        allcheckmoves = False
    End If

    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
bishop was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp

```

```

        GameForm1.ga.board(xposition, yposition) = -3
        If GameForm1.ga.board(x, y) > 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                GameForm1.ga.board(x, y) = 10 'Change space to a random
number

                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = -3
            End If
            If GameForm1.ga.board(x, y) > 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board

                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number

                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the bishop was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(x, y) = True 'allow movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

                    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    GameForm1.ga.board(x, y) = temp
                    GameForm1.ga.board(xposition, yposition) = -3
                End If
            End If
        Exit While

```

```

        End If
    Else
        Exit While
    End If
End While

'Storing position of bishop
x = xposition
y = yposition
allcheckmoves = True 'Setting boolean value to true, allowing one
calculation to happen

'diagonal right top to bottom
While (x < 7 And y < 7)
    x = x + 1
    y = y + 1
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
bishop was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -3
        If GameForm1.ga.board(x, y) > 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space

```

```

        GameForm1.ga.board(x, y) = 10 'Change space to a random
number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(x, y) = True
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(x, y) = 0
        GameForm1.ga.board(xposition, yposition) = -3
    End If
    If GameForm1.ga.board(x, y) > 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see
that if the bishop was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = -3
        End If
        Exit While
    End If
Else
    Exit While
End If
End While

'Storing position of bishop
x = xposition
y = yposition

```

```

    allcheckmoves = True 'Setting boolean value to true, allowing one
    calculation to happen

    'diagonal left bottom to top
    While (x > 0 And y > 0)
        x = x - 1
        y = y - 1
        If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
            flags(x, y) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
bishop position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
bishop was not there, would the king be in check
            If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(x, y) = True 'allow movement
            ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = -3
            If GameForm1.ga.board(x, y) > 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
                Exit While
            End If
            ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
                If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
                    GameForm1.ga.board(x, y) = 10 'Change space to a random
number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there

```

```

        flags(x, y) = True
    End If
    'Returning the value of the board back to the original
values (before the temp changes)
    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
    GameForm1.ga.board(x, y) = 0
    GameForm1.ga.board(xposition, yposition) = -3
End If
    If GameForm1.ga.board(x, y) > 0 Then 'If the bishop can take
a black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If bishop can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current bishop position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see
that if the bishop was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = -3
        End If
    Exit While
End If
Else
    Exit While
End If
End While
End Sub
Public Sub reflags()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub
Public Sub fillcheckking()
    possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then
                checktheking(i, j) = True
            End If
        Next
    Next
End Sub

```

```

        Else
            checktheking(i, j) = False
        End If
    Next
Next
allcheckmoves = False 'Reset variable
End Sub
Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)
    'Changing the position of piece, old and new coordinates are passed
    in as parameters
    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0 'Setting old space as 0

    If GameForm1.ga.canwritetoboardtracker = True Then 'Variable always
    set to true, unless the computer is doing a calculation
        Dim whatcolour As String
        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"
        Else
            whatcolour = "Black"
        End If
        Dim chessnotationletter(1) As String
        Dim chessnotationnumber(1) As Integer
        For i = 0 To 1
            Select Case yposition
                Case 0
                    chessnotationletter(i) = "A"
                Case 1
                    chessnotationletter(i) = "B"
                Case 2
                    chessnotationletter(i) = "C"
                Case 3
                    chessnotationletter(i) = "D"
                Case 4
                    chessnotationletter(i) = "E"
                Case 5
                    chessnotationletter(i) = "F"
                Case 6
                    chessnotationletter(i) = "G"
                Case 7
                    chessnotationletter(i) = "H"
            End Select

            Select Case xposition
                Case 0
                    chessnotationnumber(i) = "8"
                Case 1
                    chessnotationnumber(i) = "7"
                Case 2
                    chessnotationnumber(i) = "6"
                Case 3
                    chessnotationnumber(i) = "5"
                Case 4
                    chessnotationnumber(i) = "4"
                Case 5
                    chessnotationnumber(i) = "3"
                Case 6

```

```

        chessnotationnumber(i) = "2"
    Case 7
        chessnotationnumber(i) = "1"
    End Select
    'Setting new bishop coordinates
    xposition = x
    yposition = y
Next
    'Writing to board tracker text box
    GameForm1.BoardTracker.AppendText(whatcolour & (" Bishop ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)
    End If
End Sub
End Class

```

Game

```

Imports System.Data.OleDb
Imports System.Data
Public Class Game
    Public board(7, 7) As Integer '8 by 8 grid like the size of the board
    Public canmove(7, 7) As Boolean '^
    Public checkforwhite(7, 7) As Boolean '^
    Public checkforblack(7, 7) As Boolean '^
    Public gameflag As Boolean = True 'gameflag being true allows game to
continue, if not game is ended
    Public player As Integer = 1 'player always start of as 1
    Public blackindanger As Boolean = False
    Public whiteindanger As Boolean = False
    Public movecounter As Integer 'variable to count moves and be displayed
on the game form
    Public xpositionCheck As Integer
    Public ypositionCheck As Integer
    Public recurrisionstop As Boolean = False
    Public castleincheck(1) As Boolean
    Public canwritetoboardtracker As Boolean = True
    Public stalemate As Boolean
    Public Sub New()
        'creating an array for chess board, positive pieces are white and
negative pieces are black, 0 are also blanks
        board = {{-5, -4, -3, -2, -1, -3, -4, -5},
                {-6, -6, -6, -6, -6, -6, -6, -6},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {0, 0, 0, 0, 0, 0, 0, 0},
                {6, 6, 6, 6, 6, 6, 6, 6},
                {5, 4, 3, 2, 1, 3, 4, 5}}
        recheckblack()
        recheckwhite()
    End Sub
    Private Sub checkwinner()
        Dim checkmate(5) As Boolean
        If AICalculating = False Then
            show() 'show images

```



```

End If
If blackindanger = True Then
    gameflag = False
    checkmate(0) = True
    checkmate(1) = True
    checkmate(2) = True
    checkmate(3) = True
    checkmate(4) = True
    checkmate(5) = True
    checkforwhite(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True

    'Possible moves for the black king, if there is a move possible
then checkmate = false
    GameForm1.blackking.possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            If GameForm1.blackking.flags(i, j) = True Then
                checkmate(0) = False
            End If
        Next
    Next
Next

    'Possible moves for the black pawns, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 7
        GameForm1.blackpawn(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackpawn(k).flags(i, j) = True Then
                    checkmate(1) = False
                End If
            Next
        Next
    Next
Next

    'Possible moves for the black rook, if there is a move possible
then checkmate = false
    For k As Integer = 0 To 1 + GameForm1.brookcount
        GameForm1.blackrook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                If GameForm1.blackrook(k).flags(i, j) = True Then
                    checkmate(2) = False
                End If
            Next
        Next
    Next
Next

    'Possible moves for the black bishop, if there is a move possible
then checkmate = false

```

```

For k As Integer = 0 To 1 + GameForm1.bbishopcount
    GameForm1.blackbishop(k).possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            If GameForm1.blackbishop(k).flags(i, j) = True Then
                checkmate(3) = False
            End If
        Next
    Next
Next

'Possible moves for the black knight, if there is a move possible
then checkmate = false
For k As Integer = 0 To 1 + GameForm1.bknightcount
    GameForm1.blackknight(k).possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            If GameForm1.blackknight(k).flags(i, j) = True Then
                checkmate(4) = False
            End If
        Next
    Next
Next

'Possible moves for the black queen, if there is a move possible
then checkmate = false
For k As Integer = 0 To GameForm1.bqueencount
    GameForm1.blackqueen(k).possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            If GameForm1.blackqueen(k).flags(i, j) = True Then
                checkmate(5) = False
            End If
        Next
    Next
Next

If checkmate(0) = True And checkmate(1) = True And checkmate(2) =
True And checkmate(3) = True And checkmate(4) = True And checkmate(5) = True
Then
    If AICalculating = False Then
        'Stop game
        gameflag = False
        GameForm1.gamestarted = False
        'Stop timers
        GameForm1.P1Timer.Stop()
        GameForm1.P2Timer.Stop()
        'Hide buttons
        GameForm1.P1ResignButton.Hide()
        GameForm1.P1SettleButton.Hide()
        GameForm1.P2ResignButton.Hide()
    End If
End If

```

```

GameForm1.P2SettleButton.Hide()

'Display winner
MsgBox("Checkmate, " & GameForm1.Player1Label.Text & " is
the winner!")

'Getting both users stats and storing them temporarily
(only win and lose, as settling form has the drawn stats changed)
Dim Player1Wins As Integer
Dim Player2Loses As Integer

'Setting database location
Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
'Opening connection
Dim DBconn As String =
"Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE provider for MS Access
Dim DBsource As String = "Data Source=" & dbfilename

Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
Dim sql As String
sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]"
'SQL Statement

da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered
by adapter

Dim count As Integer

For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop
for amount of users in table, (-1) as starts from 0
    If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
        Player1Wins = ds.Tables(0).Rows(count).Item(1)
    ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
        Player2Loses = ds.Tables(0).Rows(count).Item(2)
    End If
Next

cn.Close() 'Close connection

'Updating the database with the new P1 user stats
Dim cmd As New OleDbCommand(sql, cn)
sql = "update [Table1]"
sql = sql & "SET [Wins] = '" & Player1Wins + 1 & "'"
sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
cmd = New OleDbCommand(sql, cn)
cn.Open() 'Open connection
cmd.ExecuteNonQuery()
cn.Close() 'Close connection

'Updating the database with the new P2 user stats

```

```

        sql = "update [Table1]"
        sql = sql & "SET [Loses] = '" & Player2Loses + 1 & "' "
        sql = sql & "WHERE [ID] = " & MultiplayerForm.Player2ID &
";"

        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        StoreGameReplay()
    End If
Else
    'Checkmate is not true, therefore continue game
    gameflag = True
End If

If gameflag = True Then
    stalemate = True
    'Possible moves for the black king, if there is a move possible
then stalemate = false
    GameForm1.blackking.possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If GameForm1.blackking.flags(i, j) = True Then
                stalemate = False
            End If
        Next
    Next

    'Possible moves for the black pawns, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 7
        GameForm1.blackpawn(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackpawn(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next

    'Possible moves for the black rook, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.brookcount
        GameForm1.blackrook(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackrook(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
Next

```

```

        'Possible moves for the black bishop, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.bbishopcount
        GameForm1.blackbishop(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackbishop(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
Next

    'Possible moves for the black knight, if there is a move possible
then stalemate = false
    For k As Integer = 0 To 1 + GameForm1.bknightcount
        GameForm1.blackknight(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackknight(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
Next

    'Possible moves for the black queen, if there is a move possible
then stalemate = false
    For k As Integer = 0 To GameForm1.bqueencount
        GameForm1.blackqueen(k).possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackqueen(k).flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
    Next
Next

    If stalemate = True Then
        Settle.SettleAgreed()
    End If
    'Check if it is only kings left on the board
    KingsOnly()
    If stalemate = True Then
        Settle.SettleAgreed()
    End If
End If

If whiteindanger = True Then
    show()
    gameflag = False
    checkmate(0) = True
    checkmate(1) = True
    checkmate(2) = True
    checkmate(3) = True

```

```

        checkmate(4) = True
        checkmate(5) = True
        checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        'Possible moves for the white king, if there is a move possible
then checkmate = false
        GameForm1.whiteking.possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                If GameForm1.whiteking.flags(i, j) = True Then
                    checkmate(0) = False
                End If
            Next
        Next
        'Possible moves for the white pawn, if there is a move possible
then checkmate = false
        For k As Integer = 0 To 7
            GameForm1.whitepawn(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    If GameForm1.whitepawn(k).flags(i, j) = True Then
                        checkmate(1) = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white rook, if there is a move possible
then checkmate = false
        For k As Integer = 0 To 1 + GameForm1.wrookcount
            GameForm1.whiterook(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    If GameForm1.whiterook(k).flags(i, j) = True Then
                        checkmate(2) = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white bishop, if there is a move possible
then checkmate = false
        For k As Integer = 0 To 1 + GameForm1.wbishopcount
            GameForm1.whitebishop(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    If GameForm1.whitebishop(k).flags(i, j) = True Then
                        checkmate(3) = False
                    End If
                Next
            Next
        Next

```

```

        Next
        'Possible moves for the white knight, if there is a move possible
then checkmate = false
        For k As Integer = 0 To 1 + GameForm1.wknightcount
            GameForm1.whiteknight(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    If GameForm1.whiteknight(k).flags(i, j) = True Then
                        checkmate(4) = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white queen, if there is a move possible
then checkmate = false
        For k As Integer = 0 To GameForm1.wqueencount
            GameForm1.whitequeen(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    If GameForm1.whitequeen(k).flags(i, j) = True Then
                        checkmate(5) = False
                    End If
                Next
            Next
        Next
        If checkmate(0) = True And checkmate(1) = True And checkmate(2) =
True And checkmate(3) = True And checkmate(4) = True And checkmate(5) = True
Then
            If AICalculating = False Then
                'Stop game
                gameflag = False
                GameForm1.gamestarted = False
                'Stop timers
                GameForm1.P1Timer.Stop()
                GameForm1.P2Timer.Stop()
                'Hide buttons
                GameForm1.P1ResignButton.Hide()
                GameForm1.P1SettleButton.Hide()
                GameForm1.P2ResignButton.Hide()
                GameForm1.P2SettleButton.Hide()

                'Display winner
                MsgBox("Checkmate, " & GameForm1.Player2Label.Text & " is
the winner!")

                'Getting both users stats and storing them temporarily
                (only win and lose, as settling form has the drawn stats changed)
                Dim Player1Loses As Integer
                Dim Player2Wins As Integer

                'Setting database location
                Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"

```

```

        'Openning connection
        Dim DBconn As String =
"Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE provider for MS Access
        Dim DBsource As String = "Data Source=" & dbfilename

        Dim cn As OleDbConnection = New OleDbConnection(DBconn &
DBsource) 'connection
        Dim da As OleDbDataAdapter 'adapter
        Dim ds As New DataSet 'set used with adapter
        Dim sql As String
        sql = "SELECT [Username],[Wins], [Loses] FROM [Table1]"
'SQL Statement
        da = New OleDbDataAdapter(sql, cn)
        cn.Open() 'Open connection
        da.Fill(ds, "Table1") 'Fill data set with data gathered
by adapter

        Dim count As Integer

        For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop
for amount of users in table, (-1) as starts from 0
            If LoginForm.Player1Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 1 stats
                Player1Loses = ds.Tables(0).Rows(count).Item(2)
            ElseIf MultiplayerForm.Player2Username =
ds.Tables(0).Rows(count).Item(0) Then 'For player 2 stats
                Player2Wins = ds.Tables(0).Rows(count).Item(1)
            End If
        Next

        cn.Close() 'Close connection

        'Updating the database with the new P1 user stats
        Dim cmd As New OleDbCommand(sql, cn)
        sql = "update [Table1]"
        sql = sql & "SET [Loses] = '" & Player1Loses + 1 & "' "
        sql = sql & "WHERE [ID] = '" & LoginForm.Player1ID & "';"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        'Updating the database with the new P2 user stats
        sql = "update [Table1]"
        sql = sql & "SET [Wins] = '" & Player2Wins + 1 & "' "
        sql = sql & "WHERE [ID] = '" & MultiplayerForm.Player2ID &
";"

        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

        StoreGameReplay()
    End If
Else
    gameflag = True

```



```

        End If
    End If

    If gameflag = True Then
        stalemate = True
        'Possible moves for the white king, if there is a move possible
    then stalemate = false
        GameForm1.whiteking.possiblemoves()
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.whiteking.flags(i, j) = True Then
                    stalemate = False
                End If
            Next
        Next
        'Possible moves for the white pawn, if there is a move possible
    then stalemate = false
        For k As Integer = 0 To 7
            GameForm1.whitepawn(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    If GameForm1.whitepawn(k).flags(i, j) = True Then
                        stalemate = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white rook, if there is a move possible
    then stalemate = false
        For k As Integer = 0 To 1 + GameForm1.wrookcount
            GameForm1.whiterook(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    If GameForm1.whiterook(k).flags(i, j) = True Then
                        stalemate = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white bishop, if there is a move possible
    then stalemate = false
        For k As Integer = 0 To 1 + GameForm1.wbishopcount
            GameForm1.whitebishop(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7
                    If GameForm1.whitebishop(k).flags(i, j) = True Then
                        stalemate = False
                    End If
                Next
            Next
        Next
        'Possible moves for the white knight, if there is a move possible
    then stalemate = false
        For k As Integer = 0 To 1 + GameForm1.wknightcount
            GameForm1.whiteknight(k).possiblemoves()
            For i As Integer = 0 To 7
                For j As Integer = 0 To 7

```

```

        If GameForm1.whiteknight(k).flags(i, j) = True Then
            stalemate = False
        End If
    Next
Next
Next
Next
'Possible moves for the white queen, if there is a move possible
then stalemate = false
For k As Integer = 0 To GameForm1.wqueencount
    GameForm1.whitequeen(k).possiblemoves()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If GameForm1.whitequeen(k).flags(i, j) = True Then
                stalemate = False
            End If
        Next
    Next
Next
Next

If stalemate = True And AICalculating = False Then
    Settle.SettleAgreed()
End If
'Check if it is only kings left on the board
KingsOnly()
If stalemate = True And AICalculating = False Then
    Settle.SettleAgreed()
End If
End If
End Sub

'showing images of pieces
Public Sub show()
    If GameForm1.Set1 = True Then
        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                Select Case board(i, j)
                    Case 1
                        GameForm1.pb(i, j).Image = My.Resources.Set1KingW
                    Case 2
                        GameForm1.pb(i, j).Image =
My.Resources.Set1QueenW
                    Case 3
                        GameForm1.pb(i, j).Image =
My.Resources.Set1BishopW
                    Case 4
                        GameForm1.pb(i, j).Image =
My.Resources.Set1KnightW
                    Case 5
                        GameForm1.pb(i, j).Image = My.Resources.Set1RookW
                    Case 6
                        GameForm1.pb(i, j).Image = My.Resources.Set1PawnW
                    Case -1
                        GameForm1.pb(i, j).Image = My.Resources.Set1KingB
                    Case -2
                        GameForm1.pb(i, j).Image =
My.Resources.Set1QueenB
                    Case -3

```

NEA - Chess Game

```
GameForm1.pb(i, j).Image =
My.Resources.Set1BishopB
    Case -4
        GameForm1.pb(i, j).Image =
My.Resources.Set1KnightB
    Case -5
        GameForm1.pb(i, j).Image = My.Resources.Set1RookB
    Case -6
        GameForm1.pb(i, j).Image = My.Resources.Set1PawnB
    Case Else
        GameForm1.pb(i, j).Image = Nothing
End Select
showcheck()
Next
Next
ElseIf GameForm1.Set2 = True Then
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            Select Case board(i, j)
                Case 1
                    GameForm1.pb(i, j).Image = My.Resources.Set2KingW
                Case 2
                    GameForm1.pb(i, j).Image =
My.Resources.Set2QueenW
                Case 3
                    GameForm1.pb(i, j).Image =
My.Resources.Set2BishopW
                Case 4
                    GameForm1.pb(i, j).Image =
My.Resources.Set2KnightW
                Case 5
                    GameForm1.pb(i, j).Image = My.Resources.Set2RookW
                Case 6
                    GameForm1.pb(i, j).Image = My.Resources.Set2PawnW
                Case -1
                    GameForm1.pb(i, j).Image = My.Resources.Set2KingB
                Case -2
                    GameForm1.pb(i, j).Image =
My.Resources.Set2QueenB
                Case -3
                    GameForm1.pb(i, j).Image =
My.Resources.Set2BishopB
                Case -4
                    GameForm1.pb(i, j).Image =
My.Resources.Set2KnightB
                Case -5
                    GameForm1.pb(i, j).Image = My.Resources.Set2RookB
                Case -6
                    GameForm1.pb(i, j).Image = My.Resources.Set2PawnB
                Case Else
                    GameForm1.pb(i, j).Image = Nothing
            End Select
            showcheck()
        Next
    Next
ElseIf GameForm1.Set3 = True Then
    For i As Integer = 0 To 7
```

```

        For j As Integer = 0 To 7
            Select Case board(i, j)
                Case 1
                    GameForm1.pb(i, j).Image = My.Resources.Set3KingW
                Case 2
                    GameForm1.pb(i, j).Image =
My.Resources.Set3QueenW
                Case 3
                    GameForm1.pb(i, j).Image =
My.Resources.Set3BishopW
                Case 4
                    GameForm1.pb(i, j).Image =
My.Resources.Set3KnightW
                Case 5
                    GameForm1.pb(i, j).Image = My.Resources.Set3RookW
                Case 6
                    GameForm1.pb(i, j).Image = My.Resources.Set3PawnW
                Case -1
                    GameForm1.pb(i, j).Image = My.Resources.Set3KingB
                Case -2
                    GameForm1.pb(i, j).Image =
My.Resources.Set3QueenB
                Case -3
                    GameForm1.pb(i, j).Image =
My.Resources.Set3BishopB
                Case -4
                    GameForm1.pb(i, j).Image =
My.Resources.Set3KnightB
                Case -5
                    GameForm1.pb(i, j).Image = My.Resources.Set3RookB
                Case -6
                    GameForm1.pb(i, j).Image = My.Resources.Set3PawnB
                Case Else
                    GameForm1.pb(i, j).Image = Nothing
            End Select
            showcheck()
        Next
    Next
End If
End Sub
Dim z As Integer 'variable used in the easy mode of AI

'selecting picture box
Public Function selection(ByVal x, ByVal y) As String
    If gameflag = True Then 'selection only possible if gameflag true
        (when game is not over)
            showcheck() 'if the king is in check always display it on the
board
            If board(x, y) <> 0 Then 'if value selected on board is not equal
to 0 (blank square)
                GameForm1.pb(x, y).BackgroundImage = My.Resources.Selection
'add selection resource to background image to highlight primary image
            End If

            Select Case board(x, y) 'whatever board value selected on board,
return the piece and possible moves
                Case 1

```

```

        If player = 1 Then
            showpossiblemove(GameForm1.whiteking.possiblemoves)
            Return "whiteking"
        End If
    Case 2
        If player = 1 Then
            For i As Integer = 0 To GameForm1.wqueencount
                If GameForm1.whitequeen(i).returnx = x And
GameForm1.whitequeen(i).returny = y Then

showpossiblemove(GameForm1.whitequeen(i).possiblemoves)
                End If
            Next
            Return "whitequeen"
        End If
    Case 3
        If player = 1 Then
            For i As Integer = 0 To 1 + GameForm1.wbishopcount
                If GameForm1.whitebishop(i).returnx = x And
GameForm1.whitebishop(i).returny = y Then

showpossiblemove(GameForm1.whitebishop(i).possiblemoves)

                End If
            Next
            Return "whitebishop"
        End If
    Case 4
        If player = 1 Then
            For i As Integer = 0 To 1 + GameForm1.wknightcount
                If GameForm1.whiteknight(i).returnx = x And
GameForm1.whiteknight(i).returny = y Then

showpossiblemove(GameForm1.whiteknight(i).possiblemoves)
                End If
            Next
            Return "whiteknight"
        End If
    Case 5
        If player = 1 Then
            For i As Integer = 0 To 1 + GameForm1.wrookcount
                If GameForm1.whiterook(i).returnx = x And
GameForm1.whiterook(i).returny = y Then

showpossiblemove(GameForm1.whiterook(i).possiblemoves)
                End If
            Next
            Return "whiterook"
        End If
    Case 6
        If player = 1 Then
            For i As Integer = 0 To 7
                If GameForm1.whitepawn(i).returnx = x And
GameForm1.whitepawn(i).returny = y Then

showpossiblemove(GameForm1.whitepawn(i).possiblemoves)
                End If
            Next
            Return "whitepawn"
        End If
    End Select
End Sub

```

```

        Next
    End If
    Return "whitepawn"
Case -1
    If player = 2 Then
        showpossiblemove(GameForm1.blackking.possiblemoves)
        Return "blackking"
    End If
Case -2
    If player = 2 Then
        For i As Integer = 0 To GameForm1.bqueencount
            If GameForm1.blackqueen(i).returnx = x And
GameForm1.blackqueen(i).returny = y Then

showpossiblemove(GameForm1.blackqueen(i).possiblemoves)
                z = i
            End If
        Next
        Return "blackqueen"
    End If
Case -3
    If player = 2 Then
        For i As Integer = 0 To 1 + GameForm1.bbishopcount
            If GameForm1.blackbishop(i).returnx = x And
GameForm1.blackbishop(i).returny = y Then

showpossiblemove(GameForm1.blackbishop(i).possiblemoves)
                z = i
            End If
        Next
        Return "blackbishop"
    End If
Case -4
    If player = 2 Then
        For i As Integer = 0 To 1 + GameForm1.bknightcount
            If GameForm1.blackknight(i).returnx = x And
GameForm1.blackknight(i).returny = y Then

showpossiblemove(GameForm1.blackknight(i).possiblemoves)
                'TESTING
                z = i
            End If
        Next
        Return "blackknight"
    End If
Case -5
    If player = 2 Then
        For i As Integer = 0 To 1 + GameForm1.brookcount
            If GameForm1.blackrook(i).returnx = x And
GameForm1.blackrook(i).returny = y Then

showpossiblemove(GameForm1.blackrook(i).possiblemoves)
                z = i
            End If
        Next
        Return ("blackrook")
    End If

```

```

        Case -6
            If player = 2 Then
                For i As Integer = 0 To 7
                    If GameForm1.blackpawn(i).returnx = x And
GameForm1.blackpawn(i).returny = y Then

showpossiblemove(GameForm1.blackpawn(i).possiblemoves)
                    z = i
                    End If
                Next
                Return ("blackpawn")
            End If
        Case Else
            Return Nothing
        End Select
    End If
End Function

Public Sub reback()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            GameForm1.pb(i, j).BackgroundImage = Nothing
        Next
    Next
End Sub

Public Sub recanmove()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            canmove(i, j) = False
        Next
    Next
End Sub

Public Sub recheckwhite() 'Setting Every Square Check = False
    '2 loops needed as (7, 7) array, setting all squares check false
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforwhite(i, j) = False
        Next
    Next
End Sub

Public Sub recheckblack()
    '2 loops needed as (7, 7) array, setting all squares check false
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checkforblack(i, j) = False
        Next
    Next
End Sub

Public Sub fillcheckforblack()
    If player = 2 Then
        If recurrisionstop = True Then Exit Sub
        recheckblack()
    '

```

```

'White Pawn
'
For i As Integer = 0 To 7
    If GameForm1.whitepawn(i).stillonboard = True And
board(GameForm1.whitepawn(i).xposition, GameForm1.whitepawn(i).yposition) = 6
Then
        GameForm1.whitepawn(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                If GameForm1.whitepawn(i).checktheking(j, k) =
True And GameForm1.blackking.xposition = j And GameForm1.blackking.yposition
= k Then
                    xpositionCheck =
GameForm1.whitepawn(i).xposition
                    ypositionCheck =
GameForm1.whitepawn(i).yposition
                    checkforblack(j, k) = True
                ElseIf GameForm1.whitepawn(i).checktheking(j, k)
= True Then
                    checkforblack(j, k) = True
            End If
        Next
    Next
Else
        GameForm1.whitepawn(i).reflags()
    End If
Next
'
'White Rook
'
For i As Integer = 0 To 1 + GameForm1.wrookcount
    If GameForm1.whiterook(i).stillonboard = True And
board(GameForm1.whiterook(i).xposition, GameForm1.whiterook(i).yposition) = 5
Then
        GameForm1.whiterook(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                If GameForm1.whiterook(i).checktheking(j, k) =
True And GameForm1.blackking.xposition = j And GameForm1.blackking.yposition
= k Then
                    xpositionCheck =
GameForm1.whiterook(i).xposition
                    ypositionCheck =
GameForm1.whiterook(i).yposition
                    checkforblack(j, k) = True
                ElseIf GameForm1.whiterook(i).checktheking(j, k)
= True Then
                    checkforblack(j, k) = True
            End If
        Next
    Next
Else
        GameForm1.whiterook(i).reflags()
    End If
Next
'

```



```

        'White Knight
        '
        For i As Integer = 0 To 1 + GameForm1.wknightcount
            If GameForm1.whiteknight(i).stillonboard = True And
board(GameForm1.whiteknight(i).xposition, GameForm1.whiteknight(i).yposition)
= 4 Then
                GameForm1.whiteknight(i).fillcheckking()
                For j As Integer = 0 To 7
                    For k As Integer = 0 To 7
                        If GameForm1.whiteknight(i).checktheking(j, k) =
True And GameForm1.blackking.xposition = j And GameForm1.blackking.yposition
= k Then
                            xpositionCheck =
GameForm1.whiteknight(i).xposition
                            ypositionCheck =
GameForm1.whiteknight(i).yposition
                            checkforblack(j, k) = True
                        ElseIf GameForm1.whiteknight(i).checktheking(j,
k) = True Then
                            checkforblack(j, k) = True
                        End If
                    Next
                Next
            Else
                GameForm1.whiteknight(i).reflags()
            End If
        Next
    '
    'White Bishop
    '
    For i As Integer = 0 To 1 + GameForm1.wbishopcount
        If GameForm1.whitebishop(i).stillonboard = True And
board(GameForm1.whitebishop(i).xposition, GameForm1.whitebishop(i).yposition)
= 3 Then
            GameForm1.whitebishop(i).fillcheckking()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    If GameForm1.whitebishop(i).checktheking(j, k) =
True And GameForm1.blackking.xposition = j And GameForm1.blackking.yposition
= k Then
                        xpositionCheck =
GameForm1.whitebishop(i).xposition
                        ypositionCheck =
GameForm1.whitebishop(i).yposition
                        checkforblack(j, k) = True
                    ElseIf GameForm1.whitebishop(i).checktheking(j,
k) = True Then
                        checkforblack(j, k) = True
                    End If
                Next
            Next
        Else
            GameForm1.whitebishop(i).reflags()
        End If
    Next
    '
    'White Queen

```

```

        '
        For i As Integer = 0 To GameForm1.wqueencount
            If GameForm1.whitequeen(i).stillonboard = True And
board(GameForm1.whitequeen(i).xposition, GameForm1.whitequeen(i).yposition) =
2 Then
                GameForm1.whitequeen(i).fillcheckking()
                For j As Integer = 0 To 7
                    For k As Integer = 0 To 7
                        If GameForm1.whitequeen(i).checktheking(j, k) =
True And GameForm1.blackking.xposition = j And GameForm1.blackking.yposition
= k Then
                                xpositionCheck =
GameForm1.whitequeen(i).xposition
                                ypositionCheck =
GameForm1.whitequeen(i).yposition
                                checkforblack(j, k) = True

                                ElseIf GameForm1.whitequeen(i).checktheking(j, k)
= True Then
                                        checkforblack(j, k) = True
                                End If
                            Next
                        Next
                    Else
                        GameForm1.whitequeen(i).reflags()
                    End If
                Next
            '
            'White King
            '
            GameForm1.whiteking.fillcheckking()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    If GameForm1.whiteking.checktheking(j, k) = True Then
                        checkforblack(j, k) = True
                    End If
                Next
            Next
            recurrisionstop = True
        End If
    End Sub

    Public Sub fillcheckforwhite()
        If player = 1 Then
            If recurrisionstop = True Then Exit Sub
            recheckwhite()
            '
            'Black Pawn
            '
            For i As Integer = 0 To 7
                If GameForm1.blackpawn(i).stillonboard = True And
board(GameForm1.blackpawn(i).xposition, GameForm1.blackpawn(i).yposition) = -
6 Then
                    GameForm1.blackpawn(i).fillcheckking()
                    For j As Integer = 0 To 7
                        For k As Integer = 0 To 7

```

```

                If GameForm1.blackpawn(i).checktheking(j, k) =
True And GameForm1.whiteking.xposition = j And GameForm1.whiteking.yposition
= k Then
                    xpositionCheck =
GameForm1.blackpawn(i).xposition
                    ypositionCheck =
GameForm1.blackpawn(i).yposition
                    checkforwhite(j, k) = True
                ElseIf GameForm1.blackpawn(i).checktheking(j, k)
= True Then
                    checkforwhite(j, k) = True
                End If
            Next
        Next
    Else
        GameForm1.blackpawn(i).reflags()
    End If
Next
'
'Black Rook
'
For i As Integer = 0 To 1 + GameForm1.brookcount
    If GameForm1.blackrook(i).stillonboard = True And
board(GameForm1.blackrook(i).xposition, GameForm1.blackrook(i).yposition) = -
5 Then
        GameForm1.blackrook(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                If GameForm1.blackrook(i).checktheking(j, k) =
True And GameForm1.whiteking.xposition = j And GameForm1.whiteking.yposition
= k Then
                    xpositionCheck =
GameForm1.blackrook(i).xposition
                    ypositionCheck =
GameForm1.blackrook(i).yposition
                    checkforwhite(j, k) = True
                ElseIf GameForm1.blackrook(i).checktheking(j, k)
= True Then
                    checkforwhite(j, k) = True
                End If
            Next
        Next
    Else
        GameForm1.blackrook(i).reflags()
    End If
Next
'
'Black Knight
'
For i As Integer = 0 To 1 + GameForm1.bknightcount
    If GameForm1.blackknight(i).stillonboard = True And
board(GameForm1.blackknight(i).xposition, GameForm1.blackknight(i).yposition)
= -4 Then
        GameForm1.blackknight(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7

```

```

        If GameForm1.blackknight(i).checktheking(j, k) =
True And GameForm1.whiteking.xposition = j And GameForm1.whiteking.yposition
= k Then
            xpositionCheck =
GameForm1.blackknight(i).xposition
            ypositionCheck =
GameForm1.blackknight(i).yposition
            checkforwhite(j, k) = True
            ElseIf GameForm1.blackknight(i).checktheking(j,
k) = True Then
                checkforwhite(j, k) = True
            End If
        Next
    Next
Else
    GameForm1.blackknight(i).reflags()
End If
Next
'
'Black Bishop
'
For i As Integer = 0 To 1 + GameForm1.bbishopcount
    If GameForm1.blackbishop(i).stillonboard = True And
board(GameForm1.blackbishop(i).xposition, GameForm1.blackbishop(i).yposition)
= -3 Then
        GameForm1.blackbishop(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                If GameForm1.blackbishop(i).checktheking(j, k) =
True And GameForm1.whiteking.xposition = j And GameForm1.whiteking.yposition
= k Then
                    xpositionCheck =
GameForm1.blackbishop(i).xposition
                    ypositionCheck =
GameForm1.blackbishop(i).yposition
                    checkforwhite(j, k) = True
                    ElseIf GameForm1.blackbishop(i).checktheking(j,
k) = True Then
                        checkforwhite(j, k) = True
                    End If
                Next
            Next
        Next
    Else
        GameForm1.blackbishop(i).reflags()
    End If
Next
'
'Black Queen
'
For i As Integer = 0 To GameForm1.bqueencount
    If GameForm1.blackqueen(i).stillonboard = True And
board(GameForm1.blackqueen(i).xposition, GameForm1.blackqueen(i).yposition) =
-2 Then
        GameForm1.blackqueen(i).fillcheckking()
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7

```

```

        If GameForm1.blackqueen(i).checktheking(j, k) =
True And GameForm1.whiteking.xposition = j And GameForm1.whiteking.yposition
= k Then
            xpositionCheck =
GameForm1.blackqueen(i).xposition
            ypositionCheck =
GameForm1.blackqueen(i).yposition
            checkforwhite(j, k) = True

            ElseIf GameForm1.blackqueen(i).checktheking(j, k)
= True Then
                checkforwhite(j, k) = True
            End If
        Next
    Next
Else
    GameForm1.blackqueen(i).reflags()
End If
Next
'
'Black King
'
GameForm1.blackking.fillcheckking()
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        If GameForm1.blackking.checktheking(j, k) = True Then
            checkforwhite(j, k) = True
        End If
    Next
Next

    recurrisionstop = True
End If
End Sub

Public Sub showcheck()
    whiteindanger = False
    blackindanger = False
    'black check
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If checkforblack(i, j) = True And
GameForm1.blackking.xposition = i And GameForm1.blackking.yposition = j Then
                'locating the king using x and y coords and adding check background
                GameForm1.pb(i, j).BackgroundImage = My.Resources.Check
                blackindanger = True
            End If
        Next
    Next

    'white check
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If checkforwhite(i, j) = True And
GameForm1.whiteking.xposition = i And GameForm1.whiteking.yposition = j Then
                'locating the king using x and y coords and adding check background

```

```

        GameForm1.pb(i, j).BackgroundImage = My.Resources.Check
        whiteindanger = True
    End If
Next
Next
End Sub

Public Sub showpossiblemove(ByVal b As Boolean(,))
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If b(i, j) = True Then
                GameForm1.pb(i, j).BackgroundImage = My.Resources.VMove
                canmove(i, j) = True
            End If
        Next
    Next
End Sub

Public Sub mov(ByVal selected, ByVal x, ByVal y, ByVal prex, ByVal prey)
    Select Case selected
        Case "whiteking"
            movecounter += 1
            GameForm1.MoveCounterTextBox.Text = movecounter
            If x = 7 And y = 2 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(0).AllowedToCastle = True And castleincheck(0) =
False Then

                Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
                Dim result As DialogResult
                'displaying MessageBox
                result = MessageBox.Show("Are you sure you want castle",
"Castle", button)

                'getting result of the messagebox
                If result = System.Windows.Forms.DialogResult.Yes Then
                    GameForm1.whiteking.changeposition(x, y, prex, prey)
                    GameForm1.whiterook(0).changeposition(7, 3, 7, 0)
                    GameForm1.whiteking.AllowedToCastle = False
                    GameForm1.whiterook(0).AllowedToCastle = False
                    GameForm1.whiterook(1).AllowedToCastle = False
                    GameForm1.P1Timer.Stop()
                    If GameForm1.MinutesTextBox1.Text = "00" Then
                        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                            GameForm1.SecondsTextBox1.Text += 2
                            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
                        End If
                    End If
                    GameForm1.P2Timer.Start()
                    player = 2
                    recheckwhite() 'White cannot be in check if king has
been moved
                    recheckblack() 'Black cannot be in check by the king,
therefore set all squares to not in check
                    If GameForm1.gameisoneplayer = True And gameflag =
True Then

                        If GameForm1.EasyMode = True Then
                            AIMoving()

```

```

        ElseIf GameForm1.HardMode = True Then
            AIMovingHard()
        End If
    End If
    Exit Sub
Else
    movecounter -= 1
    Exit Sub
End If
End If
If x = 7 And y = 6 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(1).AllowedToCastle = True And castleincheck(1) =
False Then

    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying MessageBox
    result = MessageBox.Show("Are you sure you want castle",
"Castle", button)

    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.whiteking.changeposition(x, y, prex, prey)
        GameForm1.whiterook(1).changeposition(7, 5, 7, 7)
        GameForm1.whiteking.AllowedToCastle = False
        GameForm1.whiterook(1).AllowedToCastle = False
        GameForm1.whiterook(0).AllowedToCastle = False
        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()
        player = 2

        recheckwhite() 'White cannot be in check if king has
been moved
        recheckblack() 'Black cannot be in check by the king,
therefore set all squares to not in check
        If GameForm1.gameisoneplayer = True And gameflag =
True Then

            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
        Exit Sub
    Else
        movecounter -= 1
        Exit Sub
    End If
End If
GameForm1.whiteking.changeposition(x, y, prex, prey)
GameForm1.whiteking.AllowedToCastle = False

```

```

        'After a white king has moved, if it has taken a black piece
set its x and y values to 0

        For L As Integer = 0 To GameForm1.bqueencount
            If GameForm1.whiteking.xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackqueen(L).yposition Then
                GameForm1.blackqueen(L).stillonboard = False
            End If
        Next

        For L As Integer = 0 To 1 + GameForm1.bknightcount
            If GameForm1.whiteking.xposition =
GameForm1.blackknight(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackknight(L).yposition Then
                GameForm1.blackknight(L).stillonboard = False
            End If
        Next

        For L As Integer = 0 To 1 + GameForm1.bbishopcount
            If GameForm1.whiteking.xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackbishop(L).yposition Then
                GameForm1.blackbishop(L).stillonboard = False
            End If
        Next

        For L As Integer = 0 To 1 + GameForm1.brookcount
            If GameForm1.whiteking.xposition =
GameForm1.blackrook(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackrook(L).yposition Then
                GameForm1.blackrook(L).stillonboard = False
            End If
        Next

        For L As Integer = 0 To 7
            If GameForm1.whiteking.xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whiteking.yposition =
GameForm1.blackpawn(L).yposition Then
                GameForm1.blackpawn(L).stillonboard = False
            End If
        Next

        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()

        recheckwhite() 'White cannot be in check if the pawn has
moved

        GameForm1.ga.recurrisionstop = False

```



```

player = 2
fillcheckforblack()
showcheck() 'Show the check if there is one

checkwinner()

If GameForm1.gameisoneplayer = True And gameflag = True Then

    If GameForm1.EasyMode = True Then
        AIMoving()
    ElseIf GameForm1.HardMode = True Then
        AIMovingHard()
    End If
End If

Case "whitequeen"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To GameForm1.wqueencount
        If GameForm1.whitequeen(i).returnx = prex And
GameForm1.whitequeen(i).returny = prey Then
            GameForm1.whitequeen(i).changeposition(x, y, prex,
prey)

                'After a white queen has moved, if it has taken a
black piece set its x and y values to 0

                    For L As Integer = 0 To GameForm1.bqueencount
                        If GameForm1.whitequeen(i).xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whitequeen(i).yposition =
GameForm1.blackqueen(L).yposition Then
                            GameForm1.blackqueen(L).stillonboard = False
                        End If
                    Next

                    For L As Integer = 0 To 1 + GameForm1.bknightcount
                        If GameForm1.whitequeen(i).xposition =
GameForm1.blackknight(L).xposition And GameForm1.whitequeen(i).yposition =
GameForm1.blackknight(L).yposition Then
                            GameForm1.blackknight(L).stillonboard = False
                        End If
                    Next

                    For L As Integer = 0 To 1 + GameForm1.bbishopcount
                        If GameForm1.whitequeen(i).xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whitequeen(i).yposition =
GameForm1.blackbishop(L).yposition Then
                            GameForm1.blackbishop(L).stillonboard = False
                        End If
                    Next

                    For L As Integer = 0 To 1 + GameForm1.brookcount
                        If GameForm1.whitequeen(i).xposition =
GameForm1.blackrook(L).xposition And GameForm1.whitequeen(i).yposition =
GameForm1.blackrook(L).yposition Then
                            GameForm1.blackrook(L).stillonboard = False
                        End If
                    End If
                End If
            End If
        End If
    End For
End Case

```

```

        Next

        For L As Integer = 0 To 7
            If GameForm1.whitequeen(i).xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whitequeen(i).yposition =
GameForm1.blackpawn(L).yposition Then
                GameForm1.blackpawn(L).stillonboard = False
            End If
        Next
        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()

        recheckwhite() 'White cannot be in check if the pawn
has moved

        GameForm1.ga.recurrisionstop = False

        player = 2
        fillcheckforblack()
        showcheck() 'Show the check if there is one

        checkwinner()

        If GameForm1.gameisoneplayer = True And gameflag =
True Then

            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
    Next

    Case "whitebishop"
        movecounter += 1
        GameForm1.MoveCounterTextBox.Text = movecounter
        For i As Integer = 0 To 1 + GameForm1.wbishopcount
            If GameForm1.whitebishop(i).returnx = prex And
GameForm1.whitebishop(i).returny = prey Then
                GameForm1.whitebishop(i).changeposition(x, y, prex,
prey)

                'After a white bishop has moved, if it has taken a
black piece set its x and y values to 0

                For L As Integer = 0 To GameForm1.bqueencount
                    If GameForm1.whitebishop(i).xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whitebishop(i).yposition =
GameForm1.blackqueen(L).yposition Then
                        GameForm1.blackqueen(L).stillonboard = False

```

```

        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.bknightcount
        If GameForm1.whitebishop(i).xposition =
GameForm1.blackknight(L).xposition And GameForm1.whitebishop(i).yposition =
GameForm1.blackknight(L).yposition Then
            GameForm1.blackknight(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.bbishopcount
        If GameForm1.whitebishop(i).xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whitebishop(i).yposition =
GameForm1.blackbishop(L).yposition Then
            GameForm1.blackbishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.brookcount
        If GameForm1.whitebishop(i).xposition =
GameForm1.blackrook(L).xposition And GameForm1.whitebishop(i).yposition =
GameForm1.blackrook(L).yposition Then
            GameForm1.blackrook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.whitebishop(i).xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whitebishop(i).yposition =
GameForm1.blackpawn(L).yposition Then
            GameForm1.blackpawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Stop()
    If GameForm1.MinutesTextBox1.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
            GameForm1.SecondsTextBox1.Text += 2
            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
        End If
    End If
    GameForm1.P2Timer.Start()

    recheckwhite() 'White cannot be in check if the pawn
has moved

    GameForm1.ga.recurrisionstop = False

    player = 2
    fillcheckforblack()
    showcheck() 'Show the check if there is one

    checkwinner()

    If GameForm1.gameisoneplayer = True And gameflag =
True Then

```

```

        If GameForm1.EasyMode = True Then
            AIMoving()
        ElseIf GameForm1.HardMode = True Then
            AIMovingHard()
        End If
    End If
End If
Next

Case "whiteknight"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To 1 + GameForm1.wknightcount
        If GameForm1.whiteknight(i).returnx = prex And
GameForm1.whiteknight(i).returny = prey Then
            GameForm1.whiteknight(i).changeposition(x, y, prex,
prey)

            'After a white knight has moved, if it has taken a
black piece set its x and y values to 0

            For L As Integer = 0 To GameForm1.bqueencount
                If GameForm1.whiteknight(i).xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whiteknight(i).yposition =
GameForm1.blackqueen(L).yposition Then
                    GameForm1.blackqueen(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.bknightcount
                If GameForm1.whiteknight(i).xposition =
GameForm1.blackknight(L).xposition And GameForm1.whiteknight(i).yposition =
GameForm1.blackknight(L).yposition Then
                    GameForm1.blackknight(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.bbishopcount
                If GameForm1.whiteknight(i).xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whiteknight(i).yposition =
GameForm1.blackbishop(L).yposition Then
                    GameForm1.blackbishop(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.brookcount
                If GameForm1.whiteknight(i).xposition =
GameForm1.blackrook(L).xposition And GameForm1.whiteknight(i).yposition =
GameForm1.blackrook(L).yposition Then
                    GameForm1.blackrook(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 7
                If GameForm1.whiteknight(i).xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whiteknight(i).yposition =
GameForm1.blackpawn(L).yposition Then
                    GameForm1.blackpawn(L).stillonboard = False

```

```

        End If
    Next
    GameForm1.P1Timer.Stop()
    If GameForm1.MinutesTextBox1.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
            GameForm1.SecondsTextBox1.Text += 2
            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
        End If
    End If
    GameForm1.P2Timer.Start()

    recheckwhite() 'White cannot be in check if the pawn
has moved

    GameForm1.ga.recurrisionstop = False

    player = 2
    fillcheckforblack()
    showcheck() 'Show the check if there is one

    checkwinner()

    If GameForm1.gameisoneplayer = True And gameflag =
True Then

        If GameForm1.EasyMode = True Then
            AIMoving()
        ElseIf GameForm1.HardMode = True Then
            AIMovingHard()
        End If
    End If
End If
Next

Case "whiterook"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    'First rook castle code
    If x = 7 And y = 3 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(0).AllowedToCastle = True And castleincheck(0) =
False Then

        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        'displaying MessageBox
        result = MessageBox.Show("Select 'Yes' to Castle, Select
'No' to move there normally", "Castle", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            GameForm1.whiterook(0).changeposition(x, y, prex,
prey)

            GameForm1.whiteking.changeposition(7, 2, 7, 4)
            GameForm1.whiterook(0).AllowedToCastle = False
            GameForm1.whiterook(1).AllowedToCastle = False
            GameForm1.whiteking.AllowedToCastle = False

            GameForm1.P1Timer.Stop()

```

```

        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
                End If
            End If
            GameForm1.P2Timer.Start()

            recheckwhite() 'White cannot be in check if the pawn
has moved
            GameForm1.ga.recurrisionstop = False

            player = 2
            fillcheckforblack()
            showcheck() 'Show the check if there is one

            checkwinner()

            If GameForm1.gameisoneplayer = True And gameflag =
True Then

                If GameForm1.EasyMode = True Then
                    AIMoving()
                ElseIf GameForm1.HardMode = True Then
                    AIMovingHard()
                End If
            End If
            Exit Sub
        Else
            If checkforwhite(7, 4) = False Then
                GameForm1.whiterook(0).changeposition(x, y, prex,
prey)
                GameForm1.whiterook(0).AllowedToCastle = False
                GameForm1.P1Timer.Stop()
                If GameForm1.MinutesTextBox1.Text = "00" Then
                    If CInt(GameForm1.SecondsTextBox1.Text) <= 5
Then
                        GameForm1.SecondsTextBox1.Text += 2
                        GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
                    End If
                End If
                GameForm1.P2Timer.Start()

                recheckwhite() 'White cannot be in check if the
pawn has moved
                GameForm1.ga.recurrisionstop = False
                player = 2

                fillcheckforblack()
                showcheck() 'Show the check if there is one

                checkwinner()

                If GameForm1.gameisoneplayer = True And gameflag
= True Then

```

```

        If GameForm1.EasyMode = True Then
            AIMoving()
        ElseIf GameForm1.HardMode = True Then
            AIMovingHard()
        End If
    End If
ElseIf checkforwhite(7, 4) = True Then
    'Cannot move there normally as king is in check
    MessageBox.Show("King in check, cannot move Rook
to D8")

    End If
Exit Sub
End If
End If
'Second rook castle code
If x = 7 And y = 5 And GameForm1.whiteking.AllowedToCastle =
True And GameForm1.whiterook(1).AllowedToCastle = True And castleincheck(1) =
False Then

    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying MessageBox
    result = MessageBox.Show("Select 'Yes' to Castle, Select
'No' to move there normally", "Castle", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then
        GameForm1.whiterook(1).changeposition(x, y, prex,
prey)

        GameForm1.whiteking.changeposition(7, 6, 7, 4)
        GameForm1.whiterook(1).AllowedToCastle = False
        GameForm1.whiterook(0).AllowedToCastle = False
        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()

        recheckwhite() 'White cannot be in check if the pawn
has moved

        GameForm1.ga.recurrisionstop = False

        player = 2
        fillcheckforblack()
        showcheck() 'Show the check if there is one
        checkwinner()

        If GameForm1.gameisoneplayer = True And gameflag =
True Then

            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
    End If
End If

```

```

        End If
    End If
    Exit Sub
Else
    If checkforwhite(7, 4) = False Then
        GameForm1.whiterook(1).changeposition(x, y, prex,
prey)

        GameForm1.whiterook(1).AllowedToCastle = False
        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5
Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()

        recheckwhite() 'White cannot be in check if the
pawn has moved

        GameForm1.ga.recurrisionstop = False

        player = 2
        fillcheckforblack()
        showcheck() 'Show the check if there is one
        checkwinner()

        If GameForm1.gameisoneplayer = True And gameflag
= True Then

            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
        ElseIf checkforwhite(7, 4) = True Then
            'Cannot move there normally as king is in check
            MessageBox.Show("King in check, cannot move Rook
to F8")
        End If
    End Sub
End If
End If

For i As Integer = 0 To 1 + GameForm1.wrookcount
    If GameForm1.whiterook(i).returnx = prex And
GameForm1.whiterook(i).returny = prey Then
        GameForm1.whiterook(i).changeposition(x, y, prex,
prey)

        GameForm1.whiterook(i).AllowedToCastle = False
        'After a white rook has moved, if it has taken a
black piece set its x and y values to 0

        For L As Integer = 0 To GameForm1.bqueencount

```



```

        If GameForm1.whiterook(i).xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whiterook(i).yposition =
GameForm1.blackqueen(L).yposition Then
            GameForm1.blackqueen(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.bknightcount
        If GameForm1.whiterook(i).xposition =
GameForm1.blackknight(L).xposition And GameForm1.whiterook(i).yposition =
GameForm1.blackknight(L).yposition Then
            GameForm1.blackknight(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.bbishopcount
        If GameForm1.whiterook(i).xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whiterook(i).yposition =
GameForm1.blackbishop(L).yposition Then
            GameForm1.blackbishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.brookcount
        If GameForm1.whiterook(i).xposition =
GameForm1.blackrook(L).xposition And GameForm1.whiterook(i).yposition =
GameForm1.blackrook(L).yposition Then
            GameForm1.blackrook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.whiterook(i).xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whiterook(i).yposition =
GameForm1.blackpawn(L).yposition Then
            GameForm1.blackpawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Stop()
    If GameForm1.MinutesTextBox1.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
            GameForm1.SecondsTextBox1.Text += 2
            GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
        End If
    End If
    GameForm1.P2Timer.Start()

    recheckwhite() 'White cannot be in check if the pawn
has moved

    GameForm1.ga.recurrisionstop = False

    player = 2
    fillcheckforblack()
    showcheck() 'Show the check if there is one

    checkwinner()

```

```

        If GameForm1.gameisoneplayer = True And gameflag =
True Then

            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
    End If
Next

Case "whitepawn"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To 7
        If GameForm1.whitepawn(i).returnx = prex And
GameForm1.whitepawn(i).returny = prey And board(x, y) <= 0 Then

            GameForm1.whitepawn(i).changeposition(x, y, prex,
prey)

            'After a white pawn has moved, if it has taken a
black piece set its x and y values to 0

            For L As Integer = 0 To GameForm1.bqueencount
                If GameForm1.whitepawn(i).xposition =
GameForm1.blackqueen(L).xposition And GameForm1.whitepawn(i).yposition =
GameForm1.blackqueen(L).yposition Then
                    GameForm1.blackqueen(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.bknightcount
                If GameForm1.whitepawn(i).xposition =
GameForm1.blackknight(L).xposition And GameForm1.whitepawn(i).yposition =
GameForm1.blackknight(L).yposition Then
                    GameForm1.blackknight(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.bbishopcount
                If GameForm1.whitepawn(i).xposition =
GameForm1.blackbishop(L).xposition And GameForm1.whitepawn(i).yposition =
GameForm1.blackbishop(L).yposition Then
                    GameForm1.blackbishop(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.brookcount
                If GameForm1.whitepawn(i).xposition =
GameForm1.blackrook(L).xposition And GameForm1.whitepawn(i).yposition =
GameForm1.blackrook(L).yposition Then
                    GameForm1.blackrook(L).stillonboard = False
                End If
            Next
    Next

```

```

        For L As Integer = 0 To 7
            If GameForm1.whitepawn(i).xposition =
GameForm1.blackpawn(L).xposition And GameForm1.whitepawn(i).yposition =
GameForm1.blackpawn(L).yposition Then
                GameForm1.blackpawn(L).stillonboard = False
            End If
        Next
        GameForm1.P1Timer.Stop()
        If GameForm1.MinutesTextBox1.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox1.Text) <= 5 Then
                GameForm1.SecondsTextBox1.Text += 2
                GameForm1.SecondsTextBox1.Text = "0" &
GameForm1.SecondsTextBox1.Text
            End If
        End If
        GameForm1.P2Timer.Start()

        PawnPromotion()

        recheckwhite() 'White cannot be in check if the pawn
has moved

        GameForm1.ga.recurrisionstop = False

        player = 2
        fillcheckforblack()
        showcheck() 'Show the check if there is one

        checkwinner()

        If GameForm1.gameisoneplayer = True And gameflag =
True Then
            If GameForm1.EasyMode = True Then
                AIMoving()
            ElseIf GameForm1.HardMode = True Then
                AIMovingHard()
            End If
        End If
    End If

Next

Case "blackking"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    If x = 0 And y = 2 And GameForm1.blackking.AllowedToCastle =
True And GameForm1.blackrook(0).AllowedToCastle = True And castleincheck(0) =
False And GameForm1.gameistwooplayer = True Then
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        'displaying MessageBox
        result = MessageBox.Show("Are you sure you want castle",
"Castle", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            GameForm1.blackking.changeposition(x, y, prex, prey)

```

```

        GameForm1.blackrook(0).changeposition(0, 3, 0, 0)
        GameForm1.blackking.AllowedToCastle = False
        GameForm1.blackrook(0).AllowedToCastle = False
        GameForm1.blackrook(1).AllowedToCastle = False
        GameForm1.P1Timer.Start()
        GameForm1.P2Timer.Stop()
        If GameForm1.MinutesTextBox2.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                GameForm1.SecondsTextBox2.Text += 2
                GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
            End If
        End If

        recheckwhite()
        recheckblack() 'Black cannot be in check if the king
has moved

        showcheck() 'Show the check if there is one

        player = 1
        checkwinner()
        Exit Sub
    Else
        movecounter -= 1
        Exit Sub
    End If
    ElseIf x = 0 And y = 2 And
GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(0).AllowedToCastle = True And castleincheck(0) = False
And GameForm1.gameisoneplayer = True Then
        'Making it so the AI castles without user prompt
        GameForm1.blackking.changeposition(x, y, prex, prey)
        GameForm1.blackrook(0).changeposition(0, 3, 0, 0)
        GameForm1.blackking.AllowedToCastle = False
        GameForm1.blackrook(0).AllowedToCastle = False

        GameForm1.P1Timer.Start()
        GameForm1.P2Timer.Stop()
        If GameForm1.MinutesTextBox2.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                GameForm1.SecondsTextBox2.Text += 2
                GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
            End If
        End If

        recheckwhite()
        recheckblack() 'Black cannot be in check if the king has
moved

        showcheck() 'Show the check if there is one

        player = 1
        checkwinner()
        Exit Sub
    End If
    If x = 0 And y = 6 And GameForm1.blackking.AllowedToCastle =
True And GameForm1.blackrook(1).AllowedToCastle = True And castleincheck(1) =
False And GameForm1.gameistwoplayer = True Then

```

```

Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
Dim result As DialogResult
'displaying MessageBox
result = MessageBox.Show("Are you sure you want castle",
"Castle", button)
'getting result of the messagebox
If result = System.Windows.Forms.DialogResult.Yes Then
    GameForm1.blackking.changeposition(x, y, prex, prey)
    GameForm1.blackrook(1).changeposition(0, 5, 0, 7)
    GameForm1.blackking.AllowedToCastle = False
    GameForm1.blackrook(0).AllowedToCastle = False
    GameForm1.blackrook(1).AllowedToCastle = False
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
            End If
        End If
        recheckwhite()
        recheckblack() 'Black cannot be in check if the king
has moved

        showcheck() 'Show the check if there is one

        player = 1
        checkwinner()
        Exit Sub
    Else
        movecounter -= 1
        Exit Sub
    End If
    ElseIf x = 0 And y = 6 And
GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(1).AllowedToCastle = True And castleincheck(1) = False
And GameForm1.gameisoneplayer = True Then
        'Making it so the AI castles without user prompt
        GameForm1.blackking.changeposition(x, y, prex, prey)
        GameForm1.blackrook(1).changeposition(0, 5, 0, 7)
        GameForm1.blackking.AllowedToCastle = False
        GameForm1.blackrook(1).AllowedToCastle = False
        GameForm1.P1Timer.Start()
        GameForm1.P2Timer.Stop()
        If GameForm1.MinutesTextBox2.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                GameForm1.SecondsTextBox2.Text += 2
                GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
                End If
            End If
            recheckwhite()
            recheckblack() 'Black cannot be in check if the king has
moved

            showcheck() 'Show the check if there is one

            player = 1

```

```

        checkwinner()
    Exit Sub
End If
GameForm1.blackking.changeposition(x, y, prex, prey)
GameForm1.blackking.AllowedToCastle = False
'After a black king has moved, if it has taken a black piece
set its x and y values to 0

    For L As Integer = 0 To GameForm1.wqueencount
        If GameForm1.blackking.xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackking.yposition =
GameForm1.whitequeen(L).yposition Then
            GameForm1.whitequeen(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wknightcount
        If GameForm1.blackking.xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackking.yposition =
GameForm1.whiteknight(L).yposition Then
            GameForm1.whiteknight(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wbishopcount
        If GameForm1.blackking.xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackking.yposition =
GameForm1.whitebishop(L).yposition Then
            GameForm1.whitebishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wrookcount
        If GameForm1.blackking.xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackking.yposition =
GameForm1.whiterook(L).yposition Then
            GameForm1.whiterook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.blackking.xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackking.yposition =
GameForm1.whitepawn(L).yposition Then
            GameForm1.whitepawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrisionstop = False

```

```

recheckwhite()
recheckblack() 'Black cannot be in check if the king has
moved

showcheck() 'Show the check if there is one

player = 1
checkwinner()

Case "blackqueen"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To GameForm1.bqueencount
        If GameForm1.blackqueen(i).returnx = prex And
GameForm1.blackqueen(i).returny = prey Then
            GameForm1.blackqueen(i).changeposition(x, y, prex,
prey)

                'After a black queen has moved, if it has taken a
black piece set its x and y values to 0

                For L As Integer = 0 To GameForm1.wqueencount
                    If GameForm1.blackqueen(i).xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackqueen(i).yposition =
GameForm1.whitequeen(L).yposition Then
                        GameForm1.whitequeen(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wknightcount
                    If GameForm1.blackqueen(i).xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackqueen(i).yposition =
GameForm1.whiteknight(L).yposition Then
                        GameForm1.whiteknight(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wbishopcount
                    If GameForm1.blackqueen(i).xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackqueen(i).yposition =
GameForm1.whitebishop(L).yposition Then
                        GameForm1.whitebishop(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wrookcount
                    If GameForm1.blackqueen(i).xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackqueen(i).yposition =
GameForm1.whiterook(L).yposition Then
                        GameForm1.whiterook(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 7

```

```

        If GameForm1.blackqueen(i).xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackqueen(i).yposition =
GameForm1.whitepawn(L).yposition Then
            GameForm1.whitepawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrisionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the king
has moved

    showcheck() 'Show the check if there is one

    checkwinner()
End If
Next

Case "blackbishop"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To 1 + GameForm1.bbishopcount
        If GameForm1.blackbishop(i).returnx = prex And
GameForm1.blackbishop(i).returny = prey Then
            GameForm1.blackbishop(i).changeposition(x, y, prex,
prey)

            'After a black bishop has moved, if it has taken a
black piece set its x and y values to 0

            For L As Integer = 0 To GameForm1.wqueencount

                If GameForm1.blackbishop(i).xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackbishop(i).yposition =
GameForm1.whitequeen(L).yposition Then
                    GameForm1.whitequeen(L).stillonboard = False
                End If

            Next

            For L As Integer = 0 To 1 + GameForm1.wknightcount
                If GameForm1.blackbishop(i).xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackbishop(i).yposition =
GameForm1.whiteknight(L).yposition Then
                    GameForm1.whiteknight(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.wbishopcount

```



```

        If GameForm1.blackbishop(i).xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackbishop(i).yposition =
GameForm1.whitebishop(L).yposition Then
            GameForm1.whitebishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wrookcount
        If GameForm1.blackbishop(i).xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackbishop(i).yposition =
GameForm1.whiterook(L).yposition Then
            GameForm1.whiterook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.blackbishop(i).xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackbishop(i).yposition =
GameForm1.whitepawn(L).yposition Then
            GameForm1.whitepawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrsionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the king
has moved

    showcheck() 'Show the check if there is one

    checkwinner()
End If
Next

Case "blackknight"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    For i As Integer = 0 To 1 + GameForm1.bknightcount
        If GameForm1.blackknight(i).returnx = prex And
GameForm1.blackknight(i).returny = prey Then
            GameForm1.blackknight(i).changeposition(x, y, prex,
prey)

            'After a black knight has moved, if it has taken a
black piece set its x and y values to 0

        For L As Integer = 0 To GameForm1.wqueencount

```

```

        If GameForm1.blackknight(i).xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackknight(i).yposition =
GameForm1.whitequeen(L).yposition Then
            GameForm1.whitequeen(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wknightcount
        If GameForm1.blackknight(i).xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackknight(i).yposition =
GameForm1.whiteknight(L).yposition Then
            GameForm1.whiteknight(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wbishopcount
        If GameForm1.blackknight(i).xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackknight(i).yposition =
GameForm1.whitebishop(L).yposition Then
            GameForm1.whitebishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wrookcount
        If GameForm1.blackknight(i).xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackknight(i).yposition =
GameForm1.whiterook(L).yposition Then
            GameForm1.whiterook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.blackknight(i).xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackknight(i).yposition =
GameForm1.whitepawn(L).yposition Then
            GameForm1.whitepawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrisionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the king
has moved

    showcheck() 'Show the check if there is one

    checkwinner()
End If

```

```

Next

Case "blackrook"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter
    'First rook castle code
    If x = 0 And y = 3 And GameForm1.blackking.AllowedToCastle =
True And GameForm1.blackrook(0).AllowedToCastle = True And castleincheck(0) =
False And GameForm1.gameistwooplayer = True Then
        Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
        Dim result As DialogResult
        'displaying MessageBox
        result = MessageBox.Show("Select 'Yes' to Castle, Select
'No' to move there normally", "Castle", button)
        'getting result of the messagebox
        If result = System.Windows.Forms.DialogResult.Yes Then
            GameForm1.blackrook(0).changeposition(x, y, prex,
prey)

            GameForm1.blackking.changeposition(0, 2, 0, 4)
            GameForm1.blackrook(0).AllowedToCastle = False
            GameForm1.blackrook(1).AllowedToCastle = False
            GameForm1.blackking.AllowedToCastle = False
            GameForm1.P1Timer.Start()
            GameForm1.P2Timer.Stop()
            If GameForm1.MinutesTextBox2.Text = "00" Then
                If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                    GameForm1.SecondsTextBox2.Text += 2
                    GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
                End If
            End If

            GameForm1.ga.recurrisionstop = False

            player = 1
            fillcheckforwhite()
            recheckblack() 'Black cannot be in check if the king
has moved

            showcheck() 'Show the check if there is one

            checkwinner()
            Exit Sub
        Else
            If checkforblack(0, 4) = False Then
                GameForm1.blackrook(0).changeposition(x, y, prex,
prey)

                GameForm1.blackrook(0).AllowedToCastle = False
                GameForm1.P1Timer.Start()
                GameForm1.P2Timer.Stop()
                If GameForm1.MinutesTextBox2.Text = "00" Then
                    If CInt(GameForm1.SecondsTextBox2.Text) <= 5
Then
                        GameForm1.SecondsTextBox2.Text += 2
                        GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
                    End If
                End If
            End If
        End If
    End Case
End Sub

```

```

        GameForm1.ga.recurrsionstop = False

        player = 1
        fillcheckforwhite()
        recheckblack() 'Black cannot be in check if the
king has moved

        showcheck() 'Show the check if there is one

        checkwinner()
    ElseIf checkforblack(0, 4) = True Then
        'Cannot move there normally as king is in check
        MessageBox.Show("King in check, cannot move Rook
to D1")

        End If
    Exit Sub
End If
ElseIf x = 0 And y = 3 And
GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(0).AllowedToCastle = True And castleincheck(0) = False
And GameForm1.gameisoneplayer = True Then
    'Making it so AI can castle without user prompt
    GameForm1.blackrook(0).changeposition(x, y, prex, prey)
    GameForm1.blackking.changeposition(0, 2, 0, 4)
    GameForm1.blackrook(0).AllowedToCastle = False
    GameForm1.blackking.AllowedToCastle = False
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrsionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the king has
moved

    showcheck() 'Show the check if there is one

    checkwinner()
Exit Sub
End If
'Second rook castle code
If x = 0 And y = 5 And GameForm1.blackking.AllowedToCastle =
True And GameForm1.blackrook(1).AllowedToCastle = True And castleincheck(1) =
False And GameForm1.gameistwooplayer = True Then
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying MessageBox
    result = MessageBox.Show("Select 'Yes' to Castle, Select
'No' to move there normally", "Castle", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.Yes Then

```

```

    GameForm1.blackrook(1).changeposition(x, y, prex,
prey)

    GameForm1.blackking.changeposition(0, 6, 0, 4)
    GameForm1.blackrook(1).AllowedToCastle = False
    GameForm1.blackrook(0).AllowedToCastle = False
    GameForm1.blackking.AllowedToCastle = False

    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
            End If
        End If
        GameForm1.ga.recurrsionstop = False

        player = 1
        fillcheckforwhite()
        recheckblack() 'Black cannot be in check if the king
has moved

        showcheck() 'Show the check if there is one

        checkwinner()
        Exit Sub
    Else
        If checkforblack(0, 4) = False Then
            GameForm1.blackrook(1).changeposition(x, y, prex,
prey)

            GameForm1.blackrook(1).AllowedToCastle = False

            GameForm1.P1Timer.Start()
            GameForm1.P2Timer.Stop()
            If GameForm1.MinutesTextBox2.Text = "00" Then
                If CInt(GameForm1.SecondsTextBox2.Text) <= 5
Then
                    GameForm1.SecondsTextBox2.Text += 2
                    GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
                End If
            End If

            GameForm1.ga.recurrsionstop = False

            player = 1
            fillcheckforwhite()
            recheckblack() 'Black cannot be in check if the
king has moved

            showcheck() 'Show the check if there is one

            checkwinner()
        ElseIf checkforblack(0, 4) = True Then
            'Cannot move there normally as king is in check

```

```

        MessageBox.Show("King in check, cannot move Rook
to F8")

        End If
        Exit Sub
    End If
    ElseIf x = 0 And y = 5 And
GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(1).AllowedToCastle = True And castleincheck(1) = False
And GameForm1.gameisoneplayer = True Then
        'Making it so AI can castle without user prompt
        GameForm1.blackrook(1).changeposition(x, y, prex, prey)
        GameForm1.blackking.changeposition(0, 6, 0, 4)
        GameForm1.blackrook(1).AllowedToCastle = False
        GameForm1.blackking.AllowedToCastle = False

        GameForm1.P1Timer.Start()
        GameForm1.P2Timer.Stop()
        If GameForm1.MinutesTextBox2.Text = "00" Then
            If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                GameForm1.SecondsTextBox2.Text += 2
                GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
            End If
        End If
        GameForm1.ga.recurrisionstop = False

        player = 1
        fillcheckforwhite()
        recheckblack() 'Black cannot be in check if the king has
moved

        showcheck() 'Show the check if there is one

        checkwinner()
        Exit Sub
    End If

    For i As Integer = 0 To 1 + GameForm1.brookcount
        If GameForm1.blackrook(i).returnx = prex And
GameForm1.blackrook(i).returny = prey Then
            GameForm1.blackrook(i).changeposition(x, y, prex,
prey)

            GameForm1.blackrook(i).AllowedToCastle = False
            'After a black rook has moved, if it has taken a
black piece set its x and y values to 0

            For L As Integer = 0 To GameForm1.wqueencount
                If GameForm1.blackrook(i).xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackrook(i).yposition =
GameForm1.whitequeen(L).yposition Then
                    GameForm1.whitequeen(L).stillonboard = False
                End If
            Next

            For L As Integer = 0 To 1 + GameForm1.wknightcount

```

```

        If GameForm1.blackrook(i).xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackrook(i).yposition =
GameForm1.whiteknight(L).yposition Then
            GameForm1.whiteknight(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wbishopcount
        If GameForm1.blackrook(i).xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackrook(i).yposition =
GameForm1.whitebishop(L).yposition Then
            GameForm1.whitebishop(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 1 + GameForm1.wrookcount
        If GameForm1.blackrook(i).xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackrook(i).yposition =
GameForm1.whiterook(L).yposition Then
            GameForm1.whiterook(L).stillonboard = False
        End If
    Next

    For L As Integer = 0 To 7
        If GameForm1.blackrook(i).xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackrook(i).yposition =
GameForm1.whitepawn(L).yposition Then
            GameForm1.whitepawn(L).stillonboard = False
        End If
    Next
    GameForm1.P1Timer.Start()
    GameForm1.P2Timer.Stop()
    If GameForm1.MinutesTextBox2.Text = "00" Then
        If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
            GameForm1.SecondsTextBox2.Text += 2
            GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text
        End If
    End If
    GameForm1.ga.recurrisionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the king
has moved

    showcheck() 'Show the check if there is one
    checkwinner()

End If
Next

Case "blackpawn"
    movecounter += 1
    GameForm1.MoveCounterTextBox.Text = movecounter

```

```

        For i As Integer = 0 To 7
            If GameForm1.blackpawn(i).returnx = prex And
GameForm1.blackpawn(i).returny = prey And board(x, y) >= 0 Then
                GameForm1.blackpawn(i).changeposition(x, y, prex,
prey)
                'After a black pawn has moved, if it has taken a
black piece set its x and y values to 0

                For L As Integer = 0 To GameForm1.wqueencount
                    If GameForm1.blackpawn(i).xposition =
GameForm1.whitequeen(L).xposition And GameForm1.blackpawn(i).yposition =
GameForm1.whitequeen(L).yposition Then
                        GameForm1.whitequeen(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wknightcount
                    If GameForm1.blackpawn(i).xposition =
GameForm1.whiteknight(L).xposition And GameForm1.blackpawn(i).yposition =
GameForm1.whiteknight(L).yposition Then
                        GameForm1.whiteknight(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wbishopcount
                    If GameForm1.blackpawn(i).xposition =
GameForm1.whitebishop(L).xposition And GameForm1.blackpawn(i).yposition =
GameForm1.whitebishop(L).yposition Then
                        GameForm1.whitebishop(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 1 + GameForm1.wrookcount
                    If GameForm1.blackpawn(i).xposition =
GameForm1.whiterook(L).xposition And GameForm1.blackpawn(i).yposition =
GameForm1.whiterook(L).yposition Then
                        GameForm1.whiterook(L).stillonboard = False
                    End If
                Next

                For L As Integer = 0 To 7
                    If GameForm1.blackpawn(i).xposition =
GameForm1.whitepawn(L).xposition And GameForm1.blackpawn(i).yposition =
GameForm1.whitepawn(L).yposition Then
                        GameForm1.whitepawn(L).stillonboard = False
                    End If
                Next

                GameForm1.P1Timer.Start()
                GameForm1.P2Timer.Stop()
                If GameForm1.MinutesTextBox2.Text = "00" Then
                    If CInt(GameForm1.SecondsTextBox2.Text) <= 5 Then
                        GameForm1.SecondsTextBox2.Text += 2
                        GameForm1.SecondsTextBox2.Text = "0" &
GameForm1.SecondsTextBox2.Text

```



```

        End If
    End If
    PawnPromotion()

    GameForm1.ga.recurrisionstop = False

    player = 1
    fillcheckforwhite()
    recheckblack() 'Black cannot be in check if the pawn
has moved
    showcheck() 'Show the check if there is one

    checkwinner()
End If
Next
Case Else

End Select
End Sub
Public Sub PawnPromotion()
    For i As Integer = 0 To 7
        If board(0, i) = 6 Then
            show()
            GameForm1.Enabled = False
            WPawnChangeForm.Show()
            Do
                Application.DoEvents()
            Loop Until GameForm1.Enabled = True
        ElseIf board(7, i) = -6 Then
            If GameForm1.gameisoneplayer = True Then
                GameForm1.bqueencount += 1
                GameForm1.ga.board(7, i) = -2
                GameForm1.blackqueen(GameForm1.bqueencount) = New
Queen(7, i, -2)
                show()
                recurrisionstop = False
                fillcheckforwhite()
            Else
                show()
                GameForm1.Enabled = False
                BPawnChangeForm.Show()
                Do
                    Application.DoEvents()
                Loop Until GameForm1.Enabled = True
            End If
        End If
    Next
End Sub
Public Sub CastlingW()

    If GameForm1.whiteking.AllowedToCastle = True And
GameForm1.whiterook(0).AllowedToCastle = True Then
        If board(7, 1) = 0 And board(7, 2) = 0 And board(7, 3) = 0 Then
            recurrisionstop = False
            fillcheckforwhite()
            If checkforwhite(7, 2) = False And
GameForm1.whiterook(0).stillonboard = True Then

```

```

        castleincheck(0) = False
        GameForm1.whiteking.flags(7, 2) = True
        GameForm1.whiterook(0).flags(7, 3) = True
    Else
        castleincheck(0) = True
    End If
End If
End If

    If GameForm1.whiteking.AllowedToCastle = True And
GameForm1.whiterook(1).AllowedToCastle = True Then
        If board(7, 5) = 0 And board(7, 6) = 0 Then
            recurrisionstop = False
            fillcheckforwhite()
            If checkforwhite(7, 6) = False And
GameForm1.whiterook(1).stillonboard = True Then
                castleincheck(1) = False
                GameForm1.whiteking.flags(7, 6) = True
                GameForm1.whiterook(1).flags(7, 5) = True
            Else
                castleincheck(1) = True
            End If
        End If
    End If
End Sub
Public Sub CastlingB()
    If GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(0).AllowedToCastle = True Then
        If board(0, 1) = 0 And board(0, 2) = 0 And board(0, 3) = 0 Then
            recurrisionstop = False
            fillcheckforblack()
            If checkforblack(0, 2) = False And
GameForm1.blackrook(0).stillonboard = True Then
                castleincheck(0) = False
                GameForm1.blackking.flags(0, 2) = True
                GameForm1.blackrook(0).flags(0, 3) = True
            Else
                castleincheck(0) = True
            End If
        End If
    End If
End If

    If GameForm1.blackking.AllowedToCastle = True And
GameForm1.blackrook(1).AllowedToCastle = True Then
        If board(0, 5) = 0 And board(0, 6) = 0 Then
            recurrisionstop = False
            fillcheckforblack()
            If checkforblack(0, 6) = False And
GameForm1.blackrook(1).stillonboard = True Then
                castleincheck(1) = False
                GameForm1.blackking.flags(0, 6) = True
                GameForm1.blackrook(1).flags(0, 5) = True
            Else
                castleincheck(1) = True
            End If
        End If
    End If
End If

```

```

End Sub

Private Sub KingsOnly()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If board(i, j) = 0 Or board(i, j) = 1 Or board(i, j) = -1
Then
                stalemate = True
            Else
                stalemate = False
                Exit Sub
            End If
        Next
    Next
End Sub

Public Sub StoreGameReplay()
    Dim button As MessageBoxButtons = MessageBoxButtons.YesNo
    Dim result As DialogResult
    'displaying messagebox
    result = MessageBox.Show(LoginForm.Player1Username & ", would you
like to store this replay?", "Storing Replay", button)
    'getting result of the messagebox
    If result = System.Windows.Forms.DialogResult.No Then
        Exit Sub
    End If

    Dim mystring As String
    Dim premove As String
    Dim premoveintotable As String = ""
    Dim move As String
    Dim moveintotable As String = ""

    'Setting database location
    Dim dbfilename As String =
System.Windows.Forms.Application.StartupPath & "\UserInfo.mdb"
    'Opening connection
    Dim DBconn As String = "Provider=Microsoft.Jet.OLEDB.4.0;" 'OLE
provider for MS Access
    Dim DBsource As String = "Data Source=" & dbfilename

    Dim cn As OleDbConnection = New OleDbConnection(DBconn & DBsource)
'connection
    Dim sql As String
    Dim lines As Integer

    For lines = 0 To GameForm1.BoardTracker.Lines.Length - 1
        mystring = (Right(GameForm1.BoardTracker.Lines(lines).ToString,
7))
        premove = Left(mystring, 2)
        premoveintotable = premoveintotable & premove

        move = Right(mystring, 2)
        moveintotable = moveintotable & move
    Next

```

```

Dim da As OleDbDataAdapter 'adapter
Dim ds As New DataSet 'set used with adapter
sql = "SELECT [Username],[PreMove1], [Move1], [PreMove2], [Move2],
[PreMove3], [Move3], [PreMove4], [Move4], [PreMove5], [Move5] FROM [Table1]"
'SQL Statement
da = New OleDbDataAdapter(sql, cn)
cn.Open() 'Open connection
da.Fill(ds, "Table1") 'Fill data set with data gathered by adapter
Dim count As Integer

Dim checkforemptyset As String
Dim emptysetnumber As Integer
For count = 0 To ds.Tables(0).Rows.Count - 1 'For loop for amount of
users in table, (-1) as starts from 0
    If LoginForm.Player1Username = ds.Tables(0).Rows(count).Item(0)
Then 'For player 1 stats

        For i As Integer = 2 To 10 Step 2
            Try
                checkforemptyset = ds.Tables(0).Rows(count).Item(i)
            Catch ex As Exception
                emptysetnumber = Int(i / 2)
            Exit For
            End Try
        Next
    End If
Next

cn.Close() 'Close connection

If emptysetnumber = 0 Then
    Dim Message, Title, MyValue
    Do
        Message = "Enter a value between 1 and 5 (Replace Replay)"
' Set prompt.
        Title = "Replay Replace" ' Set title.
        ' Display message, title, and default value.
        MyValue = InputBox(Message, Title, "1")
        Try
            MyValue = CInt(MyValue)
        Catch ex As Exception
            MyValue = "10"
        End Try
        Loop Until MyValue >= 1 And MyValue <= 5
        emptysetnumber = MyValue
    End If

    sql = "update [Table1]"
    sql = sql & "SET [PreMove" & emptysetnumber & "] = '" &
premoveintotable & "' "
    sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
    Dim cmd As New OleDbCommand(sql, cn)
    cmd = New OleDbCommand(sql, cn)
    cn.Open() 'Open connection
    cmd.ExecuteNonQuery()
    cn.Close() 'Close connection

```

```

        sql = "update [Table1]"
        sql = sql & "SET [Move] & emptysetnumber & "]" = '" & moveintotable &
"'"

        sql = sql & "WHERE [ID] = " & LoginForm.Player1ID & ";"
        cmd = New OleDbCommand(sql, cn)
        cn.Open() 'Open connection
        cmd.ExecuteNonQuery()
        cn.Close() 'Close connection

    End Sub

    Private Sub AIMoving()
        Dim AIx(1) As Integer
        Dim AIy(1) As Integer
        Dim rnd As New Random
        'Easy mode, getting the computer to select a piece and random and
        randomly move it
        Do
            Do
                recanmove()
                AIx(0) = rnd.Next(0, 8)
                AIy(0) = rnd.Next(0, 8)
                selection(AIx(0), AIy(0))
            Loop Until board(AIx(0), AIy(0)) < 0

            Select Case board(AIx(0), AIy(0))
                Case -1
                    GameForm1.selectedpiece = ("blackking")
                    For i As Integer = 0 To 7
                        For j As Integer = 0 To 7
                            If GameForm1.blackking.flags(i, j) = True Then
                                Exit Do
                            End If
                        Next
                    Next
                Case -2
                    GameForm1.selectedpiece = ("blackqueen")
                    For i As Integer = 0 To 7
                        For j As Integer = 0 To 7
                            If GameForm1.blackqueen(z).flags(i, j) = True
Then
                                Exit Do
                            End If
                        Next
                    Next
                Case -3
                    GameForm1.selectedpiece = ("blackbishop")
                    For i As Integer = 0 To 7
                        For j As Integer = 0 To 7
                            If GameForm1.blackbishop(z).flags(i, j) = True
Then
                                Exit Do
                            End If
                        Next
                    Next
                Case -4
                    GameForm1.selectedpiece = "blackknight"

```

```

        For i As Integer = 0 To 7
            For j As Integer = 0 To 7
                If GameForm1.blackknight(z).flags(i, j) = True
Then
                    Exit Do
                End If
            Next
        Next
    Next
Case -5
    GameForm1.selectedpiece = ("blackrook")
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If GameForm1.blackrook(z).flags(i, j) = True Then
                Exit Do
            End If
        Next
    Next
Case -6
    GameForm1.selectedpiece = ("blackpawn")
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If GameForm1.blackpawn(z).flags(i, j) = True Then
                Exit Do
            End If
        Next
    Next
End Select
recanmove()
reback()
GameForm1.selected = False
Loop
reback()
Do
    AIx(1) = rnd.Next(0, 8)
    AIy(1) = rnd.Next(0, 8)
    If canmove(AIx(1), AIy(1)) = True Then
        mov(GameForm1.selectedpiece, AIx(1), AIy(1), AIx(0), AIy(0))
    End If
Loop Until player = 1
GameForm1.selected = False
show()
End Sub

Dim six(5) As Integer
Dim x(5) As Integer
Dim y(5) As Integer
Dim prex(5) As Integer
Dim prey(5) As Integer
Dim selected(5) As String
Dim AICalculating As Boolean = False
Private Sub AIMovingHard()
    AICalculating = True

    Dim DontRepeat As Boolean
    Dim PieceCanMove As Boolean

```

```

For i As Integer = 0 To 7
    For j As Integer = 0 To 7
        GameForm1.pb(i, j).Enabled = False
    Next
Next
show()
'Storing current board
LoadTempBoard()
'Changing it so when pieces move the board tracker does not add text
canwritetoboardtracker = False

'Black Pawn
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 7
    '
    '
    Dim xpos As Integer = GameForm1.blackpawn(i).xposition
    Dim ypos As Integer = GameForm1.blackpawn(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recurrisionstop = False
            fillcheckforblack()
            GameForm1.blackpawn(i).possiblemoves()
            'If the pawn can move
            If GameForm1.blackpawn(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the pawn
                GameForm1.blackpawn(i).changeposition(j, k,
GameForm1.blackpawn(i).xposition, GameForm1.blackpawn(i).yposition)
                'Change the x and y position of the pawn (as write to
board = false)

                GameForm1.blackpawn(i).xposition = j
                GameForm1.blackpawn(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                ' Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackpawn(i).xposition, GameForm1.blackpawn(i).yposition) =
False Then

```

```

                                ' If black Then can check the white
king, -5 off Of q, (most post effective way, will check king unless the pawn
can take a piece)
                                q = q - 5
                                AICheckmate()
                                End If
                                End If
                                Next
Next

'Change player to 2
player = 2

'Change the x and y position of the pawn back to
original

GameForm1.blackpawn(i).xposition = xpos
GameForm1.blackpawn(i).yposition = ypos

'Store point value if pawn moves there
If DontRepeat = True Then
    six(0) = q
    x(0) = j
    y(0) = k
    prex(0) = GameForm1.blackpawn(i).xposition
    prey(0) = GameForm1.blackpawn(i).yposition
    selected(0) = "blackpawn"
    DontRepeat = False
Else
    'Calculate random number (either a 0 or a 1)
    Dim samemove As Integer
    Dim rnd As New Random
    samemove = rnd.Next(0, 2)
    If q < six(0) Then
        six(0) = q
        x(0) = j
        y(0) = k
        prex(0) = GameForm1.blackpawn(i).xposition
        prey(0) = GameForm1.blackpawn(i).yposition
        selected(0) = "blackpawn"
    End If
    'If the maximum move is the same value as the one
already stored, randomly choose a move
    If q = six(0) And samemove = 1 Then
        six(0) = q
        x(0) = j
        y(0) = k
        prex(0) = GameForm1.blackpawn(i).xposition
        prey(0) = GameForm1.blackpawn(i).yposition
        selected(0) = "blackpawn"
        DontRepeat = False
    End If
End If

'Reset board back to original
Resetboard()
End If
Next

```



```

        Next
    Next

    If PieceCanMove = False Then
        six(0) = 1000
    End If

    'Black Knight
    DontRepeat = True
    PieceCanMove = False
    '
    For i As Integer = 0 To 1 + GameForm1.bknightcount
        '
        '
        Dim xpos As Integer = GameForm1.blackknight(i).xposition
        Dim ypos As Integer = GameForm1.blackknight(i).yposition
        '
        '
        For j As Integer = 0 To 7
            For k As Integer = 0 To 7
                recursionstop = False
                fillcheckforblack()
                GameForm1.blackknight(i).possiblemoves()
                'If the knight can move
                If GameForm1.blackknight(i).flags(j, k) = True Then
                    PieceCanMove = True
                    'Changing position of the knight
                    GameForm1.blackknight(i).changeposition(j, k,
GameForm1.blackknight(i).xposition, GameForm1.blackknight(i).yposition)
                    'Change the x and y position of the knight (as write
to board = false)

                    GameForm1.blackknight(i).xposition = j
                    GameForm1.blackknight(i).yposition = k

                    'Change player to 1
                    player = 1
                    'Second Depth
                    SecondDepth()

                    'See if white king is in check
                    fillcheckforwhite()

                    ' Code will always check the king
                    For a As Integer = 0 To 7
                        For b As Integer = 0 To 7
                            If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackknight(i).xposition,
GameForm1.blackknight(i).yposition) = False Then
                                    'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the knight can
take a piece)

                                    q = q - 5
                                    AICheckmate()
                                End If
                            End If
                        End For
                    End For
                End If
            End For
        End For
    End For

```

```

        End If
    Next
Next

'Change player to 2
player = 2

'Change the x and y position of the knight back to
original
GameForm1.blackknight(i).xposition = xpos
GameForm1.blackknight(i).yposition = ypos

'Store point value if knight moves there
If DontRepeat = True Then
    six(1) = q
    x(1) = j
    y(1) = k
    prex(1) = GameForm1.blackknight(i).xposition
    prey(1) = GameForm1.blackknight(i).yposition
    selected(1) = "blackknight"
    DontRepeat = False
Else
    'Calculate random number (either a 0 or a 1)
    Dim samemove As Integer
    Dim rnd As New Random
    samemove = rnd.Next(0, 2)
    If q < six(1) Then
        six(1) = q
        x(1) = j
        y(1) = k
        prex(1) = GameForm1.blackknight(i).xposition
        prey(1) = GameForm1.blackknight(i).yposition
        selected(1) = "blackknight"
    End If
    'If the maximum move is the same value as the one
    already stored, randomly choose a move
    If q = six(1) And samemove = 1 Then
        six(1) = q
        x(1) = j
        y(1) = k
        prex(1) = GameForm1.blackknight(i).xposition
        prey(1) = GameForm1.blackknight(i).yposition
        selected(1) = "blackknight"
        DontRepeat = False
    End If
End If

'Reset board back to original
Resetboard()
End If
Next
Next
Next

If PieceCanMove = False Then
    six(1) = 1000
End If

```

```

'Black Queen
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To GameForm1.bqueencount
    '
    '
    Dim xpos As Integer = GameForm1.blackqueen(i).xposition
    Dim ypos As Integer = GameForm1.blackqueen(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recursionstop = False
            fillcheckforblack()
            GameForm1.blackqueen(i).possiblemoves()
            'If the queen can move
            If GameForm1.blackqueen(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the queen
                GameForm1.blackqueen(i).changeposition(j, k,
GameForm1.blackqueen(i).xposition, GameForm1.blackqueen(i).yposition)
                'Change the x and y position of the queen (as write
to board = false)

                GameForm1.blackqueen(i).xposition = j
                GameForm1.blackqueen(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                'Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackqueen(i).xposition, GameForm1.blackqueen(i).yposition)
= False Then
                                    '
                                    'If black can check
the white king, - 5 off of q, (most post effective way, will check king
unless the queen can take a piece)
                                        q = q - 5
                                        AICheckmate()
                                    End If
                                End If
                            Next
                        Next
                    Next

                'Change player to 2

```

```

original
    player = 2

    'Change the x and y position of the queen back to
    GameForm1.blackqueen(i).xposition = xpos
    GameForm1.blackqueen(i).yposition = ypos

    'Store point value if queen moves there
    If DontRepeat = True Then
        six(2) = q
        x(2) = j
        y(2) = k
        prex(2) = GameForm1.blackqueen(i).xposition
        prey(2) = GameForm1.blackqueen(i).yposition
        selected(2) = "blackqueen"
        DontRepeat = False
    Else
        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(2) Then
            six(2) = q
            x(2) = j
            y(2) = k
            prex(2) = GameForm1.blackqueen(i).xposition
            prey(2) = GameForm1.blackqueen(i).yposition
            selected(2) = "blackqueen"
        End If
        'If the maximum move is the same value as the one
        already stored, randomly choose a move
        If q = six(2) And samemove = 1 Then
            six(2) = q
            x(2) = j
            y(2) = k
            prex(2) = GameForm1.blackqueen(i).xposition
            prey(2) = GameForm1.blackqueen(i).yposition
            selected(2) = "blackqueen"
            DontRepeat = False
        End If
    End If

    'Reset board back to original
    Resetboard()
End If
Next
Next
Next

If PieceCanMove = False Then
    six(2) = 1000
End If

'Black Bishop
DontRepeat = True
PieceCanMove = False

```

```

For i As Integer = 0 To 1 + GameForm1.bbishopcount
    '
    '
    Dim xpos As Integer = GameForm1.blackbishop(i).xposition
    Dim ypos As Integer = GameForm1.blackbishop(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recursionstop = False
            fillcheckforblack()
            GameForm1.blackbishop(i).possiblemoves()
            'If the bishop can move
            If GameForm1.blackbishop(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the bishop
                GameForm1.blackbishop(i).changeposition(j, k,
GameForm1.blackbishop(i).xposition, GameForm1.blackbishop(i).yposition)
                'Change the x and y position of the bishop (as write
to board = false)

                GameForm1.blackbishop(i).xposition = j
                GameForm1.blackbishop(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                'Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackbishop(i).xposition,
GameForm1.blackbishop(i).yposition) = False Then
                                    'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the bishop can
take a piece)

                                        q = q - 5
                                        AICheckmate()
                                    End If
                                End If
                            Next
                        Next

                        'Change player to 2
                        player = 2

                        'Change the x and y position of the bishop back to
original

                        GameForm1.blackbishop(i).xposition = xpos
                        GameForm1.blackbishop(i).yposition = ypos

```

```

        'Store point value if bishop moves there
        If DontRepeat = True Then
            six(3) = q
            x(3) = j
            y(3) = k
            prex(3) = GameForm1.blackbishop(i).xposition
            prey(3) = GameForm1.blackbishop(i).yposition
            selected(3) = "blackbishop"
            DontRepeat = False
        Else
            'Calculate random number (either a 0 or a 1)
            Dim samemove As Integer
            Dim rnd As New Random
            samemove = rnd.Next(0, 2)
            If q < six(3) Then
                six(3) = q
                x(3) = j
                y(3) = k
                prex(3) = GameForm1.blackbishop(i).xposition
                prey(3) = GameForm1.blackbishop(i).yposition
                selected(3) = "blackbishop"
            End If
            'If the maximum move is the same value as the one
already stored, randomly choose a move
            If q = six(3) And samemove = 1 Then
                six(3) = q
                x(3) = j
                y(3) = k
                prex(3) = GameForm1.blackbishop(i).xposition
                prey(3) = GameForm1.blackbishop(i).yposition
                selected(3) = "blackbishop"
                DontRepeat = False
            End If
        End If

        'Reset board back to original
        Resetboard()
    End If
Next
Next

Next

If PieceCanMove = False Then
    six(3) = 1000
End If

'Black Rook
DontRepeat = True
PieceCanMove = False
'
For i As Integer = 0 To 1 + GameForm1.brookcount
    '
    Dim xpos As Integer = GameForm1.blackrook(i).xposition

```

```

    Dim ypos As Integer = GameForm1.blackrook(i).yposition
    '
    '
    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            recurrisionstop = False
            fillcheckforblack()
            GameForm1.blackrook(i).possiblemoves()
            'If the rook can move
            If GameForm1.blackrook(i).flags(j, k) = True Then
                PieceCanMove = True
                'Changing position of the rook
                GameForm1.blackrook(i).changeposition(j, k,
GameForm1.blackrook(i).xposition, GameForm1.blackrook(i).yposition)
                'Change the x and y position of the rook (as write to
board = false)

                GameForm1.blackrook(i).xposition = j
                GameForm1.blackrook(i).yposition = k

                'Change player to 1
                player = 1
                'Second Depth
                SecondDepth()

                'See if white king is in check
                fillcheckforwhite()

                'Code will always check the king
                For a As Integer = 0 To 7
                    For b As Integer = 0 To 7
                        If
checkforwhite(GameForm1.whiteking.xposition, GameForm1.whiteking.yposition) =
True Then
                                If
canmove(GameForm1.blackrook(i).xposition, GameForm1.blackrook(i).yposition) =
False Then
                                    'If black can check the white king, -
5 off of q, (most post effective way, will check king unless the rook can
take a piece)

                                        q = q - 5
                                        AICheckmate()
                                    End If
                                End If
                            Next
                        Next
                    Next

                    'Change player to 2
                    player = 2

                    'Change the x and y position of the rook back to
original

                    GameForm1.blackrook(i).xposition = xpos
                    GameForm1.blackrook(i).yposition = ypos
                    'Store point value if rook moves there
                    If DontRepeat = True Then
                        six(4) = q
                        x(4) = j

```

```

        y(4) = k
        prex(4) = GameForm1.blackrook(i).xposition
        prey(4) = GameForm1.blackrook(i).yposition
        selected(4) = "blackrook"
        DontRepeat = False
    Else
        'Calculate random number (either a 0 or a 1)
        Dim samemove As Integer
        Dim rnd As New Random
        samemove = rnd.Next(0, 2)
        If q < six(4) Then
            six(4) = q
            x(4) = j
            y(4) = k
            prex(4) = GameForm1.blackrook(i).xposition
            prey(4) = GameForm1.blackrook(i).yposition
            selected(4) = "blackrook"
        End If
        'If the maximum move is the same value as the one
already stored, randomly choose a move
        If q = six(4) And samemove = 1 Then
            six(4) = q
            x(4) = j
            y(4) = k
            prex(4) = GameForm1.blackrook(i).xposition
            prey(4) = GameForm1.blackrook(i).yposition
            selected(4) = "blackrook"
            DontRepeat = False
        End If
    End If

    'Reset board back to original
    Resetboard()
End If

Next
Next
Next

If PieceCanMove = False Then
    six(4) = 1000
End If

'Black King
DontRepeat = True
PieceCanMove = False
'
'
Dim xposking As Integer = GameForm1.blackking.xposition
Dim yposking As Integer = GameForm1.blackking.yposition
'
'
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        recurrisionstop = False
        fillcheckforblack()
        GameForm1.blackking.possiblemoves()
        'If the king can move

```



```

    If GameForm1.blackking.flags(j, k) = True Then
        PieceCanMove = True
        'Changing position of the king
        GameForm1.blackking.changeposition(j, k,
GameForm1.blackking.xposition, GameForm1.blackking.yposition)
        'Change the x and y position of the king (as write to
board = false)

        GameForm1.blackking.xposition = j
        GameForm1.blackking.yposition = k

        'Change player to 1
        player = 1
        'Second Depth
        SecondDepth()

        'Change player to 2
        player = 2

        'Change the x and y position of the king back to original
        GameForm1.blackking.xposition = xposking
        GameForm1.blackking.yposition = yposking

        'Store point value if king moves there
        If DontRepeat = True Then
            six(5) = q
            x(5) = j
            y(5) = k
            prex(5) = GameForm1.blackking.xposition
            prey(5) = GameForm1.blackking.yposition
            selected(5) = "blackking"
            DontRepeat = False
        Else
            'Calculate random number (either a 0 or a 1)
            Dim samemove As Integer
            Dim rnd As New Random
            samemove = rnd.Next(0, 2)
            If q < six(5) Then
                six(5) = q
                x(5) = j
                y(5) = k
                prex(5) = GameForm1.blackking.xposition
                prey(5) = GameForm1.blackking.yposition
                selected(5) = "blackking"
            End If
            'If the maximum move is the same value as the one
already stored, randomly choose a move
            If q = six(5) And samemove = 1 Then
                six(5) = q
                x(5) = j
                y(5) = k
                prex(5) = GameForm1.blackking.xposition
                prey(5) = GameForm1.blackking.yposition
                selected(5) = "blackking"
                DontRepeat = False
            End If
        End If
    End If

```

```

        'Reset board back to original
        Resetboard()
    End If
Next
Next

If PieceCanMove = False Then
    six(5) = 1000
End If

canwritetoboardtracker = True
For i As Integer = 0 To 7
    For j As Integer = 0 To 7
        GameForm1.pb(i, j).Enabled = True
    Next
Next
Resetboard()
show()
reback()
'Bubble sort algorithm for deciding best move
Dim iTemp As Integer
Dim iTemp1 As String
For iPass = 1 To 4
    For i As Integer = 0 To 4
        If six(i) < six(i + 1) Then
            iTemp = six(i)
            six(i) = six(i + 1)
            six(i + 1) = iTemp
            iTemp = x(i)
            x(i) = x(i + 1)
            x(i + 1) = iTemp
            iTemp = y(i)
            y(i) = y(i + 1)
            y(i + 1) = iTemp
            iTemp = prex(i)
            prex(i) = prex(i + 1)
            prex(i + 1) = iTemp
            iTemp = prey(i)
            prey(i) = prey(i + 1)
            prey(i + 1) = iTemp
            iTemp1 = selected(i)
            selected(i) = selected(i + 1)
            selected(i + 1) = iTemp1
        End If
    Next
Next

Dim random As Integer = -1
Dim rndd As New Random
'if multiple moves have the same minimum losing value, randomly
select one
'ignore king due to chess logic
If six(5) = six(4) Then
    random = 4
    If six(4) = six(3) Then
        random = rndd.Next(3, 5)
        If six(3) = six(2) Then

```

```

        random = rndd.Next(2, 5)
        If six(2) = six(1) Then
            random = rndd.Next(1, 5)
            If six(1) = six(0) Then
                random = rndd.Next(0, 5)
            End If
        End If
    End If
End If
End If
End If
AICalculating = False

If random = -1 Then
    mov(selected(5), x(5), y(5), prex(5), prey(5))
Else
    mov(selected(random), x(random), y(random), prex(random),
prey(random))
End If
End Sub

Dim tempboard(7, 7) As Integer
Dim points As Integer
Dim count As Integer = 0

Private Sub SecondDepth()
    LoadSecondDepthBoard()

    recurrisionstop = False
    fillcheckforwhite()
    recanmove()
    Application.DoEvents()
    'Reset count
    count = 0

    'Move all the white pawns into all possible positions
    For i As Integer = 0 To 7
        If GameForm1.whitepawn(i).stillonboard = False Then
            GameForm1.whitepawn(i).reflags()
        Else

            If board(GameForm1.whitepawn(i).xposition,
GameForm1.whitepawn(i).yposition) = 6 Then
                GameForm1.whitepawn(i).possiblemoves()
                For j As Integer = 0 To 7
                    For k As Integer = 0 To 7
                        'If the pawn can move
                        If GameForm1.whitepawn(i).flags(j, k) = True Then
                            'Move to that square

                                GameForm1.whitepawn(i).changeposition(j, k,
GameForm1.whitepawn(i).xposition, GameForm1.whitepawn(i).yposition)
                            'Reset board back to before 2nd depth move
was made

                                CalculateBoardPoints()

                                ResetSecondDepthBoard()
                            End If
                        End If
                    End For
                End For
            End If
        End If
    End For
End Sub

```

```

        Next
    Next

    GameForm1.whitepawn(i).fillcheckking()

    For j As Integer = 0 To 7
        For k As Integer = 0 To 7
            'If the pawn can move
            If GameForm1.whitepawn(i).flags(j, k) = True Then
                canmove(j, k) = True
            End If
        Next
    Next
End If
End If
Next

'Move all the white knights into all possible positions
For i As Integer = 0 To 1 + GameForm1.wknightcount
    If GameForm1.whiteknight(i).stillonboard = False Then
        GameForm1.whiteknight(i).reflags()
    Else

        If board(GameForm1.whiteknight(i).xposition,
GameForm1.whiteknight(i).yposition) = 4 Then
            GameForm1.whiteknight(i).possiblemoves()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    'If the knight can move
                    If GameForm1.whiteknight(i).flags(j, k) = True
Then
                        'Move to that square
                        GameForm1.whiteknight(i).changeposition(j, k,
GameForm1.whiteknight(i).xposition, GameForm1.whiteknight(i).yposition)
                        'Reset board back to before 2nd depth move
was made

                        CalculateBoardPoints()

                        If GameForm1.whiteknight(i).flags(j, k) =
True Then
                                canmove(j, k) = True
                                End If
                                ResetSecondDepthBoard()
                                recurrsionstop = False
                                fillcheckforwhite()
                            End If
                        Next
                    Next
                End If
            End If
        Next

'Move all the white queens into all possible positions
For i As Integer = 0 To GameForm1.wqueencount
    If GameForm1.whitequeen(i).stillonboard = False Then
        GameForm1.whitequeen(i).reflags()
    End If
Next

```

```

        Else
            If board(GameForm1.whitequeen(i).xposition,
GameForm1.whitequeen(i).yposition) = 2 Then
                GameForm1.whitequeen(i).possiblemoves()
                For j As Integer = 0 To 7
                    For k As Integer = 0 To 7
                        'If the queen can move
                        If GameForm1.whitequeen(i).flags(j, k) = True
Then
                            'Move to that square
                            GameForm1.whitequeen(i).changeposition(j, k,
GameForm1.whitequeen(i).xposition, GameForm1.whitequeen(i).yposition)
                            'Reset board back to before 2nd depth move
was made

                            CalculateBoardPoints()
                            'TESTING
                            If GameForm1.whitequeen(i).flags(j, k) = True
Then
                                canmove(j, k) = True
                                End If
                                ResetSecondDepthBoard()
                                recurrisionstop = False
                                fillcheckforwhite()
                                End If
                            Next
                        Next
                    End If
                End If
            Next

            'Move all the white bishops into all possible positions
            For i As Integer = 0 To 1 + GameForm1.wbishopcount
                If GameForm1.whitebishop(i).stillonboard = False Then
                    GameForm1.whitebishop(i).reflags()
                Else
                    If board(GameForm1.whitebishop(i).xposition,
GameForm1.whitebishop(i).yposition) = 3 Then
                        GameForm1.whitebishop(i).possiblemoves()
                        For j As Integer = 0 To 7
                            For k As Integer = 0 To 7
                                'If the bishop can move
                                If GameForm1.whitebishop(i).flags(j, k) = True
Then
                                    'Move to that square
                                    GameForm1.whitebishop(i).changeposition(j, k,
GameForm1.whitebishop(i).xposition, GameForm1.whitebishop(i).yposition)
                                    'Reset board back to before 2nd depth move
was made

                                    CalculateBoardPoints()

                                    If GameForm1.whitebishop(i).flags(j, k) =
True Then
                                        canmove(j, k) = True
                                        End If

                                    ResetSecondDepthBoard()

```

```

recursionstop = False
fillcheckforwhite()
End If
Next
Next
End If
End If
Next

'Move all the white rook into all possible positions

For i As Integer = 0 To 1 + GameForm1.wrookcount
    If GameForm1.whiterook(i).stillonboard = False Then
        GameForm1.whiterook(i).reflags()
    Else
        If board(GameForm1.whiterook(i).xposition,
GameForm1.whiterook(i).yposition) = 5 Then
            GameForm1.whiterook(i).possiblemoves()
            For j As Integer = 0 To 7
                For k As Integer = 0 To 7
                    'If the rook can move
                    If GameForm1.whiterook(i).flags(j, k) = True Then
                        'Move to that square
                        GameForm1.whiterook(i).changeposition(j, k,
GameForm1.whiterook(i).xposition, GameForm1.whiterook(i).yposition)
                        'Reset board back to before 2nd depth move
was made

                        CalculateBoardPoints()

                        If GameForm1.whiterook(i).flags(j, k) = True

Then
                            canmove(j, k) = True
                        End If
                        ResetSecondDepthBoard()
                        recursionstop = False
                        fillcheckforwhite()
                    End If
                Next
            Next
        End If
    End If
Next

'Move all the white king into all possible positions

GameForm1.whiteking.possiblemoves()
For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        'If the king can move
        If GameForm1.whiteking.flags(j, k) = True Then

            'Move to that square
            GameForm1.whiteking.changeposition(j, k,
GameForm1.whiteking.xposition, GameForm1.whiteking.yposition)
            'Reset board back to before 2nd depth move was made

```

```

        CalculateBoardPoints()
        ResetSecondDepthBoard()
        recurrisionstop = False
        fillcheckforwhite()
    End If
Next
Next

GameForm1.whiteking.fillcheckking()

For j As Integer = 0 To 7
    For k As Integer = 0 To 7
        'If the pawn can move
        If GameForm1.whiteking.flags(j, k) = True Then
            canmove(j, k) = True
        End If
    Next
Next
End Sub
Private Sub LoadTempBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            tempboard(i, j) = board(i, j)
        Next
    Next
End Sub

Private Sub Resetboard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            board(i, j) = tempboard(i, j)
        Next
    Next
End Sub
Dim tempboard2(7, 7) As Integer
Private Sub LoadSecondDepthBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            tempboard2(i, j) = board(i, j)
        Next
    Next
End Sub
Private Sub ResetSecondDepthBoard()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            board(i, j) = tempboard2(i, j)
        Next
    Next
End Sub

Dim q As Integer 'Used in minmax algorithm, minimum loss move

Private Sub CalculateBoardPoints()
    count = count + 1
    points = 0
    'Calculation
    For i As Integer = 0 To 7

```

```

For j As Integer = 0 To 7
    Select Case board(i, j)
        Case 1
            'King = 900 points
            points += 900
        Case 2
            'Queen = 90 points
            points += 90
        Case 3
            'Bishop = 30 points
            points += 30
        Case 4
            'Knight = 30 points
            points += 30
        Case 5
            'Rook = 50 points
            points += 50
        Case 6
            'Pawn = 10 point
            points += 10
        Case -1
            'King = -900 points
            points += -900
        Case -2
            'Queen = -90 points
            points += -90
        Case -3
            'Bishop = -30 points
            points += -30
        Case -4
            'Knight = -30 points
            points += -30
        Case -5
            'Rook = -50 points
            points += -50
        Case -6
            'Pawn = -10 point
            points += -10
    End Select
Next
Next

If count = 1 Then
    q = points
End If

If points > q Then
    q = points
End If

End Sub
Private Sub AICheckmate()
    Dim AICheckmate As Boolean = False
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If canmove(i, j) = False Then
                AICheckmate = True
            End If
        Next j
    Next i
End Sub

```



```

        Else
            AICancheckmate = False
        End If
    Next
Next

If AICancheckmate = True Then
    q = q - 100
End If
End Sub
End Class

```

King

```

Public Class King
    Public xposition, yposition As Integer 'Coordinates for king
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the king
    Public checkthe king(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public AllowedToCastle As Boolean = True

    'positive colour means white and negative black
    Public Sub New(ByVal x, ByVal y, ByVal pval)
        xposition = x
        yposition = y
        piecevalue = pval
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue()
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'showing possible moves
        reflags()
        If piecevalue > 0 Then
            whitemove()
        End If
        If piecevalue < 0 Then
            blackmove()
        End If
        Return flags
    End Function
    Public Sub whitemove()

        GameForm1.ga.CastlingW()

        If xposition + 1 <= 7 Then
            'one space vertically downwards

```

```

        If GameForm1.ga.board(xposition + 1, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
        Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite()
            If GameForm1.ga.checkforwhite(xposition + 1, yposition) =
False Then
                flags(xposition + 1, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition + 1, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 1

        ElseIf GameForm1.ga.board(xposition + 1, yposition) <= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition + 1, yposition) =
False Then
                flags(xposition + 1, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition + 1 <= 7 And xposition + 1 <= 7 Then
        'one space diagonally right top to bottom
        If GameForm1.ga.board(xposition + 1, yposition + 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board

```

```

        temp = GameForm1.ga.board(xposition + 1, yposition + 1)
'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.board(xposition + 1, yposition + 1) = 1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite()
        If GameForm1.ga.checkforwhite(xposition + 1, yposition + 1) =
False Then
            flags(xposition + 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition + 1, yposition + 1) <= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition + 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition + 1, yposition + 1) =
False Then
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition - 1 >= 0 And xposition + 1 <= 7 Then
        'one space diagonally left top to bottom
        If GameForm1.ga.board(xposition + 1, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition - 1)
'storing

```

```

        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.board(xposition + 1, yposition - 1) = 1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforwhite(xposition + 1, yposition - 1) =
False Then
            flags(xposition + 1, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition + 1, yposition - 1) <= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition - 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition + 1, yposition - 1) =
False Then
                flags(xposition + 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If xposition - 1 >= 0 Then
        'one space vertically upwards
        If GameForm1.ga.board(xposition - 1, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space

```

```

        GameForm1.ga.board(xposition - 1, yposition) = 1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforwhite(xposition - 1, yposition) =
False Then
            flags(xposition - 1, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition - 1, yposition) <= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition - 1, yposition) =
False Then
                flags(xposition - 1, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition + 1 <= 7 And xposition - 1 >= 0 Then
        'one space diagonally right bottom to top
        If GameForm1.ga.board(xposition - 1, yposition + 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False

```

```

        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforwhite(xposition - 1, yposition + 1) =
False Then
            flags(xposition - 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition - 1, yposition + 1) <= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition - 1, yposition + 1) =
False Then
                flags(xposition - 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition - 1 >= 0 And xposition - 1 >= 0 Then
        'one space diagonally left bottom to top
        If GameForm1.ga.board(xposition - 1, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False

```

```

        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforwhite(xposition - 1, yposition - 1) =
False Then
            flags(xposition - 1, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition - 1, yposition - 1) <= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition - 1)
' storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition - 1, yposition - 1) =
False Then
                flags(xposition - 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition + 1 <= 7 Then
        'one space horizontally to the right
        If GameForm1.ga.board(xposition, yposition + 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition + 1) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition + 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check

```

```

        If GameForm1.ga.checkforwhite(xposition, yposition + 1) =
False Then
            flags(xposition, yposition + 1) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition, yposition + 1) <= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition + 1) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition + 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition, yposition + 1) =
False Then
                flags(xposition, yposition + 1) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
            (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If

    If yposition - 1 >= 0 Then
        'one space horizontally to the left
        If GameForm1.ga.board(xposition, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false king can move
the board
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition - 1) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition - 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition, yposition - 1) =
False Then
                flags(xposition, yposition - 1) = True 'allow movement
            End If

```



```

        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 1
        ElseIf GameForm1.ga.board(xposition, yposition - 1) <= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition - 1) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition - 1) = 1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforwhite(xposition, yposition - 1) =
False Then
                flags(xposition, yposition - 1) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
            (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = 1
        End If
    End If
End Sub
Public Sub blackmove()

    GameForm1.ga.CastlingB()

    If xposition + 1 <= 7 Then
        'one space vertically downwards
        If GameForm1.ga.board(xposition + 1, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition + 1, yposition) =
False Then
                flags(xposition + 1, yposition) = True 'allow movement
            End If

```

```

        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = -1
    ElseIf GameForm1.ga.board(xposition + 1, yposition) >= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 1, yposition) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.board(xposition + 1, yposition) = -1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforblack(xposition + 1, yposition) =
False Then
            flags(xposition + 1, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition + 1, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = -1
    End If
End If

    If yposition + 1 <= 7 And xposition + 1 <= 7 Then
        'one space diagonally right top to bottom
        If GameForm1.ga.board(xposition + 1, yposition + 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition + 1)
            'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition + 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition + 1, yposition + 1) =
False Then
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
            (before the temp changes)

```

```

        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = -1
        ElseIf GameForm1.ga.board(xposition + 1, yposition + 1) >= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition + 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition + 1, yposition + 1) =
False Then
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        End If
    End If

    If yposition - 1 >= 0 And xposition + 1 <= 7 Then
        'one space diagonally left top to bottom
        If GameForm1.ga.board(xposition + 1, yposition - 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition - 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition + 1, yposition - 1) =
False Then
                flags(xposition + 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)

```

```

        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = -1
        ElseIf GameForm1.ga.board(xposition + 1, yposition - 1) >= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition + 1, yposition - 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition + 1, yposition - 1) =
False Then
                flags(xposition + 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        End If
    End If

    If xposition - 1 >= 0 Then
        'one space vertically upwards
        If GameForm1.ga.board(xposition - 1, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition - 1, yposition) =
False Then
                flags(xposition - 1, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition - 1, yposition) = temp

```

```

        GameForm1.ga.board(xposition, yposition) = -1
    ElseIf GameForm1.ga.board(xposition - 1, yposition) >= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 1, yposition) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.board(xposition - 1, yposition) = -1 'change
desired space to value of the king
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforblack(xposition - 1, yposition) =
False Then
            flags(xposition - 1, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition - 1, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = -1
    End If
End If

If yposition + 1 <= 7 And xposition - 1 >= 0 Then
    'one space diagonally right bottom to top
    If GameForm1.ga.board(xposition - 1, yposition + 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 1, yposition + 1)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.board(xposition - 1, yposition + 1) = -1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforblack(xposition - 1, yposition + 1) =
False Then
            flags(xposition - 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = -1

```

```

        ElseIf GameForm1.ga.board(xposition - 1, yposition + 1) >= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition - 1, yposition + 1) =
False Then
                flags(xposition - 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        End If
    End If

    If yposition - 1 >= 0 And xposition - 1 >= 0 Then
        'one space diagonally left bottom to top
        If GameForm1.ga.board(xposition - 1, yposition - 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition - 1, yposition - 1) =
False Then
                flags(xposition - 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition - 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1

```

```

        ElseIf GameForm1.ga.board(xposition - 1, yposition - 1) >= 0 Then
'If the king is in check, check to see if it can take the piece inflicting
that check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition - 1, yposition - 1) =
False Then
                flags(xposition - 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        End If
    End If

    If yposition + 1 <= 7 Then
        'One space horizontally to the right
        If GameForm1.ga.board(xposition, yposition + 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition + 1) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition + 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition, yposition + 1) =
False Then
                flags(xposition, yposition + 1) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        ElseIf GameForm1.ga.board(xposition, yposition + 1) >= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check

```

```

        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition, yposition + 1) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
        GameForm1.ga.board(xposition, yposition + 1) = -1 'change
desired space to value of the king
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforblack(xposition, yposition + 1) =
False Then
            flags(xposition, yposition + 1) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = -1
    End If
End If

    If yposition - 1 >= 0 Then
        'One space horizontally to the left
        If GameForm1.ga.board(xposition, yposition - 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false king can move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition - 1) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space
            GameForm1.ga.board(xposition, yposition - 1) = -1 'change
desired space to value of the king
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
            If GameForm1.ga.checkforblack(xposition, yposition - 1) =
False Then
                flags(xposition, yposition - 1) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = -1
        ElseIf GameForm1.ga.board(xposition, yposition - 1) >= 0 Then 'If
the king is in check, check to see if it can take the piece inflicting that
check
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition, yposition - 1) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
king position as a blank space

```



```

        GameForm1.ga.board(xposition, yposition - 1) = -1 'change
desired space to value of the king
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
king was not there, would it still be in check
        If GameForm1.ga.checkforblack(xposition, yposition - 1) =
False Then
            flags(xposition, yposition - 1) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = -1
    End If
End If

End Sub
Public Sub reflags()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub

Public Sub fillcheckking()
    possiblemoves()

    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then
                checktheking(i, j) = True
            Else
                checktheking(i, j) = False
            End If
        Next
    Next
End Sub

Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)

    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0
    If GameForm1.ga.canwritetoboardtracker = True Then
        Dim whatcolour As String

        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"
        Else
            whatcolour = "Black"
        End If

        Dim chessnotationletter(1) As String
        Dim chessnotationnumber(1) As Integer
        For i = 0 To 1

```

```

Select Case yposition
    Case 0
        chessnotationletter(i) = "A"
    Case 1
        chessnotationletter(i) = "B"
    Case 2
        chessnotationletter(i) = "C"
    Case 3
        chessnotationletter(i) = "D"
    Case 4
        chessnotationletter(i) = "E"
    Case 5
        chessnotationletter(i) = "F"
    Case 6
        chessnotationletter(i) = "G"
    Case 7
        chessnotationletter(i) = "H"
End Select

Select Case xposition
    Case 0
        chessnotationnumber(i) = "8"
    Case 1
        chessnotationnumber(i) = "7"
    Case 2
        chessnotationnumber(i) = "6"
    Case 3
        chessnotationnumber(i) = "5"
    Case 4
        chessnotationnumber(i) = "4"
    Case 5
        chessnotationnumber(i) = "3"
    Case 6
        chessnotationnumber(i) = "2"
    Case 7
        chessnotationnumber(i) = "1"
End Select
xposition = x
yposition = y
Next

    GameForm1.BoardTracker.AppendText(whatcolour & (" King ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)
End If

End Sub
End Class

```

Knight

```

Public Class Knight
    Public xposition, yposition As Integer 'Coordinates for knight
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the knight
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
knight, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)
    'Positive colour is white and negative is black
    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'showing possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags
    End Function

    Public Sub whitemove()
        '
        'Down 2 and left 1
        '
        If xposition + 2 <= 7 And yposition - 1 >= 0 Then
            '
            'Code to make sure black king cannot move into a check position
            allcheckmoves = True
            If GameForm1.ga.board(xposition + 2, yposition - 1) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
                flags(xposition + 2, yposition - 1) = True
                allcheckmoves = False
            End If
            '
            '
            If GameForm1.ga.board(xposition + 2, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,

```

```

GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
    Dim temp As Integer 'declaring variable to store value from
the board
    temp = GameForm1.ga.board(xposition + 2, yposition - 1)
'storing
    GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
    GameForm1.ga.board(xposition + 2, yposition - 1) = 4 'change
desired space to a random number
    GameForm1.ga.recurrsionstop = False
    GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
    If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
        flags(xposition + 2, yposition - 1) = True 'allow
movement
    ElseIf xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
        flags(xposition + 2, yposition - 1) = True 'allow
movement
    End If
    'Returning the value of the board back to the original values
(before the temp changes)
    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
    GameForm1.ga.board(xposition + 2, yposition - 1) = temp
    GameForm1.ga.board(xposition, yposition) = 4
    ElseIf GameForm1.ga.board(xposition + 2, yposition - 1) <= 0 Then
'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition + 2, yposition - 1) = 0 Then
'If the space is empty
            GameForm1.ga.board(xposition + 2, yposition - 1) = 4
'Change space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition + 2, yposition - 1) = True
            End If
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition + 2, yposition - 1) = 0
'Return the random number back to a blank space
        End If
        If GameForm1.ga.board(xposition + 2, yposition - 1) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
            If xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition + 2, yposition -
1) 'storing

```

```

        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition + 2, yposition - 1) = 4
'change desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition + 2, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(xposition + 2, yposition - 1) =
temp
        GameForm1.ga.board(xposition, yposition) = 4
    End If
End If
End If
End If
'
'down 2 and right 1
'
If xposition + 2 <= 7 And yposition + 1 <= 7 Then
    '
    'Code to make sure black king cannot move into a check position
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 2, yposition + 1) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
        flags(xposition + 2, yposition + 1) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(xposition + 2, yposition + 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 2, yposition + 1)
'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 2, yposition + 1) = 4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then

```

```

        flags(xposition + 2, yposition + 1) = True 'allow
movement
        ElseIf xposition + 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 2, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 2, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition + 2, yposition + 1) <= 0 Then
'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition + 2, yposition + 1) = 0 Then
'If the space is empty
            GameForm1.ga.board(xposition + 2, yposition + 1) = 4
'Change space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition + 2, yposition + 1) = True
            End If
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition + 2, yposition + 1) = 0
'Return the random number back to a blank space
            End If
            If GameForm1.ga.board(xposition + 2, yposition + 1) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
                If xposition + 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition + 2, yposition +
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(xposition + 2, yposition + 1) = 4
'change desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(xposition + 2, yposition + 1) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

```

```

GameForm1.ga.checkforwhite (GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
    GameForm1.ga.board(xposition + 2, yposition + 1) =
temp
    GameForm1.ga.board(xposition, yposition) = 4
    End If
    End If
    End If
End If
'
'up 2 and left 1
'
If xposition - 2 >= 0 And yposition - 1 >= 0 Then
    '
    'Code to make sure black king cannot move into a check position
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 2, yposition - 1) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
        flags(xposition - 2, yposition - 1) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(xposition - 2, yposition - 1) <= 0 And
GameForm1.ga.checkforwhite (GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 2, yposition - 1)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 2, yposition - 1) = 4 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite (GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 2, yposition - 1) = True 'allow
movement
        ElseIf xposition - 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
            flags(xposition - 2, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite (GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 2, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition - 2, yposition - 1) <= 0 Then
'If king is in check, check for available spaces

```

```

        If GameForm1.ga.board(xposition - 2, yposition - 1) = 0 Then
'If the space is empty
        GameForm1.ga.board(xposition - 2, yposition - 1) = 4
'Change space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition - 2, yposition - 1) = True
        End If
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition - 2, yposition - 1) = 0
'Return the random number back to a blank space
        End If
        If GameForm1.ga.board(xposition - 2, yposition - 1) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
            If xposition - 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition - 2, yposition -
1) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition - 2, yposition - 1) = 4
'change desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition - 2, yposition - 1) = True 'allow
movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(xposition - 2, yposition - 1) =
temp
                GameForm1.ga.board(xposition, yposition) = 4
            End If
        End If
    End If
End If
'
'up 2 and right 1
'
If xposition - 2 >= 0 And yposition + 1 <= 7 Then
    '

```



```

        'Code to make sure black king cannot move into a check position
        allcheckmoves = True
        If GameForm1.ga.board(xposition - 2, yposition + 1) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
            flags(xposition - 2, yposition + 1) = True
            allcheckmoves = False
        End If
    '
    '
        If GameForm1.ga.board(xposition - 2, yposition + 1) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition - 2, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
            GameForm1.ga.board(xposition - 2, yposition + 1) = 4 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 2, yposition + 1) = True 'allow
movement
            ElseIf xposition - 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
                flags(xposition - 2, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition - 2, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = 4
            ElseIf GameForm1.ga.board(xposition - 2, yposition + 1) <= 0 Then
'If king is in check, check for available spaces
                If GameForm1.ga.board(xposition - 2, yposition + 1) = 0 Then
'If the space is empty
                    GameForm1.ga.board(xposition - 2, yposition + 1) = 4
'Change space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                        flags(xposition - 2, yposition + 1) = True
                    End If
                    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check

```

```

        GameForm1.ga.board(xposition - 2, yposition + 1) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition - 2, yposition + 1) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition - 2, yposition +
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(xposition - 2, yposition + 1) = 4
'change desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(xposition - 2, yposition + 1) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    GameForm1.ga.board(xposition - 2, yposition + 1) =
temp
                        GameForm1.ga.board(xposition, yposition) = 4
                    End If
                End If
            End If
        End If
'
'up 1 and right 2
'
    If xposition - 1 >= 0 And yposition + 2 <= 7 Then
        '
        'Code to make sure black king cannot move into a check position
        allcheckmoves = True
        If GameForm1.ga.board(xposition - 1, yposition + 2) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
            flags(xposition - 1, yposition + 2) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(xposition - 1, yposition + 2) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board

```

```

temp = GameForm1.ga.board(xposition - 1, yposition + 2)
'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
GameForm1.ga.board(xposition - 1, yposition + 2) = 4 'change
desired space to a random number
GameForm1.ga.recurrisionstop = False
GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
    flags(xposition - 1, yposition + 2) = True 'allow
movement
ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
    flags(xposition - 1, yposition + 2) = True 'allow
movement
End If
'Returning the value of the board back to the original values
(before the temp changes)
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
GameForm1.ga.board(xposition - 1, yposition + 2) = temp
GameForm1.ga.board(xposition, yposition) = 4
ElseIf GameForm1.ga.board(xposition - 1, yposition + 2) <= 0 Then
'If king is in check, check for available spaces
    If GameForm1.ga.board(xposition - 1, yposition + 2) = 0 Then
'If the space is empty
        GameForm1.ga.board(xposition - 1, yposition + 2) = 4
'Change space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition - 1, yposition + 2) = True
        End If
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition - 1, yposition + 2) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition - 1, yposition + 2) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(xposition - 1, yposition +
2) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 2) = 4
'change desired space to a random number

```

```

        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition + 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(xposition - 1, yposition + 2) =
temp
        GameForm1.ga.board(xposition, yposition) = 4
        End If
    End If
End If
'
'down 1 and right 2
'
If xposition + 1 <= 7 And yposition + 2 <= 7 Then
    '
    'Code to make sure black king cannot move into a check position
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 1, yposition + 2) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
        flags(xposition + 1, yposition + 2) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(xposition + 1, yposition + 2) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the king is not in check, can
move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 1, yposition + 2)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition + 2) = 4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition + 1, yposition + 2) = True 'allow
movement
        ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then

```

```

        flags(xposition + 1, yposition + 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition + 2) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition + 1, yposition + 2) <= 0 Then
'If the king is in check, see if you can either block or take the piece to
resulting in a non-check
        If GameForm1.ga.board(xposition + 1, yposition + 2) = 0 Then
            GameForm1.ga.board(xposition + 1, yposition + 2) = 4
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the knight moved to the desired space, would the king still be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition + 1, yposition + 2) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition + 2) = 0
        End If
        If GameForm1.ga.board(xposition + 1, yposition + 2) < 0 Then
            If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition + 1, yposition +
2) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition + 1, yposition + 2) = 4
'change desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition + 1, yposition + 2) = True 'allow
movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(xposition + 1, yposition + 2) =
temp
                GameForm1.ga.board(xposition, yposition) = 4

```

```

        End If
    End If
End If
'
'up 1 and left 2
'
If xposition - 1 >= 0 And yposition - 2 >= 0 Then
    '
    'Code to make sure black king cannot move into a check position
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 1, yposition - 2) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
        flags(xposition - 1, yposition - 2) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(xposition - 1, yposition - 2) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 1, yposition - 2)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 1, yposition - 2) = 4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition - 2) = True 'allow
movement
        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition - 1, yposition - 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition - 2) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition - 1, yposition - 2) <= 0 Then
'If king is in check, check for available spaces
            If GameForm1.ga.board(xposition - 1, yposition - 2) = 0 Then
'If the space is empty
                GameForm1.ga.board(xposition - 1, yposition - 2) = 4
'Change space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there

```

```

        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition - 1, yposition - 2) = True
        End If
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition - 1, yposition - 2) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition - 1, yposition - 2) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(xposition - 1, yposition -
2) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 2) = 4
'change desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 1, yposition - 2) = True 'allow
movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition - 2) =
temp
            GameForm1.ga.board(xposition, yposition) = 4
        End If
    End If
End If
End If
'
'down 1 and left 2
'
If xposition + 1 <= 7 And yposition - 2 >= 0 Then
    '
    'Code to make sure black king cannot move into a check position
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 1, yposition - 2) > 0 And
allcheckmoves = True And GameForm1.ga.player = 2 Then
        flags(xposition + 1, yposition - 2) = True
        allcheckmoves = False
    End If
End If

```

```

    '
    '
    If GameForm1.ga.board(xposition + 1, yposition - 2) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 1, yposition - 2)
    'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition - 2) = 4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition + 1, yposition - 2) = True 'allow
movement
        ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 1, yposition - 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition - 2) = temp
        GameForm1.ga.board(xposition, yposition) = 4
        ElseIf GameForm1.ga.board(xposition + 1, yposition - 2) <= 0 Then
'If king is in check, check for available spaces
            If GameForm1.ga.board(xposition + 1, yposition - 2) = 0 Then
'If the space is empty
                GameForm1.ga.board(xposition + 1, yposition - 2) = 4
'Change space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition + 1, yposition - 2) = True
                End If
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(xposition + 1, yposition - 2) = 0
'Return the random number back to a blank space
            End If
            If GameForm1.ga.board(xposition + 1, yposition - 2) < 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
                If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then

```



```

        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(xposition + 1, yposition -
2) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition - 2) = 4
'change desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition + 1, yposition - 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(xposition + 1, yposition - 2) =
temp
        GameForm1.ga.board(xposition, yposition) = 4
    End If
End If
End If
End If
End Sub
Public Sub blackmove()
    '
    'down 2 and left 1
    '

    If xposition + 2 <= 7 And yposition - 1 >= 0 Then
        allcheckmoves = True
        If GameForm1.ga.board(xposition + 2, yposition - 1) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
            flags(xposition + 2, yposition - 1) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(xposition + 2, yposition - 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
            Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(xposition + 2, yposition - 1)
' storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
            GameForm1.ga.board(xposition + 2, yposition - 1) = -4 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check

```

```

        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 2, yposition - 1) = True 'allow
movement
        ElseIf xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 2, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 2, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = -4
        ElseIf GameForm1.ga.board(xposition + 2, yposition - 1) >= 0 Then
'If king is in check, check for available spaces
            If GameForm1.ga.board(xposition + 2, yposition - 1) = 0 Then
'If the space is empty
                GameForm1.ga.board(xposition + 2, yposition - 1) = -4
'Change space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition + 2, yposition - 1) = True
                End If
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(xposition + 2, yposition - 1) = 0
'Return the random number back to a blank space
            End If
            If GameForm1.ga.board(xposition + 2, yposition - 1) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
                If xposition + 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition + 2, yposition -
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(xposition + 2, yposition - 1) = -4
'change desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(xposition + 2, yposition - 1) = True 'allow
movement
                    End If

```

```

        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition + 2, yposition - 1) =
temp
        GameForm1.ga.board(xposition, yposition) = -4
    End If
End If
End If
End If

'down 2 and right 1

If xposition + 2 <= 7 And yposition + 1 <= 7 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 2, yposition + 1) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition + 2, yposition + 1) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition + 2, yposition + 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 2, yposition + 1)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 2, yposition + 1) = -4 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 2, yposition + 1) = True 'allow
movement
        ElseIf xposition + 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 2, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 2, yposition + 1) = temp
        GameForm1.ga.board(xposition, yposition) = -4
    ElseIf GameForm1.ga.board(xposition + 2, yposition + 1) >= 0 Then
        'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition + 2, yposition + 1) = 0 Then
            'If the space is empty

```

```

        GameForm1.ga.board(xposition + 2, yposition + 1) = -4
'Change space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition + 2, yposition + 1) = True
        End If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition + 2, yposition + 1) = 0
'Return the random number back to a blank space
        End If
        If GameForm1.ga.board(xposition + 2, yposition + 1) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
            If xposition + 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition + 2, yposition +
1) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition + 2, yposition + 1) = -4
'change desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(xposition + 2, yposition + 1) = True 'allow
movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                GameForm1.ga.board(xposition + 2, yposition + 1) =
temp
                GameForm1.ga.board(xposition, yposition) = -4
            End If
        End If
    End If
    End If

'up 2 and left 1
If xposition - 2 >= 0 And yposition - 1 >= 0 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 2, yposition - 1) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition - 2, yposition - 1) = True

```

```

        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition - 2, yposition - 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 2, yposition - 1)
    'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 2, yposition - 1) = -4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition - 2, yposition - 1) = True 'allow
movement
        ElseIf xposition - 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then
            flags(xposition - 2, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition - 2, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = -4
        ElseIf GameForm1.ga.board(xposition - 2, yposition - 1) >= 0 Then
'If king is in check, check for available spaces
            If GameForm1.ga.board(xposition - 2, yposition - 1) = 0 Then
'If the space is empty
                GameForm1.ga.board(xposition - 2, yposition - 1) = -4
'Change space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition - 2, yposition - 1) = True
                End If
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(xposition - 2, yposition - 1) = 0
'Return the random number back to a blank space
            End If
            If GameForm1.ga.board(xposition - 2, yposition - 1) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
                If xposition - 2 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then

```

```

        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(xposition - 2, yposition -
1) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition - 2, yposition - 1) = -4
'change desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition - 2, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition - 2, yposition - 1) =
temp
        GameForm1.ga.board(xposition, yposition) = -4
    End If
End If
End If
End If

'up 2 and right 1
If xposition - 2 >= 0 And yposition + 1 <= 7 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 2, yposition + 1) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition - 2, yposition + 1) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition - 2, yposition + 1) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 2, yposition + 1)
' storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 2, yposition + 1) = -4 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition - 2, yposition + 1) = True 'allow
movement

```

```

ElseIf xposition - 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
    flags(xposition - 2, yposition + 1) = True 'allow
movement
End If
'Returning the value of the board back to the original values
(before the temp changes)
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
GameForm1.ga.board(xposition - 2, yposition + 1) = temp
GameForm1.ga.board(xposition, yposition) = -4
ElseIf GameForm1.ga.board(xposition - 2, yposition + 1) >= 0 Then
'If king is in check, check for available spaces
    If GameForm1.ga.board(xposition - 2, yposition + 1) = 0 Then
'If the space is empty
        GameForm1.ga.board(xposition - 2, yposition + 1) = -4
'Change space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition - 2, yposition + 1) = True
        End If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition - 2, yposition + 1) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition - 2, yposition + 1) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 2 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(xposition - 2, yposition +
1) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition - 2, yposition + 1) = -4
'change desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition - 2, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

```

```

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
    GameForm1.ga.board(xposition - 2, yposition + 1) =
temp
    GameForm1.ga.board(xposition, yposition) = -4
End If
End If
End If
End If

'up 1 and right 2
If xposition - 1 >= 0 And yposition + 2 <= 7 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 1, yposition + 2) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition - 1, yposition + 2) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition - 1, yposition + 2) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 1, yposition + 2)
'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 1, yposition + 2) = -4 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition - 1, yposition + 2) = True 'allow
movement
        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition - 1, yposition + 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition + 2) = temp
        GameForm1.ga.board(xposition, yposition) = -4
    ElseIf GameForm1.ga.board(xposition - 1, yposition + 2) >= 0 Then
        'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition - 1, yposition + 2) = 0 Then
            'If the space is empty
            GameForm1.ga.board(xposition - 1, yposition + 2) = -4
            'Change space to a random number
            GameForm1.ga.recurrsionstop = False

```



```

        GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
        flags(xposition - 1, yposition + 2) = True
    End If
    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
    GameForm1.ga.board(xposition - 1, yposition + 2) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition - 1, yposition + 2) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(xposition - 1, yposition +
2) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 2) = -4
'change desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition - 1, yposition + 2) = True 'allow
movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition + 2) =
temp
            GameForm1.ga.board(xposition, yposition) = -4
        End If
    End If
End If
End If

'down 1 and right 2
If xposition + 1 <= 7 And yposition + 2 <= 7 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 1, yposition + 2) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition + 1, yposition + 2) = True
        allcheckmoves = False
    End If

```

```

        If GameForm1.ga.board(xposition + 1, yposition + 2) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the king is not in check, can
move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 1, yposition + 2)
'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition + 2) = -4 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition + 2) = True 'allow
movement
        ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 1, yposition + 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition + 2) = temp
        GameForm1.ga.board(xposition, yposition) = -4
    ElseIf GameForm1.ga.board(xposition + 1, yposition + 2) >= 0 Then
        'If the king is in check, see if you can either block or take the piece to
resulting in a non-check
        If GameForm1.ga.board(xposition + 1, yposition + 2) = 0 Then
            GameForm1.ga.board(xposition + 1, yposition + 2) = 4
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the knight moved to the desired space, would the king still be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition + 1, yposition + 2) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition + 2) = 0
        End If
        If GameForm1.ga.board(xposition + 1, yposition + 2) > 0 Then
            If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 2 = GameForm1.ga.ypositionCheck Then
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition + 1, yposition +
2) 'storing

```

```

        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition + 2) = -4
'change desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition + 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition + 1, yposition + 2) =
temp
        GameForm1.ga.board(xposition, yposition) = -4
    End If
End If
End If
End If

'up 1 and left 2
If xposition - 1 >= 0 And yposition - 2 >= 0 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition - 1, yposition - 2) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition - 1, yposition - 2) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition - 1, yposition - 2) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition - 1, yposition - 2)
'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition - 1, yposition - 2) = -4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition - 1, yposition - 2) = True 'allow
movement
        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition - 1, yposition - 2) = True 'allow
movement

```

```

        End If
        'Returning the value of the board back to the original values
        (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition - 2) = temp
        GameForm1.ga.board(xposition, yposition) = -4
        ElseIf GameForm1.ga.board(xposition - 1, yposition - 2) >= 0 Then
        'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition - 1, yposition - 2) = 0 Then
        'If the space is empty
        GameForm1.ga.board(xposition - 1, yposition - 2) = -4
        'Change space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
        If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
        flags(xposition - 1, yposition - 2) = True
        End If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition - 1, yposition - 2) = 0
        'Return the random number back to a blank space
        End If
        If GameForm1.ga.board(xposition - 1, yposition - 2) > 0 Then
        'If the knight can take a black piece and check is no longer true then flag
space
        If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(xposition - 1, yposition -
2) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition - 1, yposition - 2) = -4
        'change desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
        flags(xposition - 1, yposition - 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition - 1, yposition - 2) =
temp
        GameForm1.ga.board(xposition, yposition) = -4

```

```

        End If
    End If
End If

'down 1 and left 2
If xposition + 1 <= 7 And yposition - 2 >= 0 Then
    allcheckmoves = True
    If GameForm1.ga.board(xposition + 1, yposition - 2) < 0 And
allcheckmoves = True And GameForm1.ga.player = 1 Then
        flags(xposition + 1, yposition - 2) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(xposition + 1, yposition - 2) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false knight can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(xposition + 1, yposition - 2)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(xposition + 1, yposition - 2) = -4 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition - 2) = True 'allow
movement
        ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            flags(xposition + 1, yposition - 2) = True 'allow
movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition - 2) = temp
        GameForm1.ga.board(xposition, yposition) = -4
    ElseIf GameForm1.ga.board(xposition + 1, yposition - 2) >= 0 Then
        'If king is in check, check for available spaces
        If GameForm1.ga.board(xposition + 1, yposition - 2) = 0 Then
            'If the space is empty
            GameForm1.ga.board(xposition + 1, yposition - 2) = -4
            'Change space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition + 1, yposition - 2) = True
            End If
        End If
    End If
End If

```

```

        End If
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition + 1, yposition - 2) = 0
'Return the random number back to a blank space
    End If
    If GameForm1.ga.board(xposition + 1, yposition - 2) > 0 Then
'If the knight can take a black piece and check is no longer true then flag
space
        If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 2 = GameForm1.ga.ypositionCheck Then
            Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition + 1, yposition -
2) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(xposition + 1, yposition - 2) = -4
'change desired space to a random number
                    GameForm1.ga.recurrsionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(xposition + 1, yposition - 2) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                        GameForm1.ga.board(xposition + 1, yposition - 2) =
temp
                            GameForm1.ga.board(xposition, yposition) = -4
                        End If
                    End If
                End If
            End If
        End Sub
Public Sub reflags()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub
Public Sub fillcheckking()
    If stillonboard = True Then
        possiblemoves()
    Else
        reflags()
    End If
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then

```

```

        checktheking(i, j) = True
    Else
        checktheking(i, j) = False
    End If
Next
Next
allcheckmoves = False
End Sub
Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)
    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0
    If GameForm1.ga.canwritetoboardtracker = True Then
        Dim whatcolour As String

        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"
        Else
            whatcolour = "Black"
        End If

        Dim chessnotationletter(1) As String
        Dim chessnotationnumber(1) As Integer
        For i = 0 To 1
            Select Case yposition
                Case 0
                    chessnotationletter(i) = "A"
                Case 1
                    chessnotationletter(i) = "B"
                Case 2
                    chessnotationletter(i) = "C"
                Case 3
                    chessnotationletter(i) = "D"
                Case 4
                    chessnotationletter(i) = "E"
                Case 5
                    chessnotationletter(i) = "F"
                Case 6
                    chessnotationletter(i) = "G"
                Case 7
                    chessnotationletter(i) = "H"
            End Select

            Select Case xposition
                Case 0
                    chessnotationnumber(i) = "8"
                Case 1
                    chessnotationnumber(i) = "7"
                Case 2
                    chessnotationnumber(i) = "6"
                Case 3
                    chessnotationnumber(i) = "5"
                Case 4
                    chessnotationnumber(i) = "4"
                Case 5
                    chessnotationnumber(i) = "3"
                Case 6

```

```

        chessnotationnumber(i) = "2"
    Case 7
        chessnotationnumber(i) = "1"
    End Select
    xposition = x
    yposition = y
Next

    GameForm1.BoardTracker.AppendText(whatcolour & (" Knight ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)
    End If

End Sub
End Class

```

Pawn

```

Public Class Pawn
    Public xposition, yposition As Integer 'Coordinates for pawn
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the pawn
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
pawn, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)

    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue 'Positive P value is White, Negative P value is
Black
        reflags()
        recheckking()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'Calculation of the
possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()

```



```

End If
Return flags 'Returning all possible positions the pawn can move to
End Function
Public Sub whitemove()

    If xposition = 6 Then 'If pawn is at starting position
        '
        'one square vertically down
        '
        '
        If xposition - 1 >= 0 Then
            If GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                GameForm1.ga.board(xposition - 1, yposition) = 6 'change
desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition - 1, yposition) = True 'allow
movement
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
                GameForm1.ga.board(xposition - 1, yposition) = 0
                GameForm1.ga.board(xposition, yposition) = 6
                ElseIf GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                    GameForm1.ga.board(xposition - 1, yposition) = 6 'change
desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn moved to the desired space, would the king still be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(xposition - 1, yposition) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the original
values (before the temp changes)
                    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                    GameForm1.ga.board(xposition - 1, yposition) = 0
                    GameForm1.ga.board(xposition, yposition) = 6
                End If
            End If
        End If
    End If

```

```

        End If
        '
        'two squares vertically down
        '
        'Pawn can move two spaces upwards on first go (if the two spaces
in front of it are free)
        If GameForm1.ga.board(xposition - 2, yposition) = 0 And
GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
pawn position as a blank space
            GameForm1.ga.board(xposition - 2, yposition) = 6 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
pawn was not there, would the king be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 2, yposition) = True 'allow movement
            End If
            'Return value of the board back to original
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition - 2, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = 6
        ElseIf GameForm1.ga.board(xposition - 2, yposition) = 0 And
GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
pawn position as a blank space
            GameForm1.ga.board(xposition - 2, yposition) = 6 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if the
pawn moved to the desired space, would the king still be in check
            If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 2, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 2, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = 6
        End If
        '
        'diagonally right top downwards
        '
        If yposition + 1 <= 7 And xposition - 1 >= 0 Then

```

```

'
'Code to make sure black king cannot move into a check
position
    If GameForm1.ga.board(xposition - 1, yposition + 1) > 0 And
GameForm1.ga.player = 2 Then
        flags(xposition - 1, yposition + 1) = True
    End If
'
'
    If GameForm1.ga.board(xposition - 1, yposition + 1) < 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
        Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition - 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition - 1, yposition + 1) = 10
'change desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 1, yposition + 1) = True 'allow
movement
            ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                flags(xposition - 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition - 1, yposition + 1) = temp
            GameForm1.ga.board(xposition, yposition) = 6
            ElseIf GameForm1.ga.board(xposition - 1, yposition + 1) < 0
And GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check

                If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                    Dim temp As Integer 'declaring variable to store
value from the board
                        temp = GameForm1.ga.board(xposition - 1, yposition +
1) 'storing
                        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space

```

```

        GameForm1.ga.board(xposition - 1, yposition + 1) = 10
'change desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(xposition - 1, yposition + 1) =
temp
        GameForm1.ga.board(xposition, yposition) = 6
    End If
End If
'
'
'diagonally left top downwards
'
If yposition - 1 >= 0 And xposition - 1 >= 0 Then
    '
    'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(xposition - 1, yposition - 1) > 0 And
GameForm1.ga.player = 2 Then
            flags(xposition - 1, yposition - 1) = True
        End If
        '
        '
        If GameForm1.ga.board(xposition - 1, yposition - 1) < 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
        Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition - 1, yposition - 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition - 1, yposition - 1) = 10
'change desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 1, yposition - 1) = True 'allow
movement

```

```

        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
            flags(xposition - 1, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 6
        ElseIf GameForm1.ga.board(xposition - 1, yposition - 1) < 0
And GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
            If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition - 1, yposition -
1) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                GameForm1.ga.board(xposition - 1, yposition - 1) = 10
'change desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the pawn was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition - 1, yposition - 1) = True 'allow
movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(xposition - 1, yposition - 1) =
temp
                GameForm1.ga.board(xposition, yposition) = 6
            End If
        End If
    End If

Else
    '
    '
    'one square vertically down
    '
    '
    If xposition - 1 >= 0 Then

```

```

        If GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
        GameForm1.ga.board(xposition - 1, yposition) = 10 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition) = True 'allow
movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition) = 0
        GameForm1.ga.board(xposition, yposition) = 6
        ElseIf GameForm1.ga.board(xposition - 1, yposition) = 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0
            GameForm1.ga.board(xposition - 1, yposition) = -10
'change desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn moved to the desired space, would the king still be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition - 1, yposition) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition - 1, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = 6
        End If
    End If
    '
    '
    'diagonally right top downwards
    '
    '
    If yposition + 1 <= 7 And xposition - 1 >= 0 Then
        '
        'Code to make sure black king cannot move into a check
position

```

```

        If GameForm1.ga.board(xposition - 1, yposition + 1) > 0 And
GameForm1.ga.player = 2 Then
            flags(xposition - 1, yposition + 1) = True
        End If
    '
    '
        If GameForm1.ga.board(xposition - 1, yposition + 1) < 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
            Dim temp As Integer 'declaring variable to store value
from the board
                temp = GameForm1.ga.board(xposition - 1, yposition + 1)
            'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                GameForm1.ga.board(xposition - 1, yposition + 1) = 10
            'change desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition - 1, yposition + 1) = True 'allow
movement
                        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                            flags(xposition - 1, yposition + 1) = True 'allow
movement
                                End If
                                'Returning the value of the board back to the original
values (before the temp changes)
                                    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
                                    GameForm1.ga.board(xposition - 1, yposition + 1) = temp
                                    GameForm1.ga.board(xposition, yposition) = 6
                                ElseIf GameForm1.ga.board(xposition - 1, yposition + 1) < 0
And GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
                                    If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                                        Dim temp As Integer 'declaring variable to store
value from the board
                                            temp = GameForm1.ga.board(xposition - 1, yposition +
1) 'storing
                                                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                                                GameForm1.ga.board(xposition - 1, yposition + 1) = 10
                                            'change desired space to a random number
                                                GameForm1.ga.recurrisionstop = False
                                                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the pawn was not there, would the king be in check

```

```

        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(xposition - 1, yposition + 1) =
temp '
        GameForm1.ga.board(xposition, yposition) = 6
    End If
End If
End If
'
'
'diagonally left top downwards
'
If yposition - 1 >= 0 And xposition - 1 >= 0 Then
    '
    'Code to make sure black king cannot move into a check
position
    If GameForm1.ga.board(xposition - 1, yposition - 1) > 0 And
GameForm1.ga.player = 2 Then
        flags(xposition - 1, yposition - 1) = True
    End If
    '
    '
    If GameForm1.ga.board(xposition - 1, yposition - 1) < 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false pawn can move
        Dim temp As Integer 'declaring variable to store value
from the board
        temp = GameForm1.ga.board(xposition - 1, yposition - 1)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
        GameForm1.ga.board(xposition - 1, yposition - 1) = 10
        'change desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if
the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(xposition - 1, yposition - 1) = True 'allow
movement
        ElseIf xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
            flags(xposition - 1, yposition - 1) = True 'allow
movement

```



```

        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition - 1, yposition - 1) = temp
        GameForm1.ga.board(xposition, yposition) = 6
        ElseIf GameForm1.ga.board(xposition - 1, yposition - 1) < 0
And GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
        If xposition - 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition - 1, yposition -
1) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                GameForm1.ga.board(xposition - 1, yposition - 1) = 10
'change desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the pawn was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(xposition - 1, yposition - 1) = True 'allow
movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(xposition - 1, yposition - 1) =
temp
                GameForm1.ga.board(xposition, yposition) = 6
        End If
    End If
End If
End Sub
Public Sub blackmove()
    If xposition = 1 Then 'If pawn is at starting position
    ,
        'One square vertically upwards
    ,
        If xposition + 1 <= 7 Then
            If GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space

```

```

        GameForm1.ga.board(xposition + 1, yposition) = -6 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition) = True 'allow
movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 1, yposition) = 0
        GameForm1.ga.board(xposition, yposition) = -6
        ElseIf GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0
            GameForm1.ga.board(xposition + 1, yposition) = -6 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn moved to the desired space, would the king still be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition + 1, yposition) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = -6
        End If
    End If
    '
    'Two squares vertically upwards
    '
    '
    If GameForm1.ga.board(xposition + 2, yposition) = 0 And
GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
pawn position as a blank space
        GameForm1.ga.board(xposition + 2, yposition) = -6 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
pawn was not there, would the king be in check

```

```

        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 2, yposition) = True 'allow movement
        End If
        'Return value of the board back to original
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition + 2, yposition) = 0
        GameForm1.ga.board(xposition, yposition) = -6
        ElseIf GameForm1.ga.board(xposition + 2, yposition) = 0 And
GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0
            GameForm1.ga.board(xposition + 2, yposition) = -6 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if the
pawn moved to the desired space, would the king still be in check
            If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition + 2, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original values
(before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 2, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = -6
        End If
        '
        'diagonally right bottom upwards
        '
        If yposition + 1 <= 7 And xposition + 1 <= 7 Then
            'Code to make sure black king cannot move into a check
position
            If GameForm1.ga.board(xposition + 1, yposition + 1) < 0 And
GameForm1.ga.player = 1 Then
                flags(xposition + 1, yposition + 1) = True
            End If
            '
            '
            If GameForm1.ga.board(xposition + 1, yposition + 1) > 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
from the board
                Dim temp As Integer 'declaring variable to store value
                temp = GameForm1.ga.board(xposition + 1, yposition + 1)
            'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition + 1, yposition + 1) = -10
            'change desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check

```

```

        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition + 1) = True 'allow
movement
            ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition + 1, yposition + 1) = temp '
            GameForm1.ga.board(xposition, yposition) = -6
            ElseIf GameForm1.ga.board(xposition + 1, yposition + 1) > 0
And GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
                If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition + 1, yposition +
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                    GameForm1.ga.board(xposition + 1, yposition + 1) = -
10 'change desired space to a random number
                    GameForm1.ga.recurrsionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the pawn was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(xposition + 1, yposition + 1) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)
                    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    GameForm1.ga.board(xposition + 1, yposition + 1) =
temp '
                    GameForm1.ga.board(xposition, yposition) = -6
                End If
            End If
        End If
    '
    'diagonally left bottom upwards
    '

```

```

        If yposition - 1 >= 0 And xposition + 1 <= 7 Then
            'Code to make sure black king cannot move into a check
position
            If GameForm1.ga.board(xposition + 1, yposition - 1) < 0 And
GameForm1.ga.player = 1 Then
                flags(xposition + 1, yposition - 1) = True
            End If
            'one square diagonally left (bottom upwards)
            If GameForm1.ga.board(xposition + 1, yposition - 1) > 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
                Dim temp As Integer 'declaring variable to store value
from the board
                    temp = GameForm1.ga.board(xposition + 1, yposition - 1)
                'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                    GameForm1.ga.board(xposition + 1, yposition - 1) = -10
                'change desired space to a random number
                    GameForm1.ga.recurrsionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(xposition + 1, yposition - 1) = True 'allow
movement
                ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                    flags(xposition + 1, yposition - 1) = True 'allow
movement
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
                    GameForm1.ga.board(xposition + 1, yposition - 1) = temp
                    GameForm1.ga.board(xposition, yposition) = -6
                ElseIf GameForm1.ga.board(xposition + 1, yposition - 1) > 0
And GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
                    If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                        Dim temp As Integer 'declaring variable to store
value from the board
                            temp = GameForm1.ga.board(xposition + 1, yposition -
1) 'storing
                                GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                                GameForm1.ga.board(xposition + 1, yposition - 1) = -
10 'change desired space to a random number
                                GameForm1.ga.recurrsionstop = False

```

```

        GameForm1.ga.fillcheckforblack() 'checking to see
that if the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition - 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition + 1, yposition - 1) =
temp
        GameForm1.ga.board(xposition, yposition) = -6
    End If
End If
End If

Else
    '
    'One square vertically upwards
    '
    If xposition + 1 <= 7 Then
        If GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition + 1, yposition) = -6 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition + 1, yposition) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition + 1, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = -6
        ElseIf GameForm1.ga.board(xposition + 1, yposition) = 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can block the check from taking place
            GameForm1.ga.board(xposition, yposition) = 0
            GameForm1.ga.board(xposition + 1, yposition) = -10
        'change desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn moved to the desired space, would the king still be in check

```

```

        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition + 1, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = -6
        End If
    End If
    '
    'diagonally right bottom upwards
    '
    '
    If yposition + 1 <= 7 And xposition + 1 <= 7 Then
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(xposition + 1, yposition + 1) < 0 And
GameForm1.ga.player = 1 Then
            flags(xposition + 1, yposition + 1) = True
        End If
        If GameForm1.ga.board(xposition + 1, yposition + 1) > 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition + 1, yposition + 1)
'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
            GameForm1.ga.board(xposition + 1, yposition + 1) = -10
'change desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                flags(xposition + 1, yposition + 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition + 1, yposition + 1) = temp '

```

```

GameForm1.ga.board(xposition, yposition) = -6

ElseIf GameForm1.ga.board(xposition + 1, yposition + 1) > 0
And GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
    If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition + 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(xposition + 1, yposition +
1) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
        GameForm1.ga.board(xposition + 1, yposition + 1) = -
10 'change desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition + 1) = True 'allow
movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition + 1, yposition + 1) =
temp '
        GameForm1.ga.board(xposition, yposition) = -6
    End If
End If
End If
'
'diagonally left bottom upwards
'
If yposition - 1 >= 0 And xposition + 1 <= 7 Then
    'Code to make sure black king cannot move into a check
position
    If GameForm1.ga.board(xposition + 1, yposition - 1) < 0 And
GameForm1.ga.player = 1 Then
        flags(xposition + 1, yposition - 1) = True
    End If
    If GameForm1.ga.board(xposition + 1, yposition - 1) > 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false pawn can move
        Dim temp As Integer 'declaring variable to store value
from the board
        temp = GameForm1.ga.board(xposition + 1, yposition - 1)
        'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space

```



```

        GameForm1.ga.board(xposition + 1, yposition - 1) = -10
'change desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if
the pawn was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition + 1, yposition - 1) = True 'allow
movement
            ElseIf xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                flags(xposition + 1, yposition - 1) = True 'allow
movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(xposition + 1, yposition - 1) = temp
            GameForm1.ga.board(xposition, yposition) = -6
            ElseIf GameForm1.ga.board(xposition + 1, yposition - 1) > 0
And GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True Then 'If king is in check, check to see
if pawn can take the piece which is putting king in check
                If xposition + 1 = GameForm1.ga.xpositionCheck And
yposition - 1 = GameForm1.ga.ypositionCheck Then 'if the position the pawn
was going to move is the position the king is getting checked from, allow the
movement
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition + 1, yposition -
1) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current pawn position as a blank space
                    GameForm1.ga.board(xposition + 1, yposition - 1) = -
10 'change desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the pawn was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(xposition + 1, yposition - 1) = True 'allow
movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)
                    GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    GameForm1.ga.board(xposition + 1, yposition - 1) =
temp
                    GameForm1.ga.board(xposition, yposition) = -6
                End If

```

```

        End If
    End If
End Sub

Public Sub diagonalchecktest()
    reflags()

    'White Pawn
    If piecevalue > 0 Then
        If yposition + 1 <= 7 And xposition - 1 >= 0 Then
            flags(xposition - 1, yposition + 1) = True
        End If
        If yposition - 1 >= 0 And xposition - 1 >= 0 Then
            flags(xposition - 1, yposition - 1) = True
        End If
    End If
    'Black Pawn
    If piecevalue < 0 Then
        If allcheckmoves = True Then
            If yposition + 1 <= 7 And xposition + 1 <= 7 Then 'pawn
moving up 1 and right 1 (diagonally right)
                flags(xposition + 1, yposition + 1) = True 'therefore
move can happen
            End If
            If yposition - 1 >= 0 And xposition + 1 <= 7 Then 'pawn
moving up 1 and left 1 (diagonally left)
                flags(xposition + 1, yposition - 1) = True 'therefore
move can happen
            End If
        End If
    End If
End Sub

Public Sub reflags() 'Setting all elements false flag
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub

Public Sub recheckking()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            checktheking(i, j) = False
        Next
    Next
End Sub

Public Sub fillcheckking()
    allcheckmoves = True
    If stillonboard = True Then
        diagonalchecktest()
    Else
        reflags()
    End If
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then

```

```

        checktheking(i, j) = True
    Else
        checktheking(i, j) = False
    End If
Next
Next

End Sub

Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)
    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0
    If GameForm1.ga.canwritetoboardtracker = True Then
        Dim whatcolour As String

        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"
        Else
            whatcolour = "Black"
        End If

        Dim chessnotationletter(1) As String
        Dim chessnotationnumber(1) As Integer
        For i = 0 To 1
            Select Case yposition
                Case 0
                    chessnotationletter(i) = "A"
                Case 1
                    chessnotationletter(i) = "B"
                Case 2
                    chessnotationletter(i) = "C"
                Case 3
                    chessnotationletter(i) = "D"
                Case 4
                    chessnotationletter(i) = "E"
                Case 5
                    chessnotationletter(i) = "F"
                Case 6
                    chessnotationletter(i) = "G"
                Case 7
                    chessnotationletter(i) = "H"
            End Select

            Select Case xposition
                Case 0
                    chessnotationnumber(i) = "8"
                Case 1
                    chessnotationnumber(i) = "7"
                Case 2
                    chessnotationnumber(i) = "6"
                Case 3
                    chessnotationnumber(i) = "5"
                Case 4
                    chessnotationnumber(i) = "4"
                Case 5
                    chessnotationnumber(i) = "3"
                Case 6
                    chessnotationnumber(i) = "2"
            End Select
        Next i
    End If
End Sub

```

```

        Case 7
            chessnotationnumber(i) = "1"
        End Select
        xposition = x
        yposition = y
    Next

    GameForm1.BoardTracker.AppendText(whatcolour & (" Pawn ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)

    fillcheckking()
End If
End Sub
End Class

```

Queen

```

Public Class Queen
    Public xposition, yposition As Integer
    Public piecevalue As Integer
    Public flags(7, 7) As Boolean
    Public checktheking(7, 7) As Boolean
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean

    'positive colour is white and negative is black
    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'showing possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags
    End Function
    Public Sub whitemove()
        Dim x As Integer = xposition
        Dim y As Integer = yposition
    End Sub

```

```

allcheckmoves = True
'diagonal left top to bottom
While (x < 7 And y > 0)
    x = x + 1
    y = y - 1
    '
    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = -2 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(x, y) < 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,

```

```

GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
    flags(x, y) = True
    End If
    'Returning the value of the board back to the original
values (before the temp changes)
    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
    GameForm1.ga.board(x, y) = 0
    GameForm1.ga.board(xposition, yposition) = 2
    End If
    If GameForm1.ga.board(x, y) < 0 Then 'If the queen can take a
black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit While
End If
Else
    Exit While
End If
End While

'diagonal right bottom to top
x = xposition
y = yposition
allcheckmoves = True
While (x > 0 And y < 7)
    x = x - 1
    y = y + 1
    '
    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then

```

```

        flags(x, y) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement

        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(x, y) < 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number

                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(x, y) = 0

```

```

        GameForm1.ga.board(xposition, yposition) = 2
    End If
    If GameForm1.ga.board(x, y) < 0 Then 'If the queen can take a
black piece it can move there to get rid of check
        If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(x, y) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(x, y) = temp
            GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit While
End If
Else
    Exit While
End If
End While
'diagonal right top to bottom
x = xposition
y = yposition
allcheckmoves = True
While (x < 7 And y < 7)
    x = x + 1
    y = y + 1
    '
    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move

```



```

        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(x, y) < 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = 2
            End If
            If GameForm1.ga.board(x, y) < 0 Then 'If the queen can take a
black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing

```

```

        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit While
End If
Else
    Exit While
End If
End While
'diagonal left bottom to top
x = xposition
y = yposition
allcheckmoves = True
While (x > 0 And y > 0)
    x = x - 1
    y = y - 1
    '
    'Code to make sure black king cannot move into a check position
    If GameForm1.ga.board(x, y) > 0 And allcheckmoves = True And
GameForm1.ga.player = 2 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(x, y) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = 10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if the
queen was not there, would the king be in check

```

```

        If GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(x, y) < 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) <= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number

                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(x, y) = 0
                GameForm1.ga.board(xposition, yposition) = 2
            End If
            If GameForm1.ga.board(x, y) < 0 Then 'If the queen can take a
black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number

                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then

```

```

        flags(x, y) = True 'allow movement
    End If
    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = 2
    End If
    Exit While
End If
Else
    Exit While
End If
End While

'horizontal and vertical movement

If xposition > 0 Then
    'vertically downwards
    allcheckmoves = True
    For i As Integer = xposition - 1 To 0 Step -1
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(i, yposition) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(i, yposition) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(i, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(i, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                flags(i, yposition) = True 'allow movement
            End If

```

```

        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 2
        If GameForm1.ga.board(i, yposition) < 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(i, yposition) <= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(i, yposition) = True
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
                GameForm1.ga.board(xposition, yposition) = 2
            End If
            If GameForm1.ga.board(i, yposition) < 0 Then 'If the
queen can take a black piece it can move there to get rid of check
                If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
                    'setting current queen position as a blank space
                    GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                        flags(i, yposition) = True 'allow movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

```

```

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 2
    End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

If xposition <= 7 Then
    allcheckmoves = True
    'vertically upwards
    For i As Integer = xposition + 1 To 7
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(i, yposition) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(i, yposition) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(i, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(i, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the queen was not there, would the king be in check

            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 2

```

```

        If GameForm1.ga.board(i, yposition) < 0 Then
            Exit For
        End If
    ElseIf GameForm1.ga.board(i, yposition) <= 0 Then 'If king is
in check, check for available spaces
        If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(i, yposition) = True
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(i, yposition) = 0
            GameForm1.ga.board(xposition, yposition) = 2
        End If
        If GameForm1.ga.board(i, yposition) < 0 Then 'If the
queen can take a black piece it can move there to get rid of check
            If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
            'setting current queen position as a blank space
            GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = 2
        End If
    End If
Exit For

```

```

        End If
    Else
        Exit For
    End If
Next
End If

If yposition > 0 Then
    allcheckmoves = True
    'horizontally left
    For i As Integer = yposition - 1 To 0 Step -1
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(xposition, i) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(xposition, i) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
                flags(xposition, i) = True 'allow movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition, i) = temp
            GameForm1.ga.board(xposition, yposition) = 2
            If GameForm1.ga.board(xposition, i) < 0 Then
                Exit For
            End If
            ElseIf GameForm1.ga.board(xposition, i) <= 0 Then 'If king is
in check, check for available spaces
                If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty

```



```

        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition, i) = True
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
        GameForm1.ga.board(xposition, yposition) = 2
    End If
    If GameForm1.ga.board(xposition, i) < 0 Then 'If the
queen can take a black piece it can move there to get rid of check
        If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0
'setting current queen position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition, i) = temp
            GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

```

```

    If yposition <= 7 Then
        allcheckmoves = True
        'horizontally right
        For i As Integer = yposition + 1 To 7
            '
            'Code to make sure black king cannot move into a check
position
            If GameForm1.ga.board(xposition, i) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
                flags(xposition, i) = True
                allcheckmoves = False
            End If
            '
            '
            If GameForm1.ga.board(xposition, i) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
                Dim temp As Integer 'declaring variable to store value
from the board
                temp = GameForm1.ga.board(xposition, i) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see that if
the queen was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition, i) = True 'allow movement
                ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
                    flags(xposition, i) = True 'allow movement
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
                GameForm1.ga.board(xposition, i) = temp
                GameForm1.ga.board(xposition, yposition) = 2
                If GameForm1.ga.board(xposition, i) < 0 Then
                    Exit For
                End If
                ElseIf GameForm1.ga.board(xposition, i) <= 0 Then 'If king is
in check, check for available spaces
                    If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
                        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                        GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
                        GameForm1.ga.recurrisionstop = False
                        GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there

```

```

        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(xposition, i) = True
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
            GameForm1.ga.board(xposition, yposition) = 2
        End If
        If GameForm1.ga.board(xposition, i) < 0 Then 'If the
queen can take a black piece it can move there to get rid of check
            If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current queen position as a blank space
                GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
            GameForm1.ga.board(xposition, i) = temp
            GameForm1.ga.board(xposition, yposition) = 2
        End If
    Exit For
End If
Else
    Exit For
End If
Next
End If
End Sub
Public Sub blackmove()
    Dim x As Integer = xposition
    Dim y As Integer = yposition

    allcheckmoves = True
    'diagonal left top to bottom

```

```

While (x < 7 And y > 0)
    x = x + 1
    y = y - 1
    '
    'Code to make sure white king cannot move into a check position
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    '
    '
    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(x, y) > 0 Then 'If desired space is a
black piece, cannot move past it (cannot travel through a piece)
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,

```

```

GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
    flags(x, y) = True
End If
'Returning the value of the board back to the original
values (before the temp changes)
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
GameForm1.ga.board(x, y) = 0 'Return the random number
back to a blank space
GameForm1.ga.board(xposition, yposition) = -2
End If
If GameForm1.ga.board(x, y) > 0 Then 'If the queen can take a
black piece it can move there to get rid of check
    If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
    End If
Exit While
End If
Else
Exit While
End If
End While

'diagonal right bottom to top
x = xposition
y = yposition
allcheckmoves = True
While (x > 0 And y < 7)
    x = x - 1
    y = y + 1
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True

```

```

        allcheckmoves = False
    End If
    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement

        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(x, y) > 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number

                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(x, y) = 0 'Return the random number
back to a blank space
                GameForm1.ga.board(xposition, yposition) = -2
            End If

```

```

        If GameForm1.ga.board(x, y) > 0 Then 'If the queen can take a
black piece it can move there to get rid of check
            If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(x, y) = True 'allow movement
                End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    GameForm1.ga.board(x, y) = temp
                    GameForm1.ga.board(xposition, yposition) = -2
                End If
            Exit While
        End If
    Else
        Exit While
    End If
End While
'diagonal right top to bottom
x = xposition
y = yposition
allcheckmoves = True
While (x < 7 And y < 7)
    x = x + 1
    y = y + 1
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
            temp = GameForm1.ga.board(x, y) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
            GameForm1.ga.board(x, y) = -10 'change desired space to a
random number

```

```

        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(x, y) > 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(x, y) = 0 'Return the random number
back to a blank space
                GameForm1.ga.board(xposition, yposition) = -2
            End If
            If GameForm1.ga.board(x, y) > 0 Then 'If the queen can take a
black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
                    GameForm1.ga.recurrsionstop = False

```



```

        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
    End If
Exit While
End If
Else
    Exit While
End If
End While
'diagonal left bottom to top
x = xposition
y = yposition
allcheckmoves = True
While (x > 0 And y > 0)
    x = x - 1
    y = y - 1
    If GameForm1.ga.board(x, y) < 0 And allcheckmoves = True And
GameForm1.ga.player = 1 Then
        flags(x, y) = True
        allcheckmoves = False
    End If
    If GameForm1.ga.board(x, y) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value from
the board
        temp = GameForm1.ga.board(x, y) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting current
queen position as a blank space
        GameForm1.ga.board(x, y) = -10 'change desired space to a
random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if the
queen was not there, would the king be in check
        If GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(x, y) = True 'allow movement
        ElseIf x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then
            flags(x, y) = True 'allow movement
        End If
        'Returning the value of the board back to the original values
(before the temp changes)

```

```

        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(x, y) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(x, y) > 0 Then
            Exit While
        End If
        ElseIf GameForm1.ga.board(x, y) >= 0 Then 'If king is in check,
check for available spaces
            If GameForm1.ga.board(x, y) = 0 Then 'If the space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(x, y) = -10 'Change space to a random
number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king would
still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(x, y) = True
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(x, y) = 0 'Return the random number
back to a blank space
                GameForm1.ga.board(xposition, yposition) = -2
            End If
            If GameForm1.ga.board(x, y) > 0 Then 'If the queen can take a
black piece it can move there to get rid of check
                If x = GameForm1.ga.xpositionCheck And y =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(x, y) 'storing
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(x, y) = -10 'change desired space
to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                        flags(x, y) = True 'allow movement
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                    GameForm1.ga.board(x, y) = temp

```

```

        GameForm1.ga.board(xposition, yposition) = -2
    End If
    Exit While
End If
Else
    Exit While
End If
End While

'horizontal and vertical movement
allcheckmoves = True
If xposition > 0 Then
    'vertically downwards
    For i As Integer = xposition - 1 To 0 Step -1
        If GameForm1.ga.board(i, yposition) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(i, yposition) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(i, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(i, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(i, yposition) = -10 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the queen was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = -2
            If GameForm1.ga.board(i, yposition) > 0 Then
                Exit For
            End If
            ElseIf GameForm1.ga.board(i, yposition) >= 0 Then 'If king is
in check, check for available spaces
                If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space

```

```

        GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(i, yposition) = True
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
        GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
        GameForm1.ga.board(xposition, yposition) = -2
        End If
        If GameForm1.ga.board(i, yposition) > 0 Then 'If the
queen can take a black piece it can move there to get rid of check
            If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current queen position as a blank space
                GameForm1.ga.board(i, yposition) = -10 'change
desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(i, yposition) = True 'allow movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                GameForm1.ga.board(i, yposition) = temp
                GameForm1.ga.board(xposition, yposition) = -2
            End If
            Exit For
        End If
    Else
        Exit For
    End If
Next
End If
allcheckmoves = True
If xposition <= 7 Then

```

```

        'vertically upwards
    For i As Integer = xposition + 1 To 7
        If GameForm1.ga.board(i, yposition) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(i, yposition) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(i, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(i, yposition) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(i, yposition) = -2 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the queen was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = -2
            If GameForm1.ga.board(i, yposition) > 0 Then
                Exit For
            End If
            ElseIf GameForm1.ga.board(i, yposition) >= 0 Then 'If king is
in check, check for available spaces
                If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                    GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                    GameForm1.ga.board(i, yposition) = -2 'Change space
to a random number
                    GameForm1.ga.recurrisionstop = False
                    GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
                    If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                        flags(i, yposition) = True
                    End If
                    'Returning the value of the board back to the
original values (before the temp changes)

```

```

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
    GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
    GameForm1.ga.board(xposition, yposition) = -2
End If
    If GameForm1.ga.board(i, yposition) > 0 Then 'If the
queen can take a black piece it can move there to get rid of check
        If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current queen position as a blank space
                GameForm1.ga.board(i, yposition) = -10 'change
desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
                GameForm1.ga.board(i, yposition) = temp
                GameForm1.ga.board(xposition, yposition) = -2
            End If
        Exit For
    End If
Else
    Exit For
End If
Next
End If

allcheckmoves = True
If yposition > 0 Then
    'horizontally left
    For i As Integer = yposition - 1 To 0 Step -1
        If GameForm1.ga.board(xposition, i) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(xposition, i) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move

```

```

        Dim temp As Integer 'declaring variable to store value
from the board
        temp = GameForm1.ga.board(xposition, i) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
        GameForm1.ga.board(xposition, i) = -10 'change desired
space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if
the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(xposition, i) > 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(xposition, i) >= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition, i) = True
                End If
                'Returning the value of the board back to the
original values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                    GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
                    GameForm1.ga.board(xposition, yposition) = -2
                End If
                If GameForm1.ga.board(xposition, i) > 0 Then 'If the
queen can take a black piece it can move there to get rid of check

```

```

        If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current queen position as a blank space
        GameForm1.ga.board(xposition, i) = -10 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -2
    End If
    Exit For
End If
Else
    Exit For
End If
Next
End If
allcheckmoves = True
If yposition <= 7 Then
    'horizontally right
    For i As Integer = yposition + 1 To 7
        If GameForm1.ga.board(xposition, i) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(xposition, i) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
            GameForm1.ga.board(xposition, i) = -10 'change desired
space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the queen was not there, would the king be in check

```



```

        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -2
        If GameForm1.ga.board(xposition, i) > 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(xposition, i) >= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current queen position as a blank space
                GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition, i) = True
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
                GameForm1.ga.board(xposition, yposition) = -2
            End If
            If GameForm1.ga.board(xposition, i) > 0 Then 'If the
queen can take a black piece it can move there to get rid of check
                If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If queen can move to the black piece that
is checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
                    'setting current queen position as a blank space
                    GameForm1.ga.board(xposition, i) = -10 'change
desired space to a random number
                    GameForm1.ga.recurrisionstop = False

```

```

        GameForm1.ga.fillcheckforblack() 'checking to see
that if the queen was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(xposition, i) = temp
            GameForm1.ga.board(xposition, yposition) = -2
        End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

End Sub
Public Sub reflags()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub
Public Sub fillcheckking()
    If stillonboard = True Then
        possiblemoves()
    Else
        reflags()
    End If
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then
                checktheking(i, j) = True
            Else
                checktheking(i, j) = False
            End If
        Next
    Next
    allcheckmoves = False
End Sub
Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)
    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0

    If GameForm1.ga.canwritetoboardtracker = True Then
        Dim whatcolour As String

        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"

```

```

Else
    whatcolour = "Black"
End If

Dim chessnotationletter(1) As String
Dim chessnotationnumber(1) As Integer
For i = 0 To 1
    Select Case yposition
        Case 0
            chessnotationletter(i) = "A"
        Case 1
            chessnotationletter(i) = "B"
        Case 2
            chessnotationletter(i) = "C"
        Case 3
            chessnotationletter(i) = "D"
        Case 4
            chessnotationletter(i) = "E"
        Case 5
            chessnotationletter(i) = "F"
        Case 6
            chessnotationletter(i) = "G"
        Case 7
            chessnotationletter(i) = "H"
    End Select

    Select Case xposition
        Case 0
            chessnotationnumber(i) = "8"
        Case 1
            chessnotationnumber(i) = "7"
        Case 2
            chessnotationnumber(i) = "6"
        Case 3
            chessnotationnumber(i) = "5"
        Case 4
            chessnotationnumber(i) = "4"
        Case 5
            chessnotationnumber(i) = "3"
        Case 6
            chessnotationnumber(i) = "2"
        Case 7
            chessnotationnumber(i) = "1"
    End Select
    xposition = x
    yposition = y
Next

    GameForm1.BoardTracker.AppendText(whatcolour & (" Queen ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)
End If

End Sub
End Class

```

Rook

```

Public Class Rook
    Public xposition, yposition As Integer 'Coordinates for rook
    Public piecevalue As Integer 'negative means black, positive means white
    Public flags(7, 7) As Boolean '2D array - for the movement of the rook
    Public checktheking(7, 7) As Boolean '2D array - for the positions it can
move to check the king
    Public AllowedToCastle As Boolean = True
    Public stillonboard As Boolean = True
    Public allcheckmoves As Boolean 'used in calculations for movement of
rook, when the opposite colour is playing (used for making sure the king
cannot move to a square which isn't directly in check)

    'positive colour is white and negative is black
    Public Sub New(ByVal x, ByVal y, ByVal pvalue)
        xposition = x
        yposition = y
        piecevalue = pvalue
        reflags()
    End Sub

    Public Function returnx() As Integer
        Return xposition
    End Function
    Public Function returny() As Integer
        Return yposition
    End Function
    Public Function returnvalue() As Integer
        Return piecevalue
    End Function
    Public Function possiblemoves() As Boolean(,) 'showing possible moves
        reflags()
        If piecevalue > 0 And stillonboard = True Then
            whitemove()
        End If
        If piecevalue < 0 And stillonboard = True Then
            blackmove()
        End If
        Return flags
    End Function
    Public Sub whitemove()
        If stillonboard = True Then
            GameForm1.ga.CastlingW()
        End If

        If xposition > 0 Then
            allcheckmoves = True
            'vertically downwards
            For i As Integer = xposition - 1 To 0 Step -1
                'Code to make sure black king cannot move into a check
position
                If GameForm1.ga.board(i, yposition) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
                    flags(i, yposition) = True
                    allcheckmoves = False

```

```

End If
'
'
If GameForm1.ga.board(i, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
    Dim temp As Integer 'declaring variable to store value
from the board
    temp = GameForm1.ga.board(i, yposition) 'storing
    GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
    GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number
    GameForm1.ga.recurrisionstop = False
    GameForm1.ga.fillcheckforwhite() 'checking to see that if
the rook was not there, would the king be in check
    If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
        flags(i, yposition) = True 'allow movement
    ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
        flags(i, yposition) = True 'allow movement
    End If
    'Returning the value of the board back to the original
values (before the temp changes)
    GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
    GameForm1.ga.board(i, yposition) = temp
    GameForm1.ga.board(xposition, yposition) = 5
    If GameForm1.ga.board(i, yposition) < 0 Then
        Exit For
    End If
    ElseIf GameForm1.ga.board(i, yposition) <= 0 Then 'If king is
in check, check for available spaces
        If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
            GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(i, yposition) = True
            End If

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
        End If
        If GameForm1.ga.board(i, yposition) < 0 Then 'If the rook
can take a black piece it can move there to get rid of check

```

```

        If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
        Dim temp As Integer 'declaring variable to store
value from the board
        temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
        GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the rook was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(i, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 5
    End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

If xposition <= 7 Then
    allcheckmoves = True
    'vertically upwards
    For i As Integer = xposition + 1 To 7
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(i, yposition) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(i, yposition) = True
            allcheckmoves = False
        End If
    Next
    '
    '
    If GameForm1.ga.board(i, yposition) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value
from the board
        temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space

```

```

        GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number

        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see that if
the rook was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(i, yposition) = True 'allow movement
        ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
            flags(i, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 5
        If GameForm1.ga.board(i, yposition) < 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(i, yposition) <= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(i, yposition) = True
                End If

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
            End If
            If GameForm1.ga.board(i, yposition) < 0 Then 'If the rook
can take a black piece it can move there to get rid of check
                If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
                    'setting current rook position as a blank space
                    GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number

```

```

        GameForm1.ga.recurrsionstop = False
        GameForm1.ga.fillcheckforwhite() 'checking to see
that if the rook was not there, would the king be in check
        If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
            flags(i, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = 5
    End If
Exit For
End If
Else
    Exit For
End If
Next
End If

If yposition > 0 Then
    allcheckmoves = True
    'horizontally left
    For i As Integer = yposition - 1 To 0 Step -1
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(xposition, i) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(xposition, i) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the rook was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement

```



```

        ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = 5
        If GameForm1.ga.board(xposition, i) < 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(xposition, i) <= 0 Then 'If king is
in check, check for available spaces
        If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
            GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition, i) = True
            End If

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
        End If
        If GameForm1.ga.board(xposition, i) < 0 Then 'If the rook
can take a black piece it can move there to get rid of check
            If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
            'setting current rook position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see
that if the rook was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

```

```

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
    GameForm1.ga.board(xposition, i) = temp
    GameForm1.ga.board(xposition, yposition) = 5
    End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

If yposition <= 7 Then
    allcheckmoves = True
    'horizontally right
    For i As Integer = yposition + 1 To 7
        '
        'Code to make sure black king cannot move into a check
position
        If GameForm1.ga.board(xposition, i) > 0 And allcheckmoves =
True And GameForm1.ga.player = 2 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        '
        '
        If GameForm1.ga.board(xposition, i) <= 0 And
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforwhite() 'checking to see that if
the rook was not there, would the king be in check
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            ElseIf xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then
                flags(xposition, i) = True 'allow movement
            End If
            'Returning the value of the board back to the original
values (before the temp changes)
            GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False
            GameForm1.ga.board(xposition, i) = temp
            GameForm1.ga.board(xposition, yposition) = 5
            If GameForm1.ga.board(xposition, i) < 0 Then

```

```

        Exit For
    End If
    ElseIf GameForm1.ga.board(xposition, i) <= 0 Then 'If king is
in check, check for available spaces
        If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
            GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
            GameForm1.ga.recurrsionstop = False
            GameForm1.ga.fillcheckforwhite() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition, i) = True
            End If

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
        End If
        If GameForm1.ga.board(xposition, i) < 0 Then 'If the rook
can take a black piece it can move there to get rid of check
            If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
                GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforwhite() 'checking to see
that if the rook was not there, would the king be in check
                If
GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = False Then
                    flags(xposition, i) = True 'allow movement
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforwhite(GameForm1.whiteking.xposition,
GameForm1.whiteking.yposition) = True
                GameForm1.ga.board(xposition, i) = temp
                GameForm1.ga.board(xposition, yposition) = 5
            End If
        End If
    Exit For
End If
Else
    Exit For
End If
Next

```

```

End If
End Sub
Public Sub blackmove()
    If stillonboard = True Then
        GameForm1.ga.CastlingB()
    End If
    allcheckmoves = True
    If xposition > 0 Then
        'vertically downwards
        For i As Integer = xposition - 1 To 0 Step -1
            If GameForm1.ga.board(i, yposition) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
                flags(i, yposition) = True
                allcheckmoves = False
            End If
            If GameForm1.ga.board(i, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
                Dim temp As Integer 'declaring variable to store value
from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
                GameForm1.ga.board(i, yposition) = 10 'change desired
space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see that if
the rook was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(i, yposition) = True 'allow movement
                ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                    flags(i, yposition) = True 'allow movement
                End If
                'Returning the value of the board back to the original
values (before the temp changes)
                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
                GameForm1.ga.board(i, yposition) = temp
                GameForm1.ga.board(xposition, yposition) = -5
                If GameForm1.ga.board(i, yposition) > 0 Then
                    Exit For
                End If
                ElseIf GameForm1.ga.board(i, yposition) >= 0 Then 'If king is
in check, check for available spaces
                    If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                        GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
                        GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
                        GameForm1.ga.recurrisionstop = False
                        GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there

```

```

        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
            flags(i, yposition) = True
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space
            GameForm1.ga.board(xposition, yposition) = -5
        End If
        If GameForm1.ga.board(i, yposition) > 0 Then 'If the rook
can take a black piece it can move there to get rid of check
            If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
                GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
                GameForm1.ga.recurrsionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see
that if the rook was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = -5
        End If
    Exit For
End If
Else
    Exit For
End If
Next
End If
allcheckmoves = True
If xposition <= 7 Then
    'vertically upwards
    For i As Integer = xposition + 1 To 7
        If GameForm1.ga.board(i, yposition) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(i, yposition) = True

```

```

        allcheckmoves = False
    End If
    If GameForm1.ga.board(i, yposition) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
        Dim temp As Integer 'declaring variable to store value
from the board
        temp = GameForm1.ga.board(i, yposition) 'storing
        GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
        GameForm1.ga.board(i, yposition) = -10 'change desired
space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if
the rook was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(i, yposition) = True 'allow movement
        ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
            flags(i, yposition) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(i, yposition) = temp
        GameForm1.ga.board(xposition, yposition) = -5
        If GameForm1.ga.board(i, yposition) > 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(i, yposition) >= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(i, yposition) = 0 Then 'If the
space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
                GameForm1.ga.board(i, yposition) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(i, yposition) = True
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

                GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(i, yposition) = 0 'Return the
random number back to a blank space

```

```

        GameForm1.ga.board(xposition, yposition) = -5
    End If
    If GameForm1.ga.board(i, yposition) > 0 Then 'If the rook
can take a black piece it can move there to get rid of check
        If i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
            Dim temp As Integer 'declaring variable to store
value from the board
            temp = GameForm1.ga.board(i, yposition) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
            GameForm1.ga.board(i, yposition) = 10 'change
desired space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see
that if the rook was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(i, yposition) = True 'allow movement
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
            GameForm1.ga.board(i, yposition) = temp
            GameForm1.ga.board(xposition, yposition) = -5
        End If
    Exit For
End If
Else
    Exit For
End If
Next
End If

allcheckmoves = True
If yposition > 0 Then
    'horizontally left
    For i As Integer = yposition - 1 To 0 Step -1
        If GameForm1.ga.board(xposition, i) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(xposition, i) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space

```

```

        GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see that if
the rook was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -5
        If GameForm1.ga.board(xposition, i) > 0 Then
            Exit For
        End If
        ElseIf GameForm1.ga.board(xposition, i) >= 0 Then 'If king is
in check, check for available spaces
            If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
                GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
                GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                    flags(xposition, i) = True
                End If
                'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
                GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
                GameForm1.ga.board(xposition, yposition) = -5
            End If
            If GameForm1.ga.board(xposition, i) > 0 Then 'If the rook
can take a black piece it can move there to get rid of check
                If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                    Dim temp As Integer 'declaring variable to store
value from the board
                    temp = GameForm1.ga.board(xposition, i) 'storing

```



```

        GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
        GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
        GameForm1.ga.recurrisionstop = False
        GameForm1.ga.fillcheckforblack() 'checking to see
that if the rook was not there, would the king be in check
        If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
            flags(xposition, i) = True 'allow movement
        End If
        'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -5
    End If
Exit For
End If
Else
    Exit For
End If
Next
End If
allcheckmoves = True
If yposition <= 7 Then
    'horizontally right
    For i As Integer = yposition + 1 To 7
        If GameForm1.ga.board(xposition, i) < 0 And allcheckmoves =
True And GameForm1.ga.player = 1 Then
            flags(xposition, i) = True
            allcheckmoves = False
        End If
        If GameForm1.ga.board(xposition, i) >= 0 And
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If check is false bishop can do
any move
            Dim temp As Integer 'declaring variable to store value
from the board
            temp = GameForm1.ga.board(xposition, i) 'storing
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
            GameForm1.ga.board(xposition, i) = 10 'change desired
space to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'checking to see that if
the rook was not there, would the king be in check
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                flags(xposition, i) = True 'allow movement
            ElseIf i = GameForm1.ga.xpositionCheck And yposition =
GameForm1.ga.ypositionCheck Then
                flags(xposition, i) = True 'allow movement

```

```

        End If
        'Returning the value of the board back to the original
values (before the temp changes)
        GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False
        GameForm1.ga.board(xposition, i) = temp
        GameForm1.ga.board(xposition, yposition) = -5
        If GameForm1.ga.board(xposition, i) > 0 Then
            Exit For
        End If
    ElseIf GameForm1.ga.board(xposition, i) >= 0 Then 'If king is
in check, check for available spaces
        If GameForm1.ga.board(xposition, i) = 0 Then 'If the
space is empty
            GameForm1.ga.board(xposition, yposition) = 0 'setting
current rook position as a blank space
            GameForm1.ga.board(xposition, i) = -10 'Change space
to a random number
            GameForm1.ga.recurrisionstop = False
            GameForm1.ga.fillcheckforblack() 'See if the king
would still be in check if that piece was moved there
            If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then 'If the piece prevents the king
from being in check then the piece can move there
                flags(xposition, i) = True
            End If
            'Returning the value of the board back to the
original values (before the temp changes)

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True 'Return the king being in check
            GameForm1.ga.board(xposition, i) = 0 'Return the
random number back to a blank space
            GameForm1.ga.board(xposition, yposition) = -5
        End If
        If GameForm1.ga.board(xposition, i) > 0 Then 'If the rook
can take a black piece it can move there to get rid of check
            If xposition = GameForm1.ga.xpositionCheck And i =
GameForm1.ga.ypositionCheck Then 'If rook can move to the black piece that is
checking the king then allow it to move there
                Dim temp As Integer 'declaring variable to store
value from the board
                temp = GameForm1.ga.board(xposition, i) 'storing
GameForm1.ga.board(xposition, yposition) = 0
'setting current rook position as a blank space
                GameForm1.ga.board(xposition, i) = 10 'change
desired space to a random number
                GameForm1.ga.recurrisionstop = False
                GameForm1.ga.fillcheckforblack() 'checking to see
that if the rook was not there, would the king be in check
                If
GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = False Then
                    flags(xposition, i) = True 'allow movement
                End If

```

'Returning the value of the board back to the original values (before the temp changes)

```

GameForm1.ga.checkforblack(GameForm1.blackking.xposition,
GameForm1.blackking.yposition) = True
    GameForm1.ga.board(xposition, i) = temp
    GameForm1.ga.board(xposition, yposition) = -5
End If
Exit For
End If
Else
Exit For
End If
Next
End If
End Sub
Public Sub reflags()
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            flags(i, j) = False
        Next
    Next
End Sub

Public Sub fillcheckking()
    If stillonboard = True Then
        possiblemoves()
    Else
        reflags()
    End If
    For i As Integer = 0 To 7
        For j As Integer = 0 To 7
            If flags(i, j) = True Then
                checktheking(i, j) = True
            Else
                checktheking(i, j) = False
            End If
        Next
    Next
    allcheckmoves = False
End Sub

Public Sub changeposition(ByVal x, ByVal y, ByVal prex, ByVal prey)
    GameForm1.ga.board(x, y) = GameForm1.ga.board(prex, prey)
    GameForm1.ga.board(prex, prey) = 0
    If GameForm1.ga.canwritetoboardtracker = True Then
        Dim whatcolour As String

        If GameForm1.ga.movecounter Mod 2 = 1 Then
            whatcolour = "White"
        Else
            whatcolour = "Black"
        End If

        Dim chessnotationletter(1) As String
        Dim chessnotationnumber(1) As Integer
        For i = 0 To 1

```

```

Select Case yposition
    Case 0
        chessnotationletter(i) = "A"
    Case 1
        chessnotationletter(i) = "B"
    Case 2
        chessnotationletter(i) = "C"
    Case 3
        chessnotationletter(i) = "D"
    Case 4
        chessnotationletter(i) = "E"
    Case 5
        chessnotationletter(i) = "F"
    Case 6
        chessnotationletter(i) = "G"
    Case 7
        chessnotationletter(i) = "H"
End Select

Select Case xposition
    Case 0
        chessnotationnumber(i) = "8"
    Case 1
        chessnotationnumber(i) = "7"
    Case 2
        chessnotationnumber(i) = "6"
    Case 3
        chessnotationnumber(i) = "5"
    Case 4
        chessnotationnumber(i) = "4"
    Case 5
        chessnotationnumber(i) = "3"
    Case 6
        chessnotationnumber(i) = "2"
    Case 7
        chessnotationnumber(i) = "1"
End Select
xposition = x
yposition = y
Next

    GameForm1.BoardTracker.AppendText(whatcolour & (" Rook ") &
(chessnotationletter(0) & chessnotationnumber(0)) & (" → ") &
(chessnotationletter(1) & chessnotationnumber(1)) & Environment.NewLine)

End If

End Sub
End Class

```

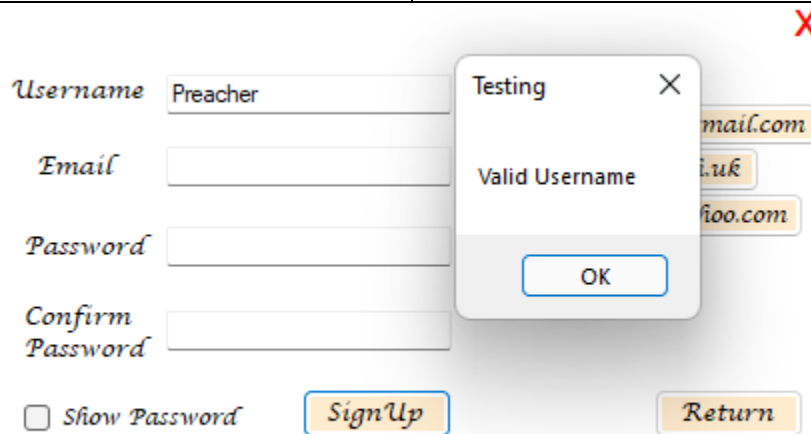
Signing-up systems testing

Username

Note - for testing purposed. After the validation of the username has been completed in the code, I added a messagebox that will display whether or not the username is valid or invalid.

Description: testing a valid username

Test Data	Type of Test Data	Expected Outcome
Preacher	Normal	Valid Username



Username: Preacher

Email: [empty]

Password: [empty]

Confirm Password: [empty]

☐ Show Password

[SignUp](#) [Return](#)

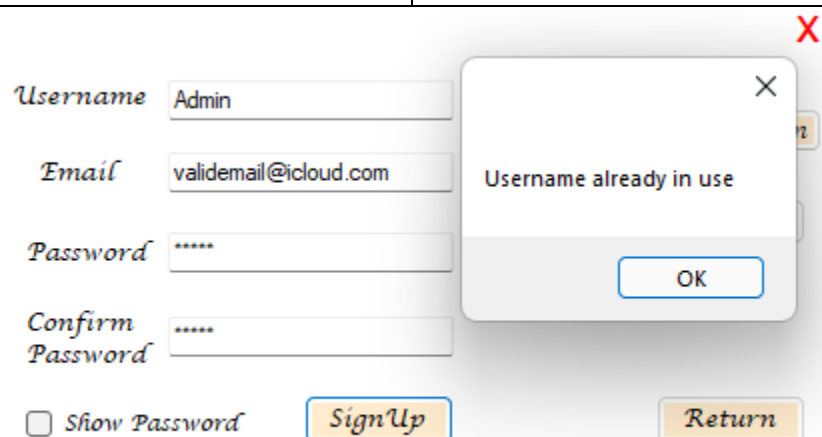
Testing

Valid Username

OK

Description: testing a username that is already taken

Test Data	Type of Test Data	Expected Outcome
Admin	Invalid	Username already taken



Username: Admin

Email: validemail@icloud.com

Password: ****

Confirm Password: ****

☐ Show Password

[SignUp](#) [Return](#)

Username already in use

OK

Description: testing a username that is unavailable as the program uses it

Test Data	Type of Test Data	Expected Outcome
Guest/AI	Invalid	Username unavailable

A screenshot of a user registration form. The form has fields for Username, Email, Password, and Confirm Password. There is a checkbox for 'Show Password' and two buttons: 'SignUp' and 'Return'. A modal dialog box is open over the form, displaying the message 'Username is not available to use' and an 'OK' button. The Username field contains the text 'AI'.

A screenshot of a user registration form, identical to the one above. The Username field now contains the text 'Guest'. The modal dialog box still displays the message 'Username is not available to use' and an 'OK' button.

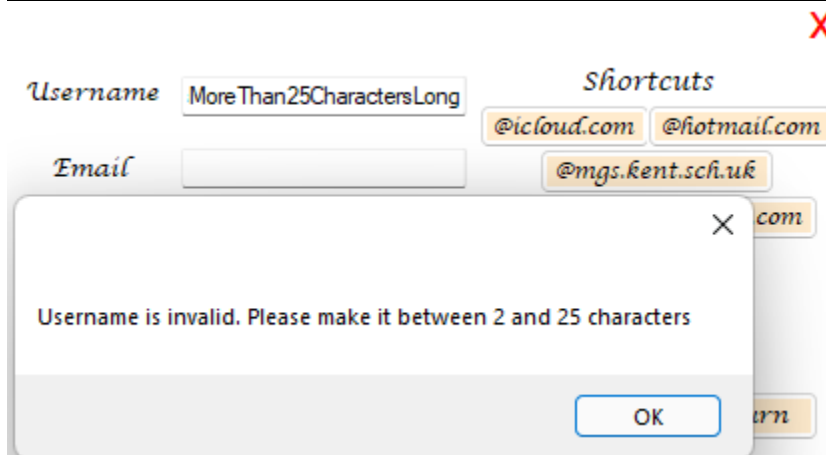
Description: testing a username that is too short

Test Data	Type of Test Data	Expected Outcome
A	Invalid	Invalid Username

A screenshot of a user registration form. The Username field contains the text 'A'. A modal dialog box is open, displaying the message 'Username is invalid. Please make it between 2 and 25 characters' and an 'OK' button. In the background, there is a 'Shortcuts' section with several email address suggestions: '@icloud.com', '@hotmail.com', '@mqs.kent.sch.uk', and another partially visible one ending in '.com'.

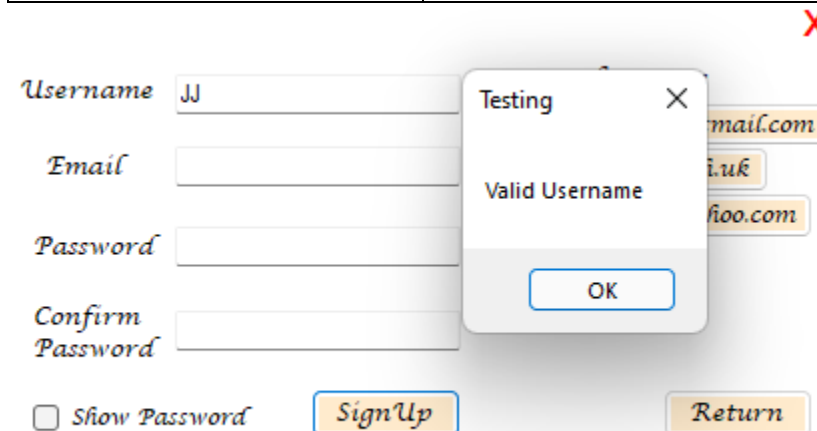
Description: testing a username that is too long

Test Data	Type of Test Data	Expected Outcome
UsernameWhichIsMoreThan25CharactersLong	Invalid	Invalid Username



Description: testing a username that is a boundary value

Test Data	Type of Test Data	Expected Outcome
JJ	Borderline	Valid Username



Password

Note - for testing purposed. After the validation of the password has been completed in the code, I added a messagebox that will display whether or not the password is valid or invalid.

Description: testing a valid password

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	JUJ24d	Normal	Valid Password

Username: Preacher

Email:

Password: JUJ24d

Confirm Password: JUJ24d

☒ Show Password

[SignUp](#) [Return](#)

Testing

Valid Password

OK

Description: testing passwords that don't match

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	1234567	Invalid	Invalid Password (Don't match)

Username: Preacher

Email:

Password: JUJ24d

Confirm Password: 1234567

☒ Show Password

[SignUp](#) [Return](#)

Testing

Passwords do not match up

OK

Description: testing a password that is too short

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
1234	1234	Invalid	Invalid Password (Too short)

Shortcuts

Username

Email

Password

Confirm Password

☒ Show Password

×

Password is too short. Please make it between 5 and 30 characters

Description: testing a password that is a boundary value

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
12345	12345	Borderline	Valid Password

Shortcuts

Username

Email

Password

Confirm Password

☒ Show Password

×

Testing

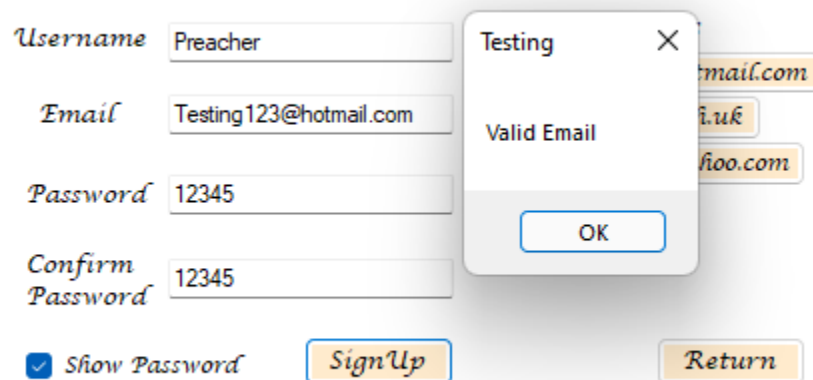
Valid Password

Email

Note - for testing purposed. After the validation of the email has been completed in the code, I added a messagebox that will display whether or not the email is valid or invalid.

Description: testing a valid email

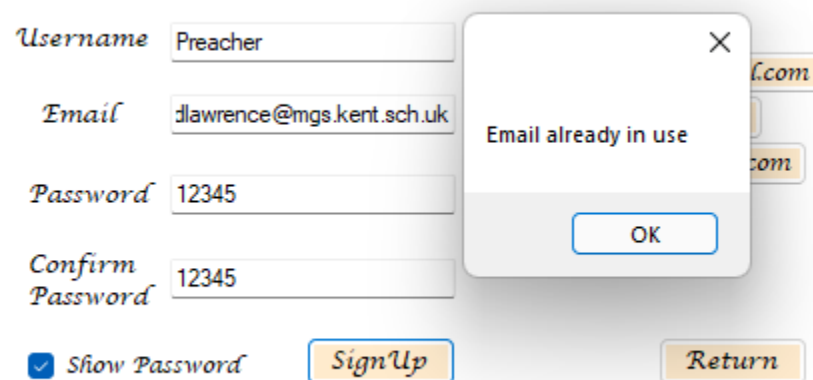
Test Data	Type of Test Data	Expected Outcome
Testing123@hotmail.com	Normal	Valid Email



The screenshot shows a registration form with the following fields: Username (Preacher), Email (Testing123@hotmail.com), Password (12345), and Confirm Password (12345). There is a checkbox for 'Show Password' which is checked. Below the fields are 'SignUp' and 'Return' buttons. A modal message box titled 'Testing' is displayed in the center, showing 'Valid Email' and an 'OK' button. A red 'X' is visible in the background of the form.

Description: testing an email which is already in use

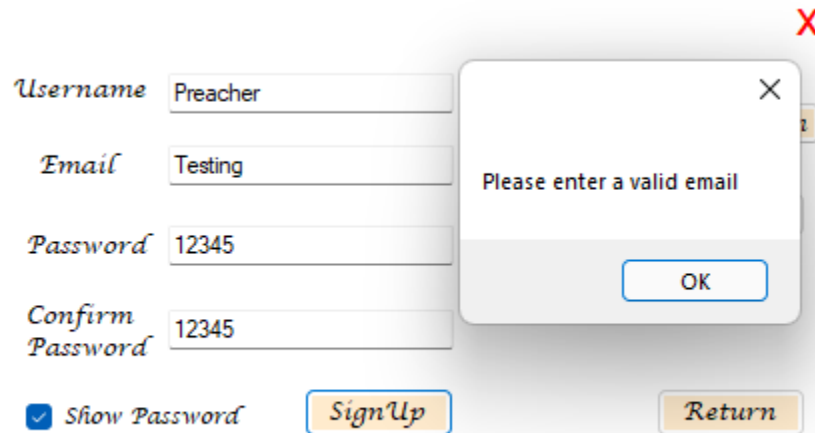
Test Data	Type of Test Data	Expected Outcome
15dlawrence@mgs.kent.sch.uk	Invalid	Invalid Email (Already Taken)



The screenshot shows the same registration form as above, but with the Email field set to '15dlawrence@mgs.kent.sch.uk'. A modal message box titled 'Testing' is displayed in the center, showing 'Email already in use' and an 'OK' button. A red 'X' is visible in the background of the form.

Description: testing an email which isn't in the correct format

Test Data	Type of Test Data	Expected Outcome
Testing	Invalid	Invalid Email (Not in the correct form)

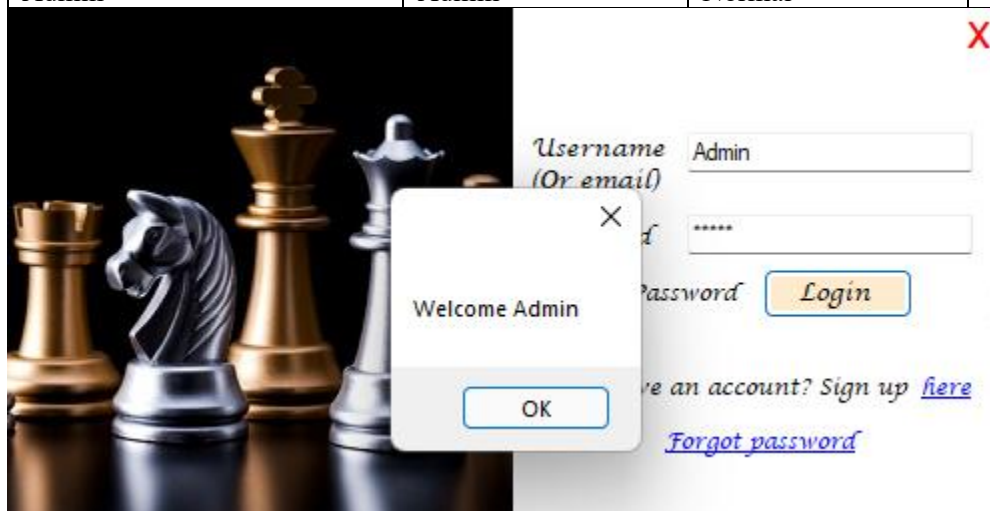


Logging in system testing

Login

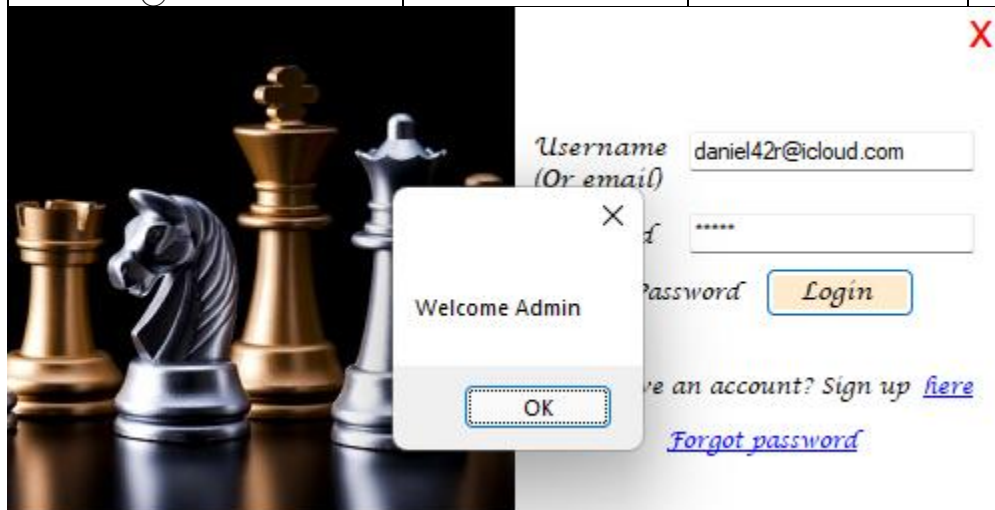
Description: testing for a valid login using a username and password

Test Data 1 - Username/Email	Test Data 2 - Password	Type of Test Data	Expected Outcome
Admin	Admin	Normal	Valid Login



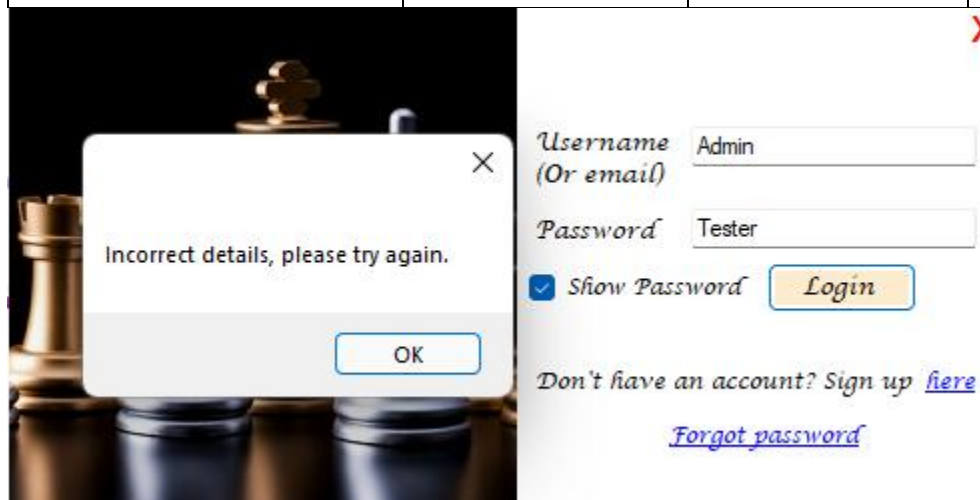
Description: testing for a valid login using an email and password

Test Data 1 - Username/Email	Test Data 2 - Password	Type of Test Data	Expected Outcome
Daniel42r@icloud.com	Admin	Normal	Valid Login



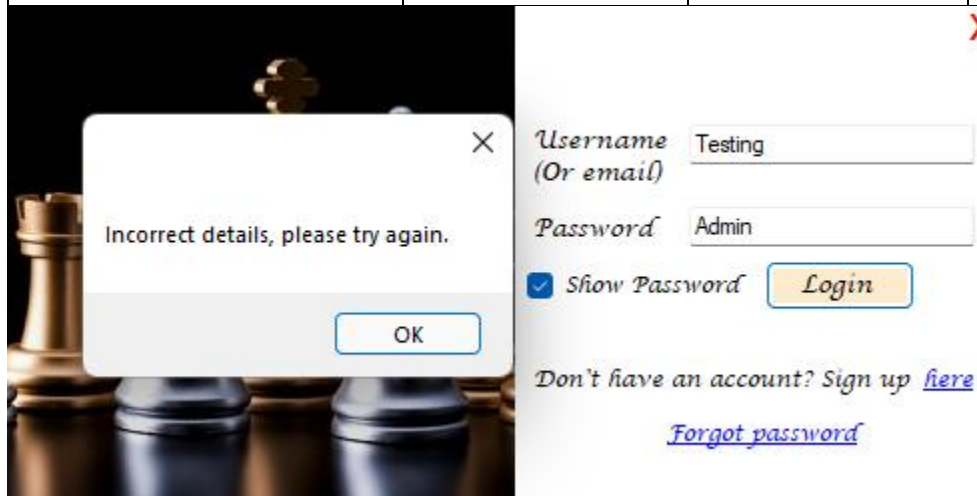
Description: testing for an invalid login using the correct username but the incorrect password

Test Data 1 - Username/Email	Test Data 2 - Password	Type of Test Data	Expected Outcome
Admin	Tester	Invalid	Invalid Login (Incorrect Password)



Description: testing for an invalid account where the username isn't found

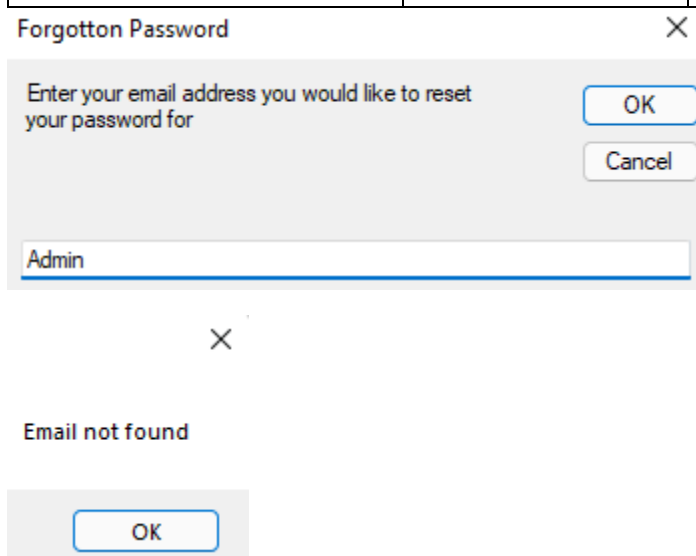
Test Data 1 - Username/Email	Test Data 2 - Password	Type of Test Data	Expected Outcome
Testing	Admin	Invalid	Invalid Login (Account not found)



Forgotten Password

Description: testing for an invalid email, where the user's input isn't in an email format

Test Data 1 - Email	Test Data 2 - Code	Type of Test Data	Expected Outcome
Admin	N/A	Normal	Invalid Email



Description: testing for a valid email and a valid code input

Test Data 1 - Email	Test Data 2 - Code	Type of Test Data	Expected Outcome
15dlawrence@mgs.kent.sch.uk	4983	Normal	Valid Login

Forgotten Password X

Enter your email address you would like to reset your password for

OK

Cancel

15dlawrence@mgs.kent.sch.uk

Forgotten Password X

Enter your 4-digit validation code that has been emailed to you. 3 attempts remaining.

OK

Cancel

4983



Change Password

New Password

Confirm New Password

☐ Show Password

Change Password

Description: testing for a valid email but the incorrect code for all 3 attempts

Test Data 1 - Email	Test Data 2 - Code	Type of Test Data	Expected Outcome
15dlawrence@mgs.kent.sch.uk	5000	Invalid	Invalid Code

Forgotten Password X

Enter your 4-digit validation code that has been emailed to you. 1 attempts remaining.

OK

Cancel

5000

Ran out of attempts

OK

Description: testing for an invalid email, where the user's input isn't a recognised email

Test Data 1 - Email	Test Data 2 - Code	Type of Test Data	Expected Outcome
testing@mgs.kent.sch.uk	N/A	Invalid	Invalid Login (Email not found)

Forgotton Password ×

Enter your email address you would like to reset your password for

×

Email not found

Game form system testing

Loading account

Description: loading the correct user's account into the game form

Username	Profile Picture	Board Colour	Piece Type	Data Type	Expected Outcome
Admin	Skin3	GreenBoard	Set2	Normal	Game successfully loaded

Username	ProfilePicture	BoardColou	PieceType
Admin	Skin3	GreenBoard	Set2

Screenshots from the database

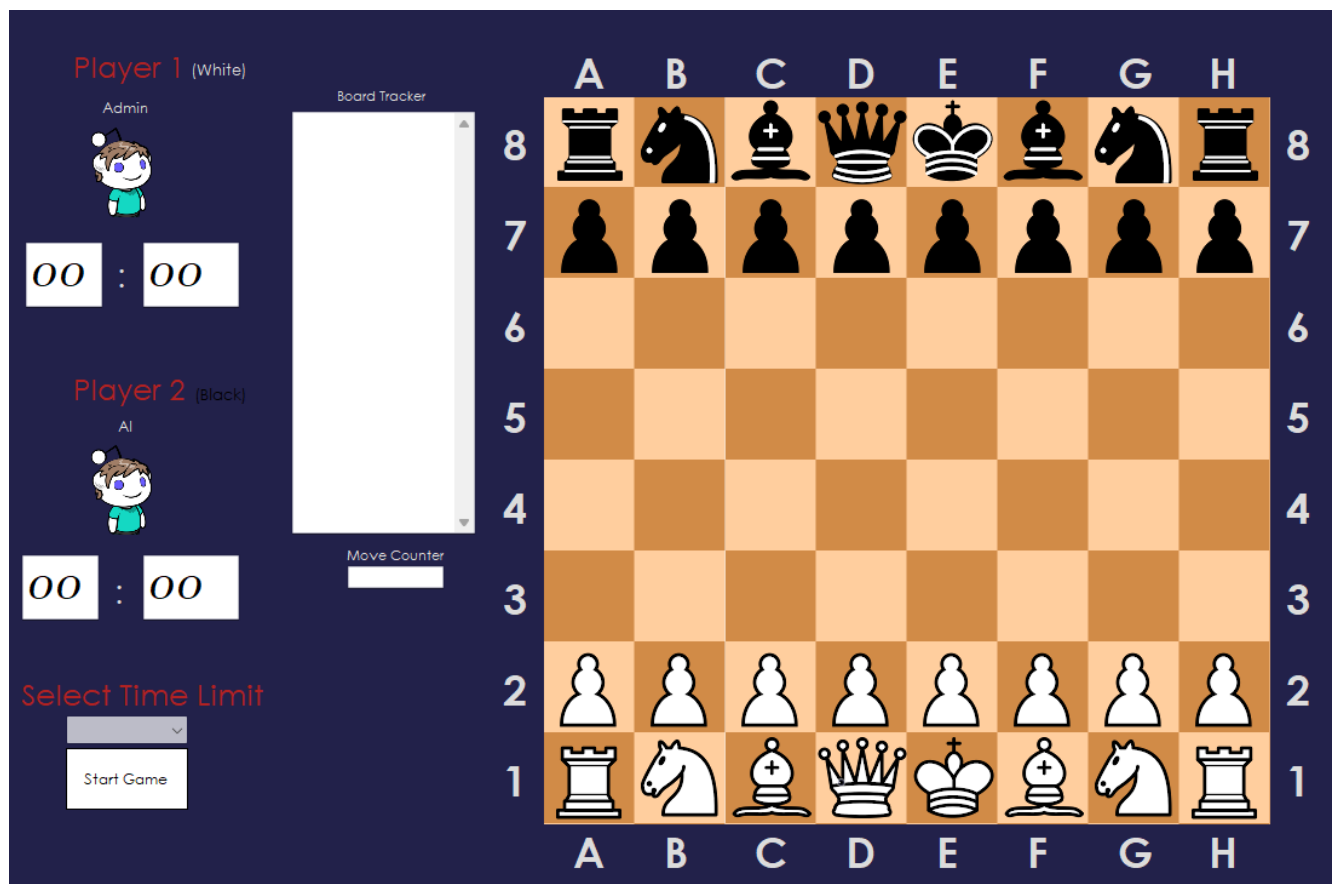


Description: loading the correct user's account into the game form for a boundary value

Username	Profile Picture	Board Colour	Piece Type	Data Type	Expected Outcome
Admin	Default	YellowBoard	Set1	Borderline	Game successfully loaded

Username ▾	ProfilePicture ▾	BoardColou ▾	PieceType ▾
Admin	Default	YellowBoard	Set1

Screenshots from the database



Username	Profile Picture	Board Colour	Piece Type	Data Type	Expected Outcome
Admin	Picture	YellowBoard	Set1	Invalid	Profile picture not found
Admin	Skin2	BrownBoard	Set1	Invalid	Board colour not found
Admin	Locked6	YellowBoard	Set9	Invalid	Piece set not found

Note - the invalid data types should never actually occur whilst the program is running, as they are never mentioned being inserted into the database.

No test available, unless manually entering values into the database.

Loading pieces onto the board

Description: testing the game form loads the chess board with the correct pieces

Key:

6 ← white pawn, 5 ← white rook, 4 ← white knight, 3 ← white bishop, 2 ← white queen, 1 ← white king

-6 ← black pawn, -5 ← black rook, -4 ← black knight, -3 ← black bishop, -2 ← black queen, -1 ← black king

	A	B	C	D	E	F	G	H	Data Type	Expected Outcome
8	-5	-4	-3	-2	-1	-3	-4	-5		
7	-6	-6	-6	-6	-6	-6	-6	-6		
6										
5									Normal	Pieces loaded successfully
4										
3										
2	6	6	6	6	6	6	6	6		
1	5	4	3	2	1	3	4	5		



Saving replays

Description: testing saving game replay into slot 2

Replay number	Data Type	Expected Outcome
2	Normal	Replay Saved

Replay Replace

Enter a value between 1 and 5 (Replace Replay)

OK

Cancel

2

PreMove2

Move2

B2G7

B3G6

Moves performed in the game

Description: testing saving the game replay into slot 9

Replay number	Data Type	Expected Outcome
9	Invalid	Invalid replay number

Replay Replace

Enter a value between 1 and 5 (Replace Replay)

OK

Cancel

9

Replay Replace

Enter a value between 1 and 5 (Replace Replay)

OK

Cancel

1

Program rejected user’s input, replacing it with a valid one, promoting the user again

Description: testing saving the game replay into slot 1

Replay number	Data Type	Expected Outcome
1	Borderline	Replay Saved

Replay Replace
×

Enter a value between 1 and 5 (Replace Replay)

OK

Cancel

1

PreMove1 ▾	Move1 ▾
B2E7	B4E5

Moves performed in the game

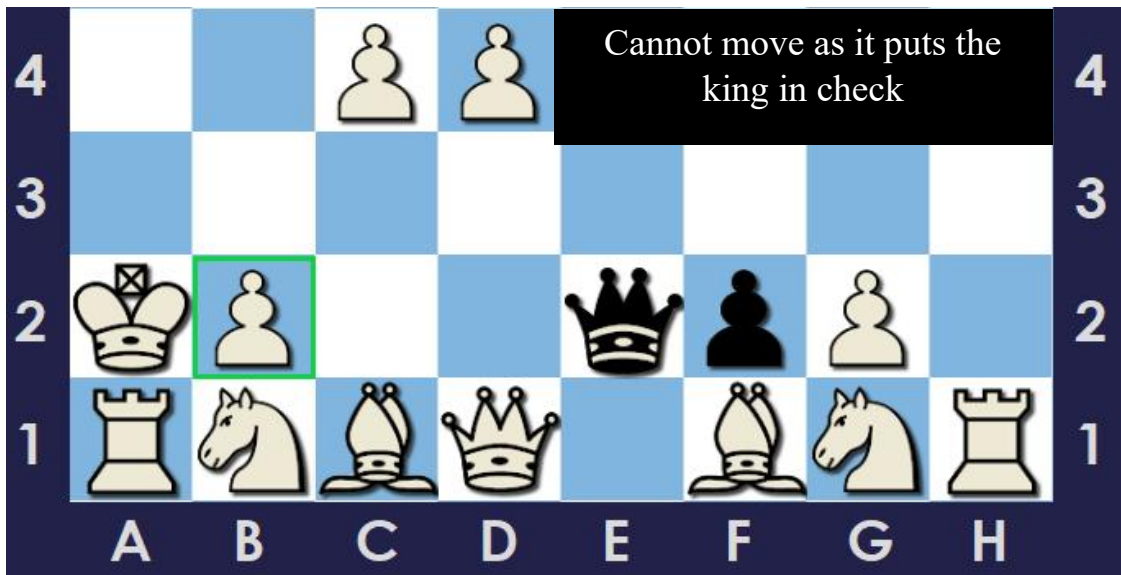
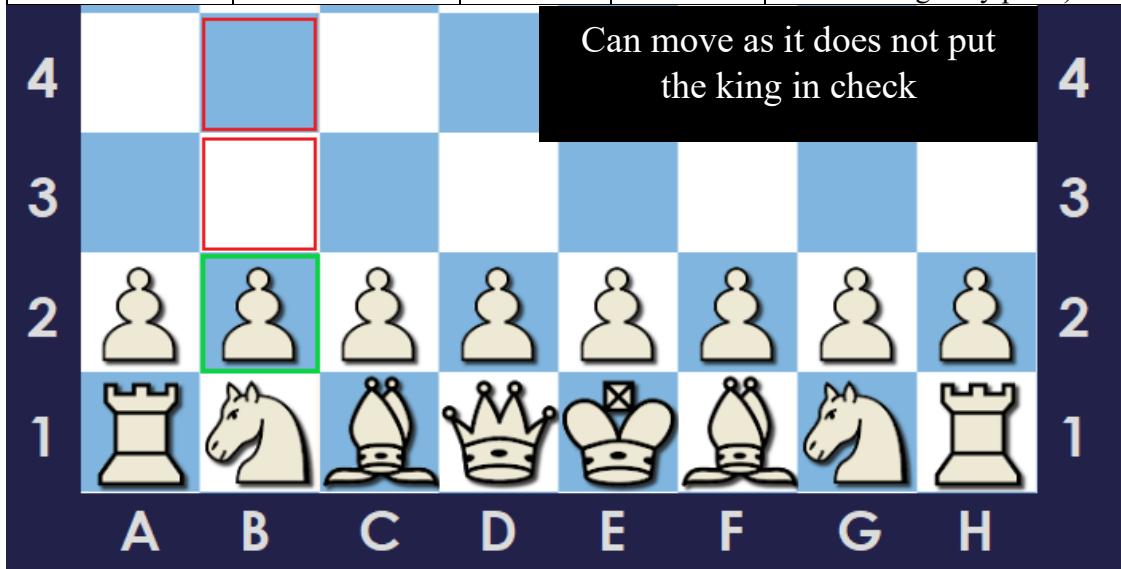
Game logic system testing

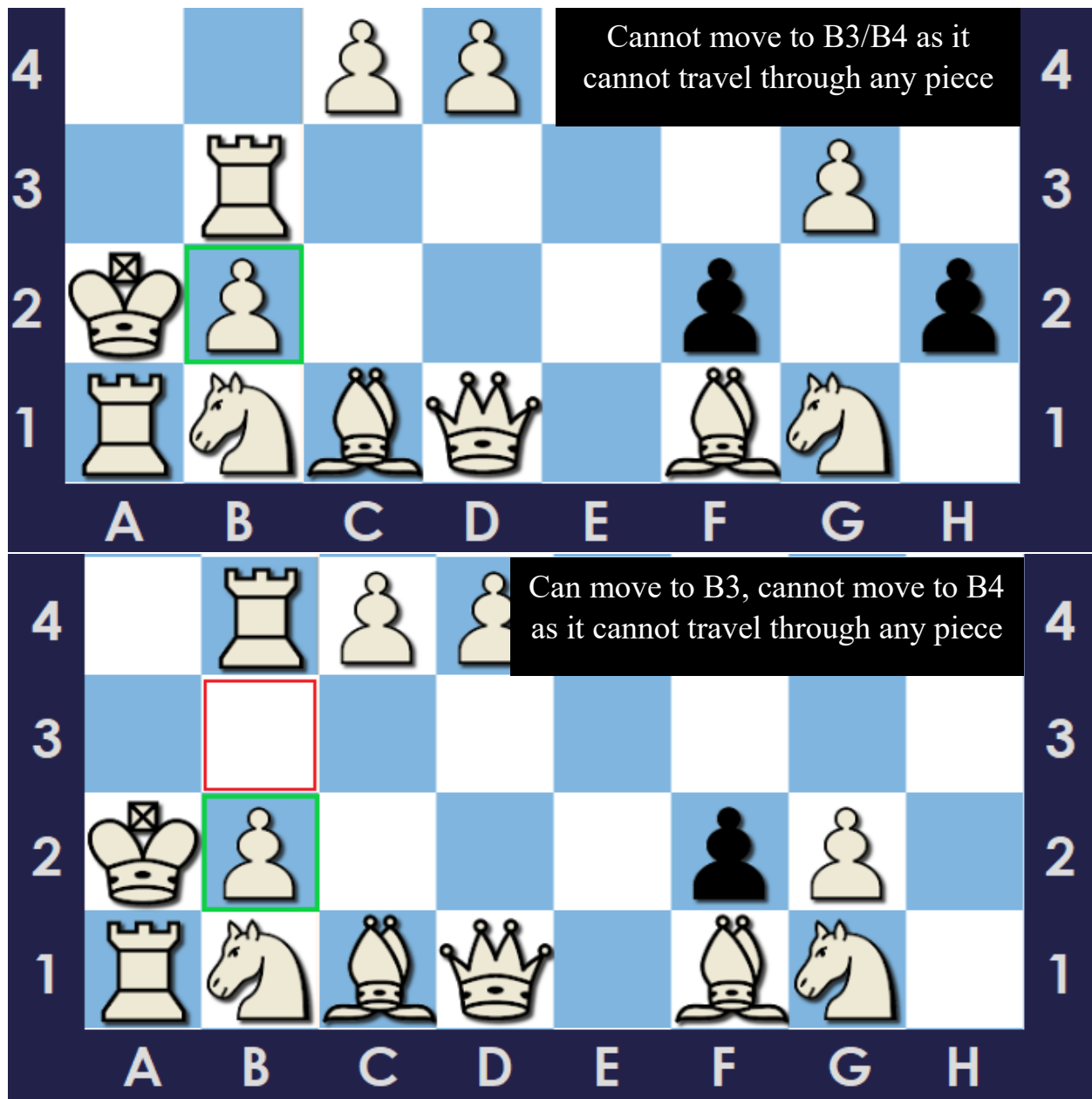
Pawn

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the pawn can move one square/two squares forward from the start position

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
B2	B3/B4	No	Normal/ Borderline	Can Move (Only if B3/B4 doesn't result in a white check & doesn't travel through any piece)

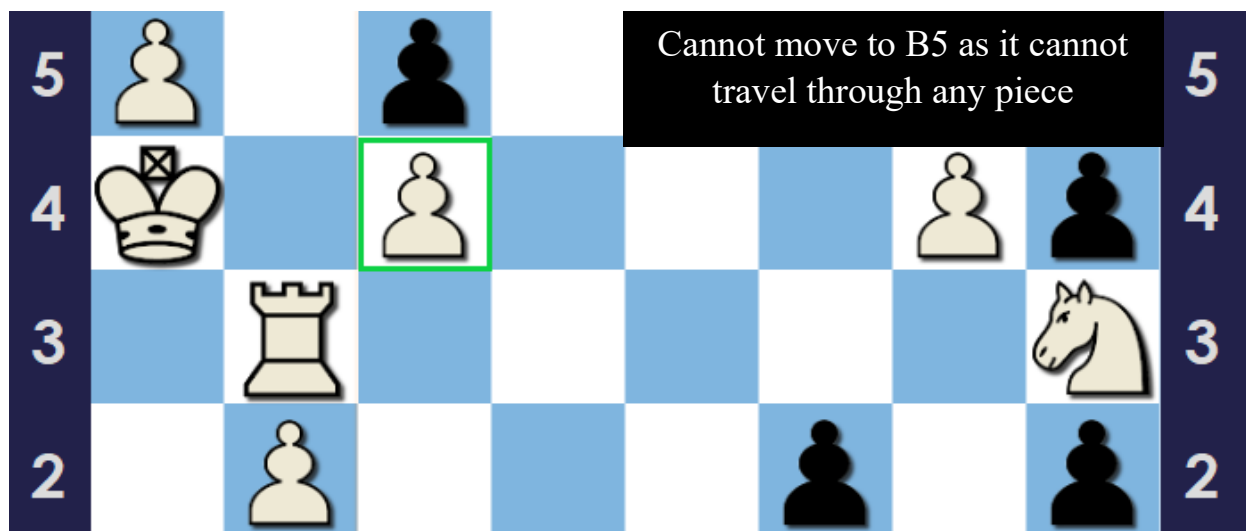




Description: testing if the pawn can move one square forward anywhere on the board (when it is not in its starting position)

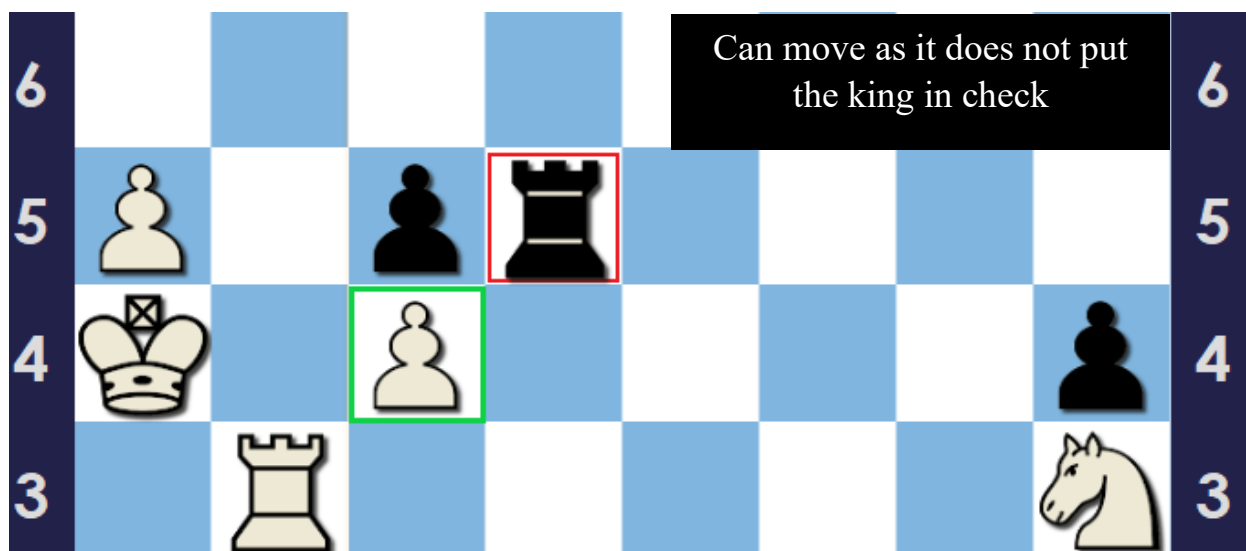
Current Position	Desired Position	In Check?	Data Type	Expected Outcome
B4	B5	No	Normal	Can Move (Only if B5 doesn't result in a white check & doesn't travel through any piece)

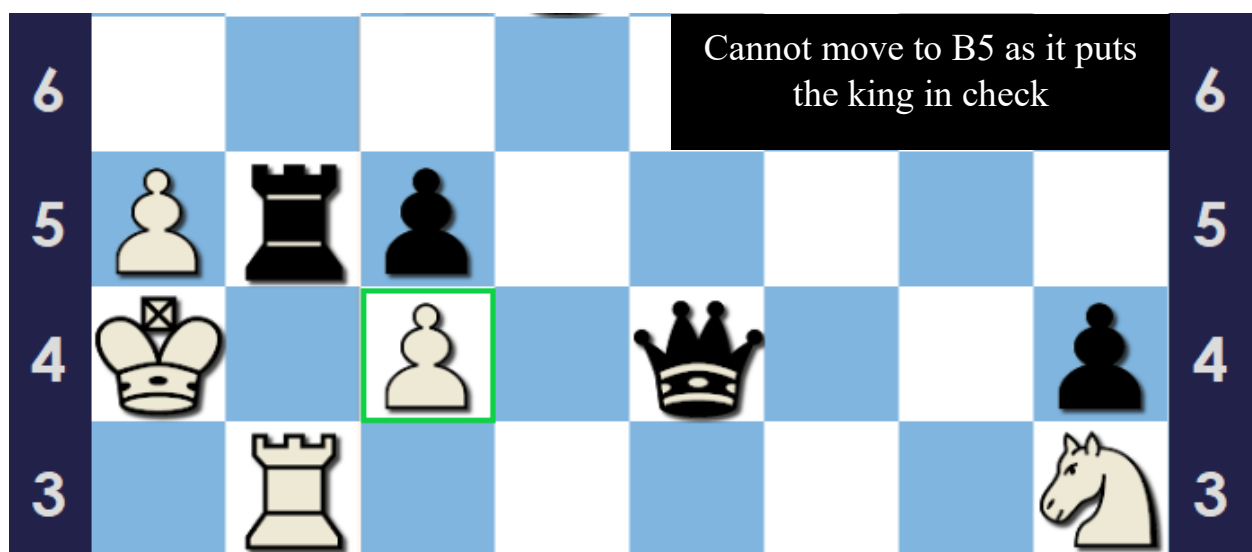
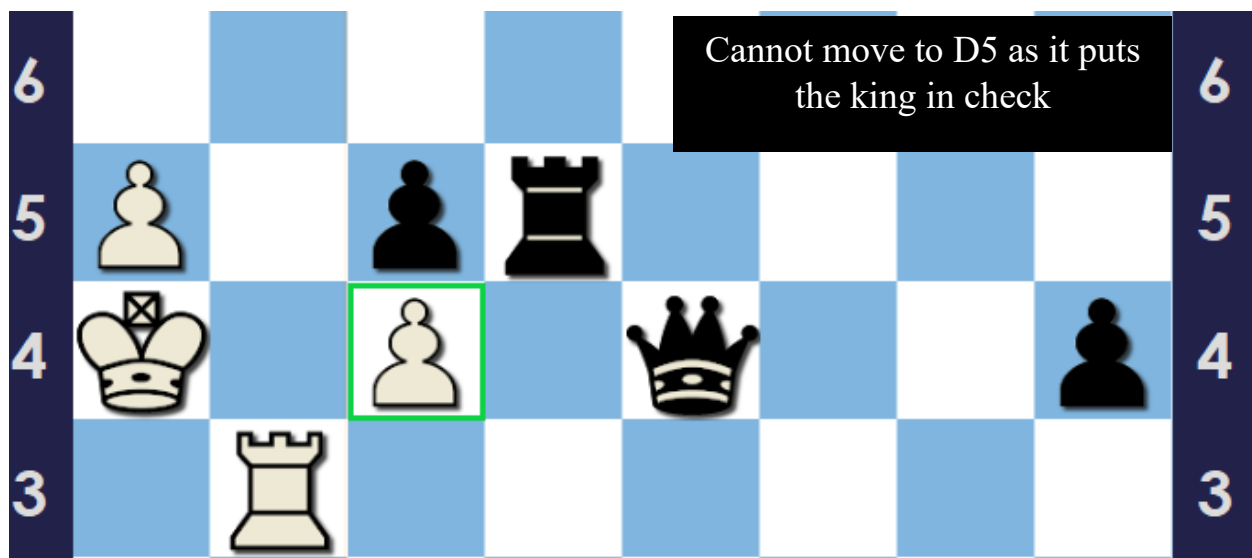
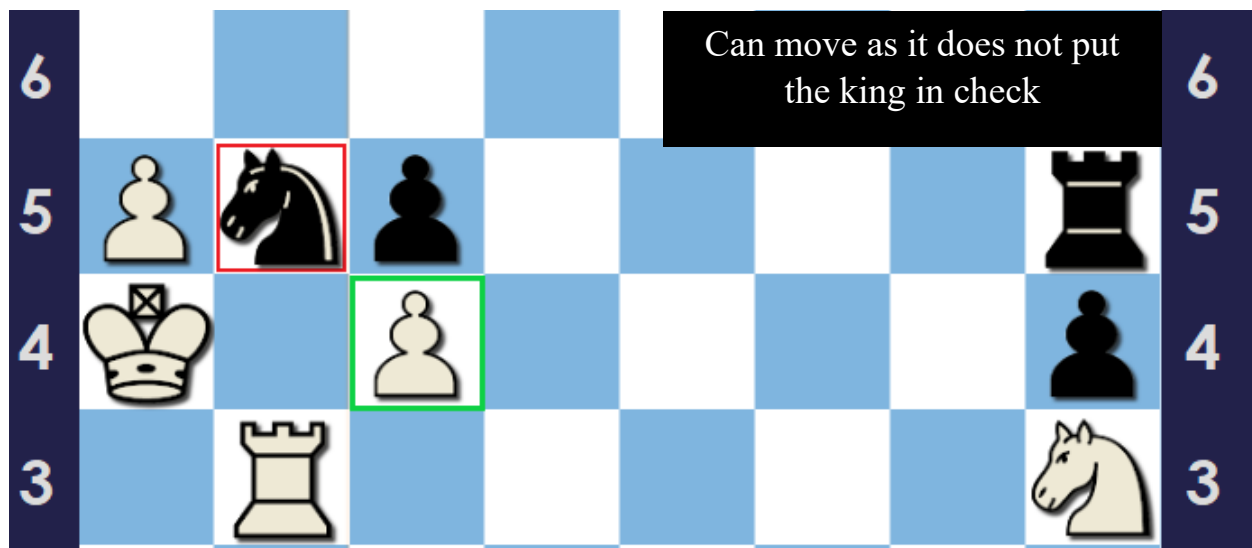




Description: testing if the pawn can move diagonally when there is a black piece in that square

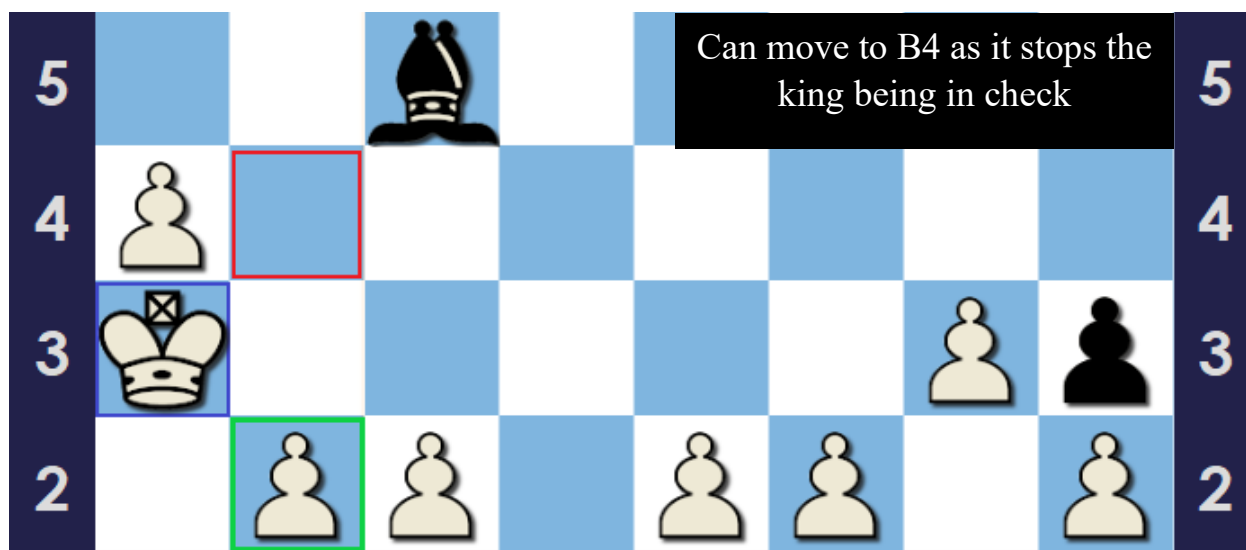
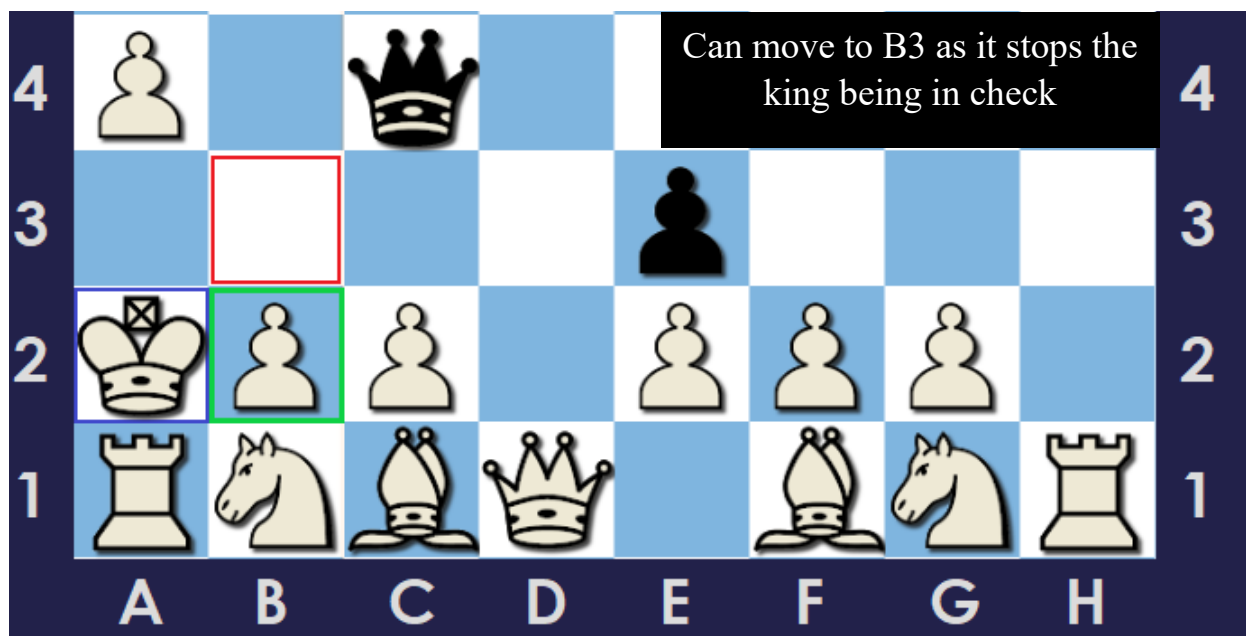
Current Position	Desired Position	In Check?	Data Type	Expected Outcome
C4	B5/D5	No	Normal	Can Move (Only if B5/D5 doesn't result in a white check & doesn't travel through any piece)

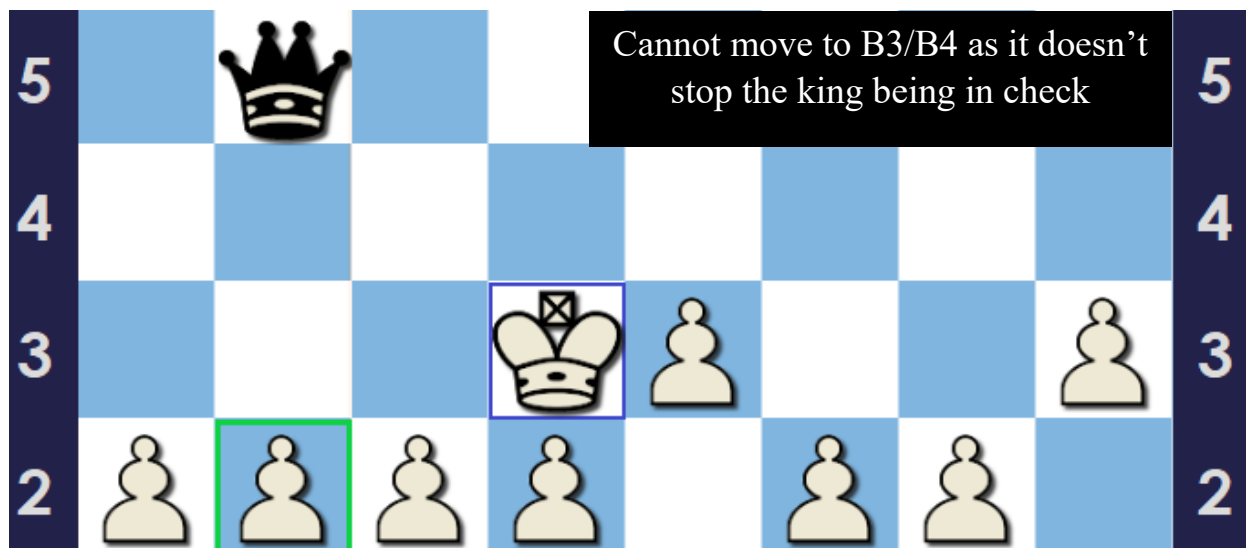




Description: testing if the pawn can move one square forward from the start position (*in check*)

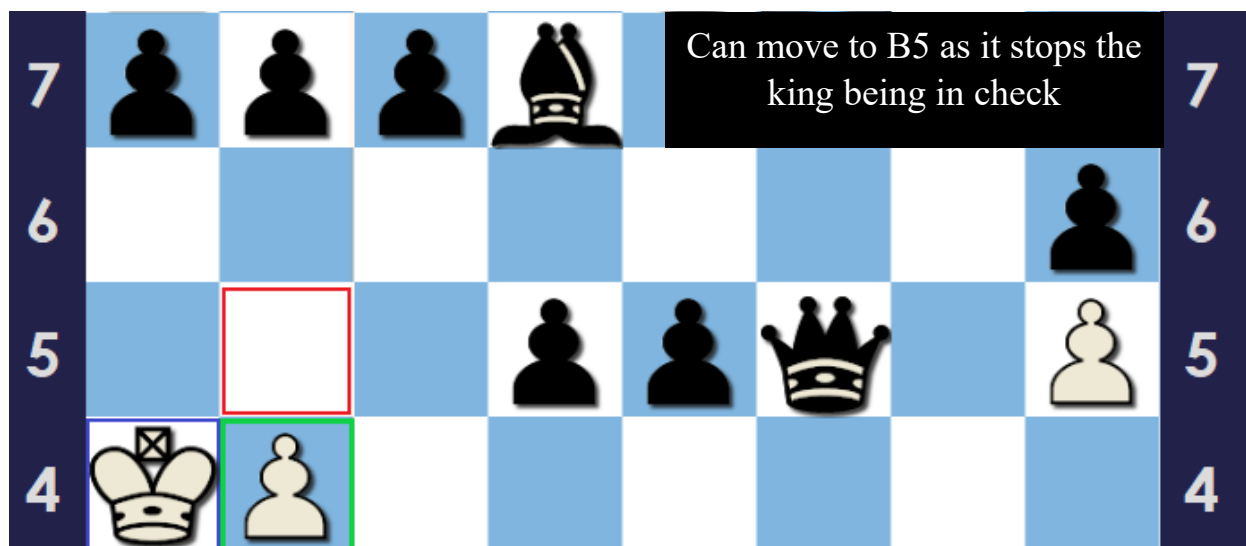
Current Position	Desired Position	In Check?	Data Type	Expected Outcome
B2	B3/B4	Yes	Normal/ Borderline	Can Move (Only if B3/B4 doesn't result in a white check & doesn't travel through any piece)

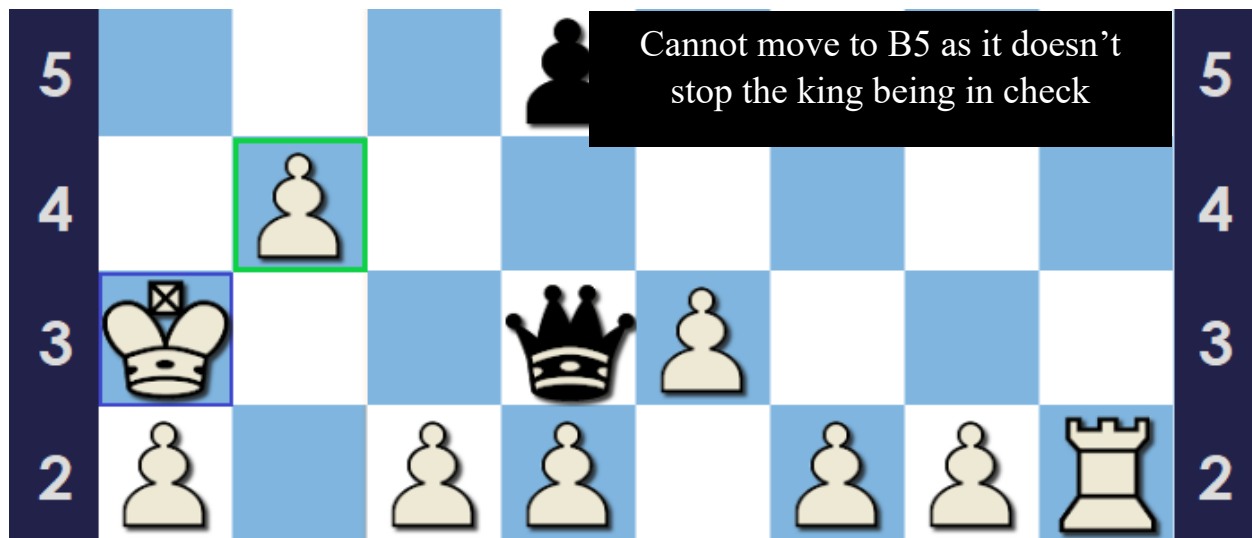




Description: testing if the pawn can move one square forward anywhere on the board (when it is not in its starting position & in check)

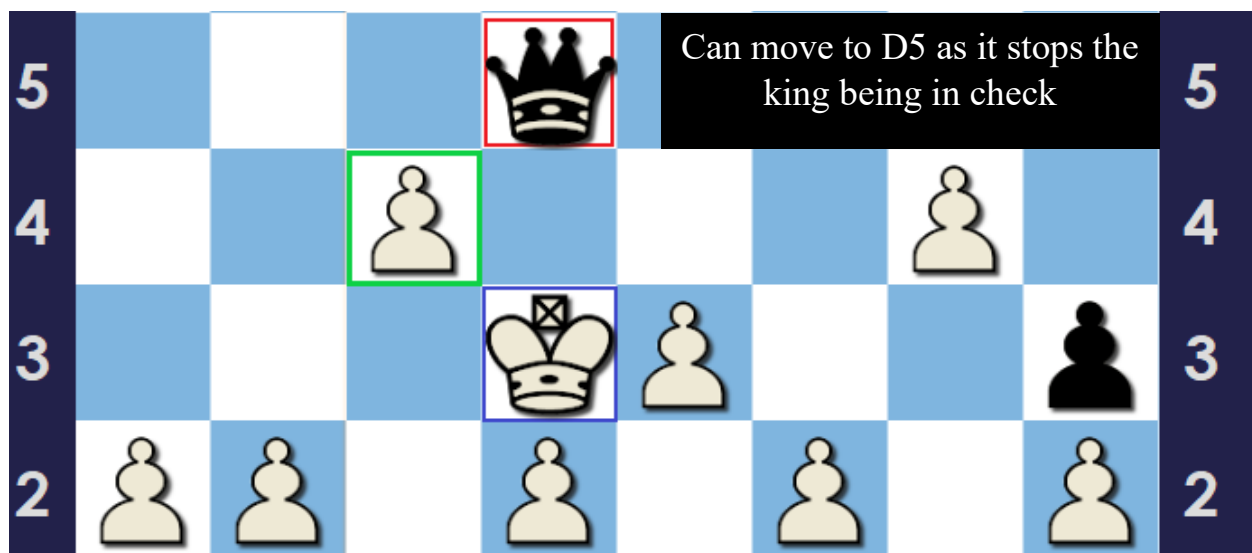
Current Position	Desired Position	In Check?	Data Type	Expected Outcome
B4	B5	Yes	Normal	Can Move (Only if B5 doesn't result in a white check & doesn't travel through any piece)

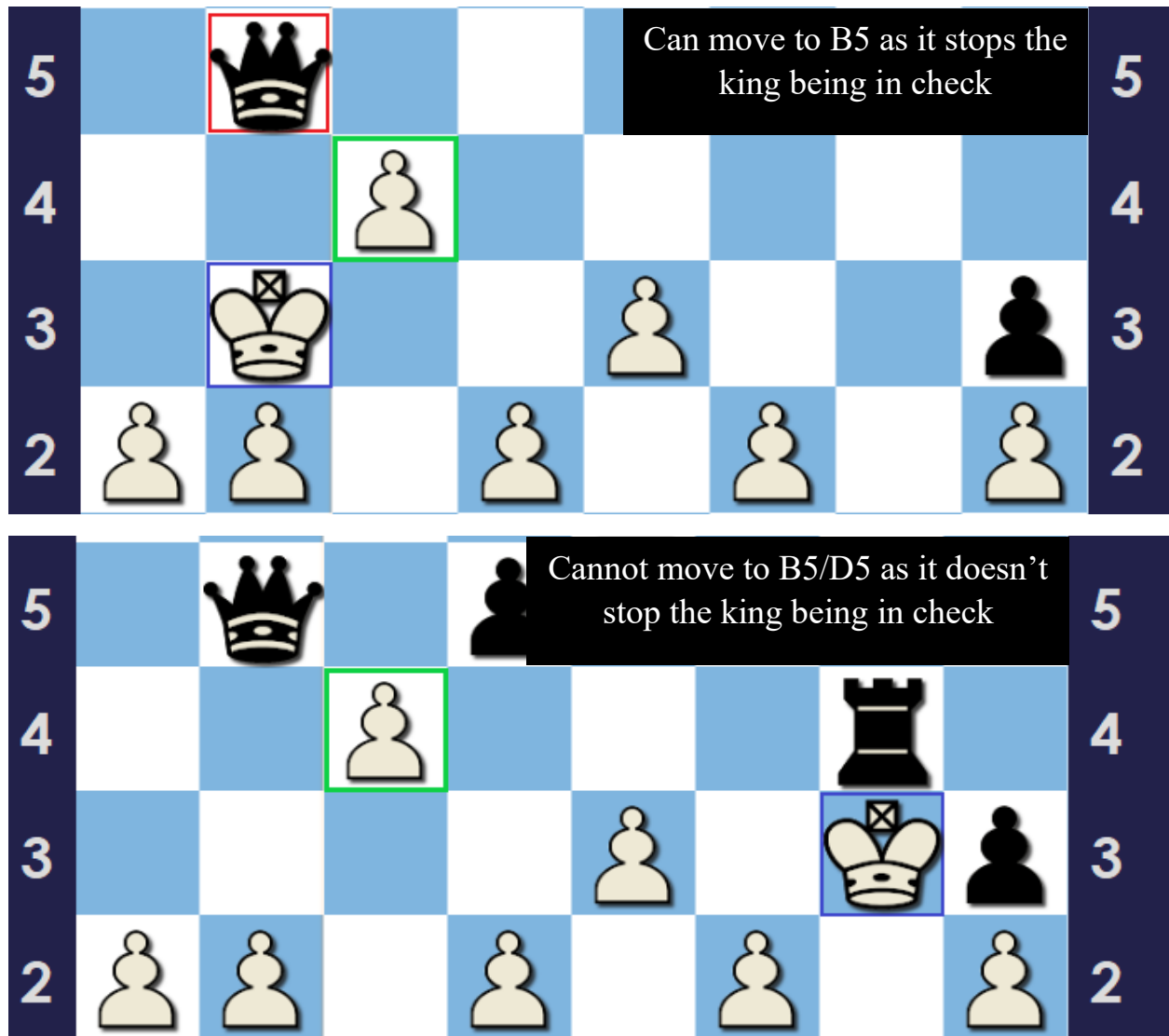




Description: testing if the pawn can move diagonally when there is a black piece in that square (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
C4	B5/D5	Yes	Normal	Can Move (Only if B5/D5 doesn't result in a white check & doesn't travel through any piece)





Rook

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the rook can move vertically

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D3	D1/D2//D4/D5/D6/D7/D8	No	Normal/Borderline	Can Move (Only if D1/D2/D4/D5/D6/D7/D8 doesn't result in a white check & doesn't travel through any piece)



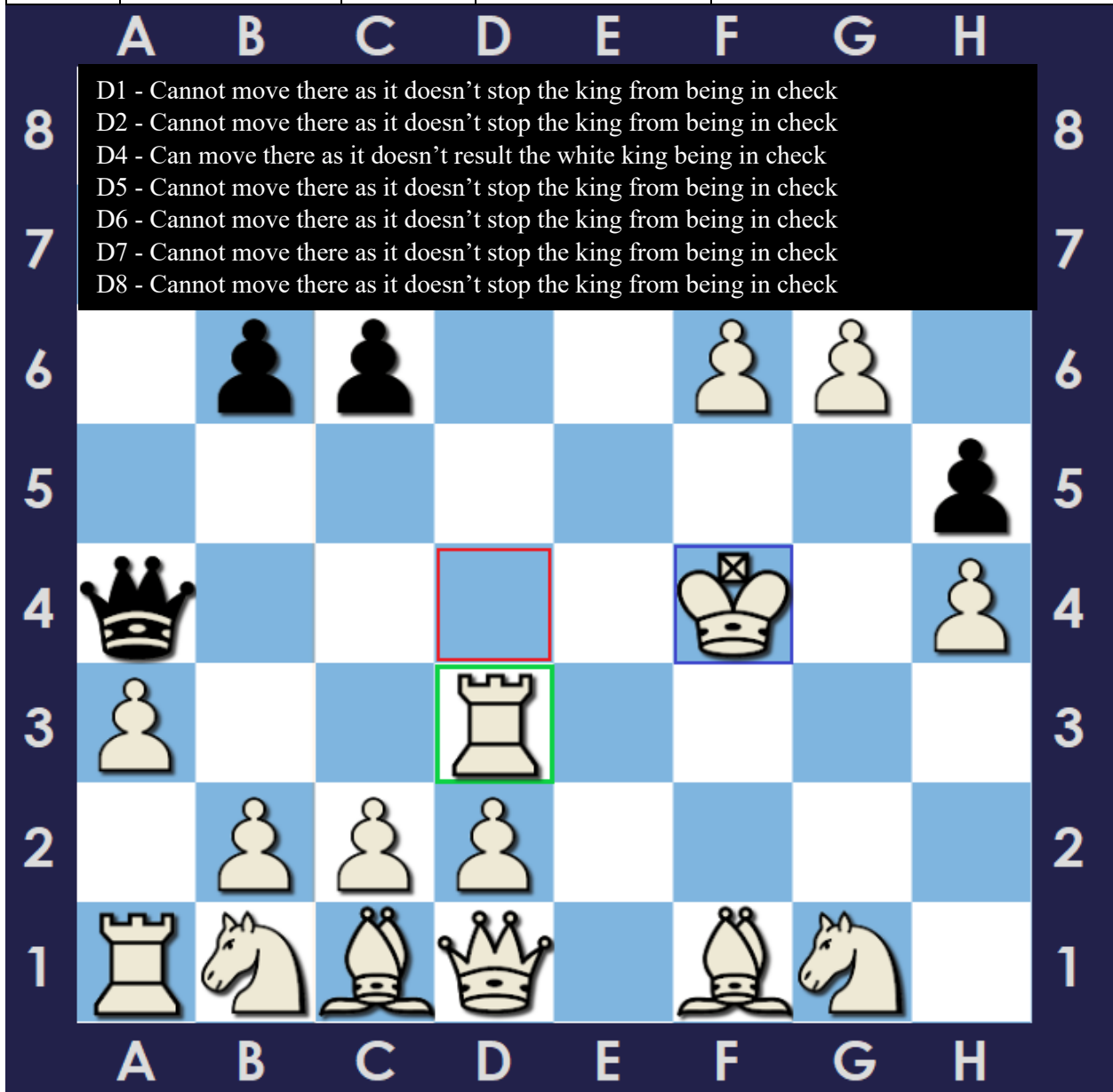
Description: testing if the rook can move horizontally

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D4	A4/B4/C4/E4/F4/G4/H4	No	Normal/Borderline	Can Move (Only if A4/B4/C4/E4/F4/G4/H4 doesn't result in a white check & doesn't travel through any piece)



Description: testing if the rook can move vertically (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D3	D1/D2//D4/D5/D6/D7/D8	No	Normal/Borderline	Can Move (Only if D1/D2/D4/D5/D6/D7/D8 doesn't result in a white check & doesn't travel through any piece)



Description: testing if the rook can move horizontally

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D4	A4/B4/C4/E4/F4/G4/H4	Yes	Normal/Borderline	Can Move (Only if A4/B4/C4/E4/F4/G4/H4 doesn't result in a white check & doesn't travel through any piece)



Knight

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the pawn can move one square/two squares forward from the start position

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D5	C7/E7/C3/E3/B6/F6/B4/F4	No	Normal/ Borderline	Can Move (Only if C7/E7/C3/E3/B6/F6/B4/F4 doesn't result in a white check)

Chessboard diagram showing a knight on D5. The knight is on a light square. Red squares highlight the possible moves for the knight: C7, E7, C3, E3, B6, F6, B4, and F4. A green square highlights the knight's current position D5. The board is labeled with files A-H and ranks 1-8.

C7 - Can move there as it doesn't result the white king being in check
 E7 - Can move there as it contains a black piece, and it doesn't result the white king being in check
 C3 - Can move there as it doesn't result the white king being in check
 E3 - Can move there as it doesn't result the white king being in check
 B6 - Can move there as it doesn't result the white king being in check
 F6 - Cannot move there as it contains a white piece
 B4 - Can move there as it doesn't result the white king being in check
 F4 - Can move there as it doesn't result the white king being in check

Description: testing if the pawn can move one square/two squares forward from the start position (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D5	C7/E7/C3/E3/B6/F6/B4/F4	Yes	Normal/ Borderline	Can Move (Only if C7/E7/C3/E3/B6/F6/B4/F4 doesn't result in a white check)



Bishop

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the bishop can move diagonally right

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D2	C1/E3/F4/G5/H6	No	Normal/ Borderline	Can Move (Only if C2/E3/F4/G5/H6 doesn't result in a white check)



Description: testing if the bishop can move diagonally left

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D2	A5/B4/C3/E1	No	Normal/ Borderline	Can Move (Only if A5/B4/C3/E1 doesn't result in a white check)



Description: testing if the bishop can move diagonally right (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D2	C1/E3/F4/G5/H6	Yes	Normal/ Borderline	Can Move (Only if C2/E3/F4/G5/H6 doesn't result in a white check)



Description: testing if the bishop can move diagonally left

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
D2	A5/B4/C3/E1	Yes	Normal/ Borderline	Can Move (Only if A5/B4/C3/E1 doesn't result in a white check)

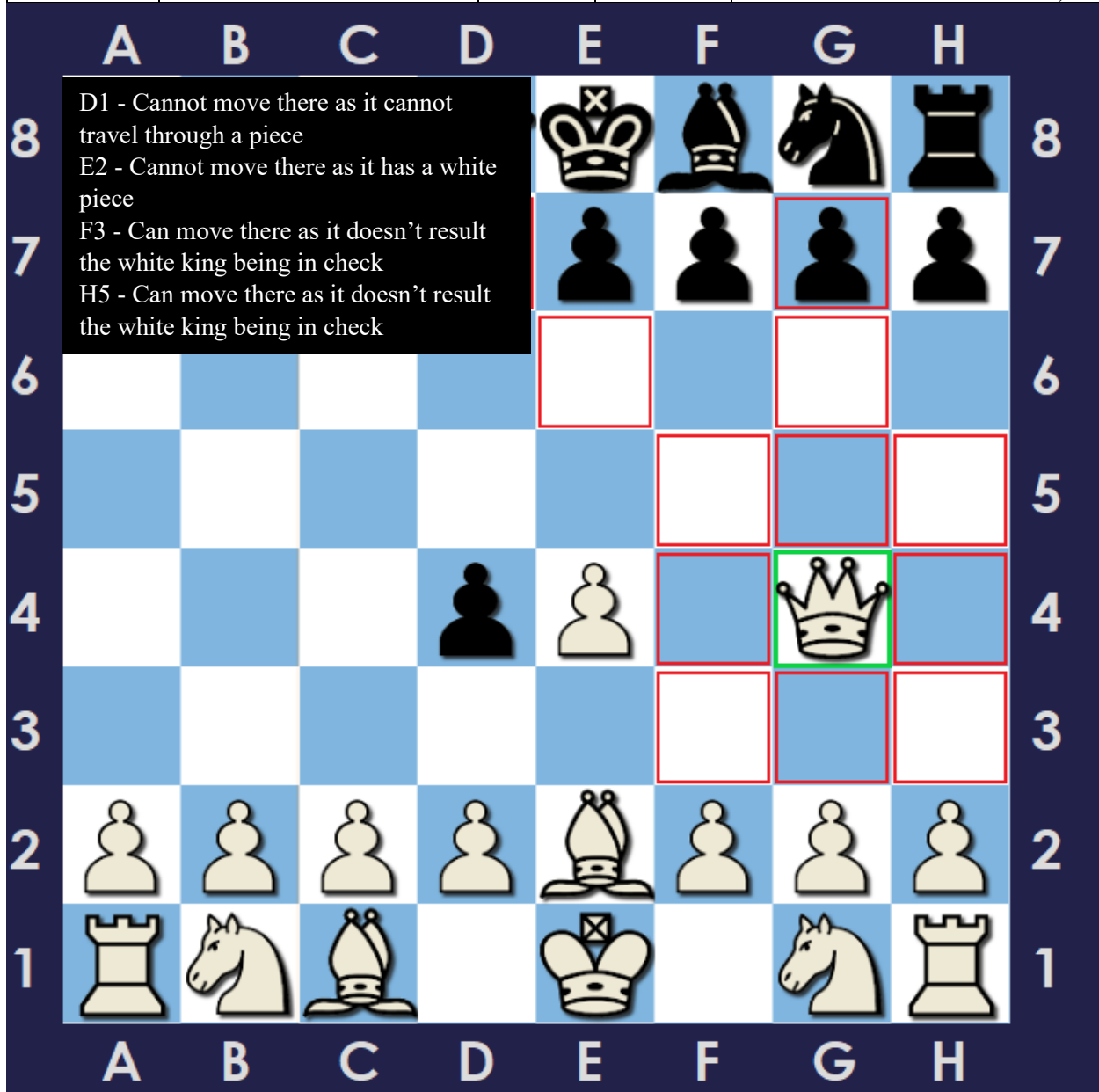


Queen

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the queen can move diagonally right

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	D1/E2/F3/H5	No	Normal/ Borderline	Can Move (Only if D1/E2/F3/H5 doesn't result in a white check)



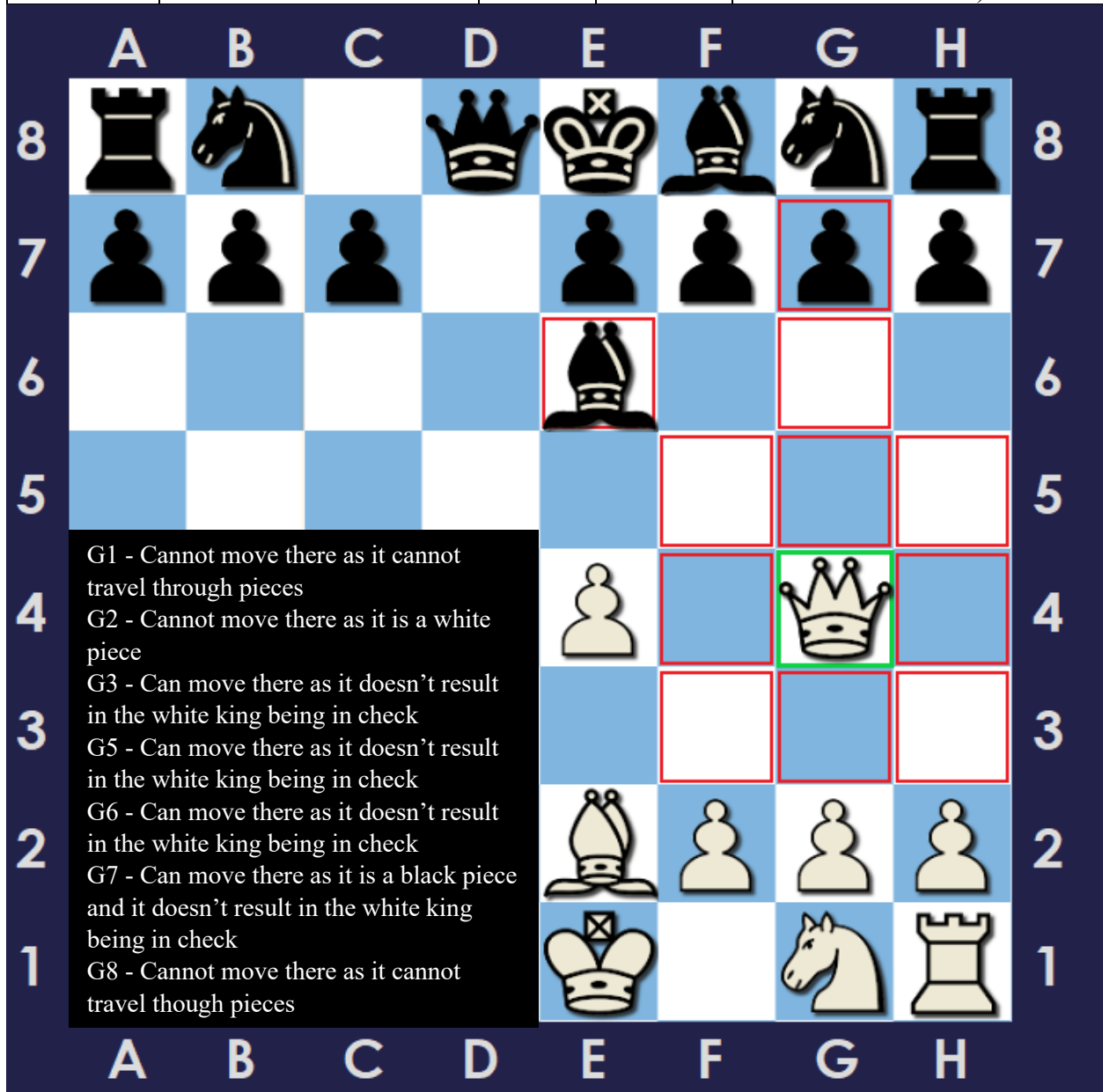
Description: testing if the queen can move diagonally left

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	C8/D7/E6/F5/H3	No	Normal/ Borderline	Can Move (Only if C8/D7/E6/F5/H3 doesn't result in a white check)



Description: testing if the queen can move vertically

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	G1/G2/G3/G5/G6/G7/G8	No	Normal/ Borderline	Can Move (Only if G1/G2/G3/G5/G6/G7/G8 doesn't result in a white check)



Description: testing if the queen can move horizontally

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	A4/B4/C4/D4/E4/F4/H4	No	Normal/ Borderline	Can Move (Only if A4/B4/C4/D4/E4/F4/H4 doesn't result in a white check)



Description: testing if the queen can move diagonally right (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	D1/E2/F3/H5	Yes	Normal/ Borderline	Can Move (Only if D1/E2/F3/H5 doesn't result in a white check)



Description: testing if the queen can move diagonally left

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	C8/D7/E6/F5/H3	Yes	Normal/ Borderline	Can Move (Only if C8/D7/E6/F5/H3 doesn't result in a white check)



Description: testing if the queen can move vertically

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	G1/G2/G3/G5/G6/G7/G8	Yes	Normal/ Borderline	Can Move (Only if G1/G2/G3/G5/G6/G7/G8 doesn't result in a white check)



Description: testing if the queen can move horizontally

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
G4	A4/B4/C4/D4/E4/F4/H4	Yes	Normal/ Borderline	Can Move (Only if A4/B4/C4/D4/E4/F4/H4 doesn't result in a white check)

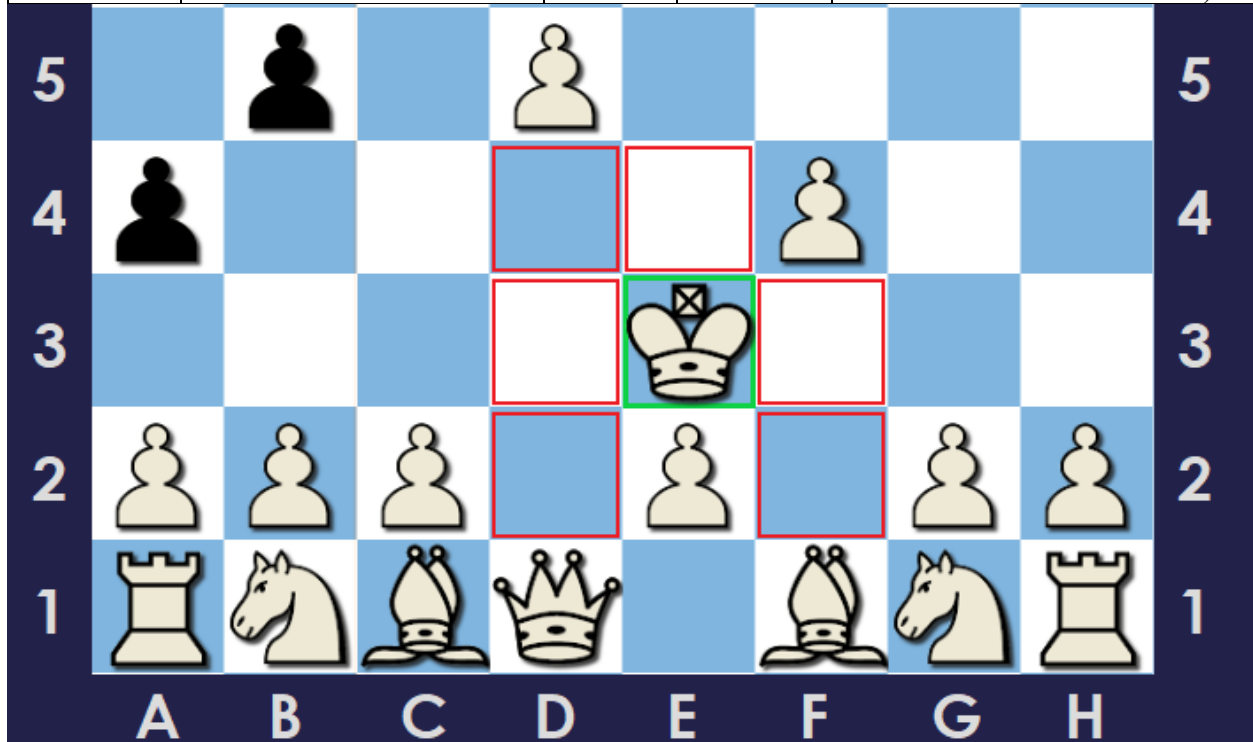


King

Note - only testing white pieces, as the code is the exact same for black, however the board is reverse, and the colour of pieces within the calculation of the moves change.

Description: testing if the king can move

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
E3	D2/E2/F2/F3/F4/E4/D4/D3	No	Normal/ Borderline	Can Move (Only if D2/E2/F2/F3/F4/E4/D4/D3 doesn't result in a white check)



D2 - Can move there as it doesn't result in the white king being in check

E2 - Cannot move there as it is a white piece

F2 - Can move there as it doesn't result in the white king being in check

F3 - Can move there as it doesn't result in the white king being in check

F4 - Cannot move there it is a white piece

E4 - Can move there as it doesn't result in the white king being in check

D4 - Can move there as it doesn't result in the white king being in check

D3 - Can move there as it doesn't result in the white king being in check

Description: testing if the king can move (*in check*)

Current Position	Desired Position	In Check?	Data Type	Expected Outcome
E3	D2/E2/F2/F3/F4/E4/D4/D3	Yes	Normal/ Borderline	Can Move (Only if D2/E2/F2/F3/F4/E4/D4/D3 doesn't result in a white check)

The chessboard diagram shows a black king on E3 in check from a white queen on D3. The king's possible escape squares (D2, E2, F2, F3, F4, E4, D4, D3) are highlighted with red borders. The board has white pieces on A1, B1, C1, D1, E1, F1, G1, H1 and black pieces on B5, C5, E5, F5, G5, H5.

D2 - Can move there as it doesn't result in the white king being in check

E2 - Cannot move there as it is a white piece

F2 - Can move there as it doesn't result in the white king being in check

F3 - Can move there as it doesn't result in the white king being in check

F4 - Cannot move there it is a white piece

E4 - Cannot move there as it doesn't stop the white king being in check

D4 - Cannot move there as it doesn't stop the white king being in check

D3 - Can move there as it doesn't result in the white king being in check

AI Opponent system testing

Minimax

Description: testing to see if the minimax algorithm chooses the correct move

Test Data		Data Type	Expected Outcome
C2 → H7		Borderline	H8 → H7

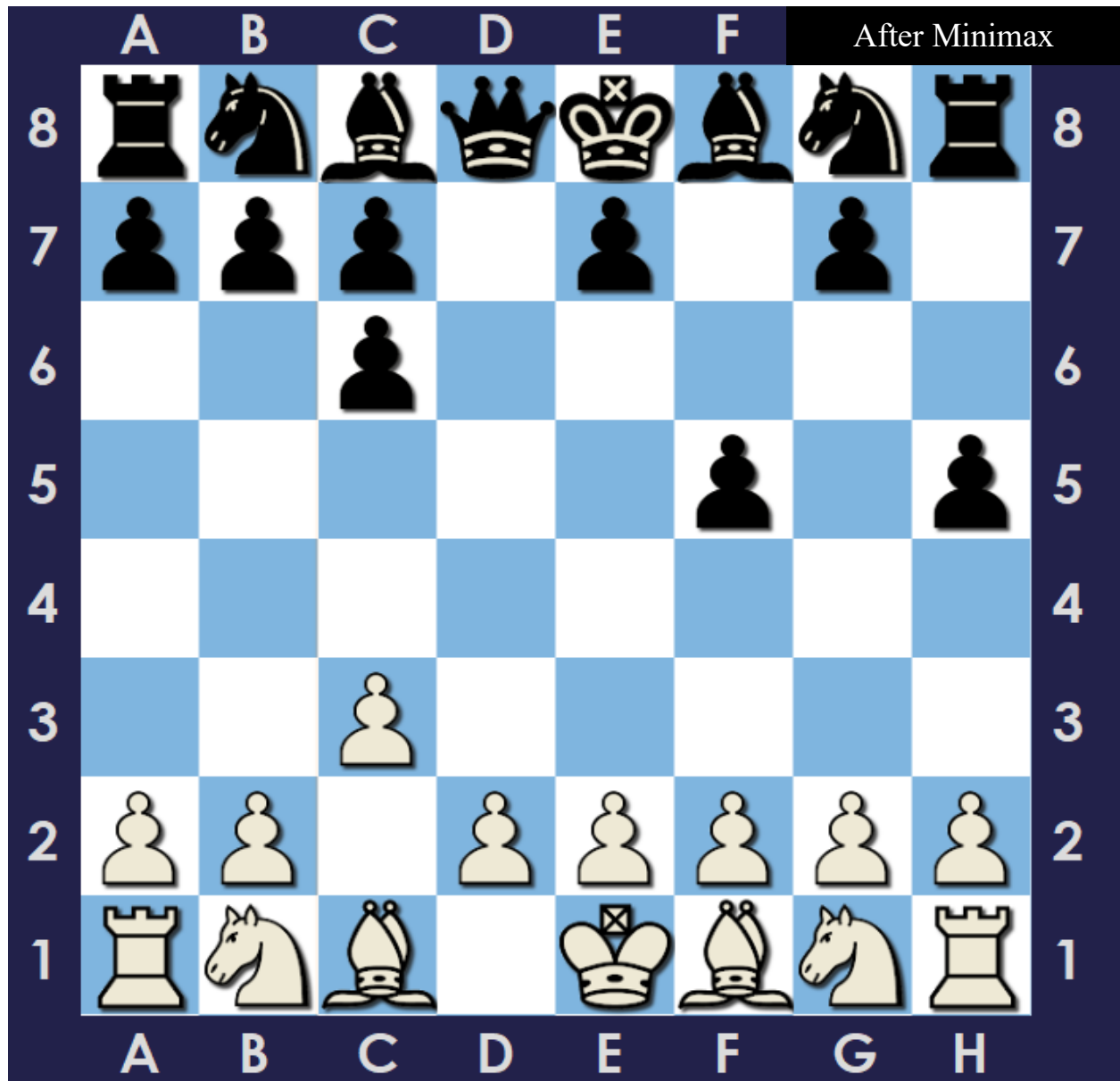
A B C D E F						Before Minimax		
8								8
7								7
6								6
5								5
4								4
3								3
2								2
1								1
A B C D E F						G H		



Description: testing to see if the minimax algorithm chooses the correct move

Test Data		Data Type	Expected Outcome
A4 → C6		Normal	B8/B7/D7 → C6

Before Minimax							
A	B	C	D	E	F		
8							
7							
6							
5							
4							
3							
2							
1							
A	B	C	D	E	F	G	H



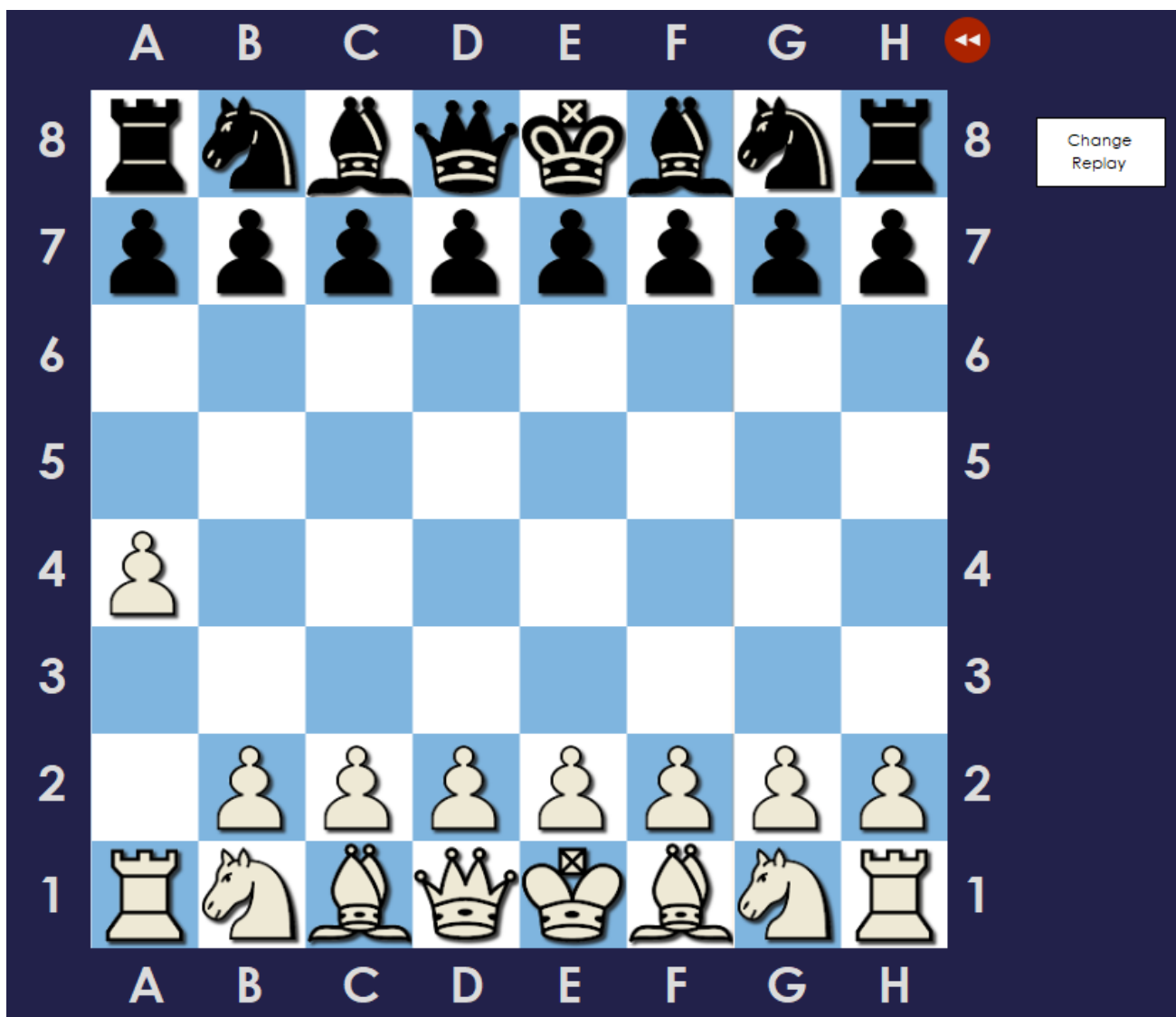
Replays system testing

Loading Replays

Description: testing whether replay 2 will load from the database to the form

Replay number	Moves inside replay number	Data Type	Expected Outcome
2	A2A4	Normal	Replay Loaded

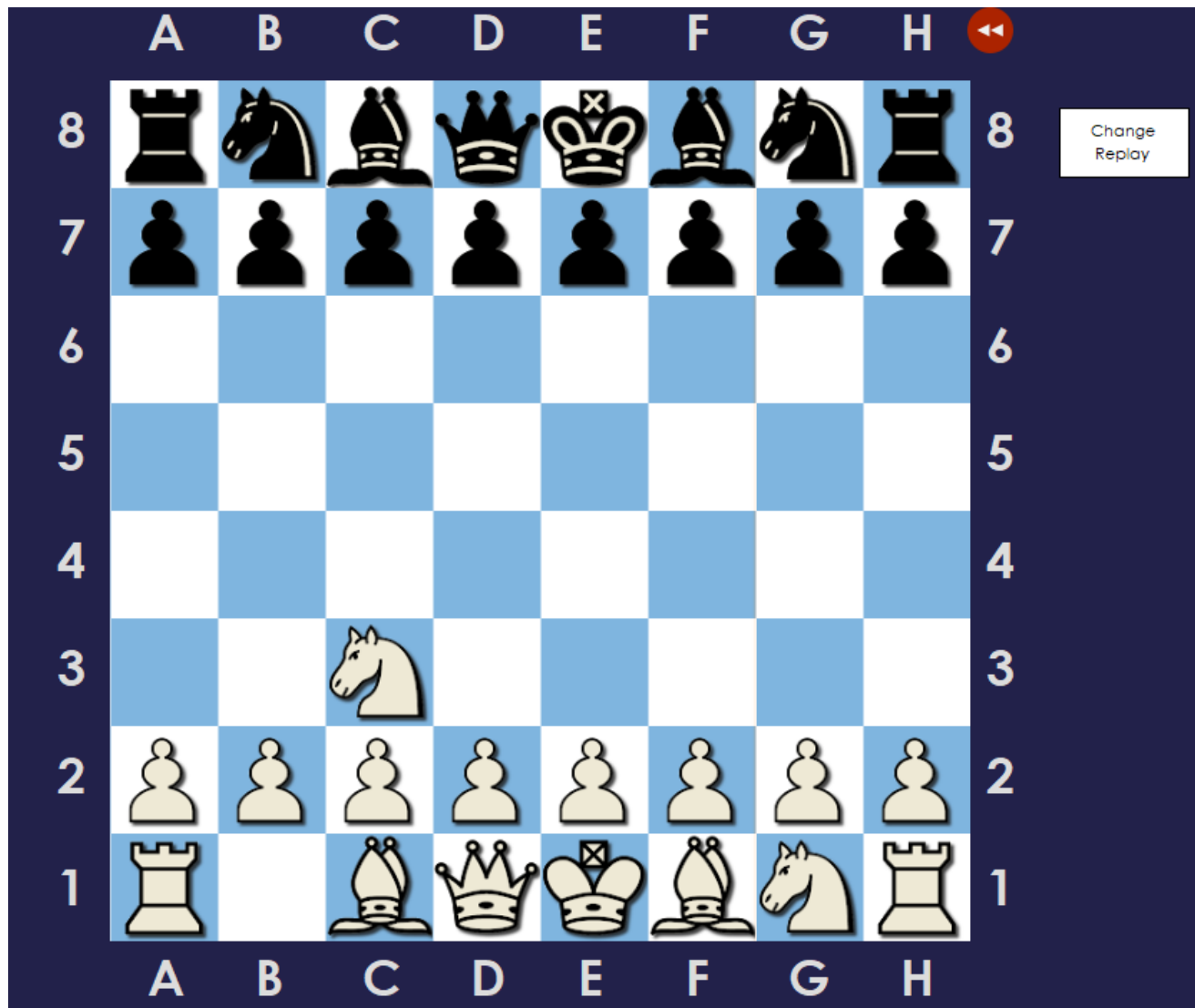
PreMove2 ▾ Move2 ▾
 A2 A4 Moves stored in database



Replay number	Moves inside replay number	Data Type	Expected Outcome
1	B1C3	Borderline	Replay Loaded

PreMove1 ▾	Move1 ▾
B1	C3

Moves stored in database



Leaderboard system testing

Loading Leaderboard

Description: testing to see if the leaderboard works

Username	Wins	Data Type	Expected Outcome
Admin, Davidl, Tester	3,5,9	Normal	Tester, Daniel, Admin

ID	Username	Wins	Draws	Loses
1	Admin	3	2	8
2	Davidl	5	1	1
3	Tester	9	0	0

Leaderboard

Username	Wins	Draws	Loses	ProfilePicture
Tester	9	0	0	Skin5
Daniel	5	0	3	Locked8
Davidl	5	1	1	Default
Admin	3	2	8	Skin4
Jenson	2	1	0	Default
Harry	2	0	1	Default
Saranyan	1	0	1	Default
Ben	1	0	0	Default
James	0	0	0	Default
Brandon	0	0	0	Default

Tester



1

Daniel



2

Davidl



3

Customisation system testing

Profile Picture

Description: testing to see if the profile picture updates

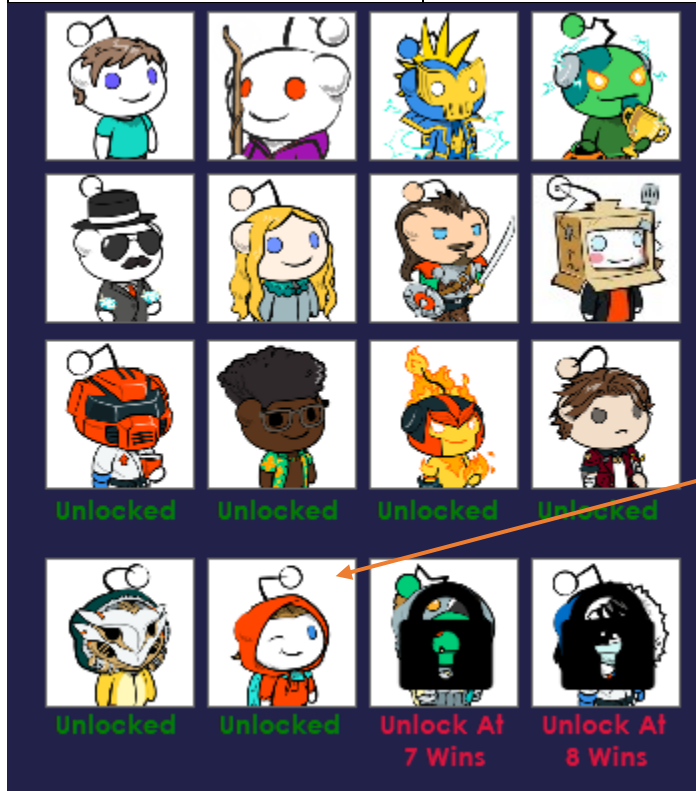
Test Data	Data Type	Expected Outcome
Locked3	Normal	Profile picture changed



ID	Username	ProfilePicture
1	Admin	Locked3

Description: testing to see if the profile picture updates

Test Data	Data Type	Expected Outcome
Locked6	Borderline	Profile picture changed

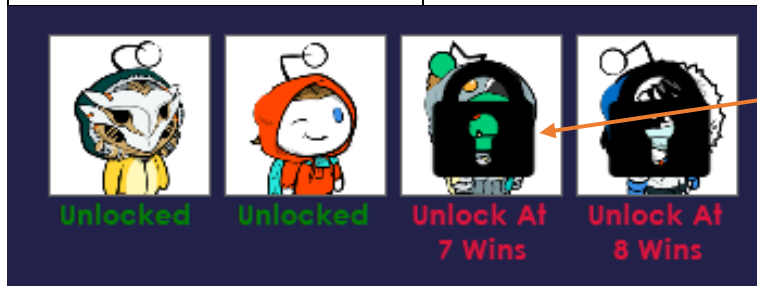


User clicks on Locked 6

ID	Username	ProfilePicture
1	Admin	Locked6

Description: testing to see if the user clicks on a profile picture they haven't unlocked

Test Data	Data Type	Expected Outcome
Locked7	Invalid	Profile picture unchanged (Not unlocked)



User clicks on Locked 7;
nothing happens as the
picture box is not enabled

Board Colour

Description: testing to see if the board colour updates

Test Data	Data Type	Expected Outcome
Red board	Normal	Board colour changed



ID	Username	BoardColou
1	Admin	RedBoard

Description: testing to see if the board changes colour

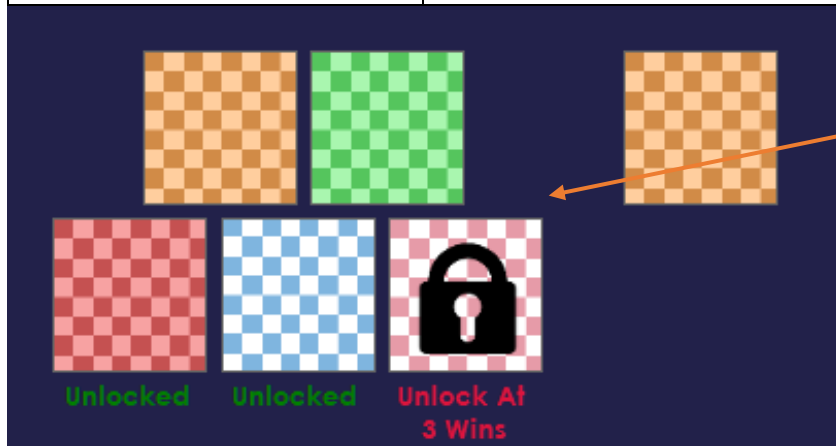
Test Data	Data Type	Expected Outcome
Blue board	Borderline	Board colour changed



ID	Username	BoardColou
1	Admin	BlueBoard

Description: testing to see if the user clicks on a board colour they haven't unlocked

Test Data	Data Type	Expected Outcome
Pink board	Invalid	Board colour unchanged (Not unlocked)

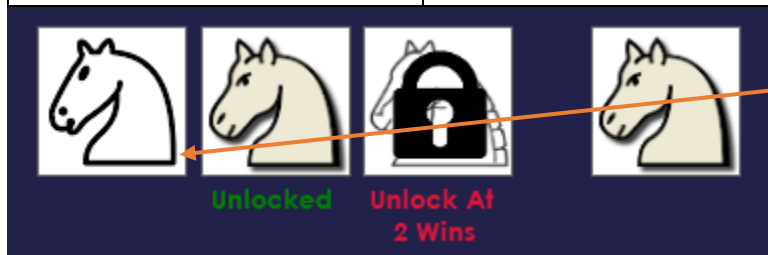


User clicks on pink board;
nothing happens as the
picture box is not enabled

Piece Set

Description: testing to see if the user clicks on set 1

Test Data	Data Type	Expected Outcome
Set1	Normal	Piece set changed

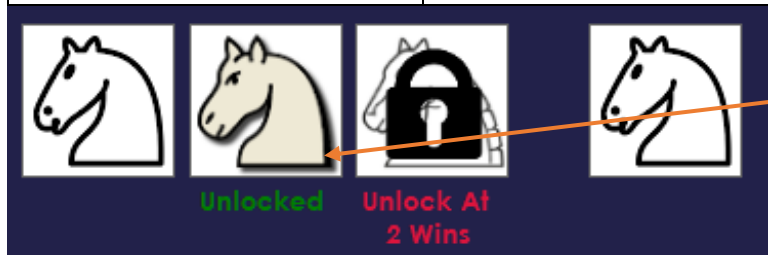


User clicks on set 1

ID	Username	PieceType
1	Admin	Set1

Description: testing to see if the user clicks on set 2

Test Data	Data Type	Expected Outcome
Set2	Borderline	Piece set changed

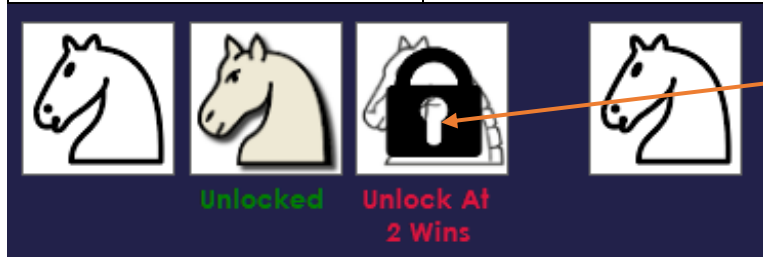


User clicks on set 2

ID	Username	PieceType
1	Admin	Set2

Description: testing to see if the user clicks on the board colour they haven't unlocked

Test Data	Data Type	Expected Outcome
Set3	Invalid	Piece set unchanged (Not unlocked)



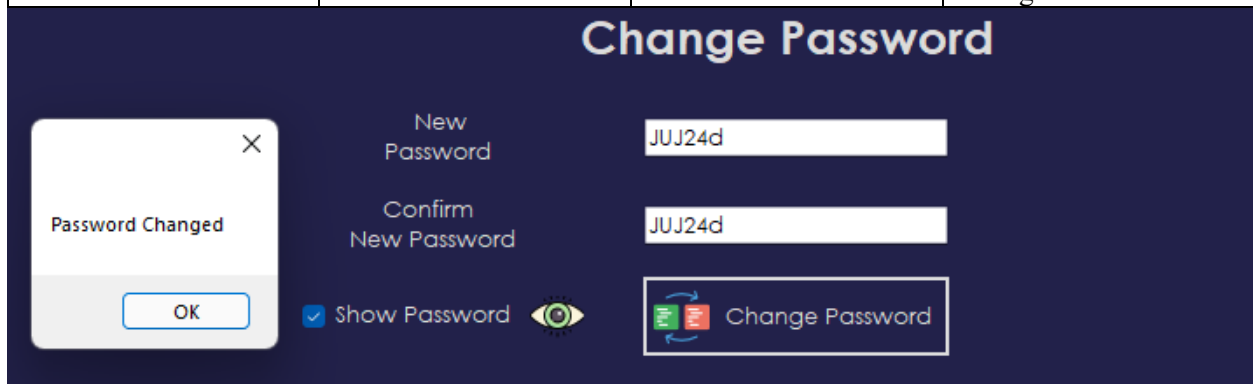
User clicks on set 3;
nothing happens as the
picture box is not enabled

Settings system testing

Change Password

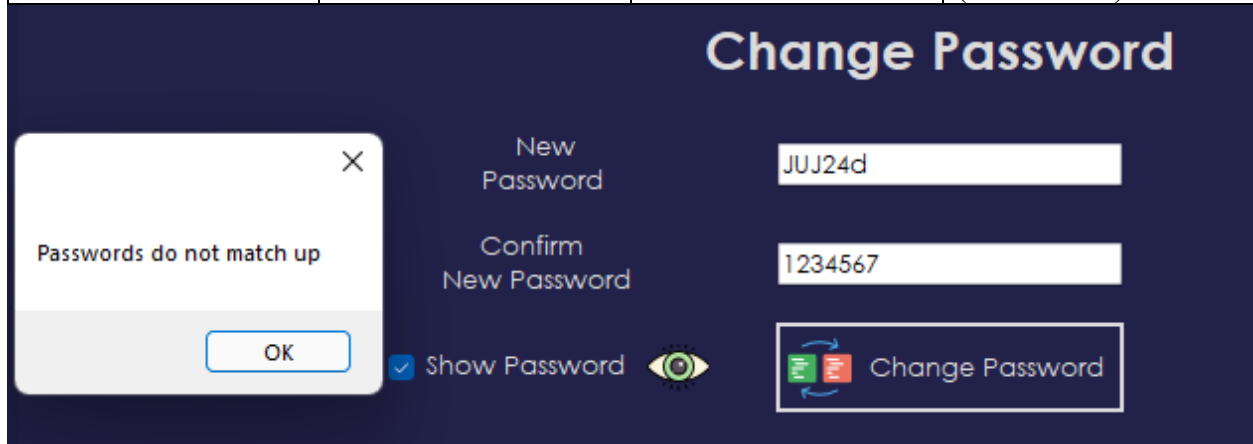
Description: testing a valid password

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	JUJ24d	Normal	Changed Password



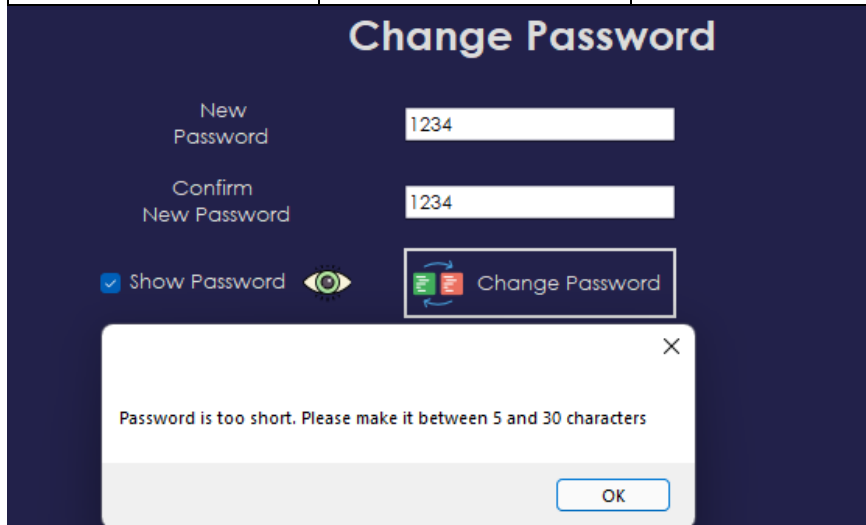
Description: testing passwords that don't match

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
JUJ24d	1234567	Invalid	Invalid Password (Don't match)



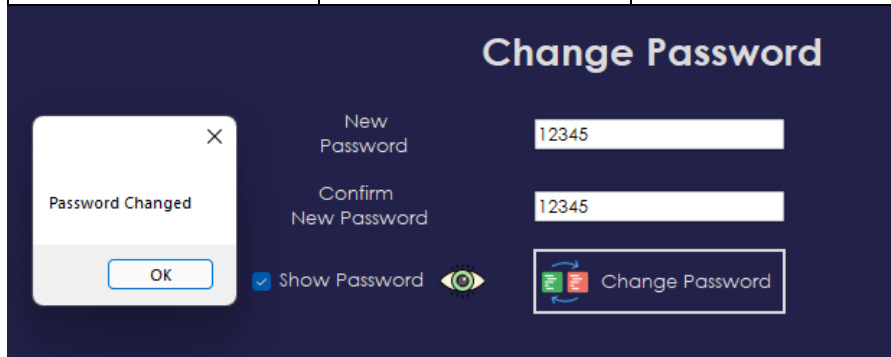
Description: testing a password that is too short

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
1234	1234	Invalid	Invalid Password (Too short)



Description: testing a password that is a boundary value

Test Data 1 (Password)	Test Data 2 (Confirm)	Type of Test Data	Expected Outcome
12345	12345	Borderline	Changed Password



Appraisal

Objectives:

Signing up - this objective was successively met, as the user can sign-up with their account being stored into the database. This is referred to on page 60. Although this objective was met, there are certain improvements which could be made with the password validation (such as stronger passwords must be created). Overall, I would say this objective was met.

Logging in - after the sign-up was complete, a login form wasn't the most complex to program, therefore this objective was definitely met. The only issue I had with this form is that if a user forgot their password, which I mentioned within the user feedback. As a whole, I thought this objective was met, and this is referred to on page 67.

Game logic - game logic was definitely the most challenging objective to complete. All the possible moves that can happen throughout a game was difficult to calculate, especially when calculating moves referring to the checking of

the king. Most of this objective was met, however I feel the way I completed this was not the most efficient way to do so, as there was a few repeated blocks of code within the 'game' class.

AI opponent - both these objectives were met correctly. The 'easy' mode was simple to implement, and the 'hard' mode took some time, however, became achievable after a few syntax errors were fixed in the classes. The minimax algorithm only goes to second depth, as when I tried third depth the program time took too long.

Replays - the replays form was a form I feel like didn't completely meet the objectives. This is because there was a few improvements to made to it, such as only storing 5 replays, and correctly loading pawn promotions. Overall, a brief understanding of the objectives was met, but not at a level the others was.

Leaderboard - this form was very simple to do as it was ordering the users regarding their wins. Therefore, this objective was correctly met.

Customisation - this was a very positive section within the user feedback section. The objectives were thoroughly met in this section, and I was very happening with the way it turned out, giving my own twist to a chess engine.

Settings - this objective was successively met, as the user can sign-up with their account being stored into the database. This is referred to on page 60. Although this objective was met, there are certain improvements which could be made with the password validation (such as stronger passwords must be created). Overall, I would say this objective was met.

There was plenty of positive user feedback when tested the program. One example of this is the user interface.

Testers said that visually the game was at a good level which I was pleased about as it's very important to have this in any game. There was a couple bits of user feedback which I implemented into the program towards the end of the project completion:

Board colours - most of my project was created with the black colour being on the bottom of the board and the white being in top. The problem with this is that is very unusual, as most boards has the white colour at the bottom of the board. Consequently, I filled the board round the correct way, adapting the code forming the correct syntax

Forgotten password - after a user had tested the program one and came back to the program, the tester had forgotten their password and had no way of signing in, unless accessing the database (which shouldn't ever happen apart from the programmer). Therefore, I created the forgotten password algorithm, which used a confirmation code and their email address for the code - which they found very useful when testing the program after this happened.

Users were really positive about the personalisation features on the program, which gave the user their own stamp on the game. To conclude, most of the users feedback was positive news, however there is always going to be a few negatives reviews. Some of the negative reviews I managed to correct during implementation of the program, however there is a few still within the final project form. Firstly, a negative side to the program was towards the creating/changing of a password. When signing-up it would have been better if not only should a password have a good length, but also a good strength (e.g., one capital latter, one number, one symbol), and when changing the password, not being able to change it to your current password should've been an option added onto the form. In addition, 'easy mode' with the AI, user's found too simple. Moving the pieces on a board randomly was the idea for an easy mode with the AI, however when testing user's found it tedious because of how simple it is. This is because even if you are brand new to chess, there is no challenge playing against that level of AI, meaning you'll most likely win. However, this is a negative implementation to the project as even new user's won't learn from that basic algorithm and therefore won't improve.

In all projects, there is always improvements to be made. One improvement to be made is changes towards the minimax algorithm. Although this algorithm works fine, the more depths I added, the processing time for each move grew substantially. Reflecting upon this, I would change this to attempting neural networks to improve efficiency with the AI. Moreover, I would change how the user moves a chess piece on the board. Despite, clicking squares on the board and moving the piece through this was working well, I feel dragging and dropping would've been a better choice to include in the program. Tester's preferred the drag and drop when moving their pieces, as it feels like 'less of a program'. For example, within chess.com they use dragging and dropping, which just feels better for the user. So, when a user plays a game where they click the positions they move to, it feels rather blocky/not smooth. Finally, in my opinion the replay form would need changing. There are a few changes I would make to this section, such as allowing the user to store more than 5 replays, a way to do this is rather than storing the data in access, store it in a test file. Also, another change I would make to the replay form would be when a pawn promotion happens to change that piece visually on the board to whatever the user had selected. Despite the improvements I listed, I think my project met the objectives to a suitable level, and as a whole was a success.

Daniel Lawrence

Candidate Number: 7300

Centre Number: 61715